The Backend Engineer's Library

APIs and Service Design

Volume 8

Contents

Chapter 1 — API Design Principles and Contracts

What this chapter covers. Every distributed system is a graph of trust boundaries, and every edge in that graph is an API. Whether the caller is a browser, a mobile app, a partner's backend, or the service three hops away in your own mesh, the contract between producer and consumer determines how fast teams can ship, how safely they can evolve, and how gracefully they fail. This chapter builds the foundation for the entire volume: what an API contract is and why it matters more than the implementation behind it, the design principles that make contracts predictable and operable at scale, the lifecycle a contract moves through from sketch to sunset, and the tooling that makes contracts machine-checkable. We anchor the discussion in a real OpenAPI 3.1 contract and introduce the patterns — versioning, pagination, error semantics, idempotency — that later chapters treat in depth.

Learning goals — after this chapter you should be able to:

- Define an API contract precisely (syntax, semantics, and evolution guarantees) and explain why the contract, not the implementation, is the unit of coupling in a distributed system.
- Apply six design principles — consistency, predictability, discoverability, minimal surprise, evolvability, and operability — when reviewing or authoring an API surface.
- Distinguish design-first from code-first workflows, name when each is appropriate, and describe the distributed-systems cost of code-first drift.
- Read and author a version-pinned OpenAPI 3.1.0 contract (with components, security schemes, and validation) and explain how it feeds code generation, documentation, and contract testing.
- Sketch the contract lifecycle (design → review → publish → version → deprecate → sunset) and map each stage to a concrete gate (lint, breaking-change check, approval).
- Explain contracts through a distributed-systems lens: deadline propagation, backward/forward compatibility windows, and the blast radius of a breaking change across dozens of consumers.

APIs as the stable seam in an unstable system

A single backend today may comprise 80 services owned by 12 teams deploying 30 times a day. The only thing that lets those teams move independently is the set of interfaces they agree not to break without coordination. An API contract is that agreement made precise.

Think of the contract as the *narrow waist* of your architecture. Implementations change — languages get replaced, databases migrate, frameworks churn — but if the contract holds, consumers do not need to care. When the contract is vague, every implementation detail leaks into every consumer, and a change in one service becomes a coordination event across ten.

For a senior backend engineer, API design is therefore not a cosmetic concern. It is the primary tool for managing coupling in a distributed organisation:

- **Coupling is proportional to contract surface.** A small, coherent contract limits the number of reasons a consumer can break. A large, inconsistent one maximises it.
- **Evolution cost is proportional to consumer count.** An internal RPC with two callers can be changed in an afternoon. A public REST endpoint with 400 consumers — some outside your company — cannot be changed without a multi-quarter deprecation.
- **Failure behaviour is part of the contract.** Timeouts, retries, rate limits, pagination semantics, and error codes determine whether a consumer can build a reliable system on top of yours. An undocumented retry policy is not a policy; it is an outage waiting to happen.

> **Distributed-systems lens.** In a monolith, a function signature is cheap to change — the compiler tells you every call site. In a distributed system, there is no global compiler. The contract *is* the compiler. A field renamed without a compatibility rule is a deserialization failure in a consumer you may not know exists, running a binary you did not build, in a region you do not operate.

What a contract actually contains

A complete contract answers three questions. Most teams formalise only the first and learn about the other two during an incident review.

1. Syntax — what bytes are valid

The structural shape: resource paths, HTTP methods, message schemas, field types, required versus optional, string formats, enum values, and wire encoding (JSON, Protobuf). This is what an IDL or specification language captures. Example: "`GET /v1/orders/{id}` returns `200` with an `Order` where `status` is one of `pending`, `paid`, `shipped`, `cancelled`."

2. Semantics — what the bytes mean and what side effects occur

Two APIs can be syntactically identical and semantically opposite. Is `POST /v1/orders` idempotent? Does `DELETE /v1/users/{id}` hard-delete or soft-delete? What ordering guarantees does `GET /v1/orders?sort=created_at` provide when two orders share a timestamp? Semantics include idempotency, ordering, consistency (read-your-writes? eventual?), auth requirements, rate limits, and pagination stability. Semantics are often documented in prose alongside the schema — the most common source of ambiguity.

3. Evolution — what is allowed to change without breaking callers

A contract without evolution rules is a snapshot, not an agreement. Rules such as "adding an optional field is backward-compatible; removing a required field is breaking; changing an enum's meaning is breaking" determine whether a producer can ship on Monday without coordinating with every consumer. Chapter 5 (Versioning) and Chapter 8 (Compatibility) formalise these rules for REST and Protobuf respectively; this chapter establishes the mental model.

Layer	Specified by	Enforced by	Example violation
Syntax	OpenAPI 3.1 / Protobuf IDL / JSON Schema	Linter, code generator, request validation	Renaming `order_id` to `id` without alias

Layer	Specified by	Enforced by	Example violation
Semantics	Documentation, RFC references, error model	Contract tests, example-based checks, review	Making `GET /orders` non-idempotent by adding a side effect
Evolution	Compatibility policy, deprecation header	Breaking-change detector (`oasdiff`, `buf breaking`)	Removing a field still read by 12% of traffic

---

Six principles for operable contracts

Principles are useful only if they can be checked in review. The six below each come with a concrete review question.

1. Consistency — one way to do one thing

If `GET /v1/users` paginates with `?page_token=&page_size=` then `GET /v1/orders` must not paginate with `?offset=&limit=&page=`. Consumers automate against your API; inconsistency forces per-endpoint special cases, which become per-endpoint bugs. Consistency applies to naming (`camelCase` vs `snake_case`), error envelopes, timestamp formats (RFC 3339 everywhere), and pagination (cursor everywhere or offset everywhere — never mixed without reason).

Review check: "Would a consumer who learned the pattern from one endpoint correctly predict the next?"

2. Predictability — honour the semantics you advertise

An operation labelled `GET` must be safe (no side effects) and idempotent. A `PUT` must be idempotent. A `DELETE` that returns `204` on success must not return `200` with a body on one resource and `204` on another. Predictability extends to status codes (Chapter 7): `400` means the client can fix the request; `500` means retrying the same bytes will not help.

3. Discoverability — a new consumer can succeed without asking you

Good contracts are self-describing: resource names match the domain language, errors carry machine-readable codes plus human-readable detail, and the contract is published where consumers already look (developer portal, schema registry, package registry). Stripe and AWS both invest heavily in discoverability not because their APIs are simple — they are not — but because scale makes one-to-one guidance impossible.

4. Least surprise — reuse what the platform already means

HTTP already defines methods, status codes, headers (`ETag`, `Cache-Control`, `Retry-After`, `Idempotency-Key`), and content negotiation. Protobuf already defines field presence and default values. An API that maps domain actions onto those platform semantics ("use `If-None-Match` for caching, not `?useCache=true`") is easier to operate because intermediaries — CDNs, gateways, service meshes — already understand the platform.

5. Evolvability — optimise for the second version, not the first

The first version of any API is wrong. The question is how expensive being wrong will be. Contracts that reserve extension points — optional fields, `additionalProperties: false` only where intentional, enumerated strings that consumers treat as open, `repeated` rather than fixed-arity — make the second version cheap. Contracts that require coordinated upgrades for every additive change make it expensive.

6. Operability — expose what operators need

Every contract should answer, before it is used in production: How is it authenticated? How is it rate-limited and what does `429` look like? What does pagination do under concurrent mutation? What is the timeout budget and is the operation retry-safe? An API whose happy-path example works but whose pagination is undefined under writes will cause correctness bugs at scale that no unit test catches.

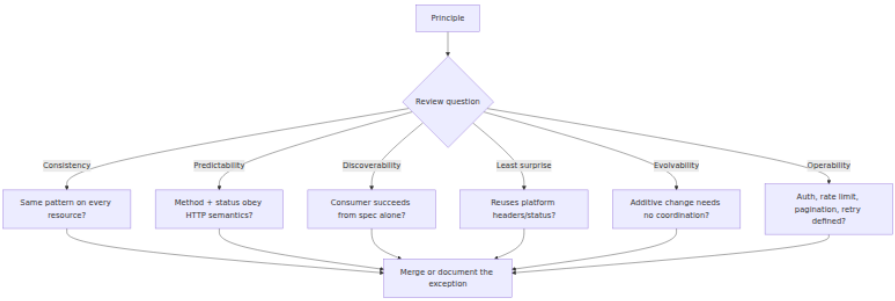


Figure 1-1: Six principles as review gates. Each principle maps to one question a reviewer can answer from the contract alone, before reading any implementation.

Contracts and schemas: choosing the description language

Different surfaces call for different contract languages. The table below is the decision you make before writing any schema.

Contract language	Version pinned in this book	Wire format	Strength	When to use
OpenAPI 3.1.0	OpenAPI 3.1.0 (2021-02-15), JSON Schema 2020-12	JSON over HTTP	Human-readable, browser-native, rich ecosystem (linters, generators, gateways)	External REST, public APIs, partner integrations
Protocol Buffers 3	protobuf 4.25.x / 5.x, `protoc` 24.x, `buf` 1.32.2	Binary (Protobuf) over HTTP/2 (gRPC) or JSON (gRPC-JSON transcoding)	Strong typing, efficient, streaming, codegen for 10+ languages	Internal service-to-service RPC

Contract language	Version pinned in this book	Wire format	Strength	When to use
AsyncAPI 3.0	AsyncAPI 3.0.0	JSON/Avro/ Protobuf over Kafka, AMQP, etc.	Describes event-driven surfaces	Event buses, webhooks (see Vol 10)
JSON Schema 2020-12	JSON Schema 2020-12	JSON	Standalone schema validation	Webhooks, config payloads, wherever OpenAPI envelope is too heavy

No single language covers every surface. A typical organisation at scale uses OpenAPI for its edge and Protobuf for its interior, with AsyncAPI for its event layer. The discipline is not picking one — it is ensuring each surface has *a* machine-readable contract and that contracts are not allowed to drift from the implementation (see "Design-first vs code-first" below).

The contract lifecycle

A contract is a living artifact with a lifecycle longer than any single deployment. Treating it as a file that is edited and forgotten is how breaking changes slip into production.

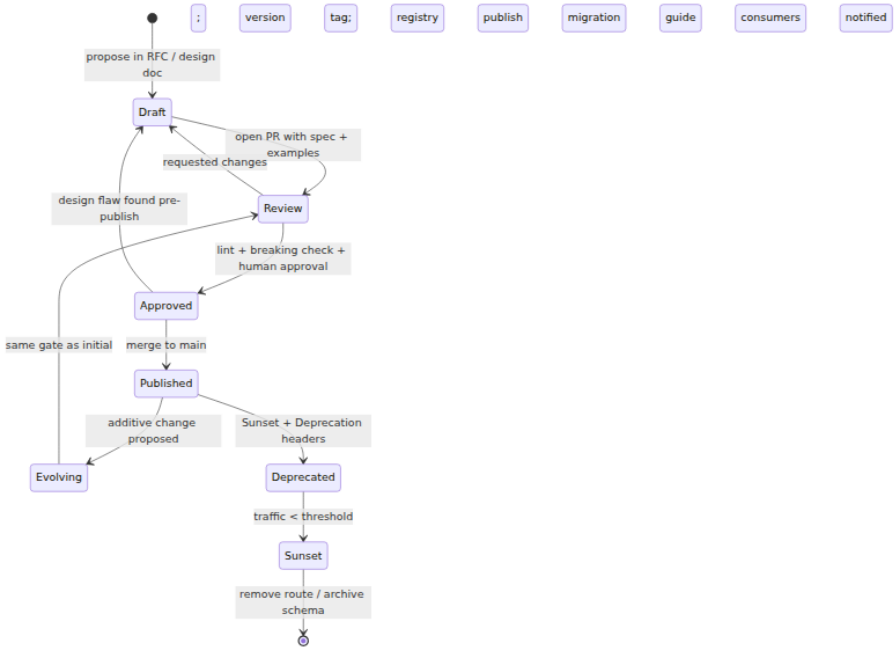


Figure 1-2: Contract lifecycle. Every transition has a gate; the most important gates are automated (lint, breaking-change detection) because human review alone does not scale.

Each stage has a concrete meaning:

- **Draft.** Written alongside a design document (see Vol 15, Chapter 2). Includes at least one happy-path and one error example per operation. No traffic.
- **Review.** A pull request containing the spec file(s), generated diff, and a changelog entry. Automated checks run: spectral lint, `oasdiff` or `buf breaking`, and example validation.
- **Approved / Published.** Merged to `main`, tagged (`apis/orders@v1.4.0`), published to the registry (SwaggerHub, Buf Schema Registry, or an internal portal). From this point, compatibility rules apply.
- **Evolving.** Additive changes (new optional field, new endpoint) go through the same review gate. Consumers can adopt at their own pace.

- **Deprecated.** The contract is still served but signals deprecation (`Deprecation: true`, `Sunset: Sat, 31 Dec 2026 23:59:59 GMT` per RFC 8594, or gRPC `deprecated` option). Documentation points to the successor. Metrics track remaining traffic.
- **Sunset.** Traffic has dropped below an agreed threshold (often <1% or zero for internal surfaces after a migration deadline). The route is removed; the schema is archived, not deleted, so historical data can still be decoded.

The lifecycle matters for distributed systems because **multiple versions are in production simultaneously**. When you deprecate `GET /v1/orders`, the old and new versions will be served by different revisions of the same service — or by different services — for weeks or months. Your routing, documentation, and observability must all handle that.

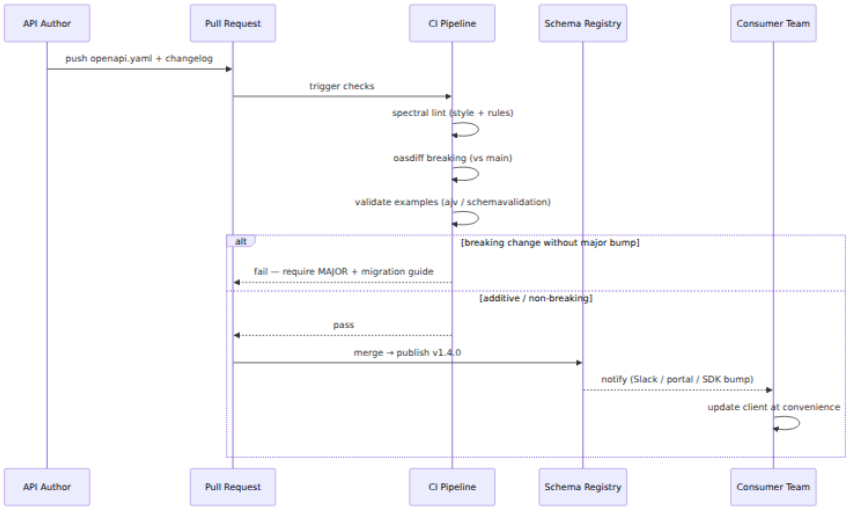


Figure 1-3: Automated contract gate in CI. Breaking changes are rejected unless the version bump and migration guide match policy; additive changes flow through.

Design-first versus code-first

Design-first means you write the contract before the implementation and generate or validate the implementation against it. **Code-first** means

you write the implementation (often with annotations) and derive the contract from code.

Code-first is faster for a single team building a single service. It fails at organisational scale because:

- The contract inherits the implementation's accidental shape (framework defaults, language idioms) rather than the domain's intended shape.
- Two services built with different frameworks produce inconsistent contracts even for the same pattern (pagination looks different in Spring versus Express).
- There is no artifact to review before code exists, so cross-team feedback arrives late — after implementation is already coupled to the shape.

Design-first costs more upfront but pays back in **reviewability and consistency**. The contract PR can be reviewed by consumers before any producer code is written. Linters enforce naming and pagination conventions uniformly regardless of implementation language. The trade-off is that the contract and implementation can drift if validation is not automated. The mitigation is to validate in CI that the implementation *conforms* to the published contract (request/response validation middleware, or generated server stubs that fail to compile when the contract changes).

> **Recommendation for this volume:** Use design-first for any surface with more than one consumer or more than one producer team, which in practice means every edge API and every internal service that is called by more than one other service. Code-first is acceptable only for single-consumer, single-team internal endpoints that are co-deployed.

A real contract: OpenAPI 3.1.0 for an Orders API

The contract below is a complete, lint-clean OpenAPI 3.1.0 document for a small Orders surface. It is deliberately small so you can read it end-to-end, but it includes every element a production contract needs: `info` with version, `servers`, reusable `components`, security, pagination, error envelope, and examples. Tool versions are pinned in comments.

```yaml
# openapi.yaml — Orders API v1.4.0
# Spec: OpenAPI 3.1.0 (https://spec.openapis.org/oas/v3.1.0.html)
# JSON Schema dialect: 2020-12 (https://json-schema.org/draft/2020-12/
json-schema-core.html)
# Lint: Spectral 6.11.0 with @stoplight/spectral-owasp-ruleset 1.0.0
# Breaking check: oasdiff 1.9.2  (oasdiff breaking https://
registry.example.com/apis/orders@v1.3.0 openapi.yaml)
openapi: 3.1.0
info:
  title: Orders API
  version: 1.4.0
  summary: Create and query customer orders
  description: |
    Design-first contract for the Orders bounded context.
    All timestamps are RFC 3339 (date-time). All IDs are ULIDs.
    Pagination is cursor-based; offset pagination is not supported.
  contact: { name: Platform API Team, url: https://docs.example.com/
orders, email: api-team@example.com }
  license: { name: Proprietary }

servers:
  - url: https://api.example.com/v1
    description: Production
  - url: https://staging-api.example.com/v1
    description: Staging

security:
  - bearerAuth: []

paths:
  /orders:
    get:
      operationId: listOrders
      summary: List orders (cursor-paginated, filterable)
      parameters:
        - $ref: '#/components/parameters/PageSize'
        - $ref: '#/components/parameters/PageToken'
        - $ref: '#/components/parameters/FilterStatus'
        - $ref: '#/components/parameters/FilterCreatedAt'
        - $ref: '#/components/parameters/SortParam'
        - $ref: '#/components/parameters/FieldsParam'
      responses:
        '200':
          description: A page of orders
          headers:
            RateLimit-Limit: { $ref: '#/components/headers/
RateLimitLimit' }
            RateLimit-Remaining: { $ref: '#/components/headers/
RateLimitRemaining' }
            RateLimit-Reset: { $ref: '#/components/headers/
```

```
RateLimitReset' }
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/OrderList'
              examples:
                firstPage:
                  value:
                    data: [{ id: "01H8X1ABCDEF1234567890AB", status: "p
aid", total_cents: 2499, created_at: "2026-03-10T14:22:31Z" }]
                    pagination: { next_page_token: "eyJpZCI6IjAxSDhYMSJ
9", has_more: true }
        '400': { $ref: '#/components/responses/BadRequest' }
        '401': { $ref: '#/components/responses/Unauthorized' }
        '429': { $ref: '#/components/responses/TooManyRequests' }
        default: { $ref: '#/components/responses/Error' }

    post:
      operationId: createOrder
      summary: Create an order (idempotent with Idempotency-Key)
      parameters:
        - name: Idempotency-Key
          in: header
          required: true
          schema: { type: string, format: uuid, maxLength: 64 }
          description: Client-generated idempotency key (RFC 7231
§4.2.2 semantics; server stores for 24h).
      requestBody:
        required: true
        content:
          application/json:
            schema: { $ref: '#/components/schemas/CreateOrderRequest' }
            examples:
              minimal:
                value: { customer_id: "01H8X1CUST1234567890ABCDE",
items: [{ sku: "WIDGET-BLUE", quantity: 2 }] }
      responses:
        '201':
          description: Created
          headers:
            Location: { schema: { type: string, format: uri },
description: Canonical URL of the new order }
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Order' }
        '400': { $ref: '#/components/responses/BadRequest' }
        '409': { $ref: '#/components/responses/Conflict' }
        default: { $ref: '#/components/responses/Error' }

  /orders/{id}:
    get:
```

```
      operationId: getOrder
      summary: Get order by ID
      parameters:
        - name: id
          in: path
          required: true
          schema: { type: string, pattern: '^[0-9A-HJKMNP-TV-Z]{26}$' }
      responses:
        '200':
          description: The order
          content:
            application/json:
              schema: { $ref: '#/components/schemas/Order' }
        '404': { $ref: '#/components/responses/NotFound' }
        default: { $ref: '#/components/responses/Error' }

components:
  securitySchemes:
    bearerAuth: { type: http, scheme: bearer, bearerFormat: JWT }

  parameters:
    PageSize: { name: page_size, in: query, schema: { type: integer,
minimum: 1, maximum: 100, default: 20 } }
    PageToken: { name: page_token, in: query, schema: { type: string,
description: Opaque cursor from previous response } }
    FilterStatus: { name: filter[status], in: query, schema: { type: st
ring, enum: [pending, paid, shipped, cancelled] } }
    FilterCreatedAt: { name: filter[created_at.gte], in: query,
schema: { type: string, format: date-time } }
    SortParam: { name: sort, in: query, schema: { type: string, enum:
[created_at, -created_at, total_cents, -total_cents], default: "-
created_at" } }
    FieldsParam: { name: fields, in: query, schema: { type: string,
description: Sparse fieldset, e.g. fields=id,status,total_cents } }

  headers:
    RateLimitLimit: { schema: { type: integer }, description:
"Requests allowed per window (RFC 6585 + draft-ietf-httpapi-ratelimit-
headers)" }
    RateLimitRemaining: { schema: { type: integer } }
    RateLimitReset: { schema: { type: integer }, description: Seconds
until window reset }

  schemas:
    Order:
      type: object
      required: [id, customer_id, status, total_cents, created_at]
      additionalProperties: false
      properties:
        id: { type: string, pattern: '^[0-9A-HJKMNP-TV-Z]{26}$',
description: ULID }
```

```
        customer_id: { type: string, pattern: '^[0-9A-HJKMNP-TV-Z]
{26}$' }
        status: { type: string, enum: [pending, paid, shipped, cancelle
d] }
        total_cents: { type: integer, minimum: 0 }
        created_at: { type: string, format: date-time }
        updated_at: { type: string, format: date-time, description: "Ad
ded in v1.4.0 — optional for backward compat" }
      example: { id: "01H8X1ABCDEF1234567890AB", customer_id: "01H8X1CU
ST1234567890ABCDE", status: "paid", total_cents: 2499, created_at: "202
6-03-10T14:22:31Z" }

    CreateOrderRequest:
      type: object
      required: [customer_id, items]
      additionalProperties: false
      properties:
        customer_id: { type: string, pattern: '^[0-9A-HJKMNP-TV-Z]
{26}$' }
        items:
          type: array
          minItems: 1
          maxItems: 100
          items:
            type: object
            required: [sku, quantity]
            additionalProperties: false
            properties:
              sku: { type: string, minLength: 1, maxLength: 64 }
              quantity: { type: integer, minimum: 1, maximum: 999 }

    OrderList:
      type: object
      required: [data, pagination]
      properties:
        data: { type: array, items: { $ref: '#/components/schemas/
Order' } }
        pagination:
          type: object
          required: [has_more]
          properties:
            next_page_token: { type: string, nullable: true }
            has_more: { type: boolean }

    Error:
      type: object
      required: [code, message]
      properties:
        code: { type: string, description: "Machine-readable, e.g. ORDE
R_NOT_FOUND, RATE_LIMITED" }
        message: { type: string, description: Human-readable detail }
```

```
        details: { type: array, items: { type: object } }
        request_id: { type: string, format: uuid }

  responses:
    BadRequest: { description: Bad request, content: { application/
json: { schema: { $ref: '#/components/schemas/Error' } } } }
    Unauthorized: { description: Unauthorized, content: { application/
json: { schema: { $ref: '#/components/schemas/Error' } } } }
    NotFound: { description: Not found, content: { application/json: {
schema: { $ref: '#/components/schemas/Error' } } } }
    Conflict: { description: Conflict (idempotency replay with
different payload), content: { application/json: { schema: { $ref: '#/
components/schemas/Error' } } } }
    TooManyRequests:
      description: Rate limited
      headers:
        Retry-After: { schema: { type: integer }, description: Seconds
to wait before retry }
      content: { application/json: { schema: { $ref: '#/components/
schemas/Error' } } }
    Error: { description: Unexpected error, content: { application/
json: { schema: { $ref: '#/components/schemas/Error' } } } }
```

What this contract gives you that prose alone does not:

- **Machine-checkable syntax.** A Spectral rule can forbid `additionalProperties: true` on responses, require `operationId`, or ban `offset` pagination. An `oasdiff` check can block removal of `status`'s `cancelled` value as a breaking change.
- **Reusable components.** `Order`, `Error`, and header definitions appear once and are referenced everywhere, which is how consistency is enforced mechanically.
- **Evolvability by construction.** `updated_at` was added in v1.4.0 as an optional field — existing consumers that ignore unknown fields (as JSON consumers should) are unaffected. Removing `total_cents` would be caught as breaking because it is `required`.
- **Operability.** Rate-limit headers, `Idempotency-Key`, and the error envelope are part of the contract, not tribal knowledge.

Validating and linting this contract locally (versions pinned):

```
# Node 20.11.0, spectral 6.11.0
npm i -D @stoplight/spectral-cli@6.11.0 @stoplight/spectral-owasp-
ruleset@1.0.0
npx spectral lint openapi.yaml --ruleset .spectral.yaml

# oasdiff 1.9.2 — breaking-change gate (compare against published
version)
oasdiff breaking https://registry.example.com/apis/orders@v1.3.0 openap
i.yaml --fail-on ERR

# Redocly 1.14.0 — bundle + validate examples against schemas
npx @redocly/cli@1.14.0 lint openapi.yaml
npx @redocly/cli@1.14.0 bundle openapi.yaml -o dist/openapi.json
```

A minimal `.spectral.yaml` that enforces the principles above:

```
# .spectral.yaml — Spectral 6.11.0
extends:
  - spectral:oas
  - "@stoplight/spectral-owasp-ruleset"
rules:
  operation-operationId: error
  no-eval-in-description: off
  oas3-schema: error
  # custom: pagination must be cursor-based
  cursor-pagination-only:
    description: "Use page_token/page_size, not offset/limit/page"
    given: "$.paths.*.*.parameters[*].name"
    severity: error
    then:
      function: pattern
      functionOptions: { notMatch: "^(offset|limit|page)$" }
  # custom: every operation must have an error response
  has-error-response:
    given: "$.paths.*.*.responses"
    severity: error
    then:
      function: schema
      functionOptions:
        schema: { required: ["default"] }
```

Consumer-driven contracts and conformance

A contract written by the producer and never tested from the consumer's perspective is a hypothesis. **Consumer-driven contract testing** (Chapter 11) inverts the perspective: each consumer publishes the subset

of the contract it actually depends on — the fields it reads, the status codes it handles, the pagination it relies on — and the producer verifies that it still satisfies all consumers before shipping.

For REST, this is typically done with Pact or with OpenAPI example validation; for gRPC, with `buf breaking` plus consumer-specific proto tests. The key insight is that **not every breaking change as defined by the spec is breaking for your actual consumers**. Removing a field that no consumer reads is technically breaking but operationally safe; changing a field that every consumer reads is breaking even if the schema calls it "optional." Consumer contracts make that distinction explicit.

At scale, the registry becomes the source of truth. Whether it is SwaggerHub, Backstage, Buf Schema Registry, or a simple Git repository with version tags, the registry must answer: "What versions are currently served? Which consumers depend on which fields? What is the deprecation timeline?" Without those answers, deprecation is guesswork and sunset is risky.

Distributed-systems lens: why contracts are harder than they look

Three properties of distributed systems make API contracts disproportionately important:

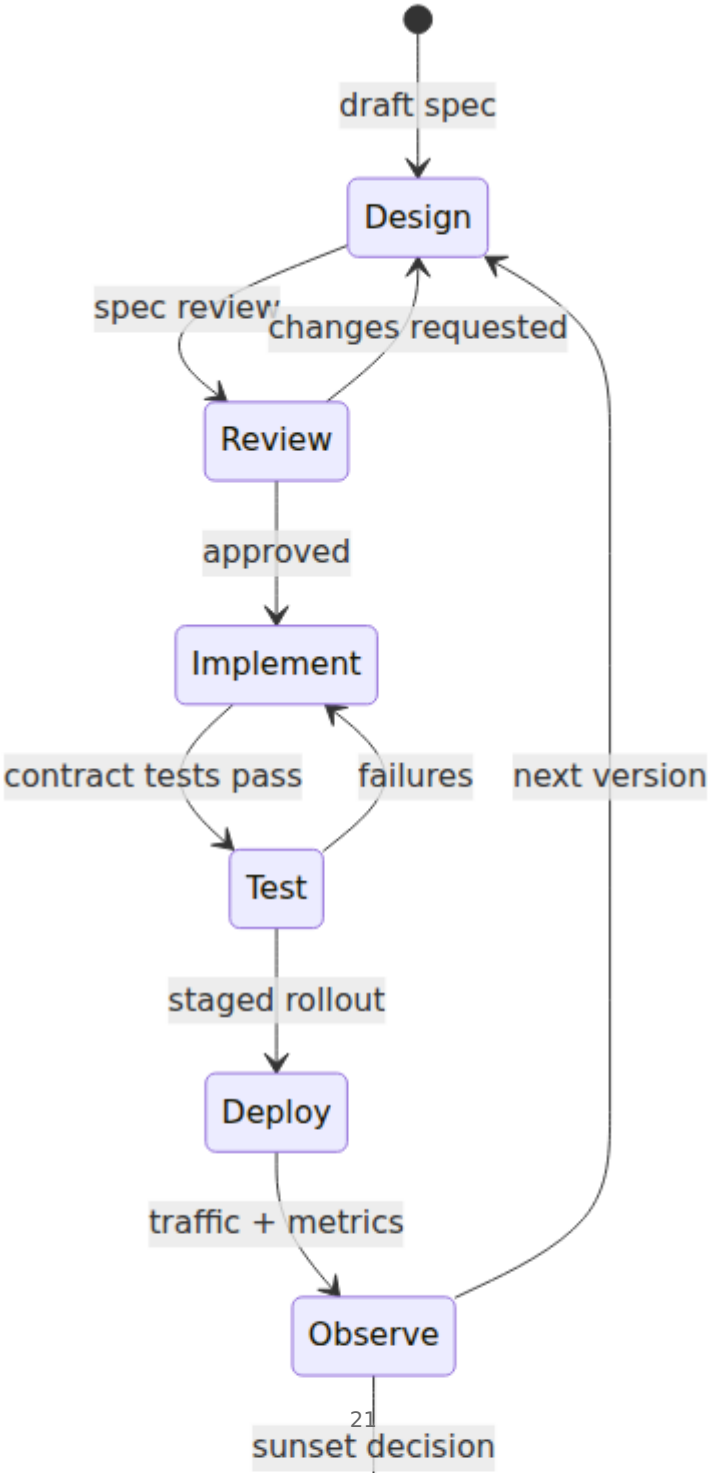
1. Partial failure is the normal case. A caller and a contract live in different processes, often different regions. The network between them can delay, drop, or duplicate requests. Contracts that are explicit about retry safety (`Idempotency-Key` required on `POST`, `PUT` is idempotent, `GET` is safe) let callers handle partial failure without creating duplicate side effects. Contracts that leave this implicit force every consumer to guess.

2. Time and version are not global. At any moment, three versions of your service may be running (old, current, canary), and consumers may be on five different client versions, some cached at the edge. Backward and forward compatibility are not theoretical — they are the steady state. A contract that requires lock-step upgrades is a contract that cannot be deployed safely.

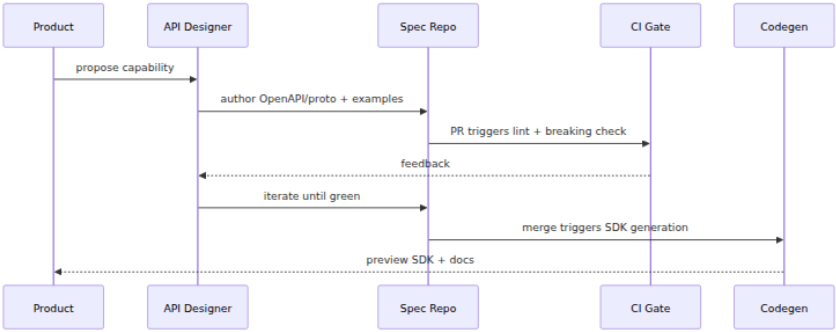
3. Coordination cost grows with consumer count. Changing an internal RPC with one caller costs one conversation. Changing a public API with hundreds of consumers requires a deprecation period, migration guide, dual-serve window, and traffic-based sunset criteria. The contract is where you pay that cost upfront (by designing for extension) or later (by coordinating a breaking change).

> **Boundary note.** Protocol-level mechanics — HTTP/2 framing, gRPC wire format, Protobuf varint encoding, TLS, and the network path a request traverses — are covered in Volume 3 (Networking), Chapter 8 (gRPC and RPC Framework Internals). This volume treats the *design* of those surfaces: resource modelling, schema layout, versioning, and evolution. Where this chapter mentions the wire, it does so only to explain why a design choice matters for compatibility or performance.

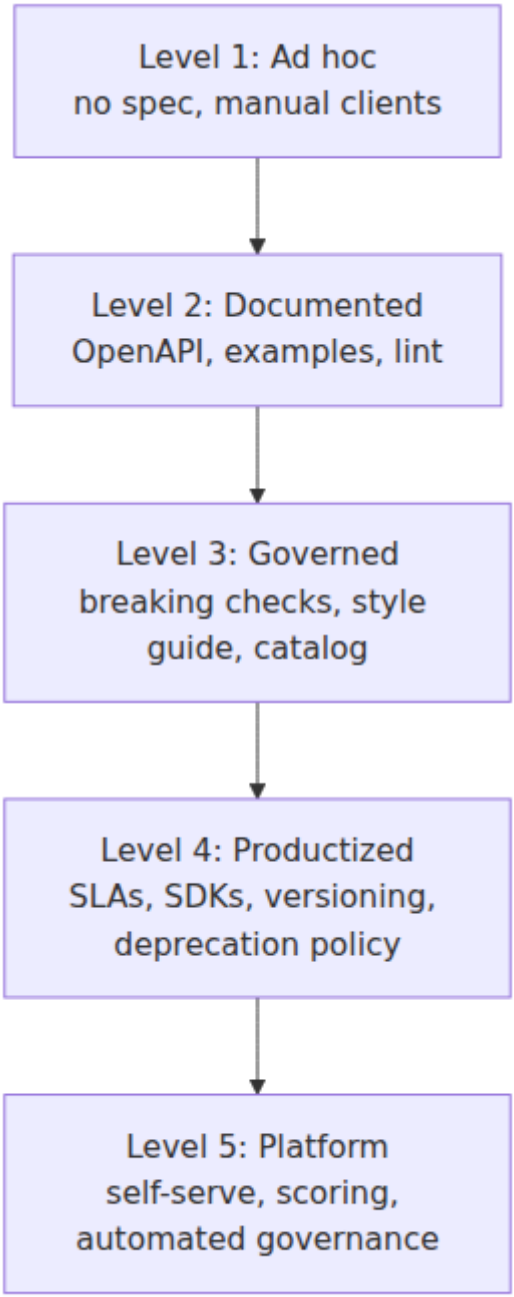
API Lifecycle State Machine


draft spec
Design
spec review
changes requested
Review
approved
Implement
contract tests pass
failures
next version
Test
staged rollout
Deploy
traffic + metrics
Observe
sunset decision

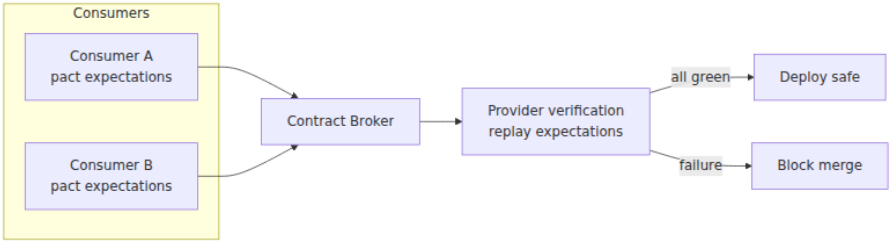
Design-First Workflow



API Maturity Model


Level 1: Ad hoc
no spec, manual clients
Level 2: Documented
OpenAPI, examples, lint
Level 3: Governed
breaking checks, style
guide, catalog
Level 4: Productized
SLAs, SDKs, versioning,
deprecation policy
Level 5: Platform
self-serve, scoring,
automated governance

Consumer-Driven Contract Flow



Key takeaways

- The contract, not the implementation, is the unit of coupling. Invest design effort where the blast radius is largest.
- A complete contract covers syntax, semantics, and evolution — and each layer needs a different enforcement mechanism (lint, docs, breaking-change detection).
- Consistency, predictability, discoverability, least surprise, evolvability, and operability are checkable in review; use them as gate criteria, not slogans.
- Design-first is the default at organisational scale; code-first is acceptable only for single-consumer, single-team surfaces where drift is cheap to detect.
- The contract lifecycle (draft → review → published → evolving → deprecated → sunset) must be automated — lint, `oasdiff` / `buf breaking`, and example validation in CI — because human review alone does not scale.
- OpenAPI 3.1.0 with Spectral and `oasdiff` gives you a machine-checkable REST contract today; the same discipline applies to Protobuf with `buf lint` and `buf breaking` (Chapter 3).
- Distributed systems require contracts to be explicit about retry safety, pagination stability, version windows, and multi-version serving — the things that are invisible in a single-process call.

Further reading

- OpenAPI Initiative. *OpenAPI Specification 3.1.0* (2021-02-15). https://spec.openapis.org/oas/v3.1.0.html — the authoritative spec; read §4 (Schema Object) and §5 (JSON Schema 2020-12 alignment) carefully.
- JSON Schema. *JSON Schema 2020-12* — https://json-schema.org/draft/2020-12/json-schema-core.html. Needed to interpret OpenAPI 3.1 schemas correctly.
- Fielding, R. *Architectural Styles and the Design of Network-based Software Architectures* (2000), Chapter 5 — the REST dissertation; still the best statement of why REST's constraints exist.
- IETF. *RFC 8594 — The Sunset HTTP Header Field* (2019) and *RFC 9745 — The Deprecation HTTP Header Field* (2024) — standard headers for lifecycle signalling.
- Vaughan-Nicholson et al. *Spectral* (Stoplight, 6.11.0) and *oasdiff* (1.9.2) — the two most practical tools for linting and breaking-change detection on OpenAPI.
- Newman, S. *Building Microservices* 2nd ed. (O'Reilly, 2021), Chapters 4–5 — pragmatic treatment of coupling, contracts, and consumer-driven testing.
- Pact Foundation. *Pact: Consumer-Driven Contract Testing* — https://docs.pact.io/ — the reference for consumer-driven workflows that this chapter introduces and Chapter 11 implements.

Chapter 2 — REST in Depth

What this chapter covers. REST is simultaneously the most widely used and most loosely defined style in backend engineering. Every team claims to build "REST APIs," but the resulting surfaces range from careful resource-oriented designs that scale to thousands of consumers to ad-hoc JSON-over-HTTP that could be RPC with path segments. This chapter makes REST precise: what the constraints actually require, how to model resources and relationships so the URI structure earns its keep, the exact semantics of methods and status codes (and where teams most often violate them), how to design pagination, filtering, sorting, and field selection so consumers can build correct and efficient clients at scale, and how caching and conditional requests turn HTTP's built-in machinery

into a performance and consistency tool. Along the way we build a realistic Express/Node handler with cursor pagination and a Go client that consumes it correctly. The chapter closes with where REST fits — and where it does not — in a distributed system that also speaks gRPC and events.

Learning goals — after this chapter you should be able to:

- State REST's six constraints, distinguish resource-oriented REST from JSON-over-HTTP, and place a given API on the Richardson Maturity Model.
- Model a domain as resources, sub-resources, and relationships — choosing collections, singletons, and actions deliberately — and design URIs, methods, and status codes that obey HTTP semantics.
- Implement and consume cursor-based pagination, compound filtering, stable sorting, sparse fieldsets, and `ETag` / `If-None-Match` caching correctly, including the distributed edge cases (concurrent mutation, clock skew).
- Decide when to use HATEOAS links, when to return `202 Accepted` versus `201 Created`, and how to model long-running operations and batch endpoints within REST.
- Explain REST's trade-offs versus gRPC (Chapter 3) and events (Vol 10) through a distributed-systems lens: cacheability, intermediary friendliness, browser reach, and debuggability.

What REST actually is — and what it is not

REST (Representational State Transfer) is an architectural *style* defined by Roy Fielding's 2000 dissertation, not a protocol or a standard you can "comply" with in a binary sense. It imposes six constraints on the interaction between client and server:

1. **Client-server** — separation of concerns; the client holds UI state, the server holds resource state.
2. **Stateless** — every request carries all context the server needs (auth, resource identity); the server stores no per-client session between requests. This is what makes horizontal scaling trivial.

3. **Cacheable** — responses declare whether they may be cached, and for how long, so intermediaries (CDNs, gateways, browsers) can serve them without contacting the origin.
4. **Uniform interface** — four sub-constraints that together are what people usually mean by "REST": resource identification in requests (URIs), resource manipulation through representations (JSON bodies), self-descriptive messages (media types, status codes, headers), and hypermedia as the engine of application state (HATEOAS).
5. **Layered system** — intermediaries (load balancers, caches, meshes) can be inserted transparently because the interface is uniform.
6. **Code on demand (optional)** — the server may ship executable code to the client. Almost never used in backend APIs.

The constraints that matter operationally are **stateless**, **cacheable**, **uniform interface**, and **layered system**. Together they explain why REST scales organisationally: statelessness lets you add origin instances without session affinity, cacheability lets you shed read load without touching application code, and the uniform interface lets any intermediary understand the traffic without custom logic.

REST versus JSON-over-HTTP versus RPC

Many surfaces labelled "REST" are really **JSON-over-HTTP**: they use HTTP as a tunnel (usually `POST` everything to `/api/execute`) and ignore method semantics, status codes, and caching. Others are **RPC-over-HTTP**: `POST /orders.cancel` looks like a procedure call, not a resource manipulation. Both can be perfectly good APIs — Stripe's API, for example, is deliberately RPC-flavoured in places (`POST /v1/refunds`) because the domain action does not map cleanly to CRUD.

The honest position is: be **resource-oriented where the domain is resource-shaped** (orders, users, invoices — things with identity and lifecycle) and **action-oriented where it is not** (cancel, refund, publish, search). Forcing every operation into `PUT` / `PATCH` / `DELETE` when the domain verb is inherently procedural creates awkwardness that consumers feel immediately.

Richardson Maturity Model — a rough ruler, not a target

Leonard Richardson's model grades an API by how fully it exploits HTTP:

- **Level 0 — Swamp of POX.** One URI, one method (`POST`), payload carries everything. Not REST at all.
- **Level 1 — Resources.** Many URIs, but still one method. You can identify resources; you cannot manipulate them with HTTP semantics.
- **Level 2 — HTTP verbs.** Resources plus correct methods and status codes. This is where most well-designed REST APIs live.
- **Level 3 — Hypermedia (HATEOAS).** Responses include links/relations that tell the client what transitions are available. Rarely achieved fully; often approximated with `_links` or `Link` headers on paginated collections and state-machine resources.

Level 2 is the pragmatic target for most backend surfaces. Level 3 is valuable for state-machine resources (an order that can transition `pending → paid → shipped` but not `shipped → pending`) and for paginated collections, where the `next` cursor link removes client-side URL construction.

Resource modelling

From domain to resources

A resource is not a database row, though it often maps to one. It is a **concept with identity that a client wants to interact with over time**. Start from the domain language and ask: "What nouns do product and support use when they talk about this area? What lifecycle do those nouns have?"

For an e-commerce ordering domain, the nouns are `Order`, `OrderItem` (not independently addressable outside an order), `Payment`, `Shipment`, and `Customer`. That suggests:

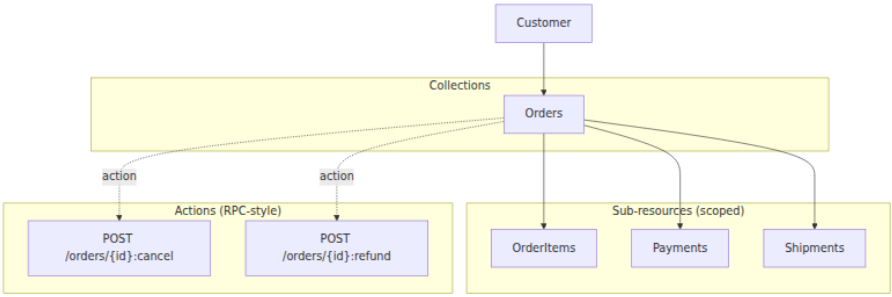


Figure 2-1: Resource model for an Orders domain. Collections are top-level; line items and nested entities are sub-resources scoped to their parent; state transitions that do not map to CRUD become custom actions.

URI design

URIs identify resources; they are not an encoding of query logic. Conventions that survive review:

- **Nouns, plural, kebab or snake consistently.** `GET /v1/orders` and `GET /v1/order-items` — not `GET /v1/getOrders` or `GET /v1/order_item`.
- **Hierarchy reflects ownership/lifecycle.** `GET /v1/orders/{orderId}/items/{itemId}` when items cannot exist without an order. If items are queryable independently, also expose `GET /v1/order-items?filter[order_id]=...`.
- **No verbs in paths — except explicit actions.** `POST /v1/orders/{id}:cancel` (Google AIP-136 style, colon-separated) or `POST /v1/orders/{id}/cancellations`. The colon convention signals "this is an action, not a sub-collection."
- **Version in the path or header — pick one and be consistent.** `GET /v1/orders` (path versioning, most common for REST) versus `Accept: application/vnd.example.v1+json` (header versioning, more correct per HTTP but worse for debuggability and CDN keying). Either way, Chapter 5 covers the evolution policy that versioning implies.
- **IDs are opaque.** Use ULIDs (`01H8X...`) or UUIDv7, not auto-increment integers that leak cardinality and make sharding painful.

Pattern	Example	When to use
Collection	`GET /v1/orders`	List/search; always paginated
Singleton (by ID)	`GET /v1/orders/{id}`	Fetch, update, delete one
Sub-collection	`GET /v1/orders/{id}/payments`	Child resources scoped to parent
Singleton sub-resource	`GET /v1/orders/{id}/shipping-address`	One-per-parent (not a collection)
Custom action	`POST /v1/orders/{id}:cancel`	State transition that is not CRUD
Batch	`POST /v1/orders:batchGet`	Fetch many by ID without N round-trips
Search (complex)	`POST /v1/orders:search`	Query too large for query-string (large filter payload)

Methods and status codes — the semantics you must honour

HTTP methods have precise semantics that intermediaries and clients rely on:

Method	Safe	Idempotent	Cacheable	Meaning
`GET`	yes	yes	yes	Retrieve representation; must not mutate
`HEAD`	yes	yes	yes	Same as `GET` but without body (for `ETag` / `Content-Length` probes)
`PUT`	no	yes	no	Replace entire resource at URI (create or overwrite)
`PATCH`	no	no*	no	Partial update (JSON Merge Patch or JSON Patch); idempotent if the patch document is

Method	Safe	Idempotent	Cacheable	Meaning
`POST`	no	no	no†	Create subordinate, trigger action, or any non-idempotent operation
`DELETE`	no	yes	no	Remove resource; repeated `DELETE` returns `204` / `404` consistently

* `PATCH` idempotency depends on the patch format: `PATCH {"status":"paid"}` is idempotent; `PATCH [{"op":"add","path":"/tags/-","value":"x"}]` with JSON Patch may not be. † `POST` responses are cacheable only if explicitly marked with `Cache-Control`.

Status codes are equally load-bearing. The minimum set a REST API should use correctly:

- `200 OK` — `GET` / `PATCH` success with body. `POST` that did not create a new URI-addressable resource.
- `201 Created` — `POST` that created a resource; include `Location: /v1/orders/{id}`.
- `202 Accepted` — request accepted for asynchronous processing; include `Location` to a status resource or `Retry-After`.
- `204 No Content` — `DELETE` or `PUT` success with no body to return.
- `304 Not Modified` — conditional `GET` with `If-None-Match` when `ETag` matches (not an error; saves bandwidth).
- `400 Bad Request` — malformed syntax the client can fix; include field-level `details`.
- `401 Unauthorized` / `403 Forbidden` — authentication vs authorisation (Vol 9, Ch 5–7).
- `404 Not Found` / `409 Conflict` / `422 Unprocessable Entity` — resource missing, state conflict, semantic validation failure.
- `429 Too Many Requests` — rate limited; include `Retry-After` and `RateLimit-*` headers.
- `500 Internal Server Error` / `503 Service Unavailable` — server fault; `503` may include `Retry-After` for shed load.

> **Common violation.** Returning `200` with `{ "error": "not_found" }` in the body breaks every intermediary and client that branches on status codes. Use `404` with a structured error body (Chapter 7) instead.

Pagination, filtering, sorting, and field selection

These four concerns are where naive REST designs collapse under real load. A `GET /orders` that returns all rows works until the table has 10 million rows; a filter that is undocumented or inconsistent is a correctness bug in every consumer that depends on it.

Pagination — cursor over offset

Offset pagination (`?offset=40&limit=20`) is simple and supports random access ("go to page 5"), but it degrades with data size (the database must still scan `offset` rows), and it breaks under concurrent mutation — if a row is inserted at position 10 while the client pages, rows shift and the client sees duplicates or misses.

Cursor pagination (`?page_token=eyJpZCI6...&page_size=20`) uses an opaque token that encodes the position (typically the last seen sort key). It is stable under concurrent inserts, efficient (a range scan on an indexed column), and the token can embed sort/filter context so tampering is detectable. The trade-off is no random access — you can only walk forward (and optionally backward with `prev_page_token`).

For any collection that is large, growing, or concurrently mutated — which is every production collection — **use cursor pagination**.

A minimal contract for pagination (OpenAPI fragment):

```
parameters:
  PageSize: { name: page_size, in: query, schema: { type: integer,
minimum: 1, maximum: 100, default: 20 } }
  PageToken: { name: page_token, in: query, schema: { type: string,
description: Opaque cursor from previous page } }
```

Response envelope:

```
{
  "data": [{ "id": "01H8X1ABC...", "status": "paid", "total_cents": 249
9 }],
  "pagination": { "next_page_token": "eyJpZCI6IjAxSDhYMSJ9",
"has_more": true }
}
```

The token should be opaque to the client (base64-encoded JSON or encrypted) even if it is just `{"last_id":"01H8X..."}` on the server side. Opaque tokens let you change the pagination strategy without breaking clients.

Filtering, sorting, and sparse fieldsets

Adopt one convention for all three and apply it everywhere (Chapter 1 — consistency). The JSON:API-inspired bracket style works well and is widely tooled:

```
GET /v1/orders?
filter[status]=paid&filter[created_at.gte]=2026-01-01T00:00:00Z
               &sort=-created_at
               &fields=id,status,total_cents
               &page_size=20&page_token=eyJp...
```

- **Filtering.** `filter[field]` for equality, `filter[field.gte]` / `lte` for ranges. Document which fields are filterable — not every column should be. Filters that require a full table scan should be rejected with `400` rather than served slowly.
- **Sorting.** `sort=field` ascending, `sort=-field` descending. When two rows share the sort key, add a deterministic tie-breaker (`id`) so pagination cursors are stable. Document the default sort.
- **Sparse fieldsets.** `fields=id,status` lets clients avoid over-fetching — the REST analogue of GraphQL field selection (Chapter 4) without the complexity. Useful for list views that only need 3 of 20 fields.

Real code — cursor pagination in Node.js (Express 4.18)

The handler below is production-shaped: it validates query params, decodes the opaque cursor, runs a keyset query (no `OFFSET`), and returns `next_page_token` only when more data exists. Versions pinned in comments.

```
// server.js — Node 20.11.0, Express 4.18.2, pg 8.11.3
// npm i express@4.18.2 pg@8.11.3 zod@3.22.4
import express from "express";
import { z } from "zod";
import { pool } from "./db.js"; // pg.Pool configured elsewhere

const app = express();
app.use(express.json());

const QuerySchema = z.object({
  page_size: z.coerce.number().int().min(1).max(100).default(20),
  page_token: z.string().optional(),
  "filter[status]": z.enum(["pending", "paid", "shipped", "cancelled"])
.optional(),
  "filter[created_at.gte]": z.string().datetime().optional(),
  sort: z.enum(["created_at", "-created_at", "total_cents", "-
total_cents"]).default("-created_at"),
  fields: z.string().optional(), // comma-separated allowlist checked
below
});

const ALLOWED_FIELDS = new Set(["id", "customer_id", "status", "total_c
ents", "created_at", "updated_at"]);

function decodeCursor(token) {
  if (!token) return null;
  try {
    const json = Buffer.from(token, "base64url").toString("utf8");
    const obj = JSON.parse(json);
    // cursor is { last_created_at: string, last_id: string }
    if (typeof obj.last_created_at === "string" && typeof obj.last_id =
== "string") return obj;
    return null;
  } catch { return null; }
}
function encodeCursor(row) {
  return Buffer.from(JSON.stringify({ last_created_at: row.created_at.t
oISOString(), last_id: row.id })).toString("base64url");
}

app.get("/v1/orders", async (req, res) => {
  const parsed = QuerySchema.safeParse(req.query);
  if (!parsed.success) {
    return res.status(400).json({ code: "BAD_REQUEST", message: "Invali
d query", details: parsed.error.issues, request_id: req.id });
  }
  const q = parsed.data;
  const cursor = q.page_token ? decodeCursor(q.page_token) : null;
  if (q.page_token && !cursor) {
    return res.status(400).json({ code: "BAD_REQUEST", message: "Invali
```

```
d page_token", request_id: req.id });
  }

  // field selection — default to all if not requested
  const fields = q.fields ? q.fields.split(",").map(s => s.trim()).filt
er(Boolean) : [...ALLOWED_FIELDS];
  const badFields = fields.filter(f => !ALLOWED_FIELDS.has(f));
  if (badFields.length) return res.status(400).json({ code: "BAD_REQUES
T", message: `Unknown fields: ${badFields.join(",")}` });
  const selectList = fields.map(f => `"${f}"`).join(", ");

  // sort — cursor pagination requires sort key in cursor; restrict to
created_at for simplicity
  // For total_cents sort you would need a composite cursor on
(total_cents, id)
  const sortDir = q.sort.startsWith("-") ? "DESC" : "ASC";
  const sortCol = q.sort.replace(/^-/, "");
  if (sortCol !== "created_at") {
    return res.status(400).json({ code: "BAD_REQUEST", message:
"Cursor pagination currently supports sort=created_at only" });
  }

  // build WHERE with keyset condition when cursor present
  const where = [];
  const params = [];
  let paramIdx = 1;

  if (q["filter[status]"]) { where.push(`status = $${paramIdx++}`); par
ams.push(q["filter[status]"]); }
  if (q["filter[created_at.gte]"]) { where.push(`created_at >= $${param
Idx++}`); params.push(q["filter[created_at.gte]"]); }
  if (cursor) {
    // keyset: (created_at, id) > (cursor_created_at, cursor_id) for
ASC, < for DESC
    const op = sortDir === "ASC" ? ">" : "<";
    where.push(`(created_at, id) ${op} ($${paramIdx}, $${paramIdx + 1})
`);
    params.push(cursor.last_created_at, cursor.last_id);
    paramIdx += 2;
  }

  const whereSQL = where.length ? `WHERE ${where.join(" AND ")}` : "";
  // fetch one extra to determine has_more without COUNT(*)
  const limit = q.page_size + 1;
  const sql = `SELECT ${selectList} FROM orders ${whereSQL} ORDER BY
created_at ${sortDir}, id ${sortDir} LIMIT $${paramIdx}`;
  params.push(limit);

  const { rows } = await pool.query(sql, params);
  const hasMore = rows.length > q.page_size;
  const page = hasMore ? rows.slice(0, q.page_size) : rows;
```

```
  const nextPageToken = hasMore ? encodeCursor(page[page.length -
1]) : null;

  // cacheable for 10s, revalidated with ETag
  res.set("Cache-Control", "private, max-age=10, stale-while-
revalidate=30");
  res.json({ data: page, pagination: { next_page_token: nextPageToken,
has_more: hasMore } });
});

app.listen(8080, () => console.log("orders API on :8080"));
```

Key choices:

- **Keyset query** `WHERE (created_at, id) > ($cursor)` with `ORDER BY created_at, id` — no `OFFSET`, stable under inserts, uses a composite index on `(created_at, id)`.
- **One-extra trick** — fetch `page_size + 1` rows; the extra row tells you `has_more` without a separate `COUNT(*)` that would be expensive and immediately stale.
- **Opaque cursor** as `base64url(JSON)` — clients cannot construct or tamper with cursors without going through the server.

Consuming pagination correctly — Go client (Go 1.22)

```go
// client.go — Go 1.22, net/http stdlib
package orders

import (
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "net/url"
)

type Order struct {
    ID         string `json:"id"`
    Status     string `json:"status"`
    TotalCents int    `json:"total_cents"`
    CreatedAt  string `json:"created_at"`
}
type ListResp struct {
    Data       []Order `json:"data"`
    Pagination struct {
        NextPageToken *string `json:"next_page_token"`
        HasMore       bool    `json:"has_more"`
    } `json:"pagination"`
}

// ListAll iterates all pages; ctx carries deadline (Vol 3, Ch 11).
func ListAll(ctx context.Context, baseURL string, pageSize int) ([]Orde
r, error) {
    var all []Order
    var pageToken string
    client := &http.Client{}
    for {
        u, _ := url.Parse(baseURL + "/v1/orders")
        q := u.Query()
        q.Set("page_size", fmt.Sprint(pageSize))
        q.Set("sort", "-created_at")
        if pageToken != "" {
            q.Set("page_token", pageToken)
        }
        u.RawQuery = q.Encode()

        req, _ := http.NewRequestWithContext(ctx, "GET", u.String(), ni
l)
        req.Header.Set("Accept", "application/json")
        resp, err := client.Do(req)
        if err != nil {
            return nil, err
        }
        if resp.StatusCode == http.StatusTooManyRequests {
            // honour Retry-After (Chapter 7); simplified — real code
```

```
    should back off
            return nil, fmt.Errorf("rate limited: retry after %s",
resp.Header.Get("Retry-After"))
        }
        if resp.StatusCode != http.StatusOK {
            resp.Body.Close()
            return nil, fmt.Errorf("list orders: %s", resp.Status)
        }
        var lr ListResp
        if err := json.NewDecoder(resp.Body).Decode(&lr); err != nil {
            resp.Body.Close()
            return nil, err
        }
        resp.Body.Close()
        all = append(all, lr.Data...)
        if !lr.Pagination.HasMore || lr.Pagination.NextPageToken ==
nil {
            break
        }
        pageToken = *lr.Pagination.NextPageToken
    }
    return all, nil
}
```

Caching and conditional requests

REST's cacheability constraint is not optional decoration — it is a scalability mechanism. A `GET /v1/orders/{id}` served with correct `Cache-Control` and `ETag` can be cached at three layers without any application change: browser, CDN, and service-mesh sidecar.

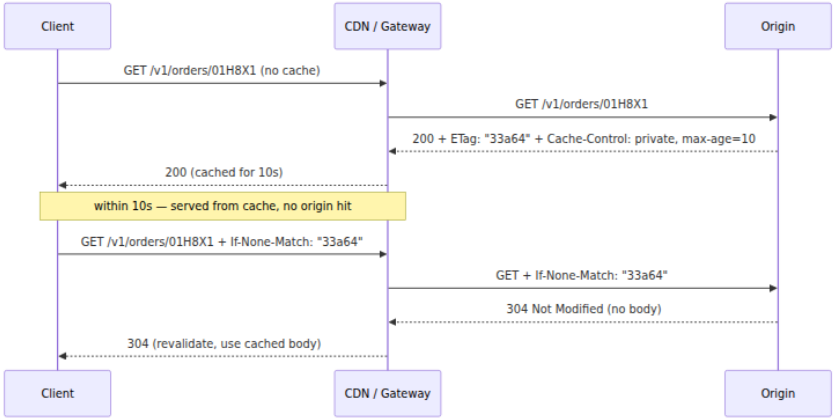


Figure 2-2: Conditional GET with ETag. The 304 saves bandwidth and origin work; with stale-while-revalidate the CDN can serve stale while refreshing in the background.

Headers that matter:

- `Cache-Control: private, max-age=10, stale-while-revalidate=30` — private (per-user), fresh for 10s, may serve stale for 30s while revalidating. Adjust per resource volatility.
- `ETag: "33a64df551425fcc55e4d42a148795d9f25f89d4"` — opaque validator, typically a hash of the representation or a version counter. Weak validators (`W/"..."`) are sufficient when byte identity is not required.
- `Last-Modified` — weaker than `ETag` (second resolution, clock-sensitive); prefer `ETag` .
- `Vary: Accept, Authorization` — tells caches that the response varies by those request headers; omitting it when it is needed causes cache poisoning.

For collections, caching is subtler. `GET /v1/orders?page_token=...` should generally be `Cache-Control: private, no-store` or very short `max-age` , because the collection is concurrently mutated. Caching a single order by ID is safe; caching a filtered list is often not worth the invalidation complexity.

Long-running operations, batch, and HATEOAS pragmatism

Not every operation fits `200` / `201` . Two patterns handle the rest:

Long-running operations. When `POST /v1/orders/{id}:cancel` triggers a workflow that takes minutes (refund, inventory restock), return `202 Accepted` with `Location: /v1/operations/abc123` and a `Retry-After` . The client polls `GET /v1/operations/abc123` (or subscribes via webhook/event) until `done: true` . Google's AIP-151 (`google.longrunning.Operation`) is a good template even outside Google.

Batch.
`POST /v1/orders:batchGet { "ids": ["01H8X1...", "01H8X2..."] }` returning `{ "orders": [...], "not_found": [...] }` avoids N round-trips. Keep batch size bounded (e.g., max 100) and document partial-

failure semantics — does one missing ID fail the whole batch, or return per-item status?

HATEOAS. Include `_links` where they carry state-machine information — e.g., an order with `status: "pending"` advertises `{"rel":"cancel","href":"/v1/orders/{id}:cancel","method":"POST"}` but not `refund` — and `Link` headers for pagination (`Link: <...?page_token=...>; rel="next"` per RFC 8288). Do not force clients to navigate solely via links; most REST consumers still construct URIs from an OpenAPI spec and treat links as hints.

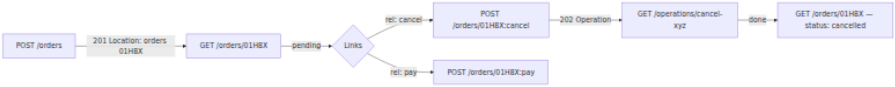


Figure 2-3: Resource lifecycle with hypermedia links and an asynchronous cancel operation. Links advertise valid transitions; the operation resource decouples the HTTP request from the workflow.

Where REST fits in a distributed system

REST's strengths at scale are:

- **Cacheability and intermediary friendliness.** Every cache, CDN, and gateway already speaks HTTP semantics. No custom protocol needed.
- **Browser and partner reach.** Any consumer with `fetch` can call your API; no codegen required.
- **Debuggability.** `curl`, browser devtools, and `jq` are universal. An on-call engineer can reproduce a failing request from a log line.

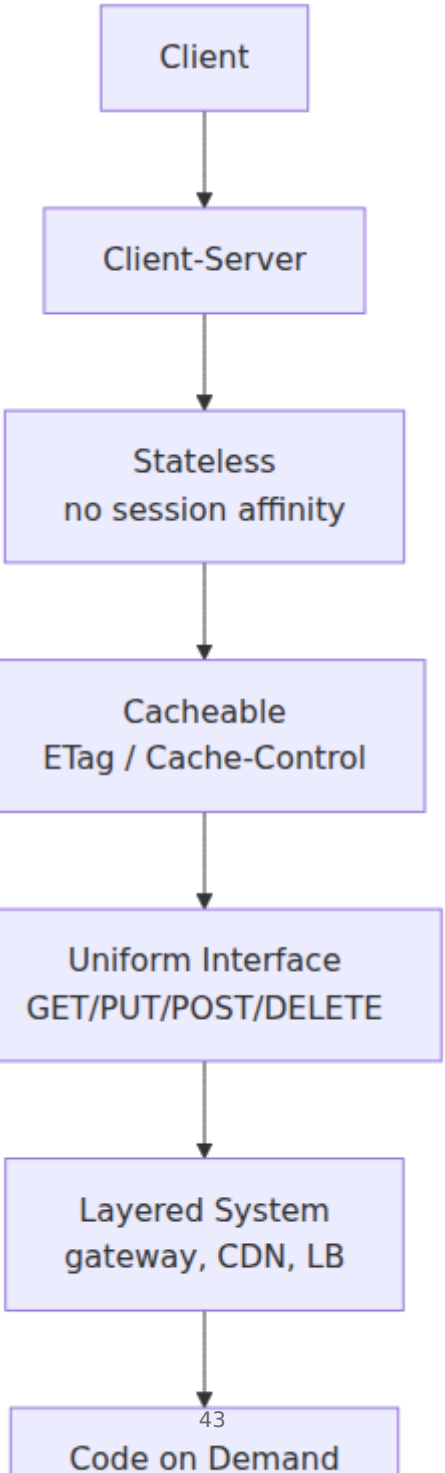
Its costs are:

- **Over/under-fetching.** `GET /orders/{id}` returns the whole order even when the client wants one field; `GET /orders` with N items either embeds or requires N follow-ups. Sparse fieldsets and batch endpoints mitigate but do not eliminate this — GraphQL (Chapter 4) exists for exactly this reason.
- **No streaming or bidirectional flow.** Server-sent events and WebSockets layer on top of HTTP but are not part of REST's uniform interface. For streaming telemetry or chat, gRPC streaming or events are better fits.

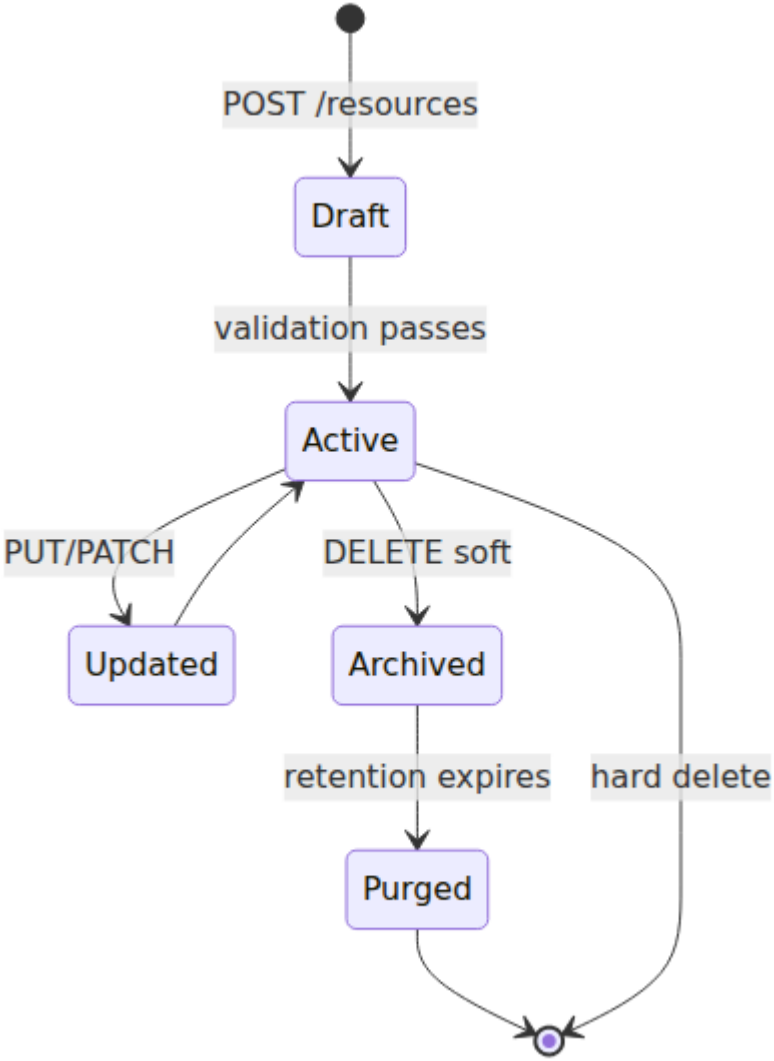
- **Text and schema overhead.** JSON field names repeat on every response; there is no built-in schema evolution contract beyond what OpenAPI documents. For high-throughput interior RPC, Protobuf over gRPC (Chapter 3) is more efficient and more strictly typed.

> **Distributed-systems lens.** At the edge, REST's cacheability and uniform interface reduce origin load and let CDNs absorb regional traffic without application changes — a direct win for multi-region latency (Vol 7, Ch 10). In the interior, where the caller and callee are both services you control, REST's text overhead and lack of streaming cost more than its debuggability is worth, which is why most large organisations use REST at the edge and gRPC in the mesh (Chapter 3). The decision is not "which is better" but "which edge of the system is this interface on."
>
> **Boundary note.** Low-level HTTP evolution — HTTP/1.1 vs HTTP/2 vs HTTP/3 framing, connection coalescing, header compression (HPACK/QPACK), and TLS negotiation — is covered in Volume 3, Chapters 7 and 10. This chapter uses HTTP as a *semantic* layer (methods, status, headers, caching) and assumes the transport beneath is correctly configured.

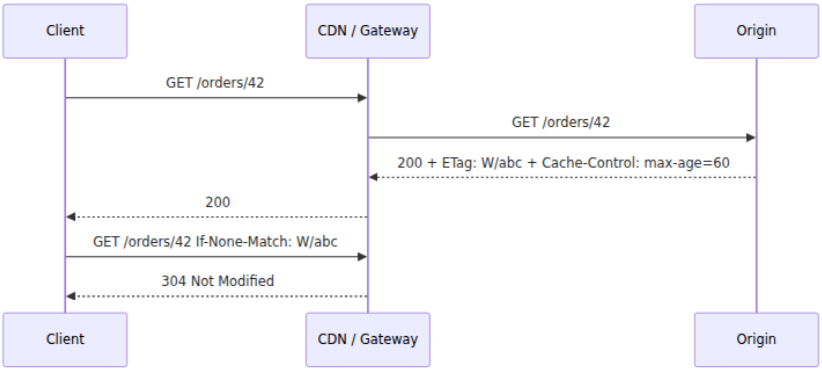
REST Constraints Applied


Client
Client-Server
Stateless
no session affinity
Cacheable
ETag / Cache-Control
Uniform Interface
GET/PUT/POST/DELETE
Layered System
gateway, CDN, LB
Code on Demand

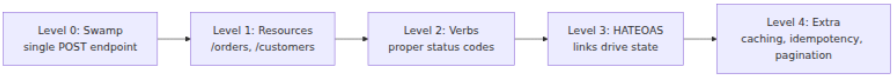
Resource Lifecycle



HTTP Caching and Conditional Requests



Richardson Maturity Model



Key takeaways

- Model domains as resources with clear ownership; use sub-resources for scoped children and colon-actions for operations that are not CRUD.
- Honour method semantics (`GET` safe, `PUT` idempotent, `POST` neither) and use the full status-code vocabulary — especially `201` / `202` / `204` / `304` / `429` — so intermediaries and clients can behave correctly without parsing bodies.
- Use cursor (keyset) pagination everywhere that data is large or concurrently mutated; employ the one-extra trick and opaque tokens, and add a deterministic tie-breaker to the sort.
- Standardise filtering (`filter[field]`), sorting (`sort=-field`), and sparse fieldsets (`fields=...`) once and enforce them with Spectral rules.
- Make cacheability explicit: `Cache-Control` , `ETag` / `If-None-Match` , and `Vary` are load-bearing for origin offload; conditional `304` saves both bytes and work.
- Return `202` with an operation resource for long-running work and bounded batch endpoints for N-item fetches; use `_links` / `Link` headers pragmatically for state machines and pagination.

- Choose REST for edge and partner surfaces where cacheability, debuggability, and browser reach matter; prefer gRPC for interior high-throughput RPC.

Further reading

- Fielding, R. *Architectural Styles and the Design of Network-based Software Architectures* (2000), Ch. 5 — the REST dissertation; the source for the six constraints and uniform-interface definition.
- Richardson, L. & Ruby, S. *RESTful Web Services* (O'Reilly, 2007) — Richardson Maturity Model and resource-oriented design in depth.
- Google Cloud. *API Improvement Proposals (AIPs)* — https://aip.dev/ — especially AIP-121 (resources), AIP-136 (custom methods), AIP-151 (long-running operations), AIP-158 (pagination). Opinionated but widely adopted conventions for resource naming and operation design.
- IETF. *RFC 8288 — Web Linking* (2017), *RFC 6585 — Additional HTTP Status Codes* (2012), *RFC 7232 — Conditional Requests* (2014), *RFC 9457 — Problem Details for HTTP APIs* (2023) — the header and error-envelope standards this chapter builds on.
- Lauret, A. *The Design of Web APIs* (Manning, 2019) — practical, example-heavy treatment of resource modelling and lifecycle.
- Express 4.18.x — https://expressjs.com/, `pg` 8.11.x — https://node-postgres.com/ — versions pinned for the handler example.

Chapter 3 — gRPC and Protobuf Schema Design

What this chapter covers. Inside the perimeter — service to service — REST's virtues (human readability, cacheability, browser reach) cost more than they are worth. Calls are frequent, payloads are structured, consumers are other services you control, and the bottleneck is not debuggability but throughput, latency, and independent evolvability across dozens of teams. That is the niche gRPC and Protocol Buffers were designed for, and where they are now the default in most large backend organisations. This chapter is about *design*: how to lay out `.proto`

schemas so they stay coherent at 200 services, how to model services, messages, and enums so compatibility is the steady state rather than the exception, how the four call shapes map to real problems, and how to operate a schema-driven workflow with `buf` that makes breaking changes fail in CI rather than in production. We treat the wire as context and the schema as the contract. If you want the bytes on the wire, HPACK tables, and HTTP/2 frame layout, that is Volume 3, Chapter 8.

Learning goals — after this chapter you should be able to:

- Explain why interior service-to-service traffic favours gRPC/Protobuf over REST/JSON, and where that trade-off reverses.
- Author a version-pinned, `buf lint`-clean Protobuf schema: packages, services, messages, enums, `oneof`, maps, well-known types, and field presence.
- Apply Protobuf compatibility rules (field numbers, `reserved`, enum handling, wire types) to evolve a schema without requiring coordinated deploys.
- Choose correctly among the four gRPC call shapes (unary, server-streaming, client-streaming, bidirectional) and model deadlines, metadata, and error details.
- Configure a `buf` workflow — `buf.yaml`, `buf.gen.yaml`, `buf lint`, `buf breaking`, `buf format` — and wire it into CI as the schema gate.
- Sketch a gRPC service flow from stub call through interceptors to handler, including deadline propagation and load-balancing implications in a distributed mesh.

Why gRPC/Protobuf inside the mesh

At the edge (Chapter 2), REST wins because every client already speaks HTTP and every intermediary understands `GET` vs `POST` and `Cache-Control`. In the interior, the client is another service you build, deploy, and observe. The calculus changes:

Concern	REST/JSON at interior	gRPC/Protobuf
Payload size	Field names repeated, numbers as ASCII, bytes as base64 (~33% overhead)	Binary TLV, varints, no field-name overhead; typically 2–6× smaller (measure your payloads)
Schema discipline	OpenAPI documents intent; nothing enforces that JSON matches at runtime without validation middleware	`.proto` *is* the contract; generated stubs make malformed requests a compile error
Streaming	Layered on (SSE, WebSocket) outside the uniform interface	First-class: server, client, and bidirectional streaming over one HTTP/2 stream
Code generation	From OpenAPI, but idioms vary by generator	One canonical generator per language (`protoc-gen-go` , `protoc-gen-grpc-java` , etc.) with consistent semantics
Debuggability	`curl` + `jq` universal	`grpcurl` , `grpcui` , server reflection; less ubiquitous than `curl`
Browser reach	Native	Requires proxy (gRPC-Web, Connect) — not the right tool for browsers

The interior choice is therefore not aesthetic but economic: at thousands of RPCs per second per service, smaller payloads, strict typing, and streaming compound into measurable latency and cost differences, and generated stubs eliminate a class of integration bugs that `JSON.parse` cannot catch.

> **Boundary note.** This chapter covers gRPC *service and schema design* — how to structure `.proto` files, model messages, version packages, and choose call shapes so dozens of teams can evolve independently. The *wire mechanics* — HTTP/2 stream mapping, length-prefixed message framing, HPACK/QPACK header compression, flow control, TLS, and connection pinning — are covered in Volume 3, Chapter 8 — gRPC and RPC Framework Internals. Where this chapter mentions the wire, it is only to explain why a design rule exists.

Protobuf schema design — the contract as source code

Protobuf's IDL is small, which is deceptive. A handful of rules determine whether your schema fleet stays coherent or accumulates breaking changes that surface as deserialization failures in production. Learn the rules before the patterns.

Packages, versions, and file layout

Every `.proto` file declares a package that must include an explicit version segment. The version is part of the import path and the service name on the wire (`acme.orders.v1.OrderService/CreateOrder`). Never put unversioned packages in production.

File layout for an organisation with multiple bounded contexts:

```
proto/
  acme/
    orders/
      v1/
        order.proto          # messages + service for orders v1
        order_events.proto   # event payloads (used over Kafka — Vol
10)
    users/
      v1/
        user.proto
    common/
      v1/
        money.proto          # shared Money type
        pagination.proto     # shared PageRequest / PageResponse
  buf.yaml
  buf.gen.yaml
  buf.lock
```

Rules:

- **One bounded context per directory, one version per sub-directory.** `acme/orders/v1` is independent of `acme/orders/v2`; `acme/users/v1` is independent of both. Consumers import the version they depend on; the producer can serve multiple versions concurrently.
- **Shared types live in `common/v1`.** `Money`, `Pagination`, and error details are shared, but domain entities (`Order`) are not — copying a small type is cheaper than coupling two bounded contexts to a shared domain model.

- **`buf.lock` is committed.** It pins transitive dependencies (`google/protobuf/timestamp.proto`, `googleapis`) so builds are reproducible.

A complete schema — Orders service

The schema below is a complete, `buf lint`-clean example covering the patterns you will use daily: service with unary and streaming RPCs, resource messages, enums with `UNSPECIFIED`, `oneof` for mutually exclusive payment methods, maps, well-known types, field presence, and deprecation. Versions pinned in comments.

```proto
// proto/acme/orders/v1/order.proto
// Protobuf: 4.25.x / 5.x  (protoc 24.x or buf 1.40.2 bundled compiler)
// Lint: buf 1.40.2 (buf lint), breaking: buf breaking against main
// gRPC: grpc-go 1.64.0, grpc-java 1.64.0, grpc-node 1.9.0
syntax = "proto3";

package acme.orders.v1;

option go_package = "github.com/acme/apis/gen/orders/v1;ordersv1";
option java_package = "com.acme.orders.v1";
option java_multiple_files = true;

import "google/protobuf/timestamp.proto";
import "google/protobuf/field_mask.proto";
import "acme/common/v1/money.proto";
import "acme/common/v1/pagination.proto";

// OrderService — resource-oriented service for the orders bounded
context.
// All methods are unary except WatchOrders (server-streaming) and
// IngestReturns (client-streaming). See § "The four call shapes".
service OrderService {
  // CreateOrder is idempotent when the client supplies an
idempotency_key.
  rpc CreateOrder(CreateOrderRequest) returns (CreateOrderResponse);

  // GetOrder fetches one order by ID.
  rpc GetOrder(GetOrderRequest) returns (GetOrderResponse);

  // ListOrders — cursor-paginated, filterable, sparse-field capable
via FieldMask.
  rpc ListOrders(ListOrdersRequest) returns (ListOrdersResponse);

  // UpdateOrder — partial update with FieldMask (see § field
presence).
  rpc UpdateOrder(UpdateOrderRequest) returns (UpdateOrderResponse);

  // CancelOrder — state transition that is not CRUD (cf.
REST :cancel).
  rpc CancelOrder(CancelOrderRequest) returns (CancelOrderResponse);

  // WatchOrders — server-streaming: tail new/updated orders for a
customer.
  rpc WatchOrders(WatchOrdersRequest) returns (stream WatchOrdersRespon
se);

  // IngestReturns — client-streaming: batch return ingestion (bulk
upload).
  rpc IngestReturns(stream IngestReturnsRequest) returns (IngestReturns
Response);
```

```
  // Chat — bidirectional: interactive order-assistance session
(example).
  rpc Chat(stream ChatMessage) returns (stream ChatMessage);
}

// ---- Resource ----

message Order {
  // Resource name (AIP-122): orders/{id} where id is a ULID.
  string name = 1; // e.g. "orders/01H8X1ABCDEF1234567890AB"

  string customer_id = 2; // ULID — opaque, not auto-increment

  Status status = 3;

  acme.common.v1.Money total = 4;

  google.protobuf.Timestamp created_at = 5;
  google.protobuf.Timestamp updated_at = 6;

  // Line items — at least one on create.
  repeated LineItem line_items = 7;

  // Mutually exclusive payment method — oneof enforces exactly one (or
none).
  oneof payment_method {
    CardPayment card = 8;
    InvoicePayment invoice = 9;
  }

  // Arbitrary labels — map is syntactic sugar for repeated entry.
  map<string, string> labels = 10;

  // Deprecated field — retained as reserved after removal (see §
evolution).
  reserved 11;
  reserved "legacy_priority";

  enum Status {
    STATUS_UNSPECIFIED = 0; // proto3 requires zero default — must be
UNSPECIFIED
    STATUS_PENDING = 1;
    STATUS_PAID = 2;
    STATUS_SHIPPED = 3;
    STATUS_CANCELLED = 4;
  }
}

message LineItem {
  string sku = 1;
```

```
  int32 quantity = 2; // validated: 1..999 in service logic (not in
schema)
  acme.common.v1.Money unit_price = 3;
}

message CardPayment {
  string card_token = 1; // tokenised — never raw PAN
  string last_four = 2;
}

message InvoicePayment {
  string po_number = 1;
  google.protobuf.Timestamp due_date = 2;
}

// ---- Request / Response messages ----
// Every RPC gets its own request/response — even when the response is
just an Order.
// This is how you add fields later without breaking the service
signature.

message CreateOrderRequest {
  // Idempotency key — client-generated UUID; server stores 24h (cf. Ch
1 & 6).
  string idempotency_key = 1;
  string customer_id = 2;
  repeated LineItem line_items = 3;
  oneof payment_method {
    CardPayment card = 4;
    InvoicePayment invoice = 5;
  }
  map<string, string> labels = 6;
}

message CreateOrderResponse {
  Order order = 1;
}

message GetOrderRequest {
  string name = 1; // "orders/{id}"
  google.protobuf.FieldMask read_mask =
2; // sparse fieldset (cf. REST fields=)
}

message GetOrderResponse {
  Order order = 1;
}

message ListOrdersRequest {
  string parent = 1; // "customers/{id}" — collection owner (AIP-132)
  int32 page_size = 2; // 1..100, default 20 — server clamps
```

```
  string page_token = 3; // opaque cursor from previous response
  string filter = 4; // CEL-like: "status=STATUS_PAID && created_at >
timestamp(\"2026-01-01T00:00:00Z\")"
  string order_by = 5; // e.g. "created_at desc"
  google.protobuf.FieldMask read_mask = 6;
}

message ListOrdersResponse {
  repeated Order orders = 1;
  string next_page_token = 2;
  int32 total_size = 3; // optional — only when caller needs count
}

message UpdateOrderRequest {
  Order order = 1;
  google.protobuf.FieldMask update_mask = 2; // which fields to apply
}

message UpdateOrderResponse {
  Order order = 1;
}

message CancelOrderRequest {
  string name = 1;
  string reason = 2;
}

message CancelOrderResponse {
  Order order = 1;
}

message WatchOrdersRequest {
  string parent = 1; // "customers/{id}"
  google.protobuf.Timestamp since = 2; // replay from this time
}

message WatchOrdersResponse {
  Order order = 1;
  ChangeType change_type = 2;
  enum ChangeType {
    CHANGE_TYPE_UNSPECIFIED = 0;
    CHANGE_TYPE_ADDED = 1;
    CHANGE_TYPE_MODIFIED = 2;
    CHANGE_TYPE_REMOVED = 3;
  }
}

message IngestReturnsRequest {
  string order_name = 1;
  string sku = 2;
  int32 quantity = 3;
```

```
}

message IngestReturnsResponse {
  int32 accepted = 1;
  int32 rejected = 2;
}

message ChatMessage {
  string session_id = 1;
  oneof payload {
    string text = 2;
    Order order_snapshot = 3;
  }
  google.protobuf.Timestamp sent_at = 4;
}
```

Shared types for completeness:

```
// proto/acme/common/v1/money.proto
syntax = "proto3";
package acme.common.v1;
option go_package = "github.com/acme/apis/gen/common/v1;commonv1";

message Money {
  string currency_code = 1; // ISO 4217, e.g. "USD"
  int64 units = 2;          // whole units
  int32 nanos = 3;          // fractional nanos, same sign as units
}
```

```
// proto/acme/common/v1/pagination.proto
syntax = "proto3";
package acme.common.v1;
option go_package = "github.com/acme/apis/gen/common/v1;commonv1";

message PageRequest {
  int32 page_size = 1;
  string page_token = 2;
}
message PageResponse {
  string next_page_token = 1;
  bool has_more = 2;
}
```

Design choices in the schema

Every RPC has its own request/response message even when the response is trivially `Order`. This is the single most important Protobuf authoring habit. If `GetOrder` returned `Order` directly and you later

needed to add `read_mask` behaviour that changes the response shape, you would need a new RPC. With `GetOrderResponse { Order order = 1; }` you add a field to the response message without touching the service definition.

Enums always have `UNSPECIFIED = 0`. Proto3 requires a zero default, and the zero value is what a client gets when it does not set the field. If `STATUS_PENDING` were zero, an unset field and an explicitly pending order would be indistinguishable. `UNSPECIFIED` makes "not set" detectable and forces server code to handle it (typically as `INVALID_ARGUMENT`).

`oneof` for mutually exclusive alternatives. `payment_method` can be `card` or `invoice`, never both. `oneof` enforces this in generated code (setting one clears the other) and on the wire (only one tag appears). Do not model this as two optional fields with application-level validation.

Well-known types for cross-language semantics. `google.protobuf.Timestamp` and `google.protobuf.FieldMask` have canonical JSON mappings and language-specific helpers (Go's `timestamppb`, Java's `Timestamps`). Using `string` for timestamps is a consistency failure that every consumer will parse differently.

Resource names as strings (`orders/{id}`) not bare IDs. AIP-122 resource names make routing, logging, and IAM policy uniform: `orders/01H8X...` is the resource, `customers/01H8Y.../orders` is the collection, and `parent` fields scope list calls. They also make gRPC-JSON transcoding produce REST-like URIs (`GET /v1/{name=orders/*}`) without a separate REST contract.

Field presence, defaults, and the zero-value trap

Proto3's most surprising behaviour for newcomers is that scalar fields set to their zero value are indistinguishable on the wire from unset fields — both are absent. `int32 quantity = 2` set to `0` encodes as nothing; a receiver sees `0` whether the sender meant "zero" or "not set."

Three mechanisms address this, each with a trade-off:

- **`optional` keyword (proto3.15+).** `optional int32 quantity = 2;` restores explicit presence tracking — the generated code has `hasQuantity()` / `Quantity != nil`. Use this when you need to

distinguish "set to zero" from "not set" on a scalar, including for `FieldMask`-driven partial updates.

- **Wrapper messages.** `google.protobuf.Int32Value` (now discouraged in favour of `optional`) wraps a scalar so absence is `null` in JSON and `nil` in Go. Prefer `optional` in new code.
- **`oneof` as presence wrapper.** A `oneof` containing a single scalar field gives it presence because the `oneof` case being set is the presence signal.

For `UpdateOrder`, the canonical pattern is `update_mask` + presence:

```
// Server — apply only fields in the mask, honouring presence.
func (s *server) UpdateOrder(ctx context.Context, req *ordersv1.UpdateO
rderRequest) (*ordersv1.UpdateOrderResponse, error) {
    if req.GetOrder() == nil || req.GetOrder().GetName() == "" {
        return nil, status.Errorf(codes.InvalidArgument, "order.name
required")
    }
    existing, err := s.store.Get(ctx, req.GetOrder().GetName())
    if err != nil { return nil, status.Errorf(codes.NotFound,
"order %q not found", req.GetOrder().GetName()) }

    mask := req.GetUpdateMask()
    if mask == nil || len(mask.GetPaths()) == 0 {
        return nil, status.Errorf(codes.InvalidArgument, "update_mask
required")
    }
    for _, path := range mask.GetPaths() {
        switch path {
        case "labels":
            existing.Labels = req.GetOrder().GetLabels()
        case "status":
            if req.GetOrder().GetStatus() == ordersv1.Order_STATUS_UNSP
ECIFIED {
                return nil, status.Errorf(codes.InvalidArgument, "statu
s: must not be UNSPECIFIED")
            }
            existing.Status = req.GetOrder().GetStatus()
        default:
            return nil, status.Errorf(codes.InvalidArgument, "update_ma
sk: unsupported path %q", path)
        }
    }
    if err := s.store.Save(ctx, existing); err != nil { return nil, sta
tus.Error(codes.Internal, err.Error()) }
    return &ordersv1.UpdateOrderResponse{Order: existing}, nil
}
```

Without `update_mask`, a client that sends `{ order: { status: STATUS_PAID } }` would implicitly clear every field it did not set — a classic partial-update bug.

Compatibility — the rules that let services evolve independently

Protobuf's wire format is what makes evolution safe, and the rules are mechanical. Internalize them as CI checks, not tribal knowledge.

Wire type and field numbers

Every field is encoded as `key = (field_number << 3) | wire_type` followed by the value (Chapter 3 of Volume 3 covers the encoding). The field *number*, not the name, is the wire identity. This implies:

- **Never change a field's number.** Data already persisted (in a queue, a log, a database column that stores serialized protos) will be misinterpreted.
- **Never reuse a retired number.** Add `reserved 11; reserved "legacy_priority";` so the compiler forbids reuse. Reusing a number silently corrupts old data.
- **Field numbers 1–15 cost one byte for the tag; 16–2047 cost two.** Reserve 1–15 for frequently populated fields. Numbers 19000–19999 are reserved for the implementation; never use them.
- **Changing a field's type is usually breaking.** `int32` / `int64` / `uint32` / `bool` / `enum` share wire type 0 (varint) and may interconvert with truncation semantics, but `int32` → `string` (wire type 0 → 2) is always breaking. Assume breaking unless you have checked the compatibility matrix.

What is safe and what is not

Change	Safe?	Why
Add new field with new number	yes	Old readers skip unknown fields by wire type

Change	Safe?	Why
Add new enum value	yes*	Old readers preserve unknown enum as integer if code handles `UNSPECIFIED`/unknown
Rename field	wire-safe, source-breaking	Name not on wire; but generated code and JSON mapping break
Remove field (reserve number+name)	wire-safe going forward	Old data with that number is still skipped; but consumers that read it break
Change field number	no	Wire identity changed — old data misinterpreted
Change wire type	no	Parser cannot skip correctly
Change `repeated` ↔ `singular`	no	Different framing
Narrow required → optional	yes	Old writers always set it; new readers handle absence
Widen optional → required	no	Old writers may omit it

* Enum addition is safe only if every consumer handles the unknown value. This is why `STATUS_UNSPECIFIED = 0` exists and why server code must treat unknown enum integers as errors or defaults, not panics.

The evolution workflow

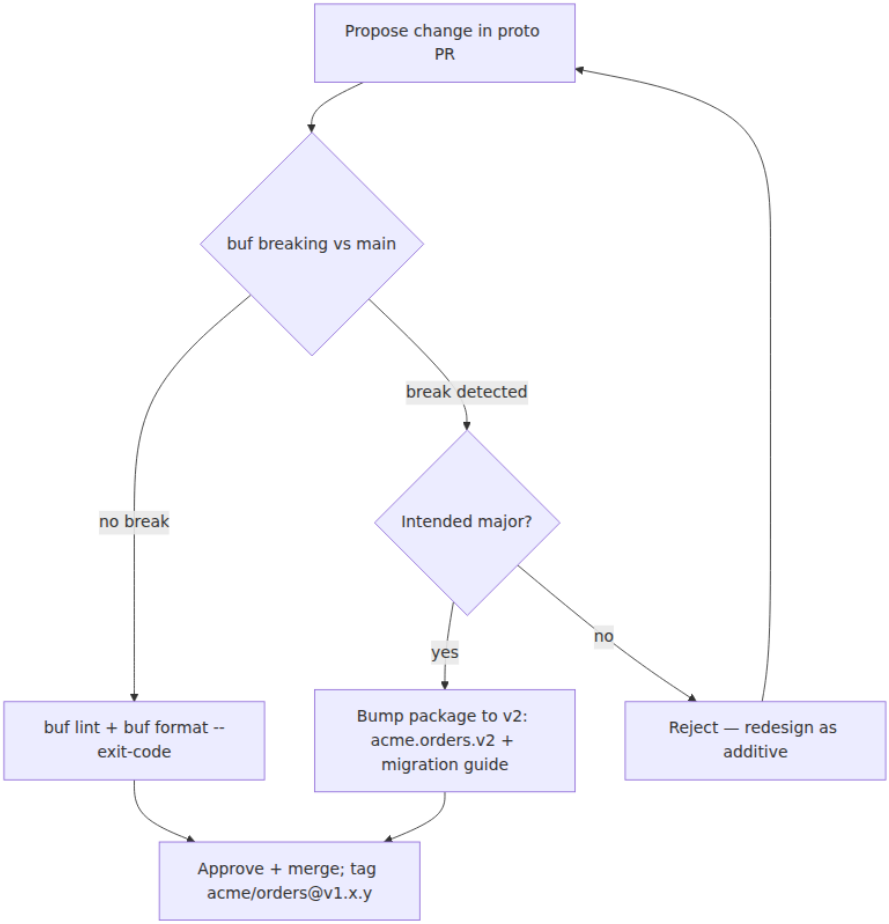
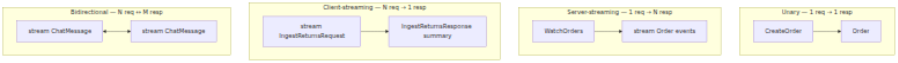


Figure 3-1: Schema evolution gate. Breaking changes require a new package version, not a justification comment.

The four call shapes — when to use each

gRPC defines four shapes distinguished by whether the client and server sides stream. All ride on one HTTP/2 stream; the difference is how many length-prefixed messages flow each way.



Each shape has a distinct fit:

Shape	Proto syntax	Use when
Unary	`rpc CreateOrder(Req) returns (Resp)`	Request/response — the default; ~90% of RPCs
Server-streaming	`rpc WatchOrders(Req) returns (stream Resp)`	Server produces a sequence: tail, pagination-as-stream, large result set
Client-streaming	`rpc IngestReturns(stream Req) returns (Resp)`	Client uploads a sequence: bulk ingestion, aggregated upload
Bidirectional	`rpc Chat(stream Req) returns (stream Resp)`	Both sides send independently: chat, interactive sessions, multiplexed telemetry

Guidance: **prefer unary** unless streaming buys something concrete. Streaming holds server resources (a goroutine or thread per stream) for the stream's lifetime, complicates load balancing (a long stream pins to one backend), and makes deadline and retry semantics subtler. Use server-streaming for `Watch`/`Tail` where the alternative is polling; use client-streaming for bulk upload where the alternative is many unary calls; use bidirectional only when both directions are genuinely independent.

---

gRPC service flow — from stub to handler

A gRPC call traverses more layers than a typical REST handler because the framework owns framing, status mapping, and cross-cutting concerns via interceptors.

Figure 3-2: gRPC service flow. Interceptors on both sides are the seam for auth, tracing, metrics, and retries without touching handler logic. The channel owns load balancing and deadline propagation.

Deadlines, metadata, and error details

Deadlines propagate as `grpc-timeout` (an absolute point in time, not a per-hop timeout). When service A calls B with a 300 ms deadline and B must call C, B passes the *same* deadline context to C — if 250 ms have elapsed, C sees ~50 ms remaining. This is the primary defence against cascading overload: work that cannot finish in the global budget is cancelled everywhere, rather than piling up.

```
// Client — derive deadline from context; gRPC sets grpc-timeout
automatically.
ctx, cancel := context.WithTimeout(context.Background(), 300*time.Milli
second)
defer cancel()
resp, err := client.CreateOrder(ctx, req)

// Server — honour ctx.Done(); pass ctx downstream so deadlines
propagate.
func (s *server) CreateOrder(ctx context.Context, req *ordersv1.CreateO
rderRequest) (*ordersv1.CreateOrderResponse, error) {
    if req.GetIdempotencyKey() == "" {
        return nil, status.Errorf(codes.InvalidArgument, "idempotency_k
ey required")
    }
    order, err := s.store.Create(ctx, req) // ctx carries deadline
    if err != nil {
        if errors.Is(err, context.DeadlineExceeded) {
            return nil, status.Error(codes.DeadlineExceeded, "store
timed out")
        }
        return nil, status.Errorf(codes.Internal, "create: %v", err)
    }
    return &ordersv1.CreateOrderResponse{Order: order}, nil
}
```

Metadata (`authorization`, `x-request-id`, `x-tenant-id`) travels as HTTP/2 headers, compressed via HPACK. Keys ending in `-bin` carry base64-encoded binary values. Metadata is the interceptor seam — do not smuggle large payloads through it.

Status codes are a closed set (~17 values), not HTTP codes. The retryability distinction matters operationally: `UNAVAILABLE` and `ABORTED` are typically retryable; `INVALID_ARGUMENT`, `NOT_FOUND`, `PERMISSION_DENIED`, and `UNIMPLEMENTED` are not. Structured details travel via `google.rpc.Status` and `grpc-status-details-bin`:

```
// Rich error with typed details (google.golang.org/genproto/
googleapis/rpc/errdetails)
st := status.New(codes.InvalidArgument, "validation failed")
ds, _ := st.WithDetails(
    &errdetails.BadRequest{
        FieldViolations: []*errdetails.BadRequest_FieldViolation{
            {Field: "customer_id", Description: "must be a ULID"},
        },
    },
    &errdetails.RetryInfo{RetryDelay: durationpb.New(0)}, // do not
retry — client bug
)
return nil, ds.Err()
```

Interceptors — the service mesh in process

Interceptors are the gRPC analogue of HTTP middleware, but they run on both sides:

```
// server.go — Go 1.22, grpc-go 1.64.0
// go get google.golang.org/grpc@v1.64.0 google.golang.org/
protobuf@v1.34.2

grpcServer := grpc.NewServer(
    grpc.ChainUnaryInterceptor(
        loggingInterceptor,   // structured log with request_id
        authInterceptor,      // validate bearer token →
context.WithValue
        validationInterceptor,// optional: protovalidate (buf.build/
protovalidate-go)
        recoveryInterceptor,  // panic → INTERNAL rather than crash
    ),
    grpc.ChainStreamInterceptor(loggingStreamInterceptor, authStreamInt
erceptor),
)
ordersv1.RegisterOrderServiceServer(grpcServer, &server{store: store})
lis, _ := net.Listen("tcp", ":50051")
grpcServer.Serve(lis)

// client — retry interceptor (retry only on retryable codes; honour
idempotency)
conn, _ := grpc.Dial("orders.internal:50051",
    grpc.WithDefaultServiceConfig(`{
        "methodConfig": [{
            "name": [{"service":"acme.orders.v1.OrderService"}],
            "retryPolicy": {
                "maxAttempts": 3,
                "initialBackoff": "0.05s",
                "maxBackoff": "1s",
                "backoffMultiplier": 2,
                "retryableStatusCodes": ["UNAVAILABLE","ABORTED"]
            }
        }]
    }`),
    grpc.WithChainUnaryInterceptor(tracingInterceptor,
retryInterceptor),
)
```

Retries are opt-in per method and must be paired with idempotency. Retrying a non-idempotent `CreateOrder` without an idempotency key duplicates the order — the same hazard as retrying a non-idempotent `POST` in REST (Chapter 6).

---

buf workflow — lint, breaking, format, generate

`buf` (1.40.2) is the de-facto toolchain for Protobuf at scale. It replaces the ad-hoc `protoc` + shell-scripts setup with versioned, reproducible operations. Pin it — `buf` compatibility checks are only as good as the version that runs in CI.

`buf.yaml` — workspace and lint/breaking config:

```
# buf.yaml — buf 1.40.2
version: v1
name: buf.build/acme/apis
deps:
  - buf.build/googleapis/googleapis
  - buf.build/grpc-ecosystem/grpc-gateway
breaking:
  use:
    - FILE  # strictest: any breaking change fails; relax to WIRE_JSON
or WIRE as needed
lint:
  use:
    - DEFAULT
    - PACKAGE_VERSION_SUFFIX  # require version suffix
(acme.orders.v1); needed because DEFAULT does not include it
  # forbid hand-written JSON names that diverge from proto naming
  allow_comment_ignores: false
```

`buf.gen.yaml` — code generation:

```
# buf.gen.yaml — buf 1.40.2
version: v1
plugins:
  - plugin: buf.build/protocolbuffers/go:v1.34.2
    out: gen/go
    opt: paths=source_relative
  - plugin: buf.build/grpc/go:v1.4.0
    out: gen/go
    opt: paths=source_relative,require_unimplemented_servers=false
  - plugin: buf.build/protocolbuffers/java:v4.26.1
    out: gen/java
```

`buf.lock` — commit this; it pins transitive deps so `buf lint` and `buf breaking` are reproducible.

CI gate (GitHub Actions — `actions/checkout` 4.1.7, `bufbuild/buf-action` 1.0.0):

```yaml
# .github/workflows/proto-ci.yaml
name: proto-ci
on:
  pull_request:
    paths: ["proto/**", "buf.yaml", "buf.gen.yaml", "buf.lock"]
jobs:
  buf:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4.1.7
        with: { fetch-depth: 0 } # buf breaking needs git history
      - uses: bufbuild/buf-setup-action@v1.34.0
        with: { github_token: ${{ secrets.GITHUB_TOKEN }} }
      - run: buf lint
      - run: buf format -d --exit-code  # fail on unformatted files
      - run: buf breaking --against 'https://github.com/acme/
apis.git#branch=main'
      - run: buf generate
      - run: git diff --exit-code gen/  # generated code must be
committed
```

```
# Local equivalents — buf 1.40.2
buf lint
buf format -w          # format in place
buf breaking --against '.git#branch=main'
buf generate           # writes gen/go, gen/java per buf.gen.yaml
buf build -o image.bin # single-file image for BSR or code review
```

> **Versioning note.** When `buf breaking` reports a break that is intentional, the fix is not to suppress the check — it is to introduce a new package version (`acme.orders.v2`) and serve both, with a migration window. Suppressions (`// buf:lint:ignore`) are for false positives, not for compatibility violations.

Distributed-systems lens and anti-patterns

Why Protobuf's compatibility model is a distributed-systems mechanism. At scale, producer and consumer deploy independently and run multiple versions simultaneously. Protobuf's rule — "unknown fields are skipped by wire type, new fields get new numbers" — is what makes independent deploys safe. Without it, every additive change would require coordinated rollout, and the coordination cost would grow with service count. The schema *is* the deployment decoupling.

Deadlines as a cross-cutting contract. A gRPC deadline is not a per-hop timeout; it is a global budget that propagates. This is load-bearing for tail latency and cascade prevention (Vol 3, Ch 11; Vol 11, Ch 10): without propagated deadlines, a slow leaf holds goroutines across the call graph until the caller exhausts its pool.

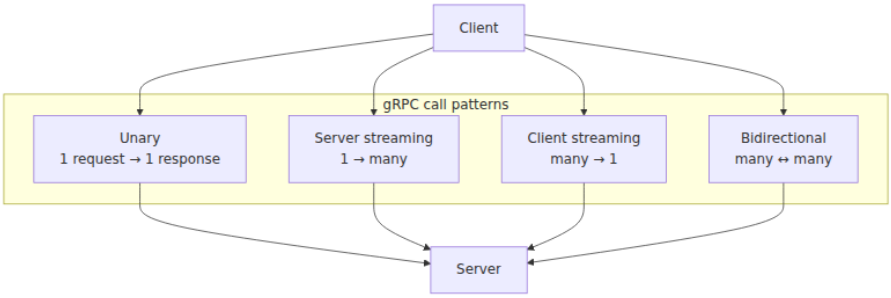
Load balancing interacts with streaming. A long-lived server-streaming `WatchOrders` stream pins to one backend (its HTTP/2 stream cannot migrate). During a rolling deploy, draining with `GOAWAY` must let streams finish or be cancelled; client-side `round_robin` or look-aside LB (xDS) must handle re-resolution. Prefer unary unless streaming is required.

Common anti-patterns to forbid in review:

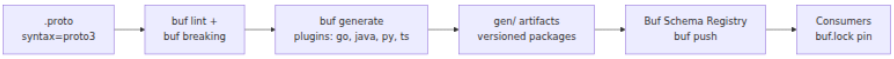
- Reusing a field number or changing a field's wire type — silent data corruption on old persisted data.
- Returning raw domain errors as `INTERNAL` without `codes.Code` — callers cannot branch on whether to retry.
- Omitting `UNSPECIFIED = 0` on enums — unset and first-value become indistinguishable.
- Using `string` for timestamps or `double` for money — locale and precision bugs that cross every language boundary.

---

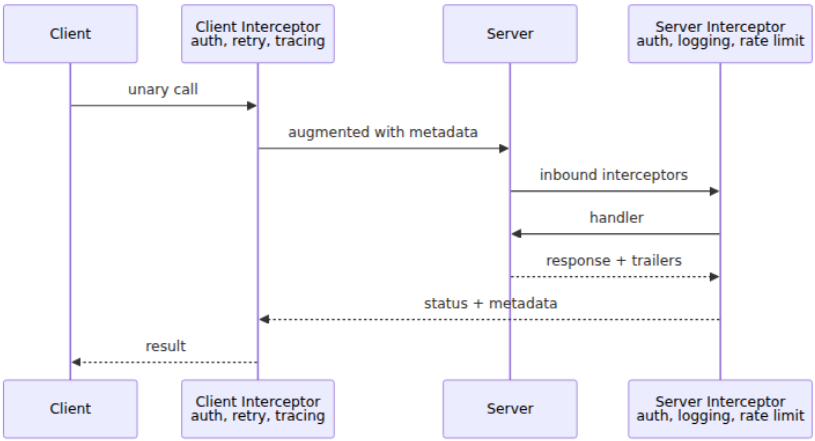
gRPC Call Types



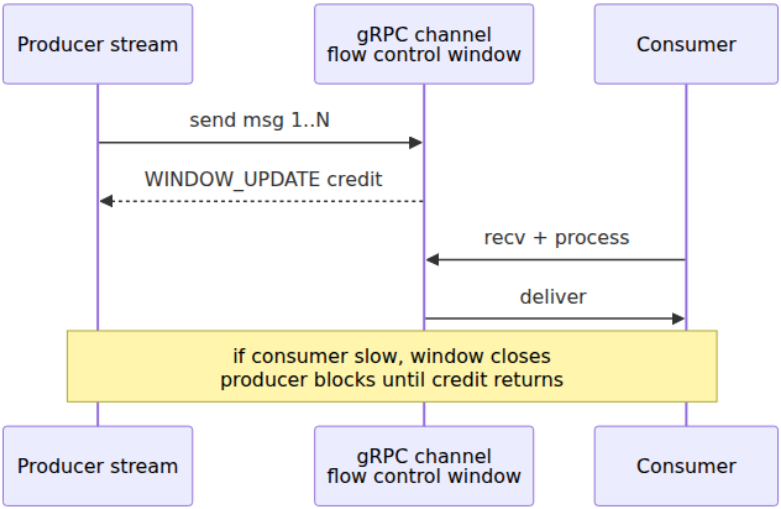
Protobuf Build Pipeline



Interceptor Chain



Streaming Backpressure



Key takeaways

- Use gRPC/Protobuf for interior service-to-service traffic where throughput, typing, and streaming outweigh REST's debuggability and cacheability; use REST at the edge.

- Every RPC gets its own request/response message; every enum gets `UNSPECIFIED = 0`; every `oneof` models a true mutual exclusion — these three habits prevent the most common schema debt.
- Field numbers are the wire identity: never change, never reuse (use `reserved`), and reserve 1–15 for hot fields; changing wire type is always breaking.
- `update_mask` + field presence (`optional` or `FieldMask`) is the correct partial-update pattern; without it, absent fields are misinterpreted as intentional clears.
- Choose unary by default; use server-streaming for tail/watch, client-streaming for bulk upload, and bidirectional only when both directions are genuinely independent.
- Deadlines propagate as `grpc-timeout` — treat them as a global budget, honour `ctx.Done()`, and map retryability to `codes.Code` (`UNAVAILABLE` / `ABORTED` retryable, `INVALID_ARGUMENT` / `NOT_FOUND` not).
- `buf` 1.40.2 gives you `lint`, `breaking`, `format`, and `generate` as a single versioned gate; wire it into CI with `fetch-depth: 0` and fail the PR on any break that is not a new package version.

Further reading

- Protocol Buffers Language Specification (proto3) — https://protobuf.dev/programming-guides/proto3/ — the authoritative reference for field presence, `oneof`, maps, and well-known types.
- gRPC Core Concepts — https://grpc.io/docs/what-is-grpc/ — service definitions, the four call shapes, status codes, and metadata.
- Buf Documentation — https://buf.build/docs/ — especially *Lint*, *Breaking Changes*, and *Code Generation* (buf 1.40.2).
- Google Cloud. *API Improvement Proposals (AIPs)* — https://aip.dev/ — especially AIP-122 (resource names), AIP-132 (parent/collection), AIP-158 (pagination), AIP-234 (batch). Resource-oriented Protobuf conventions; even outside Google they prevent the most common design drift.
- Vol 3, Chapter 8 — gRPC and RPC Framework Internals — the wire companion to this chapter: HTTP/2 framing, varint encoding, HPACK, and the Fallacies of Distributed Computing.

- Souppaya et al. *Protovalidate* (buf.build/protovalidate) — declarative field constraints for Protobuf as a complement to service-side validation.

Chapter 4 — GraphQL for Backend Engineers

What this chapter covers. REST gives you resources; gRPC gives you typed RPCs. GraphQL gives you a *queryable graph* — a single endpoint where clients describe the shape of the data they want and the server resolves it from whatever sources back it. That flexibility is why frontend teams love GraphQL (one round-trip, no over-fetching) and why backend teams approach it warily (arbitrary queries, the N+1 problem, cache-hostile POSTs, and a gateway that must orchestrate dozens of downstream services to answer one request). This chapter is the backend lens on GraphQL: when to choose it and — just as important — when *not* to; how to design a schema that remains coherent as it grows; how resolvers, batching, and DataLoader turn a naive graph traversal into an efficient execution plan; how federation and schema stitching distribute ownership without fragmenting the graph; and how to govern, secure, and observe a GraphQL gateway under production load. We anchor every pattern in real SDL and real Node.js code, pinned and runnable.

Learning goals — after this chapter you should be able to:

- Decide between GraphQL, REST (Chapter 2), and gRPC (Chapter 3) for a given surface using concrete criteria — consumer diversity, fetch patterns, cacheability, and team ownership — not fashion.
- Author a version-pinned GraphQL SDL with sound type design (objects, interfaces, unions, enums, input types), pagination, and error modelling, and explain why schema-first discipline matters.
- Trace a GraphQL execution from parse → validate → execute → resolve, diagnose the N+1 problem quantitatively, and implement DataLoader batching and caching to fix it.
- Configure Apollo Server 4 with depth/complexity limits, persisted queries, and tracing, and justify each as a production control.

- Compare schema stitching, Apollo Federation 2, and the monolithic gateway, and sketch a federated architecture that preserves team autonomy with a unified graph.
- Explain GraphQL's distributed-systems costs — gateway as a bottleneck, fan-out amplification, cache invalidation, and partial failure — and the mitigations that make it operable.

Where GraphQL fits — and where it does not

GraphQL is not a replacement for REST or gRPC. It is a *product* for a specific consumer shape: many heterogeneous clients (web, iOS, Android, partner integrations) that need different projections of the *same* underlying domain, where the set of useful projections is too large to enumerate as REST endpoints and where the cost of over-fetching or waterfall requests is felt directly in user-perceived latency.

Dimension	REST (Ch 2)	gRPC/Protobuf (Ch 3)	GraphQL
Consumer	browsers, partners, any HTTP client	interior services you control	frontend apps with diverse data needs
Fetch shape	fixed per endpoint; client chooses endpoint	fixed per RPC; strongly typed	client chooses fields per query
Round-trips	often N (list then fetch each) without batch	1 per RPC; streaming for sequences	1 for an arbitrarily nested selection
Cacheability	excellent (`GET` + `ETag` / `Cache-Control` ; CDN-friendly)	poor (binary, POST-like)	poor by default (POST, per-query shape)
Schema evolution	additive fields, versioned paths	package-versioned `.proto` , field-number compat	additive types/fields; `@deprecated` ; no versioned URL

Dimension	REST (Ch 2)	gRPC/Protobuf (Ch 3)	GraphQL
Tooling cost	low — `curl` + OpenAPI	codegen per language, `buf`	gateway, query analysis, DataLoader, federation

The honest rule:

- **Use GraphQL** when a single domain (orders, users, catalog) is consumed by multiple frontend surfaces that need different field sets and relationship traversals, and where the alternative is either endpoint proliferation (`GET /orders?fields=...&include=...` growing without bound) or client-side waterfall fetches.
- **Do not use GraphQL** for service-to-service interior traffic (use gRPC), for cache-heavy public APIs behind a CDN (use REST), or for event-driven surfaces (use AsyncAPI/Kafka — Vol 10). Inserting GraphQL between two backend services you own trades debuggability and cacheability for flexibility neither caller needs.

> **Boundary note.** This chapter covers GraphQL *service design* — schema modelling, resolver execution, gateway architecture, and backend operability. The wire mechanics of the transports GraphQL rides on — HTTP/1.1 vs HTTP/2 vs WebSocket for subscriptions, TLS, and header compression — are covered in Volume 3, Chapters 7 and 10. The persistence and indexing that make paginated GraphQL queries fast are covered in Volume 5 (Databases), especially Chapters 3 (Indexing) and 4 (Query Optimization); this chapter notes the storage contract pagination depends on but does not re-derive it.

Schema design — the graph as a contract

A GraphQL schema is the contract. Unlike REST, where the contract is spread across paths, methods, and an OpenAPI document, or gRPC where it lives in `.proto` files, the GraphQL SDL *is* the entire surface in one artifact. Every type, field, and directive is discoverable via introspection, which is both an asset (self-documenting, strongly typed clients via codegen) and a liability (the schema is a public commitment that is harder to version than a versioned REST path — see Chapter 5).

Design principles

1. **Model the domain graph, not the storage graph.** Types should reflect product concepts (`Order`, `Customer`, `LineItem`) rather than tables. If a storage join is not a product relationship, do not expose it as one.
2. **Connections for collections, always.** The Relay Connection spec (`edges { node, cursor } + pageInfo`) is the GraphQL analogue of cursor pagination in Chapter 2 and Chapter 6. Use it for every list — even when the list is currently small — because changing a `[Order!]!` to a `Connection` later is a breaking change.
3. **Input types for mutations, payload types for results.** `CreateOrderInput` and `CreateOrderPayload` give you room to add fields (e.g., `clientMutationId`, `errors`) without breaking the mutation signature.
4. **Enums with `@deprecated` and `UNSPECIFIED` discipline.** Like Protobuf enums (Chapter 3), GraphQL enums need a handling story for unknown values — clients must treat enums as open.
5. **Nullability is a design decision.** In GraphQL, `String` means nullable, `String!` means non-null. Be deliberate: a field that is non-null in the schema but nullable in storage will produce execution errors that null-bubble to the nearest nullable parent.

A complete schema — Orders domain

The SDL below is a complete, `graphql@16.8.1`-valid schema covering the patterns you will use in production: object and input types, interfaces, unions, enums, the Relay Connection, directives, and deprecation. Versions pinned in comments.

```graphql
# schema.graphql — Orders subgraph (Apollo Federation 2.5 / graphql
16.8.1)
# Tooling: graphql@16.8.1, @apollo/server@4.10.0, @as-integrations/
fastify@2.1.1
# Lint: graphql-eslint 3.20.1  (graphql-eslint --schema schema.graphql)
# Codegen: @graphql-codegen/cli@5.0.2
extend schema
  @link(url: "https://specs.apollo.dev/federation/v2.5", import: ["@ke
y", "@shareable", "@external"])

# ---- Scalars ----
scalar DateTime  # RFC 3339, e.g. "2026-03-10T14:22:31Z"
scalar ULID      # 26-char Crockford Base32, e.g.
"01H8X1ABCDEF1234567890AB"

# ---- Interfaces ----
interface Node {
  id: ULID!
}

# ---- Enums ----
enum OrderStatus {
  PENDING
  PAID
  SHIPPED
  CANCELLED
}

# ---- Objects ----
type Order implements Node @key(fields: "id") {
  id: ULID!
  customer: Customer!          # resolved via DataLoader (see § N+1)
  status: OrderStatus!
  totalCents: Int!             # Money as integer cents; Money object
alternative in text
  createdAt: DateTime!
  updatedAt: DateTime!
  lineItems: [LineItem!]!
  labels: [Label!]!
  # Deprecated field retained for one sunset window (see Ch 5)
  legacyPriority: Int @deprecated(reason: "Use labels instead. Removal
Sunsets 2026-12-31.")
}

type LineItem {
  sku: String!
  quantity: Int!
  unitPriceCents: Int!
}
```

```graphql
type Label {
  key: String!
  value: String!
}

type Customer implements Node @key(fields: "id") {
  id: ULID!
  displayName: String!
  email: String!                # gateway should gate by auth scope
}

# ---- Connection (Relay) ----
type OrderConnection {
  edges: [OrderEdge!]!
  pageInfo: PageInfo!
  totalCount: Int
# optional; expensive — only when caller needs it
}

type OrderEdge {
  node: Order!
  cursor: String!               # opaque, base64url — see Ch 6 for
construction
}

type PageInfo {
  hasNextPage: Boolean!
  hasPreviousPage: Boolean!
  startCursor: String
  endCursor: String
}

# ---- Inputs ----
input CreateOrderInput {
  customerId: ULID!
  lineItems: [LineItemInput!]!
  labels: [LabelInput!]
  idempotencyKey: String!       # UUID v4, see Ch 6
}

input LineItemInput {
  sku: String!
  quantity: Int!
}

input LabelInput {
  key: String!
  value: String!
}

input OrderFilter {
```

```
  status: OrderStatus
  createdAfter: DateTime
  customerId: ULID
}

enum OrderSort {
  CREATED_AT_ASC
  CREATED_AT_DESC
  TOTAL_CENTS_ASC
  TOTAL_CENTS_DESC
}

# ---- Payloads ----
type CreateOrderPayload {
  order: Order
  errors: [UserError!]!
}

type UserError {
  code: String!                 # machine-readable, e.g. "BAD_REQUEST"
  message: String!
  path: [String!]
}

# Union for search results that span types
union SearchResult = Order | Customer

# ---- Root operations ----
type Query {
  node(id: ULID!): Node
  order(id: ULID!): Order
  orders(
    first: Int = 20
    after: String
    filter: OrderFilter
    sort: OrderSort = CREATED_AT_DESC
  ): OrderConnection!

  # Search across types — demonstrates union
  search(query: String!, first: Int = 10): [SearchResult!]!

  # Introspection-adjacent
  _service: String!
}

type Mutation {
  createOrder(input: CreateOrderInput!): CreateOrderPayload!
  cancelOrder(id: ULID!, reason: String): CreateOrderPayload!
}

type Subscription {
```

```
  # Requires WebSocket or SSE transport; see § subscriptions
  orderStatusChanged(customerId: ULID!): Order!
}
```

Design choices worth noting:

- **Relay Connections, not bare lists.** `orders` returns `OrderConnection`, not `[Order!]!`. Changing later would break every client that paginates. `first` / `after` cursors are opaque; the resolver builds them from `(created_at, id)` keyset columns — the same construction as the REST cursor in Chapter 2, whose storage mechanics are analysed in Volume 5, Chapter 3 (Indexing).
- **`Node` interface + `@key`.** Federation entities are identified by `@key(fields: "id")`. The gateway can resolve `Customer` from the users subgraph without the orders subgraph knowing how customers are stored.
- **Payload types with `errors`.** Mutations return `CreateOrderPayload { order, errors }` rather than throwing top-level GraphQL errors for user mistakes. Top-level `errors` are reserved for execution failures (auth, rate limiting); `payload.errors` carry validation problems the client can handle field-by-field.
- **`OrderFilter` as an input object**, not a string DSL. GraphQL's type system validates filters at parse time, unlike REST's `filter[status]` string convention — but the trade-off is that the filter set must be enumerated in the schema.

Execution — from query to resolvers to DataLoader

A GraphQL request's lifecycle is more work than a REST handler's because the server must *plan* execution from the query shape:

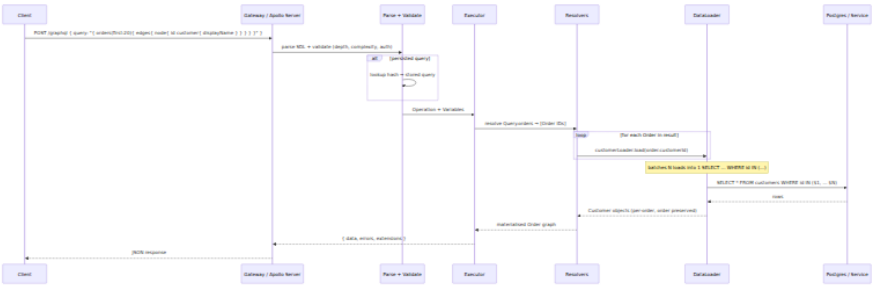


Figure 4-1: GraphQL execution lifecycle. The DataLoader batch window collapses N per-entity loads into one query — the critical optimisation for any non-trivial graph.

The N+1 problem — measured

The classic failure mode: a query asks for 20 orders and each order's customer. A naive resolver does:

```
1 query: SELECT * FROM orders ORDER BY created_at DESC LIMIT 20
20 queries: SELECT * FROM customers WHERE id = $1  (once per order)
```

At 20 orders this is 21 queries; at 200 (a large page or a nested traversal) it is 201. Under load, this is not merely slow — it exhausts connection pools and amplifies tail latency. At 1,000 RPS, 21 queries per request is 21,000 DB queries per second where 2,000 would suffice.

Figure 4-2: N+1 versus batched execution. DataLoader's batch window turns N sequential point queries into one IN query, and its per-request memoisation prevents duplicate loads for the same entity within one operation.

Solving it — DataLoader

Facebook's `dataloader@2.2.2` is the standard solution. It provides two guarantees per request: **batching** (collect `load()` calls within one tick, dispatch as one function call) and **memoisation** (duplicate `load(id)` within one request hits cache, not the data source). Both are scoped to a single GraphQL operation — a new request gets a fresh loader, so cross-request staleness is not a concern.

Real, runnable implementation — Node 20.11.0, Apollo Server 4.10.0, DataLoader 2.2.2, pg 8.11.3:

```
// loaders.js — Node 20.11.0, dataloader@2.2.2, pg@8.11.3
import DataLoader from "dataloader";
import { pool } from "./db.js"; // pg.Pool — configured in app.js

/**
 * Batch function: DataLoader calls this with all keys collected in one
tick.
 * Must return Promise<Array<V>> of the same length and order as `ids`.
 * Missing keys map to null (or an Error) at that index.
 */
async function batchCustomers(ids) {
  // ids: string[] — ULIDs
  const { rows } = await pool.query(
    `SELECT id, display_name, email FROM customers WHERE id = ANY($1)`,
    [ids],
  );
  const byId = new Map(rows.map((r) => [r.id, r]));
  return ids.map((id) => byId.get(id) ?? null);
  // Alternative for strict: return Error for missing, so GraphQL null-
bubbles correctly
  // return ids.map((id) => byId.get(id) ?? new Error(`Customer ${id}
not found`));
}

export function createLoaders() {
  return {
    customerLoader: new DataLoader(batchCustomers, {
      cache: true,          // per-request memoisation (default true)
      maxBatchSize:
100,     // split large batches to stay under query param limit
    }),
    // Add loaders per entity — orderLoader, productLoader, etc.
  };
}
```

```
// server.js — @apollo/server@4.10.0, @as-integrations/fastify@2.1.1, graphql@16.8.1
// Node 20.11.0
import Fastify from "fastify";
import { ApolloServer } from "@apollo/server";
import fastifyApollo, { fastifyApolloDrainPlugin } from "@as-integrations/fastify";
import { readFileSync } from "fs";
import { createLoaders } from "./loaders.js";
import { pool } from "./db.js";

const typeDefs = readFileSync("./schema.graphql", "utf8");

const resolvers = {
  Node: {
    __resolveType(obj) {
      if (obj.display_name !== undefined) return "Customer";
      if (obj.status !== undefined) return "Order";
      return null;
    },
  },
  SearchResult: {
    __resolveType(obj) {
      return obj.display_name !== undefined ? "Customer" : "Order";
    },
  },
  Query: {
    async order(_, { id }, { loaders }) {
      const { rows } = await pool.query(`SELECT * FROM orders WHERE id = $1`, [id]);
      return rows[0] ?? null;
    },
    async orders(_, { first = 20, after, filter, sort = "CREATED_AT_DESC" }) {
      // cursor pagination — decode opaque cursor (base64url JSON)
      let cursor = null;
      if (after) {
        try {
          cursor = JSON.parse(Buffer.from(after, "base64url").toString("utf8"));
        } catch {
          throw new Error("Invalid cursor");
        }
      }
      const limit = Math.min(first, 100) + 1; // one-extra trick
      const where = [];
      const params = [];
      let idx = 1;
      if (filter?.status) { where.push(`status = $${idx++}`); params.push(filter.status.toLowerCase()); }
```

```
      if (filter?.createdAfter) { where.push(`created_at >= ${idx++}`)
; params.push(filter.createdAfter); }
      if (filter?.customerId) { where.push(`customer_id = ${idx++}`);
params.push(filter.customerId); }
      if (cursor) {
        // keyset on (created_at, id) — requires composite index; see
Vol 5 Ch 3
        const op = sort.endsWith("_ASC") ? ">" : "<";
        where.push(`(created_at, id) ${op} ($${idx}, $${idx + 1})`);
        params.push(cursor.created_at, cursor.id);
        idx += 2;
      }
      const dir = sort.endsWith("_ASC") ? "ASC" : "DESC";
      const whereSQL = where.length ? `WHERE ${where.join(" AND ")}` :
"";
      const { rows } = await pool.query(
        `SELECT * FROM orders ${whereSQL} ORDER BY created_at ${dir},
id ${dir} LIMIT $${idx}`,
        [...params, limit],
      );
      const hasNextPage = rows.length > first;
      const page = hasNextPage ? rows.slice(0, first) : rows;
      const edges = page.map((row) => ({
        node: row,
        cursor: Buffer.from(JSON.stringify({ created_at:
row.created_at.toISOString(), id: row.id })).toString("base64url"),
      }));
      return {
        edges,
        pageInfo: {
          hasNextPage,
          hasPreviousPage: cursor !== null,
          startCursor: edges[0]?.cursor ?? null,
          endCursor: edges[edges.length - 1]?.cursor ?? null,
        },
        totalCount: null, // omit COUNT(*) at scale unless caller
explicitly needs it
      };
    },
    search: async (_, { query, first }) => {
      // simplified — real implementation fans out to search index
      return [];
    },
  },
  Order: {
    // This resolver is where N+1 would happen — DataLoader fixes it
    customer(parent, _, { loaders }) {
      return loaders.customerLoader.load(parent.customer_id);
    },
    lineItems(parent) {
      // line items stored as JSONB or separate table — no N+1 if
```

```
fetched with order
      return parent.line_items ?? [];
    },
    labels(parent) {
      return Object.entries(parent.labels ?? {}).map(([k, v]) => ({
key: k, value: v }));
    },
  },
  Mutation: {
    async createOrder(_, { input }, { loaders }) {
      // Idempotency handled by DB unique constraint on idempotency_key
— see Ch 6
      try {
        const { rows } = await pool.query(
          `INSERT INTO orders (id, customer_id, status, total_cents,
line_items, labels, idempotency_key)
           VALUES (gen_ulid(), $1, 'pending', $2, $3, $4, $5)
           ON CONFLICT (idempotency_key) DO NOTHING
           RETURNING *`,
          [input.customerId, 0, JSON.stringify(input.lineItems), JSON.s
tringify(input.labels ?? []), input.idempotencyKey],
        );
        if (rows.length === 0) {
          // replay — fetch original
          const existing = await
pool.query(`SELECT * FROM orders WHERE idempotency_key = $1`, [input.id
empotencyKey]);
          return { order: existing.rows[0], errors: [] };
        }
        return { order: rows[0], errors: [] };
      } catch (e) {
        return { order: null, errors: [{ code: "INTERNAL", message: e.m
essage, path: ["createOrder"] }] };
      }
    },
  },
};

const app = Fastify({ logger: true });

const server = new ApolloServer({
  typeDefs,
  resolvers,
  introspection: process.env.NODE_ENV !== "production",
  plugins: [fastifyApolloDrainPlugin(app)],
});

await server.start();

await app.register(fastifyApollo(server), {
  context: async (request) => ({
```

```
    loaders: createLoaders(), // fresh per request — critical for
correctness
    user: request.user,       // set by auth hook below
  }),
});

// Auth hook — parse JWT, attach user to request (see Vol 9 Ch 5)
app.addHook("onRequest", async (request) => {
  const auth = request.headers.authorization;
  if (auth?.startsWith("Bearer ")) {
    request.user = await verifyJWT(auth.slice(7)); // throws 401 on
invalid
  }
});

await app.listen({ port: 4000, host: "0.0.0.0" });
```

Key choices:

- **`createLoaders()` per request.** DataLoader's cache is per-operation. Sharing a loader across requests would serve stale data; sharing across operations within one request is exactly what you want.
- **`maxBatchSize: 100`.** Postgres has a parameter limit and large `IN (...)` degrades. Splitting into chunks of 100 keeps each query bounded.
- **Keyset pagination in the resolver, not `OFFSET`.** Offset pagination (`LIMIT 20 OFFSET 40000`) forces the database to scan and discard 40,000 rows; keyset (`WHERE (created_at, id) > (...)`) is a range scan on the composite index. The storage implications — why that index exists and how the planner uses it — are the subject of Volume 5, Chapters 3 and 4; here we honour the contract that pagination must be stable and efficient.
- **One-extra trick.** Fetch `first + 1` rows; the extra row tells you `hasNextPage` without a separate `COUNT(*)`.

Beyond DataLoader — when batching is not enough

DataLoader solves per-entity N+1. Two deeper cases remain:

- **Nested N+1.** `orders → customer → organisation → billingAccount` can still cascade loaders sequentially. The fix is either a join at the `orders` resolver (fetch customers with orders in one query when the traversal is known) or a look-ahead in the executor that batches across levels.

- **The fan-out problem.** A query `{ customers(first:100) { orders(first:50) { lineItems } } }` requests up to 5,000 orders and their line items in one operation. No batching strategy makes that cheap. The mitigation is *query complexity analysis* (next section) that rejects or budgets such queries before execution.

Federation — scaling ownership of the graph

A single `schema.graphql` owned by one team does not scale. Ten teams each owning a bounded context need to contribute types to a unified graph without coordinating on a single file. Two patterns dominate:

Approach	How it works	Trade-off
Schema stitching	Each service exposes a GraphQL schema; gateway merges them with `mergeSchemas` + `delegateToSchema`	Flexible, but gateway owns all merge logic; type conflicts are runtime errors
Apollo Federation 2 (`@apollo/subgraph@2.5`, `@apollo/gateway@2.5`, `federation@2.5`)	Each subgraph declares `@key` entities; gateway composes a supergraph via `rover supergraph compose`	Entities are first-class; composition is validated at build time; requires Federation-aware subgraphs
Monolithic gateway	One codebase, many resolvers	Simplest, but single ownership bottleneck

For most organisations beyond ~5 subgraphs, **Federation 2** is the right default: composition is validated in CI, entity ownership is explicit, and teams deploy subgraphs independently.

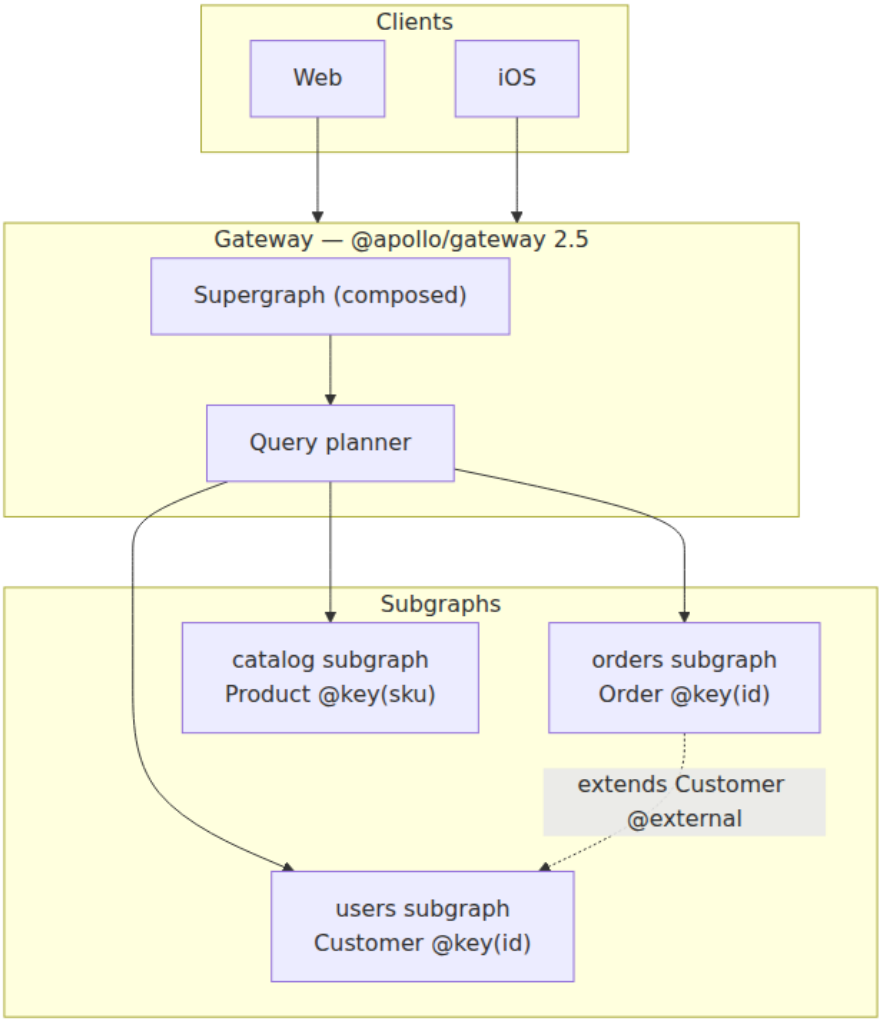


Figure 4-3: Federated graph. Each subgraph owns its entities; the gateway's query planner fans out one client query into subgraph queries and joins the results.

Minimal federated subgraph (orders) — `@apollo/subgraph@2.5.2`:

```
# orders subgraph — federation v2.5
extend schema @link(url: "https://specs.apollo.dev/federation/v2.5", import: ["@key", "@shareable", "@external"])

type Order @key(fields: "id") {
  id: ULID!
  customer: Customer!
  status: OrderStatus!
  totalCents: Int!
  createdAt: DateTime!
}

# Customer is owned by users subgraph; orders extends it
type Customer @key(fields: "id") @extends {
  id: ULID! @external
}

type Query {
  order(id: ULID!): Order
  orders(first: Int, after: String): OrderConnection!
}
```

Composition and validation in CI:

```
# rover 0.24.0 — compose supergraph, fail on composition errors
rover supergraph compose --config supergraph.yaml --output supergraph.graphql
# supergraph.yaml pins subgraph URLs and SDL locations
# rover validates: no duplicate type definitions, @key consistency, @external correctness

# Alternative: @apollo/composition without rover (Node 20)
npx @apollo/composition compose --config supergraph.yaml
```

> **Distributed-systems lens.** Federation moves the join from the database to the gateway. A query that traverses `Order → Customer → Organisation` fans out to three subgraphs, each with its own latency, failure mode, and rate limit. The gateway's query planner must handle partial failure (one subgraph times out but others succeed — return partial `data` with `errors`), deadline propagation (pass the caller's remaining budget to each subgraph), and result-size bounding. Treat the gateway as a distributed orchestrator, not a thin proxy.

Securing and operating the gateway

GraphQL's power — arbitrary query shapes over a single endpoint — is also its threat surface. Four controls are non-negotiable in production:

1. Depth and complexity limits

Without limits, a client can send `{ order { customer { orders { customer { ... } } } } }` nested 50 levels deep, or request 100 fields each costing a DB query.

```
// Apollo Server 4 — depth + complexity limits
// @apollo/server@4.10.0, graphql-depth-limit@1.1.0, graphql-validation-complexity@0.4.2
import depthLimit from "graphql-depth-limit";
import { createComplexityLimitRule } from "graphql-validation-complexity";

const server = new ApolloServer({
  typeDefs,
  resolvers,
  validationRules: [

depthLimit(10),                                          // max nesting depth
    createComplexityLimitRule(1000, {                        // max complexity score
      scalarCost: 1,
      objectCost: 2,
      listFactor: 10,
      // field costs can be overridden per field via directive
    }),
  ],
});
```

2. Persisted queries (Automatic Persisted Queries — APQ)

Instead of sending the full query string each time, clients send a hash (SHA-256) of the query; the server stores the mapping. This saves bandwidth, makes `GET`-caching possible for queries (the hash is cache-key friendly), and restricts the surface to pre-registered queries if you disable ad-hoc execution — the strongest protection against query abuse.

```
// Client (Apollo Client 3.9.4) — APQ enabled by default with HttpLink
// Server — Apollo Server 4 has APQ via cache; provide a Keyv/Redis
backing
import Keyv from "keyv"; // keyv@4.5.4

const server = new ApolloServer({
  typeDefs,
  resolvers,
  cache: new Keyv("redis://localhost:6379"), // APQ + response cache
  persistedQueries: { ttl: 86_400 },          // 24h
  allowBatchedHttpRequests: false,             // disable unless you
need it
});
```

Locking down to only persisted queries (no ad-hoc):

```
// Reject queries not in the allowlist — best for production
import { createPersistedQueryMiddleware } from "./persisted-
queries.js";
app.addHook("preHandler", createPersistedQueryMiddleware({ allowlistOnl
y: true }));
```

3. Timeouts, rate limiting, and auth

- **Per-query timeout.** Wrap execution with `Promise.race` or `AbortController`; propagate deadlines to DataLoader batch functions so slow DB queries do not hold the gateway.
- **Rate limiting by query cost, not request count.** One GraphQL request can cost 1,000× more than another. Rate-limit on complexity score or on DataLoader batch count.
- **Field-level auth.** `Customer.email` should not be resolvable without `read:customers:pii` scope. Use schema directives (`@auth(requires: READ_PII)`) checked in a `fieldResolver` wrapper, not in each resolver individually.

4. Observability

```
// Apollo Server plugin — log per-field resolver latency
const timingPlugin = {
  async requestDidStart() {
    return {
      async executionDidStart() {
        return {
          willResolveField({ info }) {
            const start = performance.now();
            return () => {
              const ms = performance.now() - start;
              if (ms > 50) console.warn(`slow resolver ${info.parentTyp
e.name}.${info.fieldName}: ${ms.toFixed(1)}ms`);
            };
          },
        };
      },
    };
  },
};
```

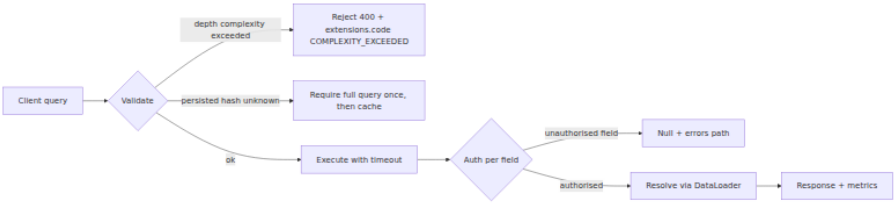


Figure 4-4: Production query lifecycle. Validation rejects abusive queries before execution; auth is field-level; DataLoader and timeouts bound the cost of what remains.

Caching — where GraphQL pays a price

REST's cacheability (Chapter 2: `GET` + `ETag` + `Cache-Control`) does not transfer cleanly to GraphQL because:

- Queries are `POST` by default (bodies are not cache keys in CDNs).
- Each query has a different shape, so response caching is per-query-hash, not per-resource.
- Mutations can invalidate many cached queries; the invalidation graph is the schema graph.

Mitigations:

Strategy	How	When
APQ over GET	`GET /graphql?extensions={"persistedQuery":{"hash":"..."}}` — hash is the cache key	When CDN caching matters and queries are persisted
Response cache per subgraph	Gateway caches subgraph responses by entity key with short TTL	Interior fan-out where subgraph data changes slowly
Client-side normalised cache	Apollo Client `InMemoryCache` keyed by `__typename + id`	Frontend deduplication; not a server concern but reduces origin load
`@cacheControl` directive	`type Order @cacheControl(maxAge: 10)` — gateway emits `Cache-Control` per field	Fine-grained TTLs when the gateway speaks HTTP caching

Do not try to replicate REST's `ETag` / `If-None-Match` semantics at the GraphQL layer. If cacheability at the edge is a primary requirement for a surface, that surface should be REST.

Subscriptions — real-time over GraphQL

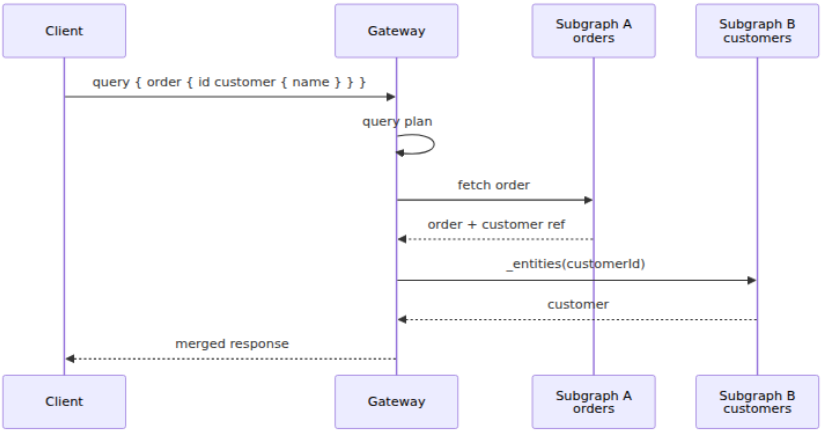
Subscriptions (`type Subscription { orderStatusChanged(customerId: ULID!): Order! }`) give clients a push stream for events. In production they ride on WebSocket (`graphql-ws@5.14.1`) or SSE, not HTTP/2 streaming.

```
// Subscriptions with graphql-ws 5.14.1 + @apollo/server 4.10.0
import { WebSocketServer } from "ws"; // ws@8.16.0
import { useServer } from "graphql-ws/lib/use/ws";
import { PubSub } from "graphql-subscriptions"; // graphql-
subscriptions@2.0.0

const pubsub = new PubSub();

const resolversWithSubs = {
  Subscription: {
    orderStatusChanged: {
      subscribe: (_, { customerId }) => pubsub.asyncIterator(`ORDER_STA
TUS:${customerId}`),
    },
  },
};

// Publish from mutation or event consumer (Vol 10)
await pubsub.publish(`ORDER_STATUS:${order.customer_id}`, { orderStatus
Changed: order });

// Wire up on Fastify — separate WS server
const wsServer = new WebSocketServer({ server: app.server, path: "/
graphql" });
useServer({ schema: server.schema, context: () => ({ loaders: createLoa
ders() }) }, wsServer);
```

> **Distributed-systems lens.** Each subscription holds a server resource (WebSocket + iterator) for its lifetime. At 100,000 concurrent subscriptions, that is 100,000 open connections — a different scaling regime from stateless `POST /graphql`. Use subscriptions only when polling is genuinely insufficient (sub-second updates, collaborative state). For everything else, short-poll the query or consume events via Kafka (Vol 10, Ch 3).

GraphQL Execution Pipeline

Federation Gateway



N+1 and DataLoader

Key takeaways

- GraphQL's strength is client-shaped fetches over a unified graph; its cost is a gateway that must plan, batch, and bound arbitrary queries. Choose it for heterogeneous frontend consumers, not for interior service-to-service traffic.
- Design the schema as the contract: Relay Connections for all lists, input/payload types for mutations, `Node` + `@key` for federation, and deliberate nullability. Validate with `graphql-eslint` and generate typed clients with `graphql-codegen`.
- The N+1 problem is not theoretical — it is the default behaviour of naive resolvers. `dataloader@2.2.2` with per-request scoping and `maxBatchSize` bounds collapses N point queries into one `IN` query. Scope loaders per request and measure before and after.
- Federation 2 (`@apollo/subgraph@2.5`) distributes graph ownership to bounded-context teams with build-time composition validation via `rover supergraph compose`. The gateway becomes a query planner that must handle fan-out, partial failure, and deadline propagation.

- Production requires depth/complexity limits, persisted queries (with allowlist-only as the strictest posture), field-level auth, per-query timeouts, and resolver-level observability. Without these, one abusive query can saturate your data sources.
- Caching in GraphQL is per-query-hash, not per-resource — weaker than REST's `ETag` model. Use APQ over `GET` for CDN reach, subgraph response caching for interior fan-out, and normalised client caches for deduplication. When edge cacheability dominates, prefer REST.
- Subscriptions scale by connection count. Use `graphql-ws@5.14.1` and `PubSub` for true real-time, but default to polling or event streaming (Vol 10) unless sub-second push is required.

Further reading

- GraphQL Foundation. *GraphQL Specification* (October 2021). https://spec.graphql.org/October2021/ — the authoritative spec; §5 (Validation) and §6 (Execution) are essential.
- Apollo. *Apollo Federation 2.5 Documentation*. https://www.apollographql.com/docs/federation/ — supergraph composition, `@key` / `@shareable` semantics, and gateway query planning.
- Apollo. *Apollo Server 4.10 Documentation*. https://www.apollographql.com/docs/apollo-server/ — `@apollo/server@4.10.0`, APQ, plugins, and Fastify integration.
- Facebook / GraphQL. *DataLoader 2.2.2*. https://github.com/graphql/dataloader — batching and per-request memoisation; read the source — it is ~300 lines.
- Relay. *GraphQL Cursor Connections Specification*. https://relay.dev/graphql/connections.htm — the `edges { node, cursor } + pageInfo` contract used throughout this chapter.
- Hartig, O. & Pérez, J. *Semantics and Complexity of GraphQL* (WWW 2018) — formal analysis of GraphQL query complexity and why depth/complexity bounding is necessary.

- `graphql-eslint@3.20.1`, `@graphql-codegen/cli@5.0.2`, `rover@0.24.0` — versions pinned for lint, codegen, and supergraph composition in this chapter.

Chapter 5 — Versioning and Evolution

What this chapter covers. Every API that reaches production acquires consumers you cannot synchronously update — other teams, other regions, cached mobile binaries, and partner integrations that will not redeploy because you asked nicely. Versioning is the discipline that lets producers and consumers evolve independently without coordination, and evolution is the set of compatibility rules that determine whether a change is safe to ship without bumping a version. This chapter makes both precise: the three versioning strategies (path, header, content negotiation) and when each fits, Semantic Versioning applied to API contracts rather than packages, the expand-contract pattern that lets you change persisted state and wire schemas without downtime, and the deprecation-and-sunset machinery (RFC 8594, RFC 9745, `Sunset`/`Deprecation` headers, migration guides, dual-serve windows) that retires old behaviour safely. We build the discussion around real OpenAPI and Protobuf examples, a version-pinned `buf breaking` gate, and SQL migrations that run while three revisions of the service are serving traffic.

Learning goals — after this chapter you should be able to:

- Apply Semantic Versioning to API contracts (not just libraries), explain what MAJOR/MINOR/PATCH mean for OpenAPI and Protobuf surfaces, and choose the correct bump for a given change.
- Compare URI, header, and content-negotiation versioning on debuggability, cacheability, gateway routing, and consumer ergonomics, and select correctly per surface.
- Execute the expand-contract pattern for both wire schemas and storage schemas, including the intermediate dual-read/dual-write window that makes zero-downtime evolution possible.

- Design a deprecation and sunset lifecycle with `Deprecation` / `Sunset` headers, `410 Gone` versus `404`, traffic-based sunset criteria, and a migration guide that consumers can actually follow.
- Wire `buf breaking` and `oasdiff breaking` into CI so breaking changes fail the build unless the version bump and migration guide match policy.
- Explain versioning through a distributed-systems lens: rolling deploys with mixed versions, version-aware routing, and the blast radius of a breaking change when old clients are still in the wild.

Why APIs version — the distributed-systems reason

In a monolith, changing a function signature is a single commit: the compiler enumerates every call site and you fix them. In a distributed system, there is no global compiler. A service may have 40 consumers, three of which are outside your organisation, five of which cache a client binary on a mobile device you cannot force-upgrade, and two of which are batch jobs that deploy quarterly. Changing `status` from a string to an enum, or renaming `customer_id` to `customerId`, is not a rename — it is a deserialization failure in a process you did not build, running in a region you do not operate, at 03:00 when you are asleep.

Versioning is the answer to a single question: *how does a producer ship a change without requiring every consumer to ship at the same time?* The mechanisms differ, but the invariant is the same: at any moment, multiple versions of the contract are *in flight* — in different clients, different gateway routes, and different service revisions doing a rolling deploy.

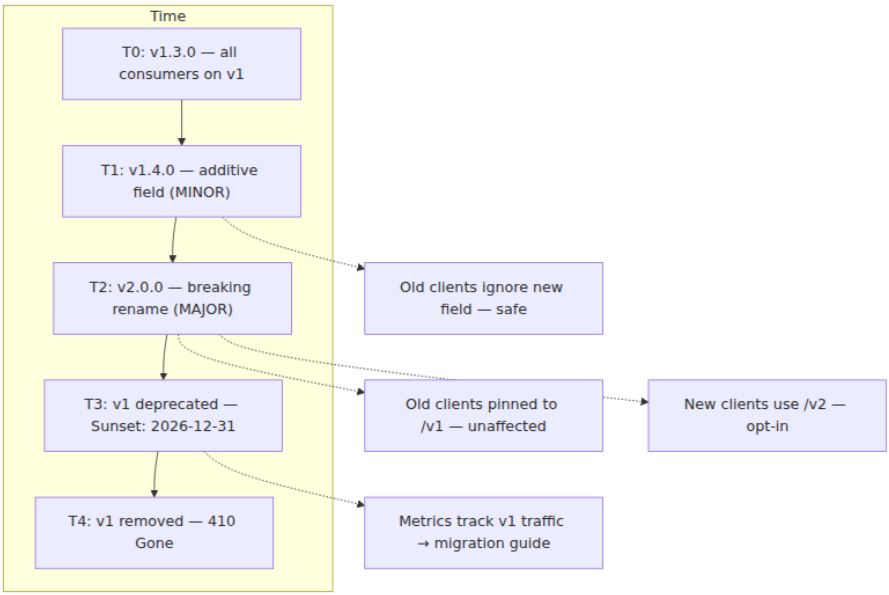


Figure 5-1: Version timeline. Additive changes ship as MINOR without coordination; breaking changes ship as MAJOR with a sunset window; old and new versions are served concurrently.

> **Boundary note.** This chapter covers *API contract* versioning and evolution — how URIs, headers, and schemas change safely. *Storage* versioning — how tables, indexes, and serialized payloads evolve without downtime — is covered in Volume 5, Chapters 4 (Query Optimization), 7 (WAL/Recovery), and 14 (Pooling/Migrations). Where this chapter shows a SQL migration, it is to illustrate the wire-storage coordination point, not to re-derive storage internals. The wire-format compatibility matrix for Protobuf field types is detailed in Volume 8, Chapter 8 (Compatibility and Wire Formats).

Semantic Versioning for contracts

Semantic Versioning (SemVer 2.0.0 — https://semver.org/) is often misapplied to APIs because it was written for packages. For a library, MAJOR means \"API-incompatible change to the package interface.\" For a service contract, the same words apply but the units differ: the version

is the *contract* version, not the service's deploy version. Service `orders` at deploy `2026.03.15` may still be serving `GET /v1/orders` (contract `v1`) and `GET /v2/orders` (contract `v2`) simultaneously.

What MAJOR, MINOR, PATCH mean for an API

Bump	OpenAPI example	Protobuf example	Consumer action
MAJOR (`1.0.0 → 2.0.0`)	Remove `GET /v1/orders`, rename `status` values, change `required` to add a new required field	Change field number, remove field without `reserved`, change wire type, new package `acme.orders.v2`	Must migrate — old contract still served until Sunset
MINOR (`1.4.0 → 1.5.0`)	Add `GET /v1/orders/{id}/shipments`, add optional `updated_at` to `Order`, add enum value `refunded`	Add field with new number, add enum value with handling for unknown	No action — old clients ignore the addition
PATCH (`1.4.0 → 1.4.1`)	Fix `description`, correct `example`, tighten `pattern` without changing valid values	Fix `deprecated` annotation, add `option deprecated = true`	No action

Rules that prevent the common mistakes:

- **Adding an optional field is MINOR** — but only if consumers treat unknown fields as ignorable (JSON) or unknown tags as skipped (Protobuf). If a consumer does strict `additionalProperties: false` validation on responses, your MINOR breaks them. Document that consumers must ignore unknown fields.
- **Adding a required field is MAJOR.** Existing clients that `POST /v1/orders` without the new field will get `400` after your deploy.
- **Changing validation (tightening) is MAJOR.** Adding `minLength: 1` to a field that previously accepted `""` rejects payloads that previously succeeded.

- **Deprecation is MINOR; removal is MAJOR.** Marking `legacyPriority` as `deprecated` warns consumers; removing it breaks them (see § Deprecation below).

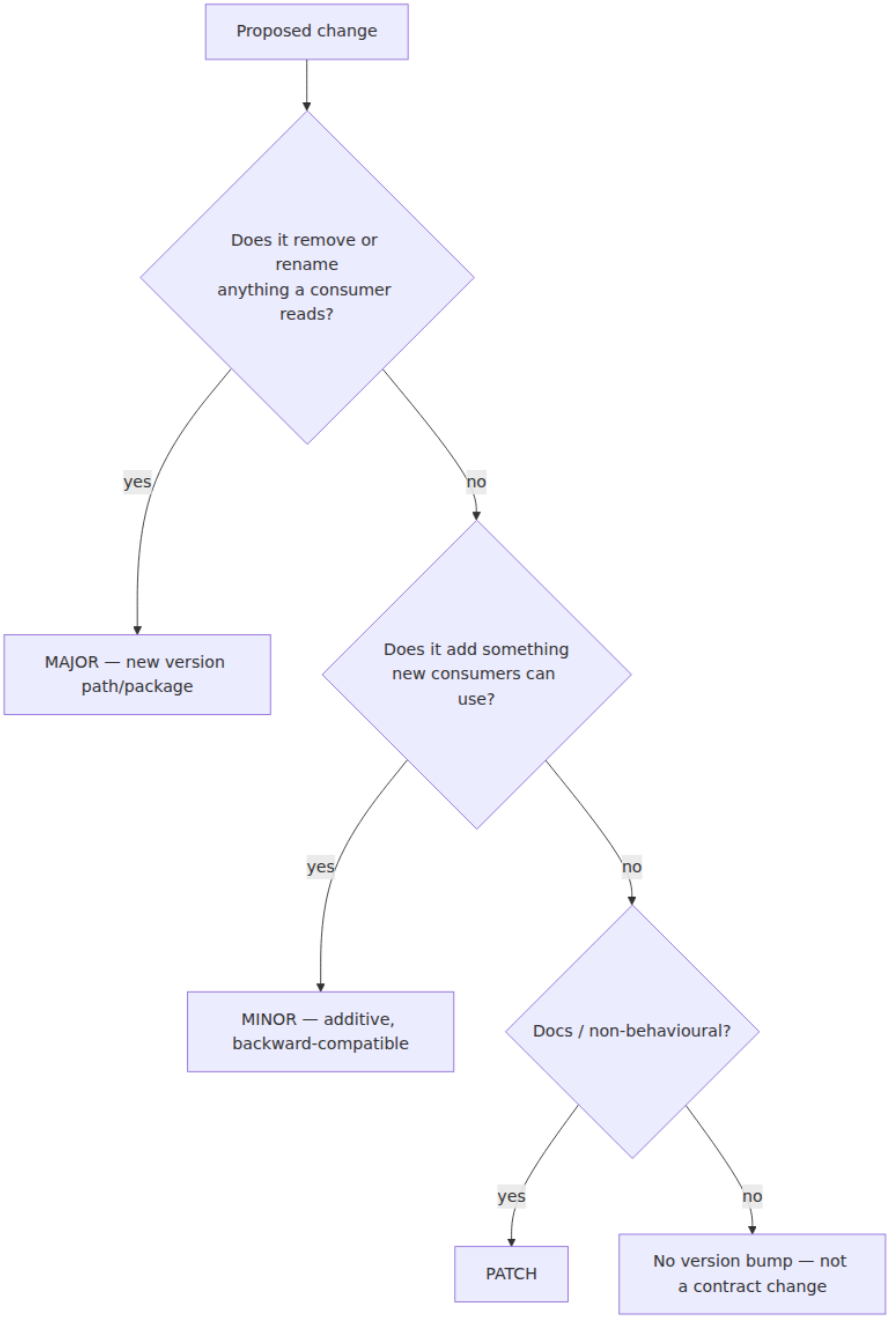

Proposed change
Does it remove or rename anything a consumer reads?
yes
no
MAJOR — new version path/package
Does it add something new consumers can use?
yes
no
MINOR — additive, backward-compatible
Docs / non-behavioural?
yes
no
PATCH
No version bump — not a contract change

Figure 5-2: SemVer decision tree for API contracts. \"Remove or rename\" includes tightening validation and making an optional field required.

Versioning the artifact, not just the URL

A disciplined setup versions the spec artifact alongside the route:

```
# Artifact layout — same pattern for OpenAPI and Protobuf
apis/
  orders/
    openapi/
      v1/openapi.yaml        # 1.4.0 — served at /v1/orders
      v2/openapi.yaml        # 2.0.0 — served at /v2/orders
    proto/
      acme/orders/v1/order.proto   # package acme.orders.v1
      acme/orders/v2/order.proto   # package acme.orders.v2 (when
needed)
```

Each version is tagged and published:

```
# Tag and publish — OpenAPI example
git tag apis/orders@v1.4.0 && git push origin apis/orders@v1.4.0
# Registry publish (choose one)
npx @redocly/cli@1.14.0 push apis/orders/openapi/v1/openapi.yaml --
branch main
# Buf Schema Registry (for Protobuf)
buf push --tag v1.4.0
```

The service binary may serve both versions from the same deploy — or v1 and v2 may be separate deployments behind a gateway that routes by path/header. Either way, the version is a property of the *contract,* not the binary.

Three strategies for carrying the version

There is no single correct place to put the version. The trade-off is debuggability and cacheability versus purity and evolvability, and the right answer depends on where the API sits.

1. URI versioning — `/v1/orders`, `/v2/orders`

The most common choice for REST at scale. The version is part of the resource path.

Pros: Visible in every log line and `curl` command; trivially routable at the gateway/CDN layer (`/v1/* → orders-v1`, `/v2/* → orders-v2`); cache keys are naturally versioned; `Host` + `path` is the cache identity — no `Vary` needed.

Cons: The version is not an HTTP-layer concern — it leaks API lifecycle into the resource identifier. Some REST purists consider it un-RESTful (Fielding's position was that versioning belongs in content negotiation). In practice, this objection has not prevented its adoption by AWS, Stripe, Google, and GitHub.

```yaml
# openapi.yaml — URI versioning
servers:
  - url: https://api.example.com/v1
    description: Orders v1
  - url: https://api.example.com/v2
    description: Orders v2 (MAJOR — breaking)
paths:
  /orders: { get: { operationId: listOrders, ... } }  # effective:
GET /v1/orders or /v2/orders
```

Gateway routing (Envoy / NGINX example — any L7 proxy applies):

```yaml
# envoy.yaml fragment — Envoy 1.30.1
routes:
  - match: { prefix: "/v1/orders" }
    route: { cluster: orders-v1, prefix_rewrite: "/orders" }
  - match: { prefix: "/v2/orders" }
    route: { cluster: orders-v2, prefix_rewrite: "/orders" }
```

2. Header versioning — `Accept: application/vnd.example.v1+json`

The version rides in a media-type header. The URI stays stable (`GET /orders`).

Pros: More correct per HTTP semantics — versioning is a representation concern, not a resource identity concern. One URI identifies the resource; `Accept` selects the representation.

Cons: Harder to debug (`curl -H "Accept: ..."` is less obvious than a path), harder to cache (requires `Vary: Accept`), harder to route at L7 without header-aware rules, and not visible in naive access logs that only record the path.

```
GET /orders HTTP/1.1
Host: api.example.com
Accept: application/vnd.example.orders.v1+json
```

```
// Express 4.18.2 — header-versioned dispatch (when you must)
app.get("/orders", (req, res) => {
  const accept = req.headers.accept ?? "";
  if (accept.includes("vnd.example.orders.v2")) return handleV2(req, re
s);
  if (accept.includes("vnd.example.orders.v1")) return handleV1(req, re
s);
  return res.status(406).json({ code: "NOT_ACCEPTABLE", message: "Unkno
wn version" });
});
```

3. Content negotiation via `Accept` with profile — `Accept: application/json; profile="v2"`

A lighter variant of header versioning using a parameter rather than a vendor media type. Rare in practice; mentioned for completeness because some standards (e.g., `application/merge-patch+json` — RFC 7396) use it.

Recommendation:

- **External/partner REST:** URI versioning (`/v1/...`). It wins on debuggability and cacheability, which dominate at the edge.
- **Interior gRPC/Protobuf:** Package versioning (`acme.orders.v1`, `acme.orders.v2`) — the `.proto` package *is* the version; no HTTP path needed.
- **GraphQL (Ch 4):** No versioned URL. Evolution is additive with `@deprecated` and field-level deprecation; breaking changes use new fields/types rather than a new endpoint. Chapter 4 covers this — GraphQL's graph is versioned by *capability*, not by *path*.
- **Header versioning:** Only when a single stable URI is a hard requirement (e.g., a resource that is also a web page whose URL must not change). Otherwise, prefer URI for REST.

> **Distributed-systems lens.** Whatever strategy you pick, the gateway must be able to route multiple versions concurrently. During a migration window, `orders-v1` and `orders-v2` may be separate deployments with separate capacity. The gateway's route table is the version registry at

> runtime; treat it as a versioned artifact (checked in, reviewed, deployed via CI) rather than a manual console edit.

Evolution without downtime — the expand-contract pattern

Most breaking changes are not wire-only. Renaming `customer_id` to `customerId` in the API touches the JSON field, the Protobuf tag, the database column, the search index, and every consumer that reads the field. Doing all of that in one deploy requires coordination — exactly what versioning is meant to avoid.

The **expand-contract** pattern (also called parallel change) eliminates coordination by splitting a breaking change into three backward-compatible deploys:

1. **Expand** — add the new representation alongside the old; producers write both, consumers can read either.
2. **Migrate** — move all producers and consumers to the new representation; backfill stored data.
3. **Contract** — remove the old representation.

At no point is there a state where a writer has stopped writing the old field but a reader still needs it.

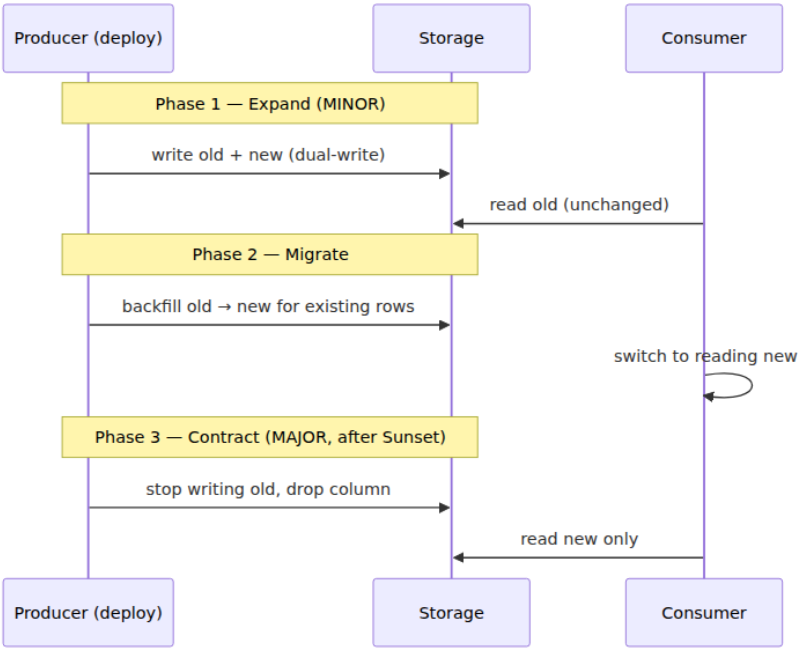


Figure 5-3: Expand-contract. Each phase is independently deployable and rollback-safe; no phase requires producer and consumer to deploy simultaneously.

Example 1 — Renaming a REST/JSON field

Goal: Rename `customer_id` (snake_case) to `customerId` (camelCase) in `POST /v1/orders` and `GET /v1/orders/{id}`.

Phase 1 — Expand (v1.5.0, MINOR): Accept and return *both* fields. Writers populate both; readers may use either.

```
# openapi.yaml — v1.5.0 (expand)
components:
  schemas:
    Order:
      type: object
      required: [id, status, created_at]
      # Neither customer field is required during expand — allows
either/both
      properties:
        customer_id:  { type: string, pattern: '^[0-9A-HJKMNP-TV-Z]
{26}$', deprecated: true }
        customerId:   { type: string, pattern: '^[0-9A-HJKMNP-TV-Z]
{26}$' }
        status: { type: string, enum: [pending, paid, shipped, cancelle
d] }
        created_at: { type: string, format: date-time }
```

```
// handler.js — Phase 1: dual-write, dual-read (Node 20.11.0, Express
4.18.2)
// Accept either field on input; return both on output.
function normalizeCustomerId(body) {
  const id = body.customerId ?? body.customer_id;
  if (!id) throw Object.assign(new Error("customerId required"), { stat
us: 400 });
  return id;
}
app.post("/v1/orders", async (req, res) => {
  const customerId = normalizeCustomerId(req.body);
  const order = await db.createOrder({ customer_id: customerId, custome
rId, ...req.body });
  // Return both for backward compatibility
  res.status(201).json({ ...order, customer_id: order.customerId, custo
merId: order.customerId });
});
```

Phase 2 — Migrate: Update all consumers to send and read `customerId`. Backfill is trivial here (no stored JSON column to rewrite — but see storage example below). Monitor traffic: when `customer_id` usage drops below threshold (e.g., <1% of requests over 7 days), proceed.

Phase 3 — Contract (v2.0.0, MAJOR): Remove `customer_id`. Return `400` with a migration hint if it is sent.

```yaml
# openapi.yaml — v2.0.0 (contract)
components:
  schemas:
    Order:
      type: object
      required: [id, customerId, status, created_at]
      additionalProperties: false
      properties:
        customerId: { type: string, pattern: '^[0-9A-HJKMNP-TV-Z]
{26}$' }
        # customer_id removed — sending it now fails validation
```

Example 2 — Renaming a database column (storage expand-contract)

This is the case most teams get wrong, because the wire and storage evolutions are often attempted in one PR.

Goal: Rename `orders.customer_id` → `orders.customer_id_v2` or simply change its semantics (e.g., from `TEXT` to `ULID`-validated `CHAR(26)`).

```
-- Phase 1 — Expand (deploy 1): add new column, dual-write, backfill
-- Postgres 16.2
ALTER TABLE orders ADD COLUMN customer_id_v2 CHAR(26);
-- Backfill in batches (avoid long exclusive lock) — pg 16+ with
CONCURRENTLY friendly
-- Application now writes both columns on every INSERT/UPDATE
UPDATE orders SET customer_id_v2 = customer_id
  WHERE customer_id_v2 IS NULL AND id IN (
    SELECT id FROM orders WHERE customer_id_v2 IS NULL LIMIT 10000
  );
-- Repeat until 0 rows remain; or use a background job

-- Dual-read in application: COALESCE(customer_id_v2, customer_id)
-- CREATE INDEX CONCURRENTLY on new column before switching reads to it
CREATE INDEX CONCURRENTLY idx_orders_customer_v2 ON orders (customer_id
_v2);

-- Phase 2 — Migrate: switch all reads to customer_id_v2
-- Application now reads customer_id_v2 exclusively

-- Phase 3 — Contract (after sunset window): drop old column
ALTER TABLE orders DROP COLUMN customer_id;
-- If column rename is desired, do it as a separate step after drop:
-- ALTER TABLE orders RENAME COLUMN customer_id_v2 TO customer_id;
-- (requires brief lock — schedule during low traffic or use pg_repack)

-- For Protobuf-backed storage (serialized protos in a column), the
same
-- expand-contract applies: write both field numbers, backfill by re-
serializing,
-- then reserve the old number (see Ch 3 & Ch 8).
```

> **Boundary note.** The indexing, locking, and WAL implications of `ADD COLUMN`, `CREATE INDEX CONCURRENTLY`, and batched `UPDATE` are the subject of Volume 5, Chapters 3 (Indexing), 5 (Transactions), and 7 (WAL/Recovery). The pattern here is the *coordination* — two columns coexisting so rolling deploys never see a missing column — not the storage engine's internal handling of the DDL.

Example 3 — Protobuf field evolution

Chapter 3 covered the wire rules; here is the expand-contract for a field that changes type (e.g., `string priority` → `enum Priority`):

```
// v1 — original
message Order {
  string priority = 11; // "low" | "medium" | "high" — stringly typed
}

// Phase 1 — Expand (MINOR): add new enum field, keep old string
message Order {
  string priority = 11 [deprecated = true]; // still written
  Priority priority_v2 = 12;                // new — producers write
both
  enum Priority {
    PRIORITY_UNSPECIFIED = 0;
    PRIORITY_LOW = 1;
    PRIORITY_MEDIUM = 2;
    PRIORITY_HIGH = 3;
  }
  reserved 13; // guard against accidental reuse
}

// Phase 3 — Contract (MAJOR, package v2): remove old string
// acme/orders/v2/order.proto
message Order {
  Priority priority = 12; // promoted to canonical number in v2 (or
keep 12)
  enum Priority { ... }
  reserved 11; reserved "priority";
}
```

Deprecation and sunset — retiring old behaviour

A breaking change without a deprecation period is a coordination failure. Consumers need three things: notice, time, and a migration path.

The lifecycle

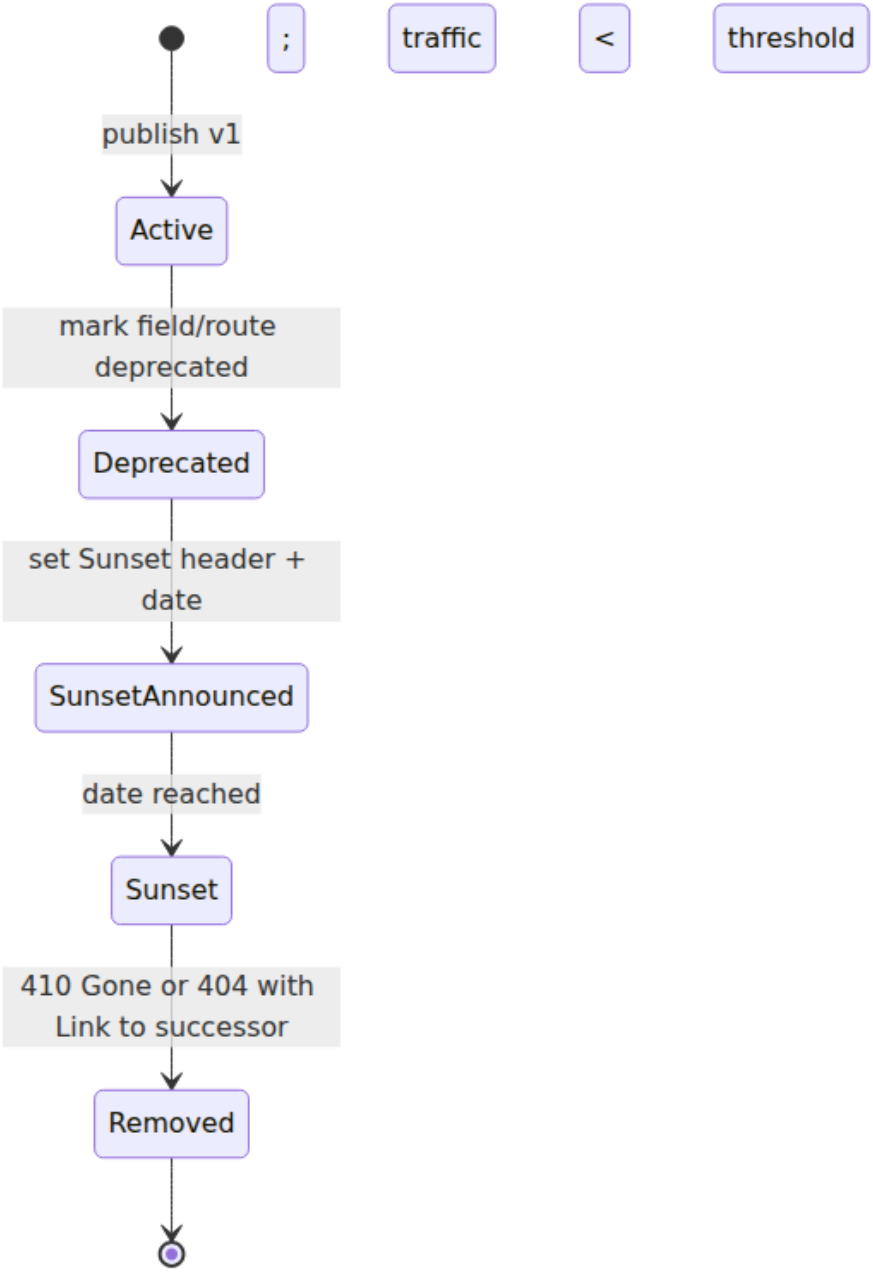

;
traffic
<
threshold
publish v1
Active
mark field/route deprecated
Deprecated
set Sunset header + date
SunsetAnnounced
date reached
Sunset
410 Gone or 404 with Link to successor
Removed

Figure 5-4: Deprecation lifecycle. Each transition is gated on traffic metrics and consumer notification, not on a calendar alone.

Headers — RFC 8594 (Sunset) and RFC 9745 (Deprecation)

Two standard headers signal lifecycle state on every response that serves deprecated behaviour:

```
HTTP/1.1 200 OK
Deprecation: true
Sunset: Sat, 31 Dec 2026 23:59:59 GMT
Sunset: Sat, 31 Dec 2026 23:59:59 GMT
Link: <https://docs.example.com/migration/v1-to-v2>; rel="successor-
version"
Link: <https://docs.example.com/migration/v1-to-v2>; rel="deprecation"
Deprecation: version="v1"
Content-Type: application/json
```

And the companion request header that lets consumers declare their intent:

```
GET /v1/orders HTTP/1.1
Sunset: Sat, 31 Dec 2026 23:59:59 GMT
```

In practice, most APIs emit `Deprecation` + `Sunset` on the response and consumers read them via middleware or gateway logs.

```
// Express 4.18.2 — deprecation middleware
function deprecationMiddleware(req, res, next) {
  if (req.path.startsWith("/v1/")) {
    res.setHeader("Deprecation", 'version="v1"');
    res.setHeader("Sunset", "Sat, 31 Dec 2026 23:59:59 GMT");
    res.setHeader("Link", '<https://docs.example.com/migration/v1-to-
v2>; rel="successor-version"');
    // Optional: emit metric for traffic tracking
    req.app.locals.metrics.increment("api.deprecated.v1.hits", {
route: req.path });
  }
  next();
}
app.use(deprecationMiddleware);
```

For Protobuf/gRPC, deprecation is at the schema level:

```
// Deprecated field — still served, but codegen emits warnings
string legacy_priority = 11 [deprecated = true];
service OrderService {
  rpc LegacySearch(LegacySearchRequest) returns (LegacySearchResponse)
{
    option deprecated = true;
  }
}
```

Removal: `410 Gone` versus `404 Not Found`

When the sunset date passes and traffic has dropped below threshold, stop serving the old version. The correct status is `410 Gone` (the resource *was* here and was intentionally removed) rather than `404` (unknown), with a `Link` to the successor:

```
HTTP/1.1 410 Gone
Link: <https://api.example.com/v2/orders>; rel="successor-version"
Content-Type: application/problem+json

{
  "type": "https://docs.example.com/errors/sunset",
  "title": "API version v1 has been sunset",
  "detail": "v1 was sunset on 2026-12-31. Migrate to v2: https://
docs.example.com/migration/v1-to-v2",
  "code": "API_VERSION_SUNSET"
}
```

Sunset criteria — not just a date

A date alone is insufficient. Require *both*:

- **Time threshold:** Sunset date has passed (e.g., 6 months after deprecation for internal surfaces, 12+ months for external).
- **Traffic threshold:** Remaining traffic on the deprecated version is below an agreed level (e.g., <0.5% of total requests over the trailing 7 days, or explicitly waived per consumer).

Track this with a dashboard that breaks down deprecated-version traffic by consumer (API key, `User-Agent`, or `X-Client-Version`). Sunset without knowing *who* still calls the old version is an outage plan, not a deprecation plan.

Enforcement in CI — making the rules mechanical

Versioning policy that lives in a wiki is not policy. Wire it into CI so breaking changes cannot merge without the correct version bump and migration guide.

OpenAPI — `oasdiff` 1.9.2

```
# Compare against published version — fail on breaking without MAJOR
oasdiff breaking https://registry.example.com/apis/orders@v1.4.0 openap
i.yaml --fail-on ERR

# In CI (GitHub Actions — actions/checkout@v4, oasdiff 1.9.2)
# .github/workflows/api-contract.yaml
name: api-contract
on: { pull_request: { paths: ["apis/orders/**"] } }
jobs:
  breaking:
    runs-on: ubuntu-22.04
    steps:
      - uses: actions/checkout@v4
      - uses: oasdiff/oasdiff-action@v1 # or install binary
        with:
          base: https://registry.example.com/apis/orders@v1.4.0
          revision: apis/orders/openapi/v1/openapi.yaml
          fail-on: ERR
```

Protobuf — `buf breaking` (buf 1.40.2)

```
# buf.yaml — buf 1.40.2
version: v2
lint:
  use: [DEFAULT]
  except: [PACKAGE_DIRECTORY_MATCH]
breaking:
  use: [FILE]  # FILE level — catches field removal, type changes,
number reuse
```

```
# Compare against main — fail if breaking
buf breaking --against '.git#branch=main'
# Or against a published module
buf breaking --against 'buf.build/acme/orders:v1.4.0'

# CI gate
buf lint && buf breaking --against '.git#branch=main' && buf format --
exit-code
```

Version-bump check

```
# Ensure the version in openapi.yaml / buf.yaml matches the expected
bump
# scripts/check-version-bump.sh — bash 5.2
BASE_VERSION=$(oasdiff version https://registry.example.com/apis/
orders@v1.4.0 | jq -r .version)
NEW_VERSION=$(yq '.info.version' apis/orders/openapi/v1/openapi.yaml)
# Simple semver compare — require MINOR for additive, MAJOR for
breaking
# (use semver CLI: npm i -g semver@7.6.0)
if oasdiff breaking --fail-on ERR ...; then
  semver --range ">${BASE_VERSION}" "${NEW_VERSION}" || exit 1
else
  # breaking detected — require MAJOR
  semver --range ">=${BASE_VERSION%%.*}.0.0" "${NEW_VERSION}" # ...
check major bump
fi
```

Distributed-systems lens — version skew in the wild

Three realities make versioning a runtime concern, not just a design-time one.

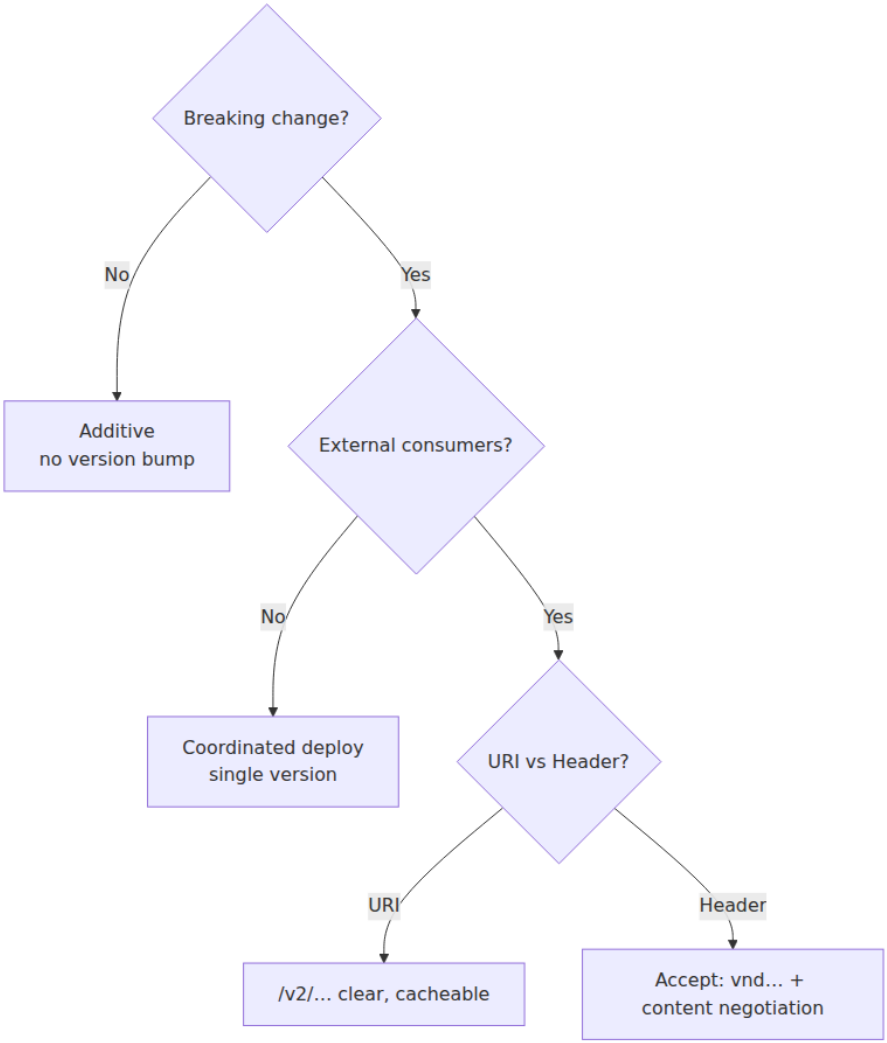
Rolling deploys create mixed-version windows. When you deploy `orders` v1.5.0 (expand phase) as a rolling update, for ~5 minutes some pods serve v1.4.0 and some serve v1.5.0. If phase 1 writes a new column that old pods do not know about, the old pods must not crash on the column's presence. The expand step guarantees this (old code ignores the new column/field). Contract-phase deploys require the opposite: old consumers must have already migrated before you remove what they read — hence the traffic-based sunset gate.

Gateways and clients cache version decisions. A mobile client that cached `GET /v1/orders` with `Cache-Control: max-age=3600` will still call `/v1/` after you sunset it, until the cache expires or the binary updates. CDNs and service-mesh sidecars add more cache layers. The `Sunset` header must be honoured by intermediaries, and the gateway should continue serving deprecated versions with deprecation headers until traffic truly drains — not merely until the calendar says so.

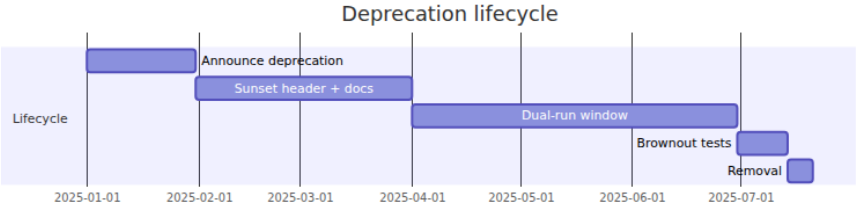
Version-aware routing is infrastructure. At scale, `/v1/` and `/v2/` may be different capacity pools. The gateway's route table must be

versioned and deployed atomically with the contract. A canary that routes 1% of `/v2/` traffic to a new `orders-v2` deployment while 99% stays on `orders-v1` is a standard technique for validating a MAJOR before full cutover — the same pattern as deployment strategies in Volume 11, Chapter 9, applied to API versions.

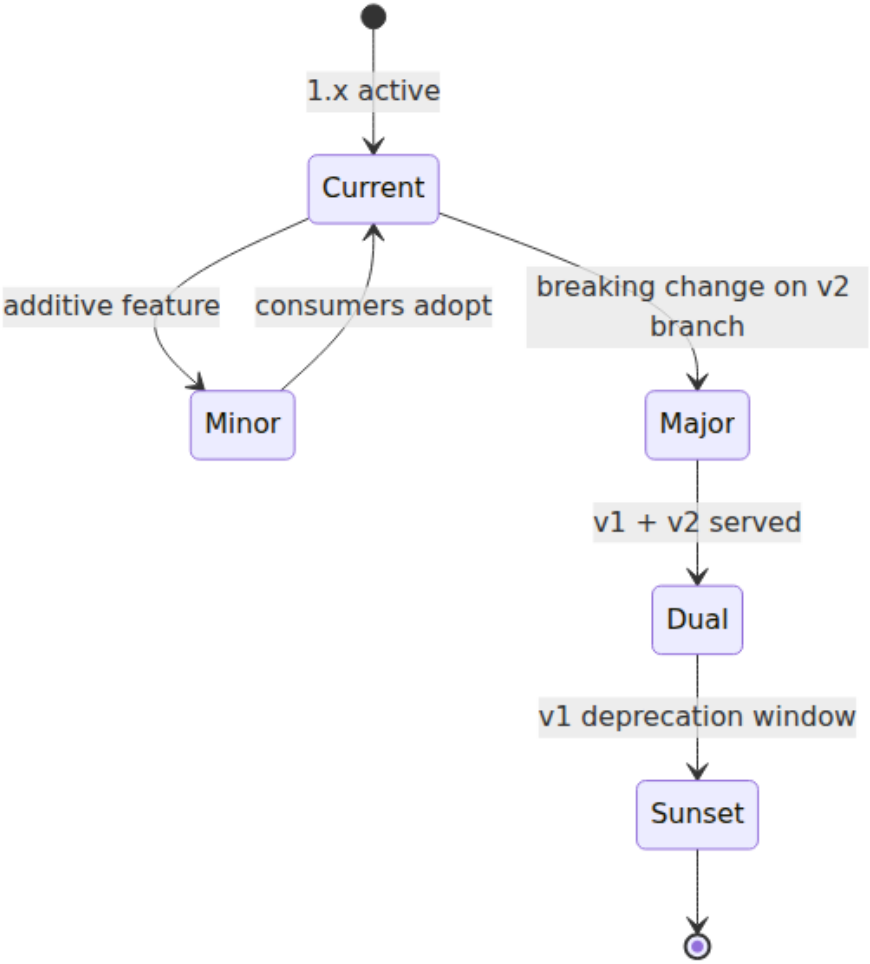
Versioning Strategy Decision



Deprecation Timeline



SemVer State Transitions



Key takeaways

- SemVer for contracts: MAJOR = breaking (remove/rename/tighten), MINOR = additive (new optional field/endpoint), PATCH = non-behavioural. Adding an optional field is MINOR only if consumers ignore unknown fields — document that requirement.
- URI versioning (`/v1/...`) wins for external REST on debuggability and cacheability; package versioning (`acme.orders.v2`) is the

Protobuf/gRPC equivalent; GraphQL evolves additively with `@deprecated` rather than versioned paths.

- Expand-contract is the only safe way to make breaking changes without coordination: expand (dual-write/both fields served, MINOR) → migrate (backfill + consumers switch) → contract (remove old, MAJOR after Sunset). Each phase is independently deployable and rollback-safe.
- Deprecation uses `Deprecation` (RFC 9745) + `Sunset` (RFC 8594) + `Link: rel="successor-version"` on every deprecated response. Sunset requires both a date and a traffic threshold (e.g., <0.5% over 7 days broken down by consumer). Removal returns `410 Gone`, not `404`.
- Enforce mechanically: `oasdiff breaking --fail-on ERR` for OpenAPI and `buf breaking --against main` for Protobuf in CI, plus a version-bump check that requires MAJOR when breaking is detected. Policy in the wiki is not policy; policy in CI is.

Further reading

- Preston-Werner, T. *Semantic Versioning 2.0.0*. https://semver.org/ — the MAJOR/MINOR/PATCH definitions applied here to contracts.
- IETF. *RFC 8594 — The Sunset HTTP Header Field* (2019) and *RFC 9745 — The Deprecation HTTP Header Field* (2024) — standard headers for deprecation and sunset signalling.
- IETF. *RFC 9457 — Problem Details for HTTP APIs* (2023) — the `application/problem+json` envelope used for `410 Gone` responses.
- Google. *API Improvement Proposals (AIPs)* — https://aip.dev/ — AIP-122 (resource names), AIP-136 (custom methods), AIP-149 (versioning), AIP-151 (long-running operations).
- `buf` 1.40.2 — https://buf.build/docs/breaking — FILE vs PACKAGE vs WIRE breaking checks for Protobuf.
- `oasdiff` 1.9.2 — https://github.com/Tufin/oasdiff — breaking-change detection for OpenAPI 3.x.
- Fowler, M. *Parallel Change* (https://martinfowler.com/bliki/ParallelChange.html) — the expand-contract pattern's original formulation.

- Newman, S. *Building Microservices* 2nd ed. (O'Reilly, 2021), Ch. 4 — versioning trade-offs and consumer-driven contracts.

Chapter 6 — Idempotency, Pagination, and Filtering

What this chapter covers. Three concerns that every production list or write endpoint must get right — and that most teams get subtly wrong until the bug surfaces as duplicate charges, missing rows during pagination, or a filter that table-scans 100 million rows. Idempotency makes retries safe, which makes *every* network call safe to retry. Pagination makes listing bounded, which makes every collection operable as it grows. Filtering, sorting, and sparse fieldsets make listing *useful*, which is the difference between an API consumers can build on and one they work around by fetching everything and filtering client-side. This chapter treats all three as infrastructure: the distributed-systems reason each matters, the contract each presents to consumers, and the production implementation that honours it under concurrency, retries, and rolling deploys. Every pattern is anchored in runnable, version-pinned code — an `Idempotency-Key` handler with Postgres and Redis, a cursor-paginated list with keyset queries and opaque tokens, and a filter/sort pipeline with validation and index awareness.

Learning goals — after this chapter you should be able to:

- Define idempotency precisely per RFC 7231, distinguish idempotent *methods* from idempotent *operations*, and decide when a `POST` must be made idempotent via `Idempotency-Key`.
- Implement a correct `Idempotency-Key` handler that is safe under concurrent duplicate requests, distinguishes replay-with-same-payload from conflict-with-different-payload (`409` / `422`), and expires keys on a bounded TTL.
- Implement cursor (keyset) pagination with opaque tokens and a deterministic tie-breaker, explain why offset pagination breaks under concurrent mutation, and return `has_more` without a separate `COUNT(*)`.

- Design a filter/sort/field-selection contract that is parseable, validatable, and index-aware, and reject or bound queries that would table-scan.
- Explain all three through a distributed-systems lens: at-least-once delivery, retry storms, gateway fan-out, and the storage contract pagination depends on.

Idempotency — making retries safe

Why idempotency is non-negotiable

Every request between two distributed processes can fail in a way where the sender does not know whether the receiver acted. The classic case:

```
Client → POST /v1/orders { customer_id, items }  →  Server
         ← timeout (no response)
Did the order get created? Retry or not?
```

Without idempotency, retrying risks duplicate side effects (two orders, two charges). Not retrying risks data loss (order never created). The only general solution is to make the operation *retry-safe* — sending the same request twice has the same effect as sending it once.

> **Distributed-systems lens.** The network provides at-least-once delivery unless you add exactly-once *semantics* on top — and exactly-once *delivery* is impossible in the presence of failures (see Vol 6, Ch 9 — Idempotency, Deduplication, and Exactly-Once, and Vol 10, Ch 2 — Delivery Semantics). Idempotency keys give you exactly-once *effect* per key: the first write wins, replays return the original result, and the system is correct even though the network retried. This is the same principle as the outbox pattern (Vol 10, Ch 6) and deduplication at the consumer — the edge that makes retries a reliability tool instead of a correctness hazard.

HTTP method idempotency versus operation idempotency

RFC 7231 §4.2.2 defines an idempotent method as one where `N > 0` identical requests have the same *effect* as a single request:

Method	Idempotent per RFC 7231?	Safe?	What it means
`GET`, `HEAD`	yes	yes	No side effects; retry freely
`PUT`, `DELETE`	yes	no	Side effect, but repeating is safe (put same bytes, delete same resource)
`POST`, `PATCH`	no	no	Repeating may create duplicates or apply partial updates twice

For `PUT` / `DELETE`, idempotency is inherent in the method: `PUT /orders/{id}` with the same body twice writes the same state; `DELETE /orders/{id}` twice leaves the resource absent. For `POST` (create) and sometimes `PATCH`, the *operation* must be made idempotent explicitly — that is what `Idempotency-Key` does.

The `Idempotency-Key` contract

The pattern, popularised by Stripe and now the de facto standard:

1. **Client generates** a unique key per *intent* (UUID v4, ULID, or hash of the operation) and sends it in `Idempotency-Key: <key>`.
2. **Server stores** the key with the request fingerprint (hash of method + path + body) and the response, scoped to the authenticated principal.
3. **Replay with same key + same fingerprint** → return the stored response (`200` or `201`, not an error).
4. **Replay with same key + different fingerprint** → `422 Unprocessable Entity` or `409 Conflict` — the client reused a key for a different operation, which is a bug.
5. **Key expires** after a TTL (24 hours is conventional; Stripe uses 24h). After expiry, the same key is treated as new.

Critically, the key must be **scoped to the caller** (API key, user, tenant). Two different users sending the same UUID must not collide. And the server must handle **concurrent duplicate requests** — two identical `POST`s arriving within milliseconds, before either has stored a result.

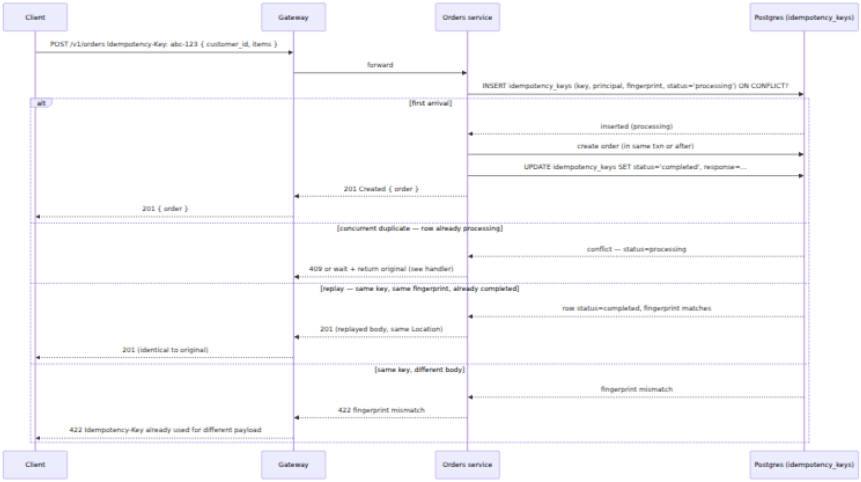


Figure 6-1: Idempotency-Key lifecycle. The `processing` state is the concurrency guard; the fingerprint check prevents key reuse for a different operation; TTL bounds storage.

Real code — Idempotency-Key handler (Node 20.11.0, Express 4.18.2, pg 8.11.3, Redis 7.2 / ioredis 5.3.2)

This handler is production-shaped: concurrent-safe via `INSERT ... ON CONFLICT`, fingerprint-validated, TTL-bounded, and scoped to the authenticated principal. Two storage options are shown — Postgres (durable, transactional with the domain write) and Redis (lower latency, suitable when the idempotency store is separate from the domain store).

Postgres DDL — the idempotency table:

```
-- Postgres 16.2 — idempotency_keys (one row per key per principal)
CREATE TABLE idempotency_keys (
  key          TEXT        NOT NULL,
  principal    TEXT        NOT NULL,  -- api_key_id or user_id — scopes
the key
  fingerprint TEXT        NOT NULL,  -- sha256(method + path + body)
  status       TEXT        NOT NULL CHECK (status IN ('processing','comp
leted','failed')),
  response_status INT,
  response_body JSONB,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at   TIMESTAMPTZ NOT NULL DEFAULT now() + INTERVAL '24 hours',
  PRIMARY KEY (principal, key)
);
CREATE INDEX idx_idempotency_expires ON idempotency_keys (expires_at);
-- Expire via periodic DELETE or pg_cron; alternatively use pg_partman
TTL partition
```

Express middleware — Postgres-backed (transactional with the domain write):

```
// idempotency.js — Node 20.11.0, Express 4.18.2, pg@8.11.3
import crypto from "crypto";
import { pool } from "./db.js";

function fingerprintOf(req) {
  // Hash method + path + raw body — canonical, order-independent for
JSON
  const body = typeof req.body === "string" ? req.body :
JSON.stringify(req.body ?? {});
  return crypto.createHash("sha256").update(`${req.method}:${req.path}:
${body}`).digest("hex");
}

/**
 * Idempotency middleware for POST/PATCH that require Idempotency-Key.
 * Mount before the route handler: app.post("/v1/orders", idempotency,
createOrder)
 */
export async function idempotency(req, res, next) {
  const key = req.headers["idempotency-key"];
  if (!key) {
    return res.status(400).json({
      code: "IDEMPOTENCY_KEY_REQUIRED",
      message: "Idempotency-Key header is required for this operation",
      request_id: req.id,
    });
  }
  if (typeof key !== "string" || key.length === 0 || key.length > 64) {
    return res.status(400).json({ code: "BAD_REQUEST", message: "Idempo
tency-Key must be 1..64 chars" });
  }

  const principal = req.auth?.apiKeyId ?? req.auth?.userId ?? "anonymou
s";
  const fp = fingerprintOf(req);

  // Try to claim the key — INSERT with ON CONFLICT handles concurrency
  const claimed = await pool.query(

`INSERT INTO idempotency_keys (key, principal, fingerprint, status,
expires_at)
     VALUES ($1, $2, $3, 'processing', now() + INTERVAL '24 hours')
     ON CONFLICT (principal, key) DO NOTHING
     RETURNING (xmax = 0) AS inserted`,
    [key, principal, fp],
  );

  if (claimed.rows.length === 0) {
    // Key already exists — fetch it
    const { rows } = await pool.query(
```

```
      `SELECT fingerprint, status, response_status, response_body FROM
idempotency_keys
       WHERE principal = $1 AND key = $2`,
      [principal, key],
    );
    const row = rows[0];
    if (!row) return next(); // race — retry insert path (or return
500)

    if (row.fingerprint !== fp) {
      return res.status(422).json({
        code: "IDEMPOTENCY_KEY_REUSE",
        message: "Idempotency-Key already used for a different request
payload",
        request_id: req.id,
      });
    }
    if (row.status === "processing") {
      // Concurrent duplicate still in flight — 409 with Retry-After
      // Alternative: poll/wait for completion (shown in Redis variant
below)
      res.setHeader("Retry-After", "1");
      return res.status(409).json({
        code: "CONFLICT_IN_PROGRESS",
        message: "A request with this Idempotency-Key is already being
processed",
        request_id: req.id,
      });
    }
    if (row.status === "completed") {
      // Replay — return original response identically
      return res.status(row.response_status).json(row.response_body);
    }
    if (row.status === "failed") {
      // Previous attempt failed server-side — allow retry by resetting
to processing
      await pool.query(
        `UPDATE idempotency_keys SET fingerprint=$3,
status='processing', response_status=NULL, response_body=NULL
         WHERE principal=$1 AND key=$2 AND status='failed'`,
        [principal, key, fp],
      );
      // fall through to handler
    }
  }

  // Claim succeeded (or reset from failed) — wrap res.json to capture
response for storage
  const originalJson = res.json.bind(res);
  let capturedStatus, capturedBody;
  res.json = (body) => {
```

```
    capturedStatus = res.statusCode;
    capturedBody = body;
    return originalJson(body);
  };

  // After handler finishes, persist the result. Use 'finish' so we
capture even on errors.
  res.on("finish", async () => {
    try {
      // Only store definitive outcomes — 2xx/4xx are terminal; 5xx ->
mark failed so client can retry with same key
      const terminal = capturedStatus >= 200 && capturedStatus < 500;
      await pool.query(
        `UPDATE idempotency_keys
         SET status = $3, response_status = $4, response_body =
$5::jsonb
         WHERE principal = $1 AND key = $2 AND status = 'processing'`,
        [principal, key, terminal ? "completed" : "failed", capturedSta
tus, JSON.stringify(capturedBody ?? {})],
      );
    } catch (e) {
      // Log but do not fail the response — it was already sent
      console.error("idempotency persist failed", e);
    }
  });

  next();
}
```

Redis variant — lower latency, with wait-for-completion on concurrent duplicate:

```
// idempotency-redis.js — ioredis@5.3.2, Redis 7.2
import Redis from "ioredis";
import crypto from "crypto";

const redis = new Redis(process.env.REDIS_URL); // redis://
localhost:6379
const TTL_SECONDS = 24 * 3600;

function fingerprintOf(req) {
  const body = typeof req.body === "string" ? req.body :
JSON.stringify(req.body ?? {});
  return crypto.createHash("sha256").update(`${req.method}:${req.path}:
${body}`).digest("hex");
}

export async function idempotencyRedis(req, res, next) {
  const key = req.headers["idempotency-key"];
  if (!key) return res.status(400).json({ code: "IDEMPOTENCY_KEY_REQUIR
ED", message: "Idempotency-Key required" });

  const principal = req.auth?.apiKeyId ?? "anonymous";
  const fp = fingerprintOf(req);
  const redisKey = `idem:${principal}:${key}`;

  // SET NX — atomic claim
  const claimed = await redis.set(redisKey, JSON.stringify({ status: "p
rocessing", fingerprint: fp }), "EX", TTL_SECONDS, "NX");
  if (claimed === null) {
    // Already exists — fetch
    const raw = await redis.get(redisKey);
    if (!raw) return next();
    const row = JSON.parse(raw);
    if (row.fingerprint !== fp) {
      return res.status(422).json({ code: "IDEMPOTENCY_KEY_REUSE", mess
age: "Key already used for different payload" });
    }
    if (row.status === "processing") {
      // Wait briefly for the in-flight request to finish (up to 5s),
then return its result
      for (let i = 0; i < 10; i++) {
        await new Promise((r) => setTimeout(r, 500));
        const polled = await redis.get(redisKey);
        if (!polled) break;
        const p = JSON.parse(polled);
        if (p.status === "completed") return res.status(p.response_stat
us).json(p.response_body);
        if (p.status === "failed") break; // fall through to retry
      }
      res.setHeader("Retry-After", "1");
      return res.status(409).json({ code: "CONFLICT_IN_PROGRESS", messa
```

```
ge: "Request with this key is still processing" });
    }
    if (row.status === "completed") return res.status(row.response_stat
us).json(row.response_body);
  }

  // Captured response — same pattern as Postgres variant
  const originalJson = res.json.bind(res);
  let capturedStatus, capturedBody;
  res.json = (body) => { capturedStatus = res.statusCode; capturedBody
= body; return originalJson(body); };
  res.on("finish", async () => {
    const terminal = capturedStatus >= 200 && capturedStatus < 500;
    const payload = JSON.stringify({
      status: terminal ? "completed" : "failed",
      fingerprint: fp,
      response_status: capturedStatus,
      response_body: capturedBody,
    });
    await redis.set(redisKey, payload, "EX", TTL_SECONDS);
  });
  next();
}
```

Key choices:

- **`INSERT ... ON CONFLICT DO NOTHING` / `SET NX`** — the concurrency guard. Two concurrent requests with the same key: one inserts, one conflicts. No `SELECT`-then-`INSERT` race.
- **Fingerprint as SHA-256 of method + path + body** — distinguishes replay (same intent) from misuse (different payload, same key). Never compare bodies with `===` on JSON — key order differs.
- **`processing` state** — avoids the thundering-herd where 10 retries all try to create the order. The first wins; the rest get `409` (or wait and return the winner's result in the Redis variant).
- **TTL (24h)** — bounds storage. Keys must expire; otherwise the table grows without bound. After expiry, the same key is treated as new — which is correct because the caller's retry window has closed.
- **Scoped to principal** — prevents cross-tenant collisions on the same UUID.
- **Failed → retryable** — a `500` that marked `failed` allows the client to retry with the same key, which is the whole point. Do not mark `500` as `completed`; otherwise retries would replay the error forever.

> **Distributed-systems lens.** The idempotency store must be as available as the domain store. If `idempotency_keys` is in Postgres and Postgres is down, idempotency checks fail — but so does the domain write, so the request fails anyway. If the idempotency store is in Redis and Redis is down, decide explicitly: fail closed (`503` — do not process the write without deduplication) or fall back to Postgres. Failing open (process the write without the key) risks duplicates under retry — the worse failure mode for money-moving operations.

Pagination — listing at scale

Why pagination is infrastructure, not convenience

An unbounded `GET /v1/orders` that returns every row works until the table has 10,000 rows, then it times out, OOMs, or saturates the database. Every collection that can grow — which is every production collection — must be paginated, and the pagination contract determines whether clients can build correct views over data that is concurrently mutated.

Offset versus cursor — the decision

Offset pagination (`?offset=40&limit=20` → `LIMIT 20 OFFSET 40`) is intuitive and supports random access (\"go to page 7\"), but it breaks in two ways that matter:

1. **Performance.** `OFFSET 40000` forces the database to scan and discard 40,000 rows on every page. Cost grows linearly with page number. At large offsets, the query is a table scan that happens to return few rows. The fix (Postgres `OFFSET` still scans; MySQL similar) is to avoid offset entirely at scale.
2. **Correctness under mutation.** If a row is inserted at position 10 while the client pages through results, rows shift: the client sees duplicates or misses. Offset pagination is a snapshot of a moving target with no stability guarantee.

Cursor (keyset) pagination (`?page_token=eyJpZ...&page_size=20` → `WHERE (sort_key, id) > (cursor)`) fixes both:

- **Performance** — a range scan on an indexed column, cost independent of page depth.

- **Stability** — the cursor encodes the last seen sort key, so inserts before the cursor do not affect the next page. Inserts after the cursor appear on a subsequent page (or not at all if they sort before the cursor — which is correct for a forward walk).

Rule: **use cursor pagination for every collection that is large, growing, or concurrently mutated** — which is every production collection. Reserve offset for small, static, admin-only lists where random access matters and the dataset is bounded (<1,000 rows).

> **Boundary note.** *Why* cursor pagination is fast and offset pagination is slow is a storage-engine question — B-Tree range scans versus heap scans, composite index design, index-only scans, and how the query planner chooses a path. That analysis belongs to Volume 5, Chapter 3 (Indexing) and Chapter 4 (Query Processing and Optimization). This chapter uses the conclusion — *keyset on an indexed* `(sort_key, id)` *is an efficient range scan; offset is not* — and focuses on the API contract: token shape, tie-breaker, `has_more`, and consumer iteration.

The contract

```
GET /v1/orders?
filter[status]=paid&sort=-created_at&page_size=20&page_token=eyJp...

Response 200:
{
  "data": [{ "id": "01H8X1...", "status": "paid", ... }, ...],
  "pagination": { "next_page_token": "eyJpZC...MSJ9", "has_more": tru
e }
}
```

- **`page_token` is opaque.** Base64url-encoded JSON on the server side, but clients must treat it as an opaque string and never construct or decode it. Opaque tokens let you change the pagination strategy without breaking clients.
- **`has_more` + `next_page_token`** — `has_more: false` and `next_page_token: null` on the last page. The client loops until `!has_more`.
- **No `total_count` by default.** `COUNT(*)` over a filtered large table is expensive and immediately stale. Return `total_count` only when callers explicitly need it and accept the cost (and note it as approximate).

Real code — cursor pagination in Express 4.18.2 (pg 8.11.3)

This handler is the REST counterpart to the GraphQL resolver in Chapter 4, now combined with filtering and sorting. It shares the same keyset construction and one-extra trick.

```
// orders-list.js — Node 20.11.0, Express 4.18.2, pg@8.11.3, zod@3.22.4
import { z } from "zod";

const QuerySchema = z.object({
  page_size: z.coerce.number().int().min(1).max(100).default(20),
  page_token: z.string().optional(),
  "filter[status]": z.enum(["pending", "paid", "shipped", "cancelled"])
.optional(),
  "filter[created_at.gte]": z.string().datetime().optional(),
  sort: z.enum(["created_at", "-created_at", "total_cents", "-
total_cents"]).default("-created_at"),
  fields: z.string().optional(), // sparse fieldset — comma-separated
allowlist
});

const ALLOWED_FIELDS = new Set(["id", "customer_id", "status", "total_c
ents", "created_at", "updated_at"]);
const ALLOWED_SORTS = new Set(["created_at", "-created_at", "total_cent
s", "-total_cents"]);

function decodeCursor(token) {
  if (!token) return null;
  try {
    const json = Buffer.from(token, "base64url").toString("utf8");
    const obj = JSON.parse(json);
    if (typeof obj.last_created_at === "string" && typeof obj.last_id =
== "string") return obj;
    if (typeof obj.last_total_cents === "number" && typeof obj.last_id
=== "string") return obj;
    return null;
  } catch { return null; }
}
function encodeCursor(row, sortCol) {
  const payload = sortCol === "total_cents"
    ? { last_total_cents: row.total_cents, last_id: row.id }
    : { last_created_at: row.created_at.toISOString(), last_id:
row.id };
  return Buffer.from(JSON.stringify(payload)).toString("base64url");
}

export async function listOrders(req, res) {
  const parsed = QuerySchema.safeParse(req.query);
  if (!parsed.success) {
    return res.status(400).json({ code: "BAD_REQUEST", message: "Invali
d query", details: parsed.error.issues, request_id: req.id });
  }
  const q = parsed.data;

  // Validate cursor
  const cursor = q.page_token ? decodeCursor(q.page_token) : null;
```

```
  if (q.page_token && !cursor) {
    return res.status(400).json({ code: "BAD_REQUEST", message: "Invali
d page_token", request_id: req.id });
  }

  // Sparse fieldset — allowlist, default to all
  const fields = q.fields ? q.fields.split(",").map((s) => s.trim()).fi
lter(Boolean) : [...ALLOWED_FIELDS];
  const badFields = fields.filter((f) => !ALLOWED_FIELDS.has(f));
  if (badFields.length) return res.status(400).json({ code: "BAD_REQUES
T", message: `Unknown fields: ${badFields.join(",")}` });
  const selectList = fields.map((f) => `"${f}"`).join(", ");

  // Sort — cursor pagination requires the sort key in the cursor
  const sortDir = q.sort.startsWith("-") ? "DESC" : "ASC";
  const sortCol = q.sort.replace(/^-/, "");
  // For compound sort, extend cursor to include both keys; kept simple
here
  const cursorCol = sortCol === "total_cents" ? "total_cents" : "create
d_at";

  // WHERE with keyset condition when cursor present
  const where = [];
  const params = [];
  let idx = 1;
  if (q["filter[status]"]) { where.push(`status = $${idx++}`); params.p
ush(q["filter[status]"]); }
  if (q["filter[created_at.gte]"]) { where.push(`created_at >= $${idx+
+}`); params.push(q["filter[created_at.gte]"]); }
  if (cursor) {
    const op = sortDir === "ASC" ? ">" : "<";
    // Tie-breaker on id ensures deterministic ordering when sort key
is not unique
    if (cursorCol === "total_cents") {
      where.push(`(total_cents, id) ${op} ($${idx}, $${idx + 1})`);
      params.push(cursor.last_total_cents, cursor.last_id);
    } else {
      where.push(`(created_at, id) ${op} ($${idx}, $${idx + 1})`);
      params.push(cursor.last_created_at, cursor.last_id);
    }
    idx += 2;
  }
  const whereSQL = where.length ? `WHERE ${where.join(" AND ")}` : "";
  const limit = q.page_size + 1; // one-extra trick
  const sql = `SELECT ${selectList} FROM orders ${whereSQL} ORDER BY $
{cursorCol} ${sortDir}, id ${sortDir} LIMIT $${idx}`;
  params.push(limit);

  const { rows } = await pool.query(sql, params);
  const hasMore = rows.length > q.page_size;
  const page = hasMore ? rows.slice(0, q.page_size) : rows;
```

```
  const nextPageToken = hasMore ? encodeCursor(page[page.length - 1], c
ursorCol) : null;

  // List responses are short-lived cacheable only with caution —
collection is concurrently mutated
  res.setHeader("Cache-Control", "private, no-
store"); // or very short max-age if acceptable
  res.json({ data: page, pagination: { next_page_token: nextPageToken,
has_more: hasMore } });
}
```

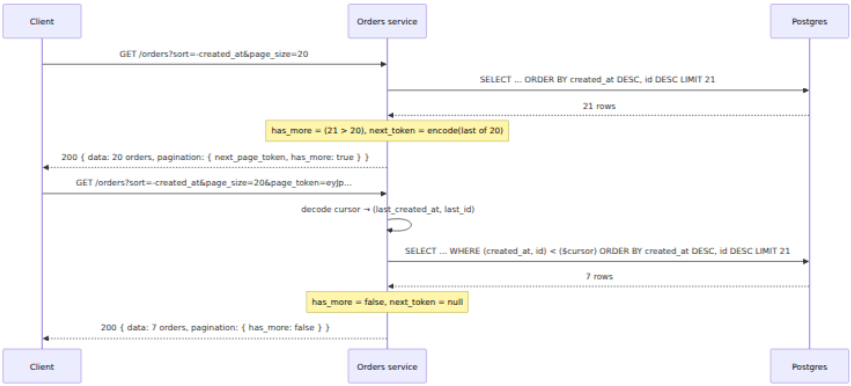


Figure 6-2: Cursor pagination walk. Each page fetches one extra row to determine `has_more` without a separate count query; the cursor encodes the last seen sort key and tie-breaker.

Key choices:

- **Composite condition `(created_at, id) > (...)` with tie-breaker on `id`.** When two orders share the same `created_at`, ordering by `id` alone ensures the cursor is stable. Without the tie-breaker, rows with identical sort keys would be skipped or duplicated across pages.
- **One-extra trick.** Fetch `page_size + 1`; the extra row tells you `has_more` without `COUNT(*)`. `COUNT(*)` with filters over large tables is a sequential scan that is both expensive and stale by the time you return it.
- **Opaque `base64url(JSON)` cursor.** Clients cannot construct cursors; the server can change the encoding (e.g., encrypt the cursor to prevent tampering) without breaking clients.
- **Cursor encodes sort context.** A cursor from `sort=-created_at` cannot be used with `sort=total_cents` — the server should reject the

mismatch. The simplest enforcement is to encode the sort column in the cursor or require the client to pass the same `sort` on every page.

Consuming pagination — Go client (Go 1.22)

```go
// client.go — Go 1.22, net/http stdlib
package orders

import (
    "context"
    "encoding/json"
    "fmt"
    "net/http"
    "net/url"
)

type Order struct {
    ID        string `json:"id"`
    Status    string `json:"status"`
    CreatedAt string `json:"created_at"`
}
type ListResp struct {
    Data       []Order `json:"data"`
    Pagination struct {
        NextPageToken *string `json:"next_page_token"`
        HasMore       bool    `json:"has_more"`
    } `json:"pagination"`
}

func ListAll(ctx context.Context, baseURL string, pageSize int) ([]Orde
r, error) {
    var all []Order
    var pageToken string
    client := &http.Client{}
    for {
        u, _ := url.Parse(baseURL + "/v1/orders")
        q := u.Query()
        q.Set("page_size", fmt.Sprint(pageSize))
        q.Set("sort", "-created_at")
        if pageToken != "" {
            q.Set("page_token", pageToken)
        }
        u.RawQuery = q.Encode()
        req, _ := http.NewRequestWithContext(ctx, "GET", u.String(), ni
l)
        resp, err := client.Do(req)
        if err != nil { return nil, err }
        if resp.StatusCode == http.StatusTooManyRequests {
            return nil, fmt.Errorf("rate limited: retry after %s",
resp.Header.Get("Retry-After"))
        }
        if resp.StatusCode != http.StatusOK {
            resp.Body.Close()
            return nil, fmt.Errorf("list orders: %s", resp.Status)
        }
```

```
        var lr ListResp
        if err := json.NewDecoder(resp.Body).Decode(&lr); err != nil {
            resp.Body.Close()
            return nil, err
        }
        resp.Body.Close()
        all = append(all, lr.Data...)
        if !lr.Pagination.HasMore || lr.Pagination.NextPageToken ==
nil {
            break
        }
        pageToken = *lr.Pagination.NextPageToken
    }
    return all, nil
}
```

Filtering, sorting, and sparse fieldsets

Pagination bounds the result size; filtering, sorting, and field selection make the bounded result *useful*. Without them, clients fetch every page and filter locally — the most expensive possible implementation of a query.

The contract — one convention, applied everywhere

Adopt one style for all three and enforce it with a Spectral rule (Chapter 1). The bracket style (`filter[status]=paid`, `sort=-created_at`, `fields=id,status`) is widely tooled and maps cleanly to query-string parsing:

```
GET /v1/orders?
filter[status]=paid&filter[created_at.gte]=2026-01-01T00:00:00Z
               &sort=-created_at
               &fields=id,status,total_cents
               &page_size=20&page_token=eyJp...
```

Concern	Param	Example
Equality filter	`filter[field]=value`	`filter[status]=paid`

Concern	Param	Example
Range filter	`filter[field.gte/lte]=value`	`filter[created_at.gte]=2026-01-01T00:00:00Z`
Sort	`sort=field` / `sort=-field`	`sort=-created_at`
Sparse fieldset	`fields=a,b,c`	`fields=id,status`

For GraphQL, the same concerns are typed inputs (Chapter 4): `filter: OrderFilter`, `sort: OrderSort`, selection set for field selection. The validation is at the GraphQL type level rather than query-string parsing, but the index and pagination implications are identical.

Validation and index awareness

Not every column should be filterable. A filter on an unindexed column (`filter[notes]=%foo%`) that triggers a sequential scan over 100 million rows is a denial-of-service vector, not a feature. The API must:

1. **Allowlist filterable fields.** Reject `filter[arbitrary_column]` with `400`.
2. **Bound range filters.** Reject `filter[created_at.gte]=1970-01-01` that would return the entire table — require a minimum selectivity or a mandatory time window.
3. **Reject unindexed sort.** If `sort=total_cents` has no index, either add the index or reject the sort with a message that names the supported sorts.

```
// Validation fragment — extends the handler above
const FILTERABLE = new Set(["status", "created_at.gte", "customer_id"])
;
const SORTABLE = new Set(["created_at", "-created_at", "total_cents", "
-total_cents"]);

function validateFilters(query) {
  for (const key of Object.keys(query)) {
    if (key.startsWith("filter[")) {
      const field = key.slice(7, -1); // "status" from "filter[status]"
      if (!FILTERABLE.has(field)) {
        throw Object.assign(new Error(`filter[${field}] is not
filterable`), { status: 400 });
      }
    }
  }
}
```

> **Boundary note.** *Which* filters can be served efficiently depends on the indexes that exist and how the planner uses them — composite indexes, partial indexes, and index-only scans. The storage-engine reasoning for why `filter[status]=paid AND created_at >= ...` needs a composite index on `(status, created_at, id)` versus two single-column indexes is covered in Volume 5, Chapters 3 (Indexing) and 4 (Query Processing). The API contract here is the seam: it declares *which* filters are supported, and the storage layer must honour that declaration with an index. If the contract advertises a filter without a backing index, the API is correct but slow — a bug that surfaces as a latency regression, not a functional failure.

Combining pagination, filtering, and sorting — the full pipeline

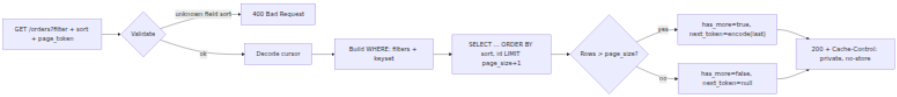


Figure 6-3: Combined list pipeline. Validation rejects unsupported filters/sorts before touching the database; the keyset condition and filters share the same WHERE clause so one index can cover both when designed correctly.

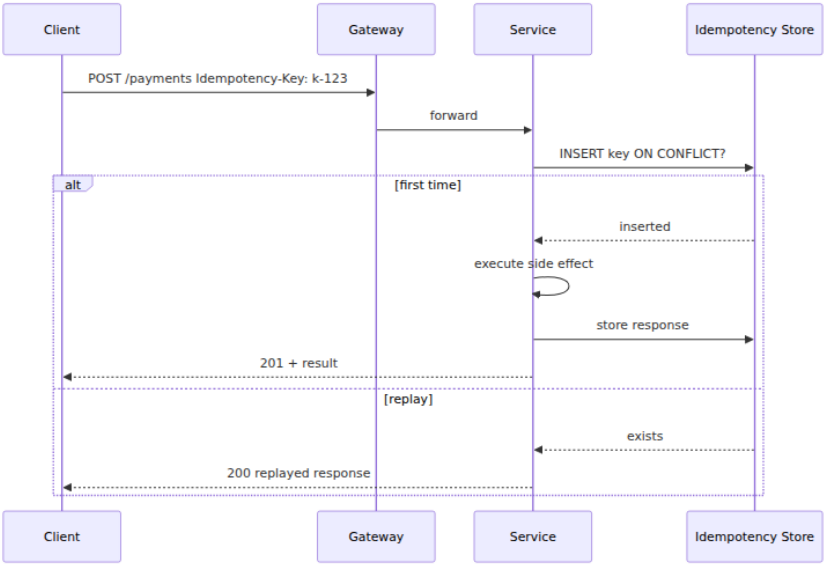
Idempotency meets pagination — the retry story

Pagination and idempotency interact: a client that pages through a large collection may retry a page fetch after a timeout. Because `GET` is idempotent and safe, retrying a page fetch with the same `page_token` is correct — it returns the same page. But two subtleties arise:

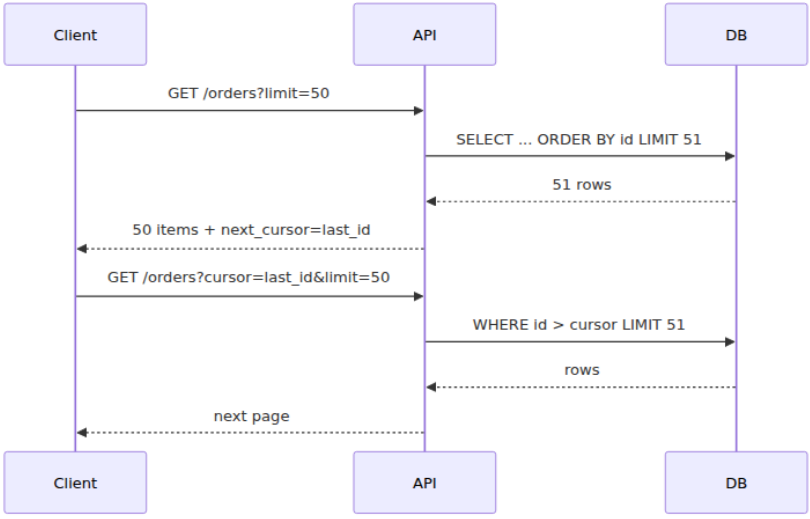
- **Cursors are not idempotency keys.** A `page_token` identifies a *position*, not an *intent`. Retrying` GET /orders?page_token=abc `is safe because` GET `is inherently idempotent. Do not require` Idempotency-Key `on` GET`.
- **Mutating while paginating.** If the client creates orders while paginating the same collection, cursor pagination guarantees the walk does not miss or duplicate rows that existed at the start — but newly created rows that sort before the current cursor will not appear (they are behind the cursor). This is correct for a forward walk; document it.

> **Distributed-systems lens.** At scale, list endpoints are called by batch jobs that page through millions of rows. Those jobs must be restartable: if the job crashes on page 842, it resumes from the last cursor, not from the beginning. Opaque cursors make this trivial (persist the cursor), while offset pagination makes it expensive (re-scan from zero). Design pagination for the batch consumer, not just the UI.

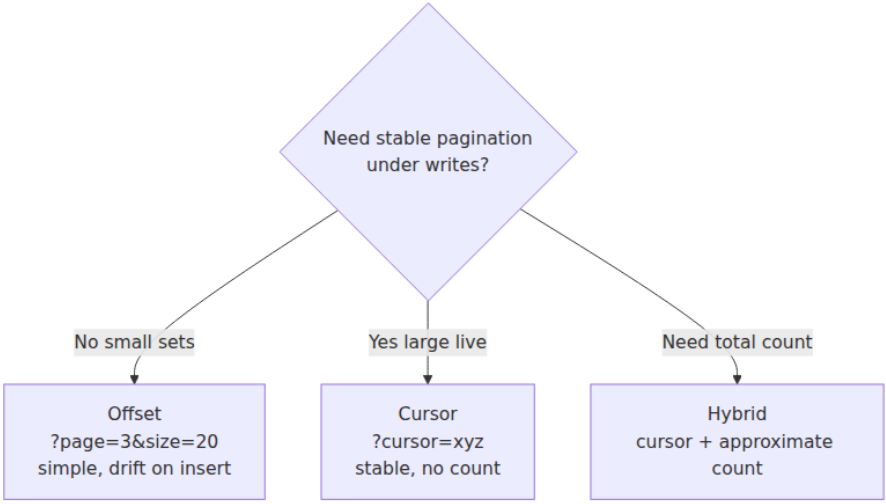
Idempotency Key Flow



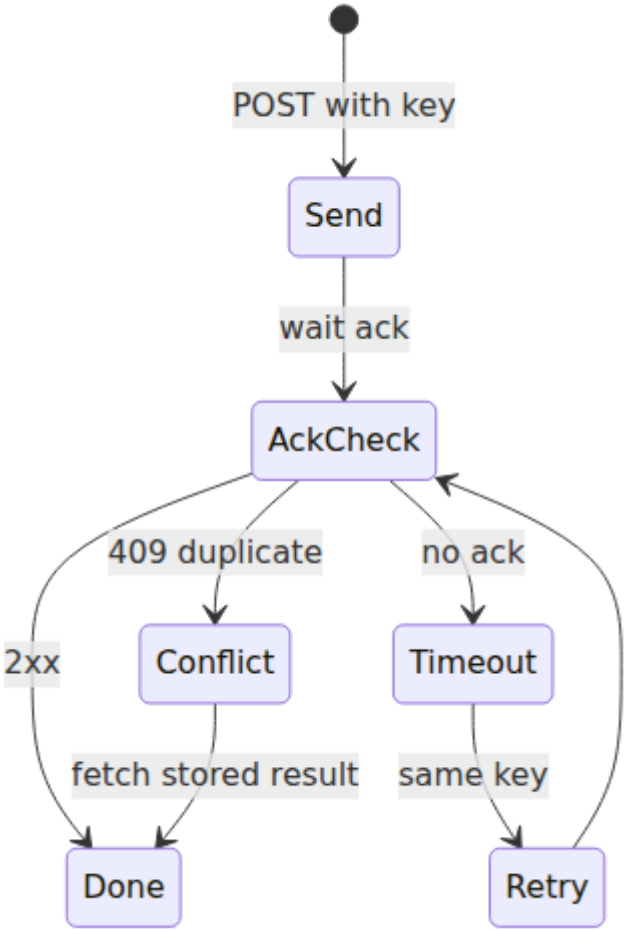
Cursor Pagination Sequence



Offset vs Cursor Tradeoff



Retry with Idempotency Safety



Key takeaways

- Make every `POST` that creates or mutates state idempotent via `Idempotency-Key`. The server must handle concurrent duplicate arrivals (`INSERT ... ON CONFLICT` / `SET NX`), distinguish replay (same fingerprint → return stored response) from misuse (different fingerprint → `422`), scope keys to the principal, and expire them on a bounded TTL (24h).
- Use the `processing` state to guard against thundering retries — one request proceeds, concurrent duplicates get `409` (or wait and return

the winner's result). Mark `5xx` as `failed` so the client can retry with the same key.

- Use cursor (keyset) pagination for every production collection: `WHERE (sort_key, id) > (cursor)` as a range scan on a composite index, opaque `base64url(JSON)` tokens, a deterministic tie-breaker on `id`, and the one-extra trick (`LIMIT page_size+1`) for `has_more` without `COUNT(*)`.
- Offset pagination (`LIMIT/OFFSET`) is reserved for small, static, admin-only lists. At scale it is a performance and correctness hazard (scan cost grows with offset; rows shift under concurrent mutation).
- Filtering and sorting must be allowlisted and index-aware. Advertise only filters and sorts that have backing indexes; reject unindexed queries with `400` rather than serving them slowly. The index design that makes a filter fast is the subject of Volume 5, Chapters 3–4; the API contract is where you declare the promise.
- Sparse fieldsets (`fields=id,status`) reduce over-fetching for list views; validate against an allowlist and default to all fields if omitted.
- `GET` pagination is inherently retry-safe; do not require `Idempotency-Key` on reads. Batch consumers that page through millions of rows depend on cursor stability for restartability — design for them.

Further reading

- IETF. *RFC 7231 §4.2.2 — Idempotent Methods* (2014) — the definition of method idempotency that this chapter builds on.
- Stripe. *Idempotent Requests* — https://stripe.com/docs/api/idempotent_requests — the industry reference for the `Idempotency-Key` pattern, including the `processing` / `completed` lifecycle.
- IETF. *RFC 6648 — Deprecating Use of the \"X-\" Prefix* (2012) and *RFC 6648 successors* — why `Idempotency-Key` (no `X-`) is the correct header name.
- `pg` 8.11.3 — https://node-postgres.com/, `ioredis` 5.3.2 — https://github.com/redis/ioredis, `zod` 3.22.4 — https://zod.dev/ — versions pinned for the handler examples.
- Redis 7.2 — https://redis.io/docs/latest/commands/set/ — `SET key value EX seconds NX` semantics for the atomic claim.

- Postgres 16.2 — https://www.postgresql.org/docs/16/sql-insert.html — `INSERT ... ON CONFLICT` for the concurrent-safe claim.
- Volume 5, Chapters 3 (Indexing) and 4 (Query Processing and Optimization) — the storage-engine analysis of why keyset pagination is a range scan and offset pagination is not, and how composite indexes cover filtered, sorted pagination.
- Volume 6, Chapter 9 (Idempotency, Deduplication, Exactly-Once) — the theory of exactly-once effect that idempotency keys implement.
- Volume 10, Chapter 6 (The Outbox Pattern) — deduplication at the event layer, the streaming counterpart to request-level idempotency.

Chapter 7 — Error Handling and Status Semantics

What this chapter covers. Every API call can fail, and the question is never *whether* it will fail but *how clearly* the failure will be communicated to a caller that cannot attach a debugger to your service. In a monolith, a stack trace is a few frames away. In a distributed system with five hops, a gateway, a gRPC-to-HTTP transcoding layer, and three different client SDKs, a lazy `500 Internal Server Error` with no machine-readable code is an incident multiplier: callers cannot distinguish "retry" from "fix your request," operators cannot alert without false positives, and canaries cannot auto-roll back. This chapter makes error handling a design discipline. It defines the contract an error *is* — code, message, details — and shows how to model it consistently across REST, gRPC, and mixed environments. We dissect HTTP status semantics under RFC 9110, RFC 9457 Problem Details, and the gRPC status model (17 codes, `google.rpc.Status`, and rich `ErrorDetails`), build a bidirectional mapping between the two that preserves retry semantics, and implement production patterns: structured problem+json envelopes, gRPC rich errors in Go and Java, interceptor-based error translation, and client-side handling that classifies retryable versus fatal errors. Every pattern is grounded in runnable, version-pinned code and viewed through the distributed-systems lens where retries, idempotency, and observability make or break the design.

Learning goals — after this chapter you should be able to:

- Choose correct HTTP status codes per RFC 9110 and explain why `400`, `404`, `409`, `422`, `429`, and `503` are routinely misused — and what an API-aware load balancer or gateway *does* with the code you return.
- Design and implement an RFC 9457 (`application/problem+json`) error envelope that carries a stable machine-readable `type`/`code`, a human-readable `detail`, and extension members — and reject vague `message`-only contracts.
- Use the gRPC status model correctly — the 17 canonical `codes.Code` values, `status.Status`, and rich `ErrorDetails` (`BadRequest`, `ErrorInfo`, `RetryInfo`, `QuotaFailure`, `PreconditionFailure`, `LocalizedMessage`) — to return errors that clients can program against.
- Map errors bidirectionally between HTTP/REST and gRPC (`google.rpc.Code` ↔ HTTP status) without losing retry semantics, and explain why `grpc-status` trailers and `google.api.http` transcoding matter.
- Implement server-side error creation and translation (Go `status.Errorf` / `status.ErrorDetails`, Java `StatusRuntimeException`, middleware/interceptor normalization) and client-side handling that classifies `UNAVAILABLE` / `DEADLINE_EXCEEDED` / `RESOURCE_EXHAUSTED` correctly for retries.
- Reason about errors in a distributed call graph — propagation versus translation at service boundaries, correlation with `request_id` / `trace_id`, error budgets, and why `Retry-After` and `RetryInfo` must agree.

Why errors are a contract, not an afterthought

An API's success path gets the design review. Its error path gets whatever the framework defaults to — usually a stack trace in development and a bare `500` in production. The cost surfaces months later: a partner team cannot programmatically distinguish "your `payment_method_id` is invalid" from "our payments service is down," so they retry both — one floods logs, the other risks duplicate charges. A gateway cannot decide whether to fail fast or retry on the next upstream.

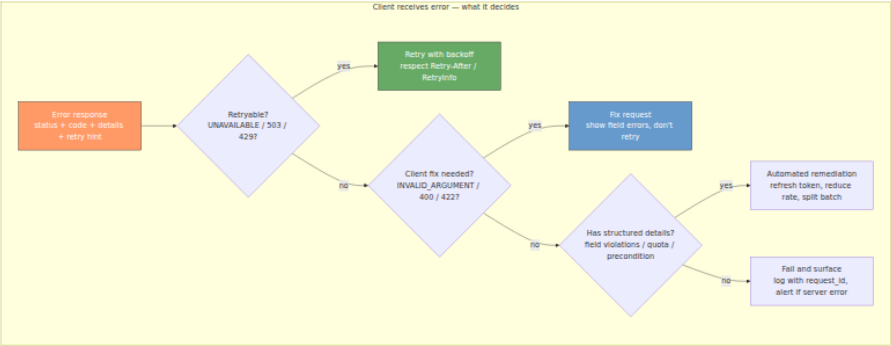
An SRE cannot write an alert that fires on *server* errors without firing on *client* errors.

The distributed-systems reason is sharper: in a call graph that fans out across five services, each hop must make a *local* decision — retry, fail, or translate — using only the error the downstream returned. If that error is imprecise, the wrong decision propagates. A `503 UNAVAILABLE` should be retried with backoff; a `404 NOT_FOUND` on a resource that was just created should be retried briefly for read-after-write consistency; a `400 INVALID_ARGUMENT` must never be retried without changing the request. Collapsing all three into `500` defeats the retry, circuit-breaker, and backpressure machinery that keeps the system alive (see Vol 6, Ch 9 — Idempotency; Vol 7, Ch 11 — Designing for Failure; Vol 11, Ch 10 — Resilience Patterns).

An error contract answers five questions for every failure:

1. **What happened?** A stable, machine-readable code (`code` / `type`) the client switches on.
2. **Whose fault?** Client error (fix the request) versus server error (retry or escalate) — encoded in the status code family and the code itself.
3. **Is it retryable?** Whether the same request, unchanged, could succeed — and after how long (`Retry-After` / `RetryInfo`).
4. **What to do next?** Field-level violations, quota that was exceeded, a precondition that failed — structured details so the client can act, not parse English.
5. **How to correlate?** A `request_id` / `trace_id` that ties the client-side error to the server's logs and traces.

> **Invariant.** An error response is part of the API's versioned contract exactly like a success response. Changing a machine-readable error `code` , reclassifying a `400` as `404` , or removing a detail field is a breaking change — see Chapter 8 (Compatibility). Treat errors with the same versioning discipline you give resources.



HTTP status semantics — what RFC 9110 actually says

HTTP status codes are not marketing labels ("we use `200` for everything and put the real status in the JSON"). They are routing and retry signals read by every intermediary between client and server: CDNs, API gateways, service meshes, and `fetch` itself.

The five classes

Class	Range	Meaning	Retry?
`1xx`	100–199	Informational	—
`2xx`	200–299	Success (`200 OK` , `201 Created` , `202 Accepted` , `204 No Content`)	—
`3xx`	300–399	Redirection (`301` , `302` , `307` , `308` — method-preserving vs not; `304 Not Modified`)	—
`4xx`	400–499	Client error — fix the request, don't retry unchanged	No (except `408` , `409` , `429` with hints)
`5xx`	500–599	Server error — retry *may* succeed	Conditional

The single most important distinction is `4xx` versus `5xx` . A `4xx` says *the client must change something*; a gateway that retries `4xx` on the next upstream will just fail there again. A `5xx` says *the server failed to fulfill a valid request*; a gateway that does not retry `503` when the next replica is

healthy leaves availability on the floor. Mixing them — returning `500` for validation failures because "it was an exception on the server" — poisons every retry and alert downstream.

The codes you will use (and misuse)

Code	Name	When to return	Common misuse
`200`	OK	Successful read/update	Returning `200` for errors with `{ "success": false }` — breaks intermediaries
`201`	Created	`POST` that created a resource — include `Location: /v1/orders/{id}`	Returning `200` with the created resource and no `Location`
`202`	Accepted	Request accepted for async processing (`Location` or status resource)	Using `200` for async work so the client thinks it is done
`204`	No Content	Successful `DELETE` or `PUT` with no body	Returning `200` with `{}`
`400`	Bad Request	Malformed syntax — invalid JSON, unparseable field	Using `400` for semantic validation (prefer `422`)
`401`	Unauthorized	Missing or invalid authentication — client should re-authenticate	`403` for "not logged in" — `401` means *unauthenticated*
`403`	Forbidden	Authenticated but not authorized — do not retry with same credentials	`401` for permission denied — `403` means *authenticated, not allowed*
`404`	Not Found	Resource does not exist — be careful to not leak existence (see security note)	Returning `404` for "not found because your filter matched nothing" (return `200` with empty list)

Code	Name	When to return	Common misuse
`405`	Method Not Allowed	Method not supported on this resource — include `Allow` header	`404` or `400` when the path is right but the verb is wrong
`406`	Not Acceptable	`Accept` / `Accept-Encoding` negotiation failed	Rarely used directly; content negotiation belongs in Ch 8
`408`	Request Timeout	Server timed out waiting for the request	Client may retry
`409`	Conflict	State conflict — duplicate `Idempotency-Key` with different payload, concurrent modification (`If-Match` failed), already exists	Using `400` for idempotency conflicts — see Ch 6
`410`	Gone	Resource permanently gone (sunset/deprecated) — cacheable	`404` for sunset — `410` tells the client to stop looking (Ch 5)
`412`	Precondition Failed	`If-Match` / `If-None-Match` / `If-Unmodified-Since` failed	`409` for preconditions — `412` is condition-specific
`413`	Payload Too Large	Body exceeds `max_bytes`	—
`415`	Unsupported Media Type	`Content-Type` not accepted	—
`422`	Unprocessable Entity	Well-formed but semantically invalid — validation failures, often with RFC 9457 detail	Overloading `400` — `422` signals *parseable but invalid*, ideal for field violations
`429`	Too Many Requests	Rate limited — **must** include `Retry-After`	`503` for rate limits — `429` lets the client distinguish quota from outage (Ch 7/Vol 7 Ch 9)

Code	Name	When to return	Common misuse
`500`	Internal Server Error	Unexpected server failure — no retry hint	Using `500` for validation or auth failures
`501`	Not Implemented	Method/endpoint not implemented	Rarely; usually `404` or `405` is closer
`502`	Bad Gateway	Gateway received invalid response from upstream	Distinguish from `503` / `504` — `502` is protocol/parse failure upstream
`503`	Service Unavailable	Temporary overload or maintenance — **should** include `Retry-After`	Using `500` for overload — `503` is the retry signal
`504`	Gateway Timeout	Upstream did not respond in time	Retries depend on whether the upstream is idempotent — see distributed lens

> **Security note — enumeration via `404` vs `403`.** Returning `404` for "you lack permission to see whether this resource exists" versus `403` for "it exists but you cannot access it" leaks existence. For sensitive resources, return `404` (or `403` uniformly) regardless of existence — document the choice (Vol 9, Ch 7 — Authorization, Ch 9 — AppSec).

Headers that belong on errors

- **`Retry-After: <http-date>` or `Retry-After: <seconds>`** on `429` and `503` — clients, gateways, and SDKs respect it; without it they guess.
- **`WWW-Authenticate`** on `401` — tells the client *how* to authenticate.
- **`Allow`** on `405` — lists the allowed methods.
- **`Deprecation` / `Sunset`** (RFC 8594/RFC 9745) — on errors that signal a deprecated or sunset path (Ch 5).
- **`X-Request-Id` / `traceparent`** — always echo the request's correlation ID on *errors* so the caller can ask "what happened to `req_abc123`?"

RFC 9457 Problem Details — the error envelope

RFC 9457 (`application/problem+json`, successor to RFC 7807) gives HTTP errors a standard envelope so clients can handle them generically. The alternative — every endpoint inventing `{ "message": "...", "error": "..." }` differently — forces client code to branch per endpoint. A problem+json envelope is:

```
{
  "type": "https://api.example.com/problems/out-of-stock",
  "title": "Out of stock",
  "status": 409,
  "code": "OUT_OF_STOCK",
  "detail": "SKU 'SKU-881' has 0 units available; requested 2.",
  "instance": "/v1/orders/req_abc123",
  "request_id": "req_abc123",
  "errors": [
    { "field": "items[0].quantity", "code": "INSUFFICIENT_STOCK", "mess
age": "Only 0 units available" }
  ]
}
```

RFC 9457 defines `type` (URI identifying the problem type), `title` (human-readable summary), `status` (mirrors the HTTP status), `detail` (human-readable, specific to this occurrence), and `instance` (URI for this occurrence). The spec explicitly permits **extension members** — `code`, `request_id`, `errors` above — which is where the machine-readable contract lives. Use them.

Design rules:

1. **`type` is a URI.** It should dereference to human-readable docs (`https://api.example.com/problems/out-of-stock`) — not `"OUT_OF_STOCK"`. The `code` extension member carries the switchable enum.
2. **`code` is enum-stable.** `OUT_OF_STOCK`, `PAYMENT_DECLINED`, `QUOTA_EXCEEDED` — clients `switch` on `code`, never on `detail`. Adding a `code` is minor; changing one is breaking.
3. **`detail` is for humans.** It may contain interpolated values ("SKU 'SKU-881'..."), but never structured data the client must parse.
4. **`errors[]` for field violations.** Each entry names `field` (JSON Pointer or dotted path), `code`, and `message`. This powers form-level UX without parsing `detail`.

5. `status` **mirrors the HTTP status.** Don't diverge — caches and generic HTTP clients read `status` without parsing the body.
6. **Content type matters.** Return `Content-Type: application/problem+json` so a generic problem+json client (`problem+json` aware `fetch` wrapper) can deserialize without knowing your API.
7. **Never leak internals.** No stack traces, no SQL, no file paths — even in `detail` — outside debug builds.

A real middleware pair (Express, Go) looks like:

```typescript
// Express (TypeScript) — problem+json envelope helper (RFC 9457)
import type { Request, Response, NextFunction } from "express";

type FieldViolation = { field: string; code: string; message: string };
type Problem = {
  type: string; title: string; status: number;
  code: string; detail: string;
  instance: string; request_id: string;
  errors?: FieldViolation[];
};

export function problem(
  res: Response, req: Request,
  status: number,
  code: string,
  title: string,
  detail: string,
  opts: { type?: string; errors?: FieldViolation[] } = {},
) {
  const requestId = (req.headers["x-request-id"] as string) ?? res.getH
eader("x-request-id") as string;
  const body: Problem = {
    type: opts.type ?? `https://api.example.com/problems/${code.toLower
Case().replace(/_/g,"-")}`,
    title, status, code, detail,
    instance: req.originalUrl,
    request_id: requestId,
    ...(opts.errors ? { errors: opts.errors } : {}),
  };
  return res.status(status)
    .type("application/problem+json")
    .json(body);
}

// Usage:
// problem(res, req, 422, "VALIDATION_FAILED", "Validation failed",
//   "One or more fields are invalid.", { errors: [{ field:"email",
code:"INVALID_FORMAT", message:"Not a valid email" }] });
// problem(res, req, 429, "RATE_LIMITED", "Too many requests", "Quota
100/min exceeded. Retry after 37s.");
```

```go
// Go — RFC 9457 problem+json helper
package httpx

import (
    "encoding/json"
    "net/http"
)

type FieldViolation struct {
    Field   string `json:"field"`
    Code    string `json:"code"`
    Message string `json:"message"`
}

type Problem struct {
    Type      string           `json:"type"`
    Title     string           `json:"title"`
    Status    int              `json:"status"`
    Code      string           `json:"code"`
    Detail    string           `json:"detail"`
    Instance  string           `json:"instance"`
    RequestID string           `json:"request_id"`
    Errors    []FieldViolation `json:"errors,omitempty"`
}

func WriteProblem(w http.ResponseWriter, r *http.Request, status int, c
ode, title, detail string, violations []FieldViolation) {
    w.Header().Set("Content-Type", "application/problem+json")
    w.Header().Set("X-Request-Id", r.Header.Get("X-Request-Id"))
    w.WriteHeader(status)
    _ = json.NewEncoder(w).Encode(Problem{
        Type:      "https://api.example.com/problems/" + code,
        Title:     title,
        Status:    status,
        Code:      code,
        Detail:    detail,
        Instance:  r.URL.Path,
        RequestID: r.Header.Get("X-Request-Id"),
        Errors:    violations,
    })
}
```

The gRPC status model

gRPC errors are not exceptions tacked on — they are typed values on the wire. Every gRPC response is either *success* (status `OK = 0`, trailers absent of `grpc-status`) or *error* (non-zero `grpc-status` trailer, HTTP/2

`200` over the wire with gRPC status in trailers — see Vol 3, Ch 8). The model has three layers:

Layer 1 — the 17 canonical codes

From `google.golang.org/grpc/codes` / `io.grpc.Status` / `grpc/status`:

gRPC code	Value	HTTP equiv	Retryable?	Use when
`OK`	0	200	—	Success
`CANCELLED`	1	499*	No	Client cancelled the call
`UNKNOWN`	2	500	No	Unknown — avoid; use a specific code
`INVALID_ARGUMENT`	3	400	No	Bad field / malformed
`DEADLINE_EXCEEDED`	4	504/408*	Conditional (idempotent only)	Deadline/ timeout hit
`NOT_FOUND`	5	404	No	Resource not found
`ALREADY_EXISTS`	6	409	No	Create that already exists
`PERMISSION_DENIED`	7	403	No	Authenticated but not authorized
`RESOURCE_EXHAUSTED`	8	429	After `RetryInfo`	Quota/rate limit, out of space
`FAILED_PRECONDITION`	9	400/412	No	System not in required state
`ABORTED`	10	409	Yes (with backoff)	Concurrency conflict, transaction aborted

gRPC code	Value	HTTP equiv	Retryable?	Use when
`OUT_OF_RANGE`	11	400	No	Cursor/offset past range
`UNAVAILABLE`	14	503	Yes (with backoff)	Transient — safe to retry
`UNIMPLEMENTED`	12	501	No	Method not implemented
`INTERNAL`	13	500	No (unless known transient)	Invariant broken
`DATA_LOSS`	15	500	No	Unrecoverable data corruption
`UNAUTHENTICATED`	16	401	No (after re-auth)	Missing/invalid auth

`*` `CANCELLED` maps to HTTP `499` (nginx/client closed) or is not HTTP-mapped; `DEADLINE_EXCEEDED` maps to `504 Gateway Timeout` externally and `408 Request Timeout` in some mappings — document your gateway's choice.

> **Rule.** `UNKNOWN` means "the author did not choose a code." Never return it intentionally. `INTERNAL` is strictly for "this should never happen and did" — bugs and invariant violations. If you can name what went wrong, name it (`INVALID_ARGUMENT` , `FAILED_PRECONDITION` , `OUT_OF_RANGE`).

Layer 2 — `google.rpc.Status` and layer 3 — rich ErrorDetails

A gRPC error with details is a `google.rpc.Status` message whose `details` carry typed `Any` payloads (`google.rpc.*`):

```
// google/rpc/status.proto (simplified)
message Status {
  int32 code = 1;    // google.rpc.Code (same 17)
  string message = 2; // human-readable, not for switching
  repeated google.protobuf.Any details = 3;
}
```

The standard detail types (`google/rpc/error_details.proto`) are:

- **`BadRequest`** + `FieldViolation` — field-level validation (`field` , `description`).
- **`ErrorInfo`** — stable `reason` + `domain` + `metadata` map — the switchable identity (like `code` in problem+json). Example: `reason: "ORDER_OUT_OF_STOCK" domain: "api.example.com" metadata: { "sku": "SKU-881" }` .
- **`RetryInfo`** — `retry_delay` (`Duration`) — how long to wait before retrying. The gRPC equivalent of `Retry-After` .
- **`QuotaFailure`** + `Violation` — which quota (`subject`) was exceeded.
- **`PreconditionFailure`** + `Violation` — which precondition failed (`type` / `subject` / `description`).
- **`ResourceInfo`** — which `resource_type` / `resource_name` triggered the error.
- **`Help`** + `Link` — human help links.
- **`LocalizedMessage`** — locale-aware `message` (separate from the English `Status.message`).
- **`DebugInfo`** — `stack_entries` + `detail` — only in debug builds.

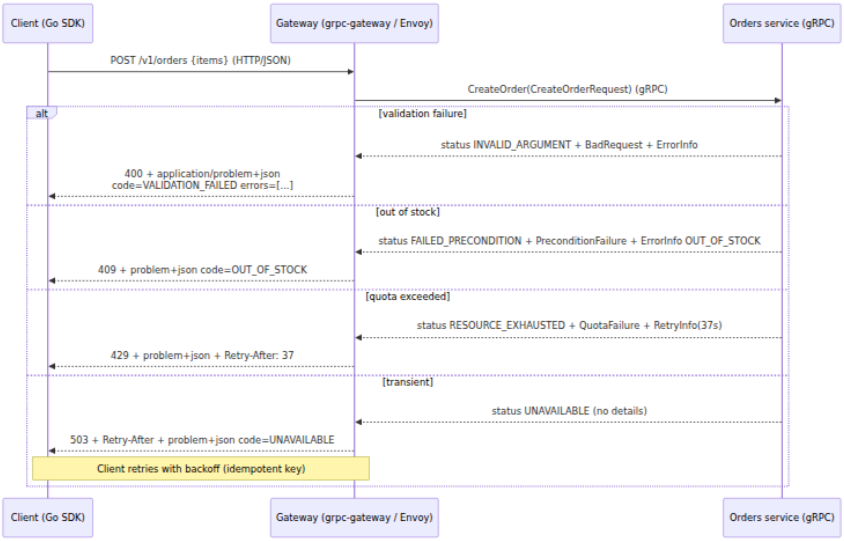
Creating gRPC errors (Go)

```go
// Go — rich gRPC errors (grpc-go 1.64+, google.golang.org/grpc,
google.golang.org/genproto/googleapis/rpc/errdetails)
package orders

import (
    "context"
    "time"

    "google.golang.org/grpc/codes"
    "google.golang.org/grpc/status"
    "google.golang.org/genproto/googleapis/rpc/errdetails"
    "google.golang.org/protobuf/types/known/durationpb"
)

func (s *Server) CreateOrder(ctx context.Context, req *CreateOrderReque
st) (*Order, error) {
    if err := req.Validate(); err != nil {
        // INVALID_ARGUMENT with BadRequest field violations — client
can map to form errors
        st := status.New(codes.InvalidArgument, "validation failed")
        br := &errdetails.BadRequest{}
        for _, v := range err.FieldViolations() {
            br.FieldViolations = append(br.FieldViolations,
&errdetails.BadRequest_FieldViolation{
                Field: v.Field, Description: v.Message,
            })
        }
        // Also attach ErrorInfo for stable switching
        ei := &errdetails.ErrorInfo{
            Reason: "VALIDATION_FAILED", Domain: "api.example.com",
            Metadata: map[string]string{"request_id": reqIdFrom(ctx)},
        }
        st, _ = st.WithDetails(br, ei)
        return nil, st.Err()
    }

    if !s.inventory.HasStock(req.Items) {
        st := status.New(codes.FailedPrecondition, "insufficient
stock")
        pf := &errdetails.PreconditionFailure{
            Violations: []*errdetails.PreconditionFailure_Violation{
                {Type: "STOCK", Subject: "sku/SKU-881", Description:
"0 units available, requested 2"},
            },
        }
        ei := &errdetails.ErrorInfo{Reason: "OUT_OF_STOCK", Domain: "ap
i.example.com",
            Metadata: map[string]string{"sku": "SKU-881"}}
        st, _ = st.WithDetails(pf, ei)
        return nil, st.Err()
```

```
    }

    if limited, retryAfter := s.limiter.Check(ctx); limited {
        st := status.New(codes.ResourceExhausted, "quota exceeded:
orders.create 100/min")
        st, _ = st.WithDetails(
            &errdetails.QuotaFailure{
                Violations: []*errdetails.QuotaFailure_Violation{
                    {Subject: "quota:orders.create/100 per minute", Des
cription: "quota exceeded"},
                },
            },
            &errdetails.RetryInfo{RetryDelay:
durationpb.New(retryAfter)},
        )
        return nil, st.Err()
    }

    order, err := s.store.Create(ctx, req)
    if err != nil {
        return nil, status.Errorf(codes.Internal, "create order: %v", e
rr)
    }
    return order, nil
}

// Client side — handling rich errors
func placeOrder(ctx context.Context, c OrdersClient, req *CreateOrderRe
quest) error {
    _, err := c.CreateOrder(ctx, req)
    if err == nil {
        return nil
    }
    st, ok := status.FromError(err)
    if !ok {
        return err
    }
    switch st.Code() {
    case codes.InvalidArgument:
        for _, d := range st.Details() {
            if br, ok := d.(*errdetails.BadRequest); ok {
                for _, v := range br.FieldViolations {
                    log.Printf("field %q: %s", v.Field, v.Description)
                }
            }
        }
        return fmt.Errorf("validation failed: %s", st.Message())
    case codes.ResourceExhausted:
        for _, d := range st.Details() {
            if ri, ok := d.(*errdetails.RetryInfo); ok {
                log.Printf("retry after %s", ri.RetryDelay.AsDuration()
```

```
)
                time.Sleep(ri.RetryDelay.AsDuration())
            }
        }
        return fmt.Errorf("rate limited: %s", st.Message())
    case codes.Unavailable, codes.Aborted:
        // retry with backoff (idempotent only — see Ch 6)
        return errRetryable{err: err}
    default:
        return err
    }
}
```

```
// Java — same model (grpc-java 1.66+, google.rpc.* from grpc-proto)
import io.grpc.Status;
import io.grpc.StatusRuntimeException;
import io.grpc.protobuf.StatusProto;
import com.google.rpc.BadRequest;
import com.google.rpc.ErrorInfo;
import com.google.rpc.RetryInfo;

public Order createOrder(CreateOrderRequest req) {
    if (!req.isValid()) {
        com.google.rpc.Status status =
com.google.rpc.Status.newBuilder()
            .setCode(Status.INVALID_ARGUMENT.getCode().value())
            .setMessage("validation failed")
            .addDetails(Any.pack(BadRequest.newBuilder()
                .addFieldViolations(BadRequest.FieldViolation.newBuilde
r()
                    .setField("email").setDescription("invalid format")
.build())
                .build()))
            .addDetails(Any.pack(ErrorInfo.newBuilder()
                .setReason("VALIDATION_FAILED").setDomain("api.example.
com").build()))
            .build();
        throw StatusProto.toStatusRuntimeException(status);
    }
    // ... rate-limit branch adds QuotaFailure + RetryInfo similarly
    return store.create(req);
}
```



Mapping HTTP and gRPC without losing meaning

Most systems are not pure REST or pure gRPC — a gRPC service sits behind an HTTP gateway (Envoy, `grpc-gateway`, `Connect`) that transcodes `google.api.http` bindings to JSON, or a REST service is called by a gRPC client via `grpc-status` trailers. The mapping must preserve two things: the *category* (`4xx` vs `5xx`, retryable vs not) and the *retry hint*.

gRPC code → HTTP status (server → gateway → client)

Canonical mapping used by `grpc-gateway` and `google.rpc.Code` docs:

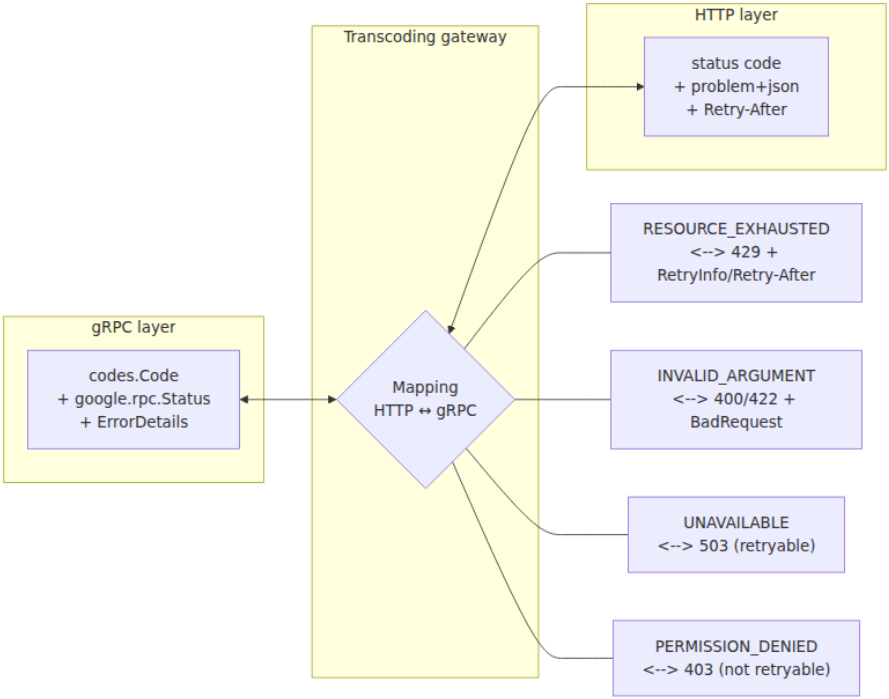
gRPC code	HTTP	Notes
`OK`	200 (or 201/204 per binding)	Success
`CANCELLED`	499	Client closed request
`INVALID_ARGUMENT`	400	Use `422` through an envelope if the API prefers `422` for validation — overrides are fine if documented

gRPC code	HTTP	Notes
`DEADLINE_EXCEEDED`	504	Also 408 in some proxies — pick one and document
`NOT_FOUND`	404	
`ALREADY_EXISTS`	409	
`PERMISSION_DENIED`	403	
`RESOURCE_EXHAUSTED`	429	Preserve `RetryInfo` → `Retry-After`
`FAILED_PRECONDITION`	400 (or 409/412 for state conflicts)	Split `FAILED_PRECONDITION` finely in the envelope
`ABORTED`	409	Retryable with backoff
`OUT_OF_RANGE`	400	
`UNIMPLEMENTED`	501	
`INTERNAL`	500	
`UNAVAILABLE`	503	Retryable
`DATA_LOSS`	500	
`UNAUTHENTICATED`	401	Include `WWW-Authenticate` when transcoding

HTTP → gRPC is the inverse (e.g., `429` → `RESOURCE_EXHAUSTED` with `RetryInfo`). If your gateway does not propagate `RetryInfo` to `Retry-After` and back, fix the gateway — not the service. The contract is the *cross-protocol* pair:

```
gRPC:  status RESOURCE_EXHAUSTED + RetryInfo{ retry_delay: 37s }
HTTP:  429 Too Many Requests + Retry-After: 37 + problem+json { code:
"RATE_LIMITED", retry_after: 37 }
```

Both say the same thing: "retry after 37 seconds."


HTTP layer
Transcoding gateway
status code
+ problem+json
+ Retry-After
RESOURCE_EXHAUSTED
<--> 429 +
RetryInfo/Retry-After
gRPC layer
codes.Code
+ google.rpc.Status
+ ErrorDetails
Mapping
HTTP ↔ gRPC
INVALID_ARGUMENT
<--> 400/422 +
BadRequest
UNAVAILABLE
<--> 503 (retryable)
PERMISSION_DENIED
<--> 403 (not retryable)

`google.api.http` transcoding (where the mapping is declared)

```
// orders.proto — the mapping lives in the proto, not in ad-hoc gateway
config
syntax = "proto3";
package api.orders.v1;
import "google/api/annotations.proto";
import "google/api/http.proto";

service OrdersService {
  rpc CreateOrder(CreateOrderRequest) returns (Order) {
    option (google.api.http) = {
      post: "/v1/orders"
      body: "*"
    };
  }
  rpc GetOrder(GetOrderRequest) returns (Order) {
    option (google.api.http) = {
      get: "/v1/orders/{order_id}"
    };
  }
  rpc ListOrders(ListOrdersRequest) returns (ListOrdersResponse) {
    option (google.api.http) = {
      get: "/v1/orders"
    };
  }
}
```

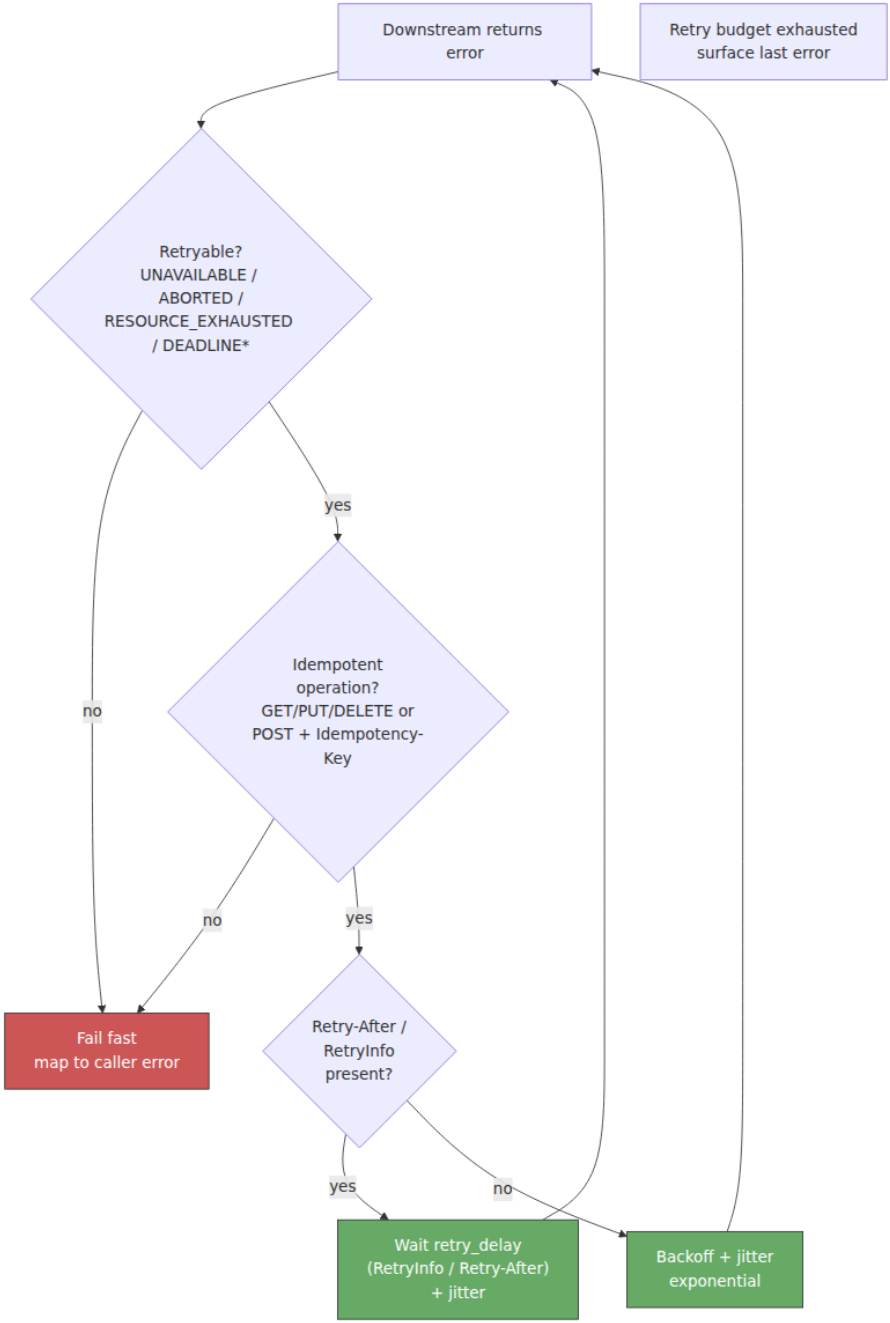
The gateway reads `google.api.http` at generation time (`protoc-gen-grpc-gateway` / `protoc-gen-connect-openapi` / Envoy `grpc_json_transcoder`) and emits routes whose error transcoding respects the `gRPC code → HTTP status` table. Hand-rolling routes without this annotation is how teams end up mapping `RESOURCE_EXHAUSTED` to `503` "because it felt like a server error."

The distributed-systems lens — errors in a call graph

Retryability is the only classification that matters for reliability

Every automated actor — client SDK, service mesh, gateway, workflow engine — must decide *retry or fail* without human judgement. Split codes into two sets and wire the automation off the split:

- **Retryable (same request may succeed):** `UNAVAILABLE` / `503`, `ABORTED` / `409` (with backoff), `RESOURCE_EXHAUSTED` / `429` (after `Retry-After`), `DEADLINE_EXCEEDED` / `504` *only* if the handler is known idempotent. Use exponential backoff with jitter and a bounded attempt count (Vol 3, Ch 11; Vol 7, Ch 11).
- **Not retryable without changing the request:** `INVALID_ARGUMENT` / `400`, `NOT_FOUND` / `404`, `PERMISSION_DENIED` / `403`, `UNAUTHENTICATED` / `401`, `ALREADY_EXISTS` / `409` (idempotency conflict — see Ch 6), `FAILED_PRECONDITION` / `400` / `412`, `OUT_OF_RANGE` / `400`, `UNIMPLEMENTED` / `501`.
- **`INTERNAL` / `500`:** Treat as non-retryable by default. If a specific `INTERNAL` is known transient (brief invariant violation during a deploy), mark it with an `ErrorInfo` reason that the retry layer whitelists — don't make all `500`s retryable.


Downstream returns error
Retry budget exhausted surface last error
Retryable? UNAVAILABLE / ABORTED / RESOURCE_EXHAUSTED / DEADLINE*
yes
no
Idempotent operation? GET/PUT/DELETE or POST + Idempotency-Key
no
yes
Fail fast map to caller error
Retry-After / RetryInfo present?
yes
no
Wait retry_delay (RetryInfo / Retry-After) + jitter
Backoff + jitter exponential

> **Critical.** Non-idempotent `POST` without an `Idempotency-Key` must not be retried automatically on `UNAVAILABLE` / `DEADLINE_EXCEEDED`. The caller's retry turns a transient error into a duplicate order. Gateways that retry blindly on `503` must check `Idempotency-Key` or the operation's idempotency claim (Ch 6) before retrying.

Propagation versus translation — the boundary rule

When service A calls service B and B returns an error, A has two choices:

- **Propagate:** return B's error unchanged (same code, same details). Correct when A is a thin proxy and the client can handle B's errors.
- **Translate:** map B's error to A's contract. Correct when A abstracts B — the client should not learn that `api.orders` internally calls `api.inventory` and that inventory returned `OUT_OF_STOCK`.

Default to *translate* at ownership boundaries and *propagate* inside a bounded context. In both cases, preserve `request_id` / `trace_id` and add context — wrapping a `NOT_FOUND` from `inventory.GetSKU` into an `INVALID_ARGUMENT` on `orders.CreateOrder` with `detail: "SKU 'SKU-881' not found"` is translation done well. Wrapping it into `INTERNAL` "failed to create order" is a lie that breaks observability.

Logging: log the *original* downstream error (with `trace_id`, downstream `code`, `retryable` bit) at the call site; return the *translated* contract to the caller. Never log the translated message and discard the cause.

Observability — errors as metrics, not just messages

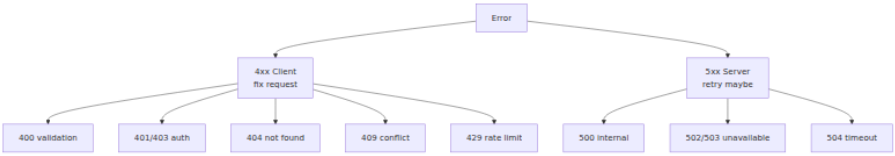
- **Cardinality control.** Metric label `code` should be the stable `ErrorInfo.reason` / problem+json `code` (`OUT_OF_STOCK`), not `detail`. A `detail` that contains SKU IDs would explode cardinality.
- **SLOs.** Count `5xx` and gRPC `INTERNAL` / `UNAVAILABLE` / `DATA_LOSS` against the server error budget; count `4xx` / `INVALID_ARGUMENT` / `NOT_FOUND` etc. separately. Alerting on *all* non-`2xx` fires on client bugs.
- **Sampling.** Sample or always trace error responses (OpenTelemetry `StatusCode.ERROR` with the RPC code as an attribute). Gateways should echo `traceparent` on errors so the client can hand you an ID.

- **Idempotency-aware dashboards.** Track `409` / `ALREADY_EXISTS` with `Idempotency-Key` conflicts distinctly from business-logic conflicts — they require different runbooks.

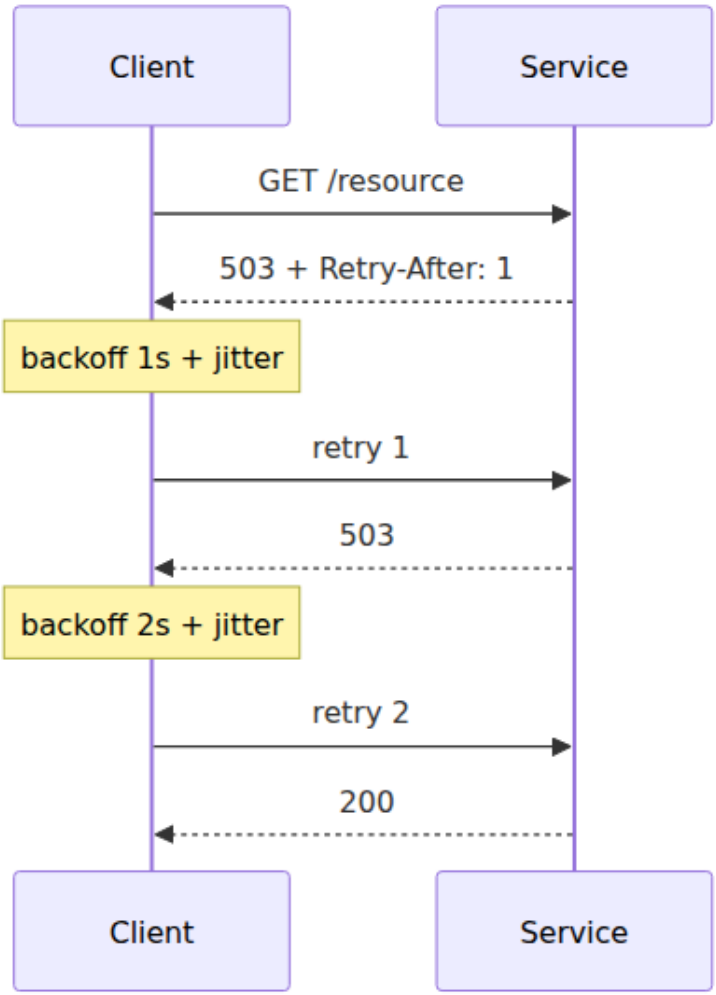
Anti-patterns (and what to do instead)

Anti-pattern	Why it hurts	Fix
`200` with `{ "success": false, "error": "..." }`	Intermediaries, caches, and SDKs treat it as success; retries and alerts break	Use real status codes + problem+json / gRPC codes
`500` for everything "exceptional" on the server	Client cannot distinguish retryable from fatal; error budget polluted	Classify: `4xx` / `INVALID_ARGUMENT` vs `5xx` / `INTERNAL`
Machine-readable text in `detail` / `message`	Clients `string.contains("out of stock")` and break on rewording	Use stable `code` / `ErrorInfo.reason`; `detail` is for humans
Returning stack traces / SQL in production errors	Information disclosure (Vol 9, Ch 9); payload bloat	Strip `DebugInfo` in production; emit a `request_id` instead
`UNKNOWN` / `INTERNAL` for known cases (quota, validation)	Loses retry signal and field details	Return `RESOURCE_EXHAUSTED` + `RetryInfo`, `INVALID_ARGUMENT` + `BadRequest`
`400` for validation and `404` for "validation: SKU not found" inconsistently	Callers cannot program against errors	Policy: unknown SKU on *create* is `INVALID_ARGUMENT` (client fix); on *get* is `NOT_FOUND`
No `Retry-After` on `429` / `503`	Clients hammer retries	Always attach `Retry-After` (HTTP) / `RetryInfo` (gRPC)

Error Taxonomy



Retry with Backoff and Jitter



ProblemDetails Propagation

Key takeaways

- Treat every error response as versioned contract. Stable `code` / `ErrorInfo.reason` values are what clients switch on; `detail` / `message` are for humans. Changing a code is a breaking change.
- Respect RFC 9110: `4xx` means "fix the request," `5xx` means "server failed, retry *may* work." Intermediaries, gateways, and meshes route and retry based on this split — misclassification is a reliability bug.
- Use RFC 9457 `application/problem+json` for HTTP APIs: `type` (docs URI), `title`, `status`, `code`, `detail`, `instance`, `request_id`, `errors[]` with `Retry-After` where applicable. Use `codes.Code` + `google.rpc.Status` + typed `ErrorDetails` (`BadRequest`, `ErrorInfo`, `RetryInfo`, `QuotaFailure`, `PreconditionFailure`) for gRPC.
- Wire a bidirectional HTTP ↔ gRPC mapping that preserves retryability and carries `RetryInfo` ↔ `Retry-After`. Declare it in `google.api.http` and let the gateway (`grpc-gateway` / `Connect` /Envoy) enforce it — do not hand-map per route.
- Classify retryable versus non-retryable sharply and only retry idempotent operations (or `POST` with an `Idempotency-Key` — Ch 6). Retry `UNAVAILABLE` / `ABORTED` / `RESOURCE_EXHAUSTED` (after delay) and non-transactional `DEADLINE_EXCEEDED` on idempotent handlers; fail fast on `INVALID_ARGUMENT` / `NOT_FOUND` / `PERMISSION_DENIED` / `UNAUTHENTICATED` / `FAILED_PRECONDITION`.
- At service boundaries translate errors to the caller's contract while preserving correlation (`request_id` / `trace_id`) and logging the untranslated cause. Propagate raw downstream errors only inside a bounded context.
- Observe errors as typed signals: metric label on stable `code`, not on `detail`; error-budget SLOs on `5xx` /retryable gRPC codes only; always sample/trace error responses with OpenTelemetry (`StatusCode.ERROR` + RPC code attribute).

Further reading

- RFC 9110 — HTTP Semantics (status code definitions). https://www.rfc-editor.org/rfc/rfc9110.html
- RFC 9457 — Problem Details for HTTP APIs (obsoletes RFC 7807, `application/problem+json`). https://www.rfc-editor.org/rfc/rfc9457.html
- RFC 6585 — Additional HTTP Status Codes (`429`, `511`). https://www.rfc-editor.org/rfc/rfc6585.html
- RFC 8594 / RFC 9745 — `Deprecation` / `Sunset` headers for API lifecycle (Ch 5). https://www.rfc-editor.org/rfc/rfc8594.html / https://www.rfc-editor.org/rfc/rfc9745.html
- gRPC Status Codes — canonical list and semantics. https://grpc.io/docs/guides/status/
- `google.rpc.Status` / `google.rpc.Code` — `googleapis/google/rpc/status.proto`, `code.proto`. https://github.com/googleapis/googleapis/tree/master/google/rpc
- `google.rpc.ErrorDetails` — `BadRequest`, `ErrorInfo`, `RetryInfo`, `QuotaFailure`, `PreconditionFailure`, `ResourceInfo`, etc. `google/rpc/error_details.proto`. https://github.com/googleapis/googleapis/blob/master/google/rpc/error_details.proto
- AIP-193 — Errors (Google API Improvement Proposals) — guidance on `google.rpc.Status` usage. https://google.aip.dev/193
- gRPC-Gateway — HTTP/JSON transcoding for gRPC (`google.api.http`). https://grpc-ecosystem.github.io/grpc-gateway/
- ConnectRPC — `Connect` protocol error model and HTTP mapping. https://connectrpc.com/docs/protocol/
- OpenTelemetry — Semantic conventions for RPC and HTTP status. https://opentelemetry.io/docs/specs/semconv/rpc/

- Envoy — `grpc_json_transcoder` filter (gRPC ↔ JSON error translation). https://www.envoyproxy.io/docs/envoy/latest/configuration/http/http_filters/grpc_json_transcoder_filter

Chapter 8 — Compatibility and Wire Formats

What this chapter covers. An API's wire format is the contract that outlives every deploy: the bytes that cross the network after the producer has upgraded but before every consumer has. Choosing JSON versus Protobuf versus Avro is not a style preference — it determines what kinds of evolution are safe, how large payloads are on the wire, whether a consumer that has not yet upgraded can ignore fields it does not understand, and whether a gateway can validate and route without knowing every schema version. Compatibility is the discipline that keeps the system correct when *mixed versions* are in flight — during rolling deploys, canary analysis, blue/green cuts, and the months-long tails of mobile and partner upgrades that never converge to a single version (see Chapter 5 — Versioning and Evolution). This chapter makes both precise. We dissect the Protobuf wire format at the byte level (varints, tags, length-delimited fields, unknown-field preservation), contrast it with JSON and Avro/Thrift/FlatBuffers on size, speed, schema discipline, and browser reachability, define backward, forward, and full compatibility crisply and show which breaks each format tolerates, and build the production machinery: the expand-contract pattern for storage and wire schemas, the `buf breaking` and `oasdiff` / `openapi-diff` gates that fail CI on breaking changes, and the schema-registry designs (Confluent/Apicurio/Buf Schema Registry) that enforce compatibility at publish time. Every pattern is anchored in runnable, version-pinned proto/Avro/JSON examples and viewed through the distributed-systems lens where rolling deploys, serialization-aware routing, and dual-version serving make or break the design.

Learning goals — after this chapter you should be able to:

- Describe the Protobuf binary wire format (tag = `field_number << 3 | wire_type`, varint/ZigZag, length-delimited, unknown-field preservation) and explain why it enables forward compatibility by default — and where it does not.

- Compare JSON, Protobuf, Avro, Thrift, FlatBuffers, and MessagePack on payload size, encode/decode cost, schema discipline, browser support, and compatibility guarantees — and choose correctly per surface (public REST, internal gRPC, event log, edge/browser).
- Classify any schema change as backward-compatible, forward-compatible, full-compatible, or breaking — and apply the correct expand-contract step rather than a flag-day migration.
- Apply Protobuf compatibility rules precisely: safe additions (new `optional` / `repeated` fields with fresh numbers), safe `reserved` discipline, `oneof` evolution, `enum` aliasing, and the five changes that are always breaking (number reuse, type-widening, `required`-like semantics, `oneof` removal, enum value reuse).
- Apply JSON/OpenAPI compatibility rules: additive fields and `x-` extensions are backward-compatible, type changes/renames/removals and tightening `required` / `enum` are breaking, and why JSON's schemaless convenience trades away the safety Protobuf gives you for free.
- Wire compatibility enforcement into CI and the registry: `buf breaking` against `main` (or BSR), `oasdiff breaking` / `openapi-diff` for OpenAPI, Confluent/Apicurio compatibility modes (`BACKWARD` / `FORWARD` / `FULL` / `TRANSITIVE`), and Schema Registry validation on Kafka/BSR publish.

Why wire format and compatibility are the same problem

A wire format defines *how* data is serialized; compatibility defines *when* a change to that serialization breaks someone who has not yet upgraded. The two are inseparable: the wire format's design determines which evolutions are invisible to old readers and old writers.

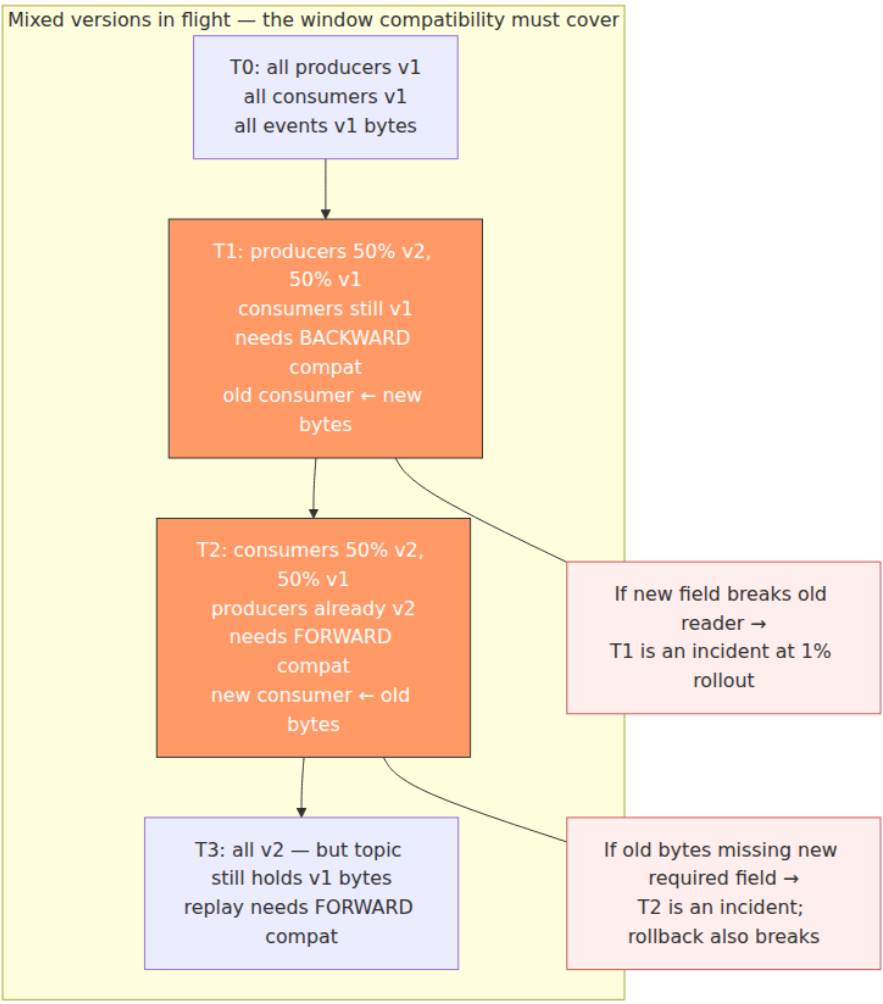
In a single-process system, changing a struct field from `string status` to `enum Status status` is a compile-time error you fix in one commit. In a distributed system, that field is serialized, pushed to a topic, written to a database, cached at the edge, and polled by a mobile binary you last released three months ago. Changing it is *three* deployments (writer, reader, storage) across *two* time windows with *mixed versions* in flight. The only question that matters is: *will the bytes produced by version N*

still be readable by version N−1, and will the bytes produced by version N−1 still be readable by version N?

That question splits into three guarantees:

Guarantee	Question	Consumer that benefits
Backward compatible	Can an *old* consumer read data produced by a *new* producer?	Old readers during rollout (the common case)
Forward compatible	Can a *new* consumer read data produced by an *old* producer?	New readers that rolled first, or replays of old events
Full compatible	Both directions — old↔new interop	Safe rolling deploys in either direction; event replays

> **Distributed-systems lens.** Every rolling deploy, blue/green cut, and Kafka topic has a window where *both* versions coexist. If the new producer's bytes break the old consumer, the rollout turns into an incident the moment 1% of traffic hits the new version. Backward compatibility is what lets you roll forward; forward compatibility is what lets you roll back or replay. Full compatibility is what lets you do either without coordination — and it is the invariant a schema registry *enforces* (see Vol 6, Ch 7 — Quorum Systems, on why coordination-free evolution matters, and Vol 10, Ch 3 — Kafka Architecture, on log immutability).



The rest of the chapter shows how each wire format answers this question — and how to verify the answer mechanically.

Wire formats compared — what to use where

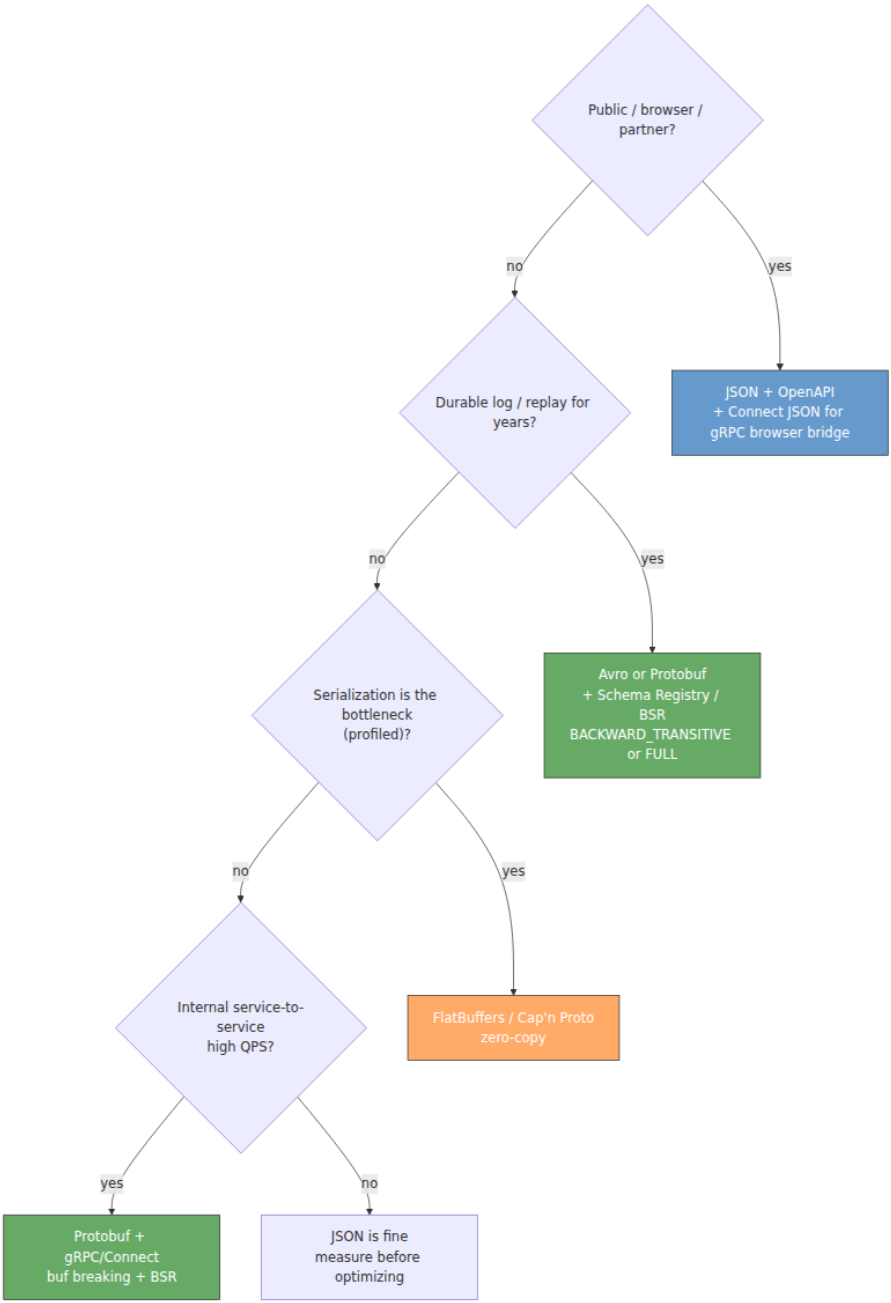
No single format wins everywhere. The trade-off is *size/speed* versus *human readability and ubiquity* versus *schema discipline and compatibility safety*.

Format	Encoding	Schema?	Size (vs JSON)	Speed	Browser-native	Co saf
JSON	Text, self-describing	Optional (JSON Schema / OpenAPI)	1× (baseline)	Baseline	Yes	Lov add saf not enf the
Protobuf (proto3)	Binary, tag-value	Required (`.proto`)	~0.25–0.5×	2–5× faster than JSON	No (via `grpc-web` / Connect JSON or `protobuf-es`)	Hig unk fiel `re` `bu` `bre`
Avro	Binary, schema + data	Required (Avro schema, often with Registry)	~0.2–0.4× (with registry, no schema per message)	Fast; schema resolution at read time	No	Hig rea sch res `BA` `FU` enf Reg
Thrift (compact/ binary)	Binary, field IDs	Required (`.thrift`)	~0.3–0.5×	Fast	No	Me — s Pro `re` dar
FlatBuffers / Cap'n Proto	Binary, zero-copy	Required	Smallest; zero-copy read	Fastest (no parse)	No	Me zer tra evo flex

Format	Encoding	Schema?	Size (vs JSON)	Speed	Browser-native	Co saf
MessagePack / CBOR	Binary, self-describing	No	~0.5–0.7×	~1.5× JSON	Partial (JS libs)	Lov JSO sch
gRPC JSON (`protojson`) / Connect JSON	Text (JSON mapping of proto)	Required (proto) but JSON on wire	~1×	JSON speed	Yes (Connect)	Me fiel rul app pro

Selection heuristics for a senior backend engineer:

1. **Public API consumed by browsers/partners:** JSON over HTTPS (OpenAPI), with a JSON Schema/BSR-compatible check. Add Connect/gRPC-JSON transcoding if the same service also serves internal gRPC (see Ch 3, Ch 5 `google.api.http`).
2. **Internal service-to-service (low latency, high QPS):** Protobuf over gRPC (HTTP/2) or Connect (HTTP/1.1/2/3). Binary, code-generated, `buf`-checked.
3. **Durable event log / streaming (Kafka, Pulsar, Kinesis):** Avro with a Schema Registry (Confluent `BACKWARD_TRANSITIVE` / `FULL`) or Protobuf with BSR — never schemaless JSON on a log you will replay for years (Vol 10, Ch 3).
4. **Ultra-low-latency or zero-copy (feature store, game, device):** FlatBuffers/Cap'n Proto — only if profiling shows serialization is the bottleneck.
5. **Cache/memoization payloads (Redis, Memcached):** MessagePack/CBOR if size matters and the reader is controlled; otherwise JSON for debuggability.


Public / browser / partner?
no
yes
JSON + OpenAPI + Connect JSON for gRPC browser bridge
Durable log / replay for years?
no
yes
Avro or Protobuf + Schema Registry / BSR BACKWARD_TRANSITIVE or FULL
Serialization is the bottleneck (profiled)?
no
yes
FlatBuffers / Cap'n Proto zero-copy
Internal service-to-service high QPS?
yes
no
Protobuf + gRPC/Connect buf breaking + BSR
JSON is fine measure before optimizing

Protobuf wire format — the bytes that make compatibility work

Protobuf's compatibility story is not magic — it falls out of the wire encoding. Understanding the bytes explains *why* unknown fields are ignored, why field numbers must never be reused, and why `reserved` exists.

Encoding at a byte level

Every field on the wire is a **tag** followed by a **value**. The tag is `field_number << 3 | wire_type` encoded as a varint.

Wire type	Value	Used for	Value encoding
`0`	Varint	`int32`, `int64`, `uint32`, `uint64`, `sint32`, `sint64`, `bool`, `enum`	Varint (7 bits per byte, MSB = continuation)
`1`	64-bit	`fixed64`, `sfixed64`, `double`	8 bytes little-endian
`2`	Length-delimited	`string`, `bytes`, embedded messages, `repeated` packed	Varint length + N bytes
`5`	32-bit	`fixed32`, `sfixed32`, `float`	4 bytes little-endian

Varint with **ZigZag** for signed types: `sint32` / `sint64` map `(0, -1, 1, -2, 2, …)` → `(0, 1, 2, 3, 4, …)` so small negatives do not become 10-byte varints.

Example. This proto:

```
// orders.proto
syntax = "proto3";
package api.orders.v1;

message CreateOrderRequest {
  string customer_id = 1;  // tag = 1<<3|2 = 10 (0x0A)
  int32  quantity    = 2;  // tag = 2<<3|0 = 16 (0x10)
  bool   expedited   = 3;  // tag = 3<<3|0 = 24 (0x18)
}
```

Request `{ customer_id: "42", quantity: 300, expedited: true }` encodes as:

```
0A 02 34 32    field 1, length-delimited, len=2, bytes "42"
10 AC 02       field 2, varint, 300 = 0xAC 0x02 (varint)
18 01          field 3, varint, true = 1
```

If a *new* producer adds `string promo_code = 4` and an *old* consumer does not know field 4, the old consumer sees tag `34` (`4<<3|2`) — unrecognized — and **skips** `length` bytes. New field, old reader, no failure: backward compatibility for free. Conversely, an old producer that never writes field 4 leaves it at its default (`""`) on the new consumer — forward compatibility, provided the new field is not treated as required.

Unknown fields are **preserved** on parse and re-serialized by proto3 runtimes (since 3.5) — a proxy that does not understand field 4 can still forward it. That is full compatibility at the byte level — as long as you never reuse numbers or change wire types.

What is allowed, what breaks — the Protobuf rules

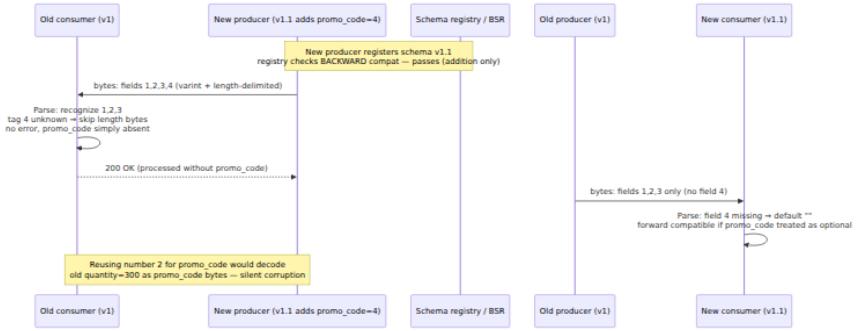
Safe (non-breaking) changes:

- **Add a new field** with a fresh number, `optional` or `repeated` , with a sensible default. Old readers ignore it; new readers see default when reading old bytes.
- **`reserved` a number or name you will never reuse.** `reserved 6, 9 to 11; reserved "old_field";` prevents accidental reuse.
- **Add a value to an `enum`** — but only if consumers handle unknown enum values (they arrive as the numeric value; generated `UNRECOGNIZED` in Java/Go). Always include `UNSPECIFIED = 0` as the default.
- **Add a new `oneof` alternative** (old readers ignore the new case as unknown field on the `oneof` 's tag).
- **Widen from `optional` to `repeated` via a new field** — never change the same number's cardinality.

Always-breaking changes:

Breaking change	Why it breaks
Reuse a field number (even with a different name/type)	Old bytes for the old field are decoded as the new field — silent data corruption
Change wire type (`int32` → `string` , `int32` → `fixed32`)	Decoder reads the wrong number of bytes
Change `int32` ↔ `uint32` ↔ `sint32` semantics without care	Varint/ZigZag mismatch; negative values corrupt
Remove a `oneof` or change its kind	`oneof` discriminant lost; unknown-field handling differs
Remove or rename an `enum` value that old producers still emit	Old numeric value becomes `UNRECOGNIZED` ; switch statements that lack a `default` misbehave
Tighten a constraint the wire does not enforce (`string` → `enum` , add `required` -like validation)	Wire-compatible but *semantically* breaking — old producer sends a value the new consumer rejects (see expanding below)

```proto
// api/orders/v1/order.proto — safe evolution discipline (version-
pinned: buf 1.40+, protobuf 5.x)
syntax = "proto3";
package api.orders.v1;

import "google/protobuf/timestamp.proto";

message Order {
  string customer_id = 1;
  int32  quantity    = 2;
  bool   expedited   = 3;

  // Added in v1.1 — safe: fresh number, optional semantics (proto3
optional)
  optional string promo_code = 4;

  // Added in v1.2 — safe: new enum value at end
  enum Status {
    STATUS_UNSPECIFIED = 0;
    STATUS_PENDING     = 1;
    STATUS_SHIPPED     = 2;
    STATUS_DELIVERED   = 3;
    // Added in v1.2:
    STATUS_RETURNED    = 4;
  }
  Status status = 5;

  google.protobuf.Timestamp created_at = 6;

  // Numbers 7-9 reserved — never reuse if you remove a field
  reserved 7, 8, 9;
  reserved "legacy_discount_code";
}

// Breaking — DO NOT DO:
// message Order { string customer_id = 1; string quantity = 2; }  //
type change: int32 -> string
// message Order { string promo_code = 2; }  // reuse of number 2
```



JSON / OpenAPI compatibility — additive is safe, everything else is suspect

JSON has no wire-level field numbers — compatibility is purely conventional and must be enforced by tooling.

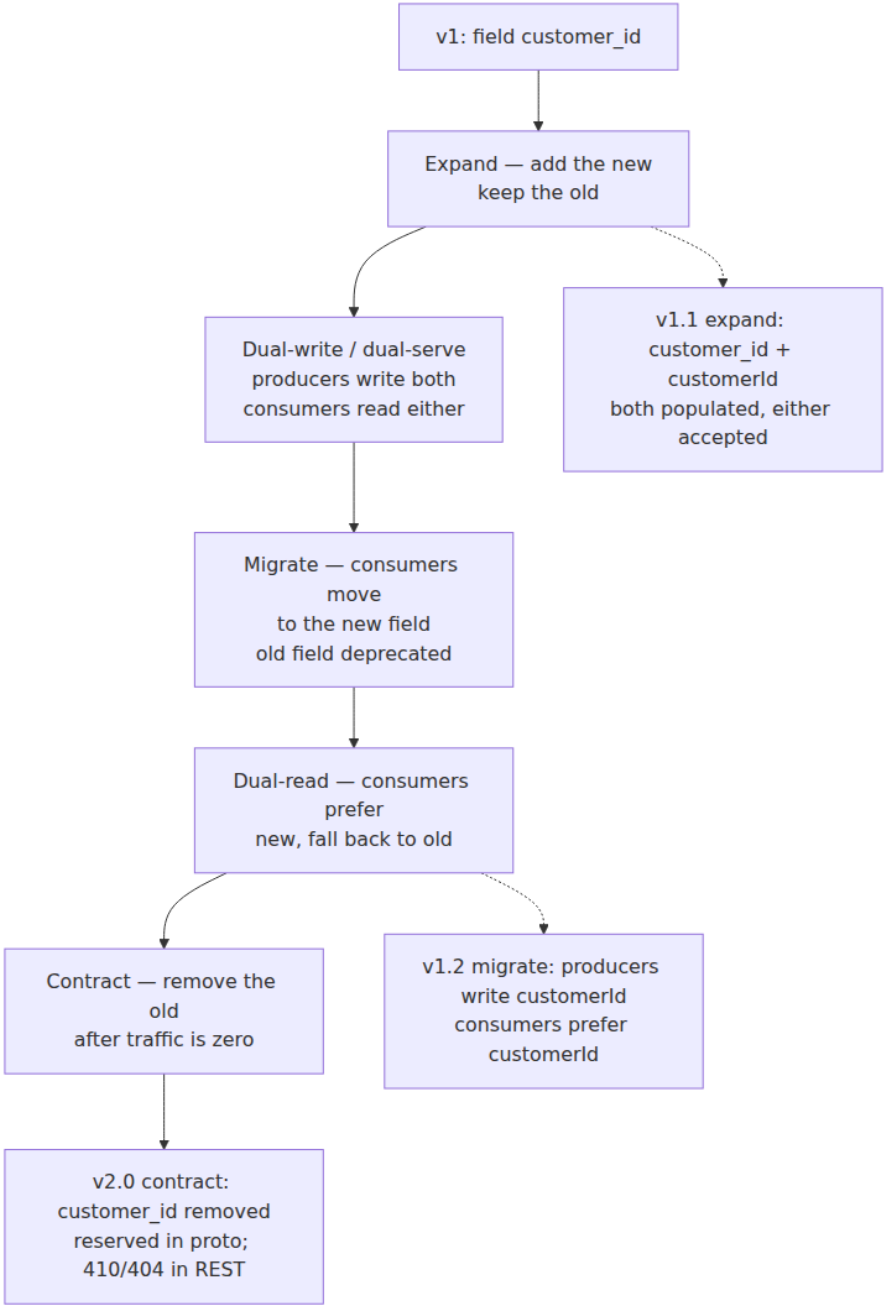
Change	Backward? (old reader ← new bytes)	Forward? (new reader ← old bytes)	Verdict
Add optional field	Yes — old reader ignores unknown keys	Yes — new reader sees missing → default/null	Safe
Add `required` field	**No** — old writer omits it, new reader may reject	—	Breaking (tightening)
Remove field (even optional)	—	**No** — new reader may still expect it	Breaking if any client still sends it
Rename field	**No** — old reader looks for old name	**No** — new reader looks for new name	Breaking (two-field expand-contract instead)
Change type (`string` → `number` , `string` → `enum`)	**No** — old reader parses wrong type	**No**	Breaking

Change	Backward? (old reader ← new bytes)	Forward? (new reader ← old bytes)	Verdict
Tighten `enum` (remove value)	—	**No** — old writer emits removed value	Breaking
Widen `enum` (add value)	**No** if old reader lacks default case	Yes	Conditionally safe — document unknown-value handling
Tighten numeric range / `pattern` / `maxLength`	—	**No** — old valid values now invalid	Breaking
Add `x-` extension / additive header	Yes	Yes	Safe

The implication: JSON APIs need the same discipline Protobuf gives for free — and a CI gate that catches what the wire does not. That gate is `oasdiff` / `openapi-diff` (see below). Without it, "just add a field" discipline is a handshake agreement waiting to be violated.

Expand-contract — the only safe migration

When the change *is* breaking (rename, type change, tighten), the flag-day alternative — "everyone upgrades at once" — fails at any non-trivial consumer count. Expand-contract (also called parallel change) is the mechanical alternative.


v1: field customer_id
Expand — add the new
keep the old
Dual-write / dual-serve
producers write both
consumers read either
v1.1 expand:
customer_id +
customerId
both populated, either
accepted
Migrate — consumers
move
to the new field
old field deprecated
Dual-read — consumers
prefer
new, fall back to old
Contract — remove the
old
after traffic is zero
v1.2 migrate: producers
write customerId
consumers prefer
customerId
v2.0 contract:
customer_id removed
reserved in proto;
410/404 in REST

Concrete storage + wire example:

```sql
-- Phase 1 — Expand (backward compatible). Old code still reads
customer_id; new code writes both.
ALTER TABLE orders ADD COLUMN customer_id_v2 TEXT;  -- nullable
-- Application dual-writes: SET customer_id_v2 = customerId,
customer_id = customer_id_v2

-- Phase 2 — Migrate + Dual-read. Backfill then switch reads.
UPDATE orders SET customer_id_v2 = customer_id WHERE customer_id_v2 IS
NULL;
-- Application: read customer_id_v2 if present else customer_id

-- Phase 3 — Contract. Only after old producer traffic is 0 and
backfill is verified.
ALTER TABLE orders DROP COLUMN customer_id;  -- proto: reserved
"customer_id"; reserved 1;
```

```protobuf
// Proto expand-contract — rename customer_id -> customer_id_v2 without
breaking
message Order {
  // Phase 1: keep old, add new
  string customer_id    = 1; // deprecated = true
  string customer_id_v2 = 7; // new canonical

  // Phase 3 after migration:
  // reserved 1;
  // reserved "customer_id";
  // string customer_id_v2 = 7;
}
```

Wire rules during expand: producers populate *both* fields with the same value; consumers read the *new* field if present, else the old. Validation accepts either but prefers the new. Traffic metrics confirm the old field's `present` rate falls to zero before `reserved` is applied — see observability below.

Enforcement — failing the build on breaking changes

Discipline without enforcement drifts. Two gates cover the two surfaces:

Protobuf — `buf breaking` (Buf 1.40+, BSR)

```
# buf.yaml — module at api/orders
version: v2
modules:
  - path: proto
lint:
  use: [DEFAULT]
  # Enforce reserved discipline, enum zero value, field naming
breaking:
  use: [FILE]           # FILE = strict; PACKAGE/WIRE_JSON are looser
  except: []            # never except breaking without a reason
```

```
# buf.lock is committed; CI compares against main's BSR or git
# .github/workflows/buf.yaml
name: buf
on: [pull_request]
jobs:
  breaking:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: fetch-depth: 0
      - uses: bufbuild/buf-action@v1
        with:
          setup_only: true
      - run: buf lint proto
      - run: buf breaking proto --against '.git#branch=main'
        # or against BSR: --against 'buf.build/acme/orders:main'
```

A PR that reuses field number `2`, changes `int32 quantity` to `string quantity`, or removes `STATUS_PENDING` fails with a line-precise diagnostic:

```
proto/api/orders/v1/order.proto:8:3: Field "2" on message "Order"
changed type from "int32" to "string".
proto/api/orders/v1/order.proto:7:3: Field "2" on message "Order" chang
ed name from "quantity" to "promo_code".
Failure: 2 breaking changes detected.
```

Wire-compatibility categories map to `buf breaking` groups: `WIRE` (wire-type changes) versus `WIRE_JSON` (+ JSON name changes) — choose `FILE` for public APIs; `WIRE_JSON` is tolerable for internal-only.

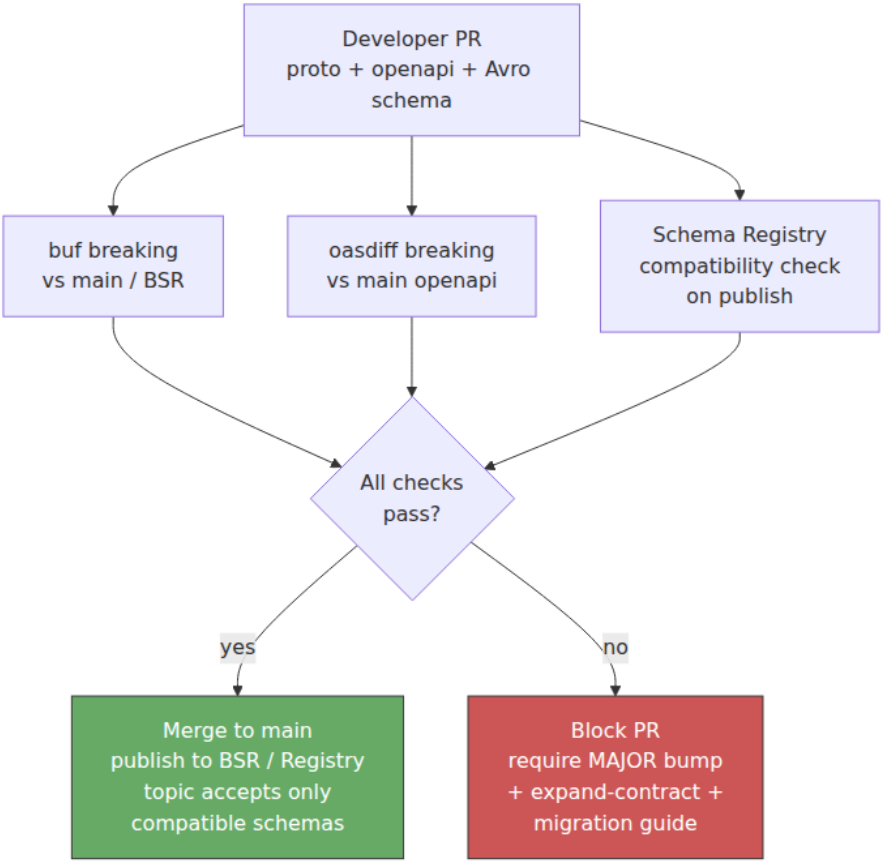
OpenAPI — `oasdiff` / `openapi-diff` (oasdiff 1.10+, openapi-diff 2.1+)

```
# Compare PR's openapi.yaml against main — fail on breaking changes
oasdiff breaking https://raw.githubusercontent.com/acme/orders/main/
openapi.yaml \
                ./openapi.yaml \
                --fail-on ERR \
                --composed  # also catch allOf/anyOf/oneOf breaks

# Alternative (Java, Redocly ecosystem):
openapi-diff --fail-on-incompatible \
  https://raw.githubusercontent.com/acme/orders/main/openapi.yaml \
  ./openapi.yaml
```

Outputs are stability-graded: `ERR` (breaking: removed field, new `required`, type change) fails CI; `WARN` (added optional field) is informational. Back that with a Schema Registry for event surfaces:

```
# Confluent Schema Registry / Apicurio — topic-level compatibility
# Admin API or Terraform (confluent_schema, apicurio registry)
confluent.value.schema.validation=true
# Per-subject compatibility (the invariant the registry enforces on
publish)
# BACKWARD: new schema can read data written by old schema (old
consumer ← new bytes)
# FORWARD: old schema can read data written by new schema (new consumer
← old bytes)
# FULL: both; TRANSITIVE variants check against *all* prior versions,
not just the latest
subject.orders-value.compatibilityLevel=FULL_TRANSITIVE
```

Publishing a schema that reuses a field number, tightens `required`, or narrows an `enum` is rejected at `POST /subjects/orders-value/versions` with a compatibility error — before any bytes hit the topic. Apicurio's `FULL_TRANSITIVE` and Buf Schema Registry's `breaking` check are the same idea: the registry is the compatibility gate for the log.



The distributed-systems lens — living with mixed versions

Rolling deploys and the two invariants

A rolling deploy is the steady state, not the exception — especially with canaries, blue/green, and multi-region traffic shifting (Vol 7, Ch 10–11). At any moment, two service revisions serve traffic. Compatibility

determines whether the gateway can safely route either version to either revision:

- **New gateway + old service** needs forward compatibility (new consumer ← old bytes).
- **Old gateway + new service** needs backward compatibility (old consumer ← new bytes).

Full compatibility lets you roll in either direction without coordinating producer/consumer deployment order. Anything weaker forces deploy ordering ("producers first" for backward, "consumers first" for forward") — and ordering assumptions break the first time someone deploys out of order.

Deserialization-aware routing and validation

Gateways and meshes that validate `Content-Type` or decode Protobuf for authZ/routing (Envoy `ext_authz`, `grpc_json_transcoder`, JSON Schema validation) are *deserializers*. A gateway that was not redeployed with the new schema may reject new fields it does not recognize — even though the destination service would ignore them safely. Mitigate by: (a) treating gateway validation as part of the compatibility surface and deploying it with the schema, or (b) configuring gateways to `ignore_unknown_fields` and pass through bytes they do not understand.

Caching, storage, and replays

Wire bytes outlive the request that produced them:

- **CDN / gateway cache** (Vol 7, Ch 8) may serve old serialized responses to new clients — cache keys must not assume a single schema version, and `Vary: Accept` / versioned `ETag` prevents serving stale-serialized bytes.
- **Database / object store** holds the *storage schema*, which evolves separately from the wire schema — apply expand-contract to storage (nullable column / dual-read) and wire (new field) in the same window, and do not `DROP COLUMN` / `reserved` until both are migrated.
- **Kafka / event log** holds bytes for months or years. `FULL_TRANSITIVE` is the right default for topics you replay; `BACKWARD` alone lets new consumers break on old events after a new required field is added. Measure replay: a single old event that fails to deserialize on a new consumer halts the partition.

Observability that earns its keep

- **Unknown-field counters.** Protobuf runtimes can log/count unknown fields (or a proxy can). A spike in unknown fields after a deploy confirms the new producer is live before the new consumer — use it as a canary signal.
- **Deserialize-failure rate.** Track separately from business errors — a rise in `InvalidProtocolBufferException` / `JsonMappingException` during a rollout is a compatibility break, not a product bug.
- **Field-presence metrics.** During expand-contract, emit `field_present{field="customer_id_v2"}` — contract only when the old field's presence is zero across all producers for longer than the retention window.

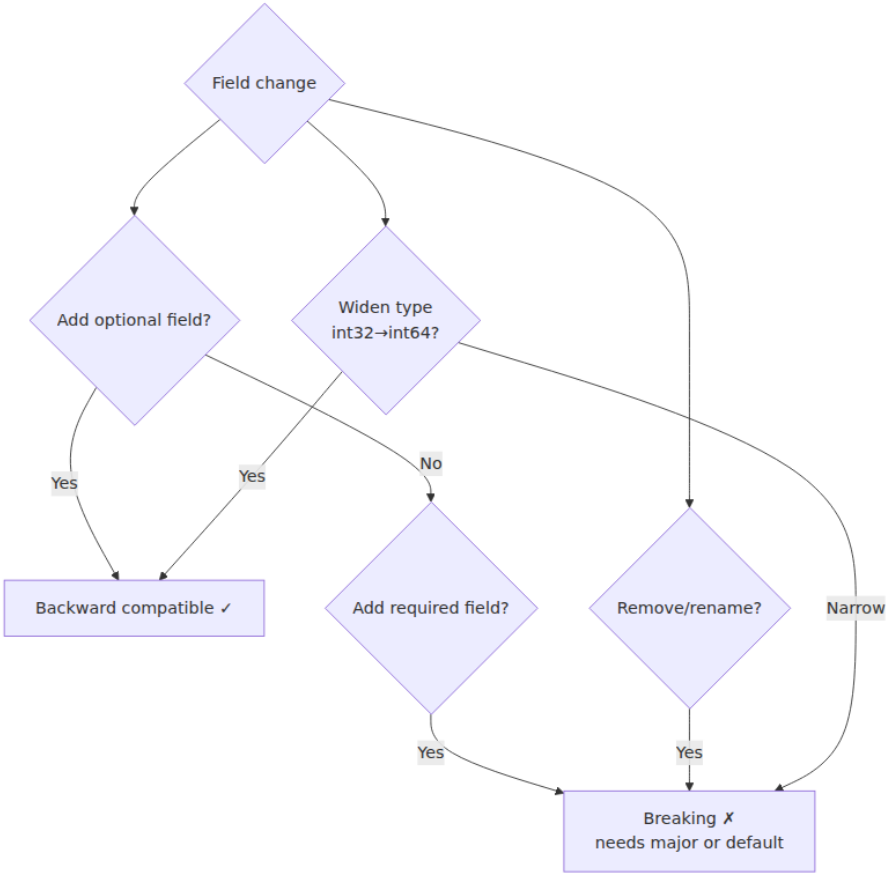
Anti-patterns (and what to do instead)

Anti-pattern	Why it hurts	Fix
Reusing a field number "since the old field is deleted"	Old bytes silently decode as the new field — data corruption	`reserved` every deleted number/name forever
Changing `int32` → `int64` on the same number "because it's wider"	Wire-type compatible but value semantics change; JSON mapping changes; truncation on old readers	New field with new number; expand-contract
`required` in proto2 / "required" by validation on a new field	Old writers omit it; new readers reject every old event — breaks forward compat and replays	All wire fields optional; validation only tightens *after* dual-write window
Schemaless JSON on a durable topic "for flexibility"	No compatibility gate; renames/type changes are invisible until a replay fails months later	Avro/Protobuf + Schema Registry with `FULL_TRANSITIVE`
Hand-editing `buf.yaml` `except:` to silence a breaking error	Bypasses the gate that prevents incidents	`except` only with a `MAJOR` bump + migration guide + `Sunset`

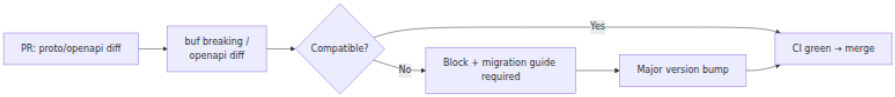
Anti-pattern	Why it hurts	Fix
"Wire and storage schemas are the same thing"	Conflates two evolution surfaces with different lifetimes and rollback windows	Version wire and storage independently; expand-contract both

---

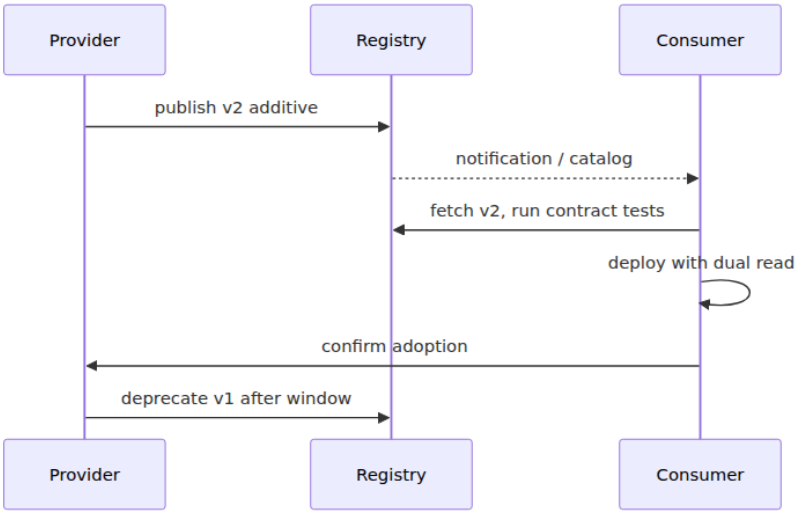
Compatibility Matrix



Breaking Change Detection Pipeline



Consumer Upgrade Safe Path



Key takeaways

- Compatibility is about *mixed versions in flight*, not single-version correctness. Backward (old ← new) lets you roll forward; forward (new ← old) lets you roll back and replay; full (both) lets you do either without deploy ordering.
- Wire format choice determines the compatibility you get for free. Protobuf gives tag-based unknown-field skipping and `reserved` discipline; Avro gives reader/writer schema resolution via the Registry; JSON gives nothing for free and must be gated by `oasdiff` / `openapi-diff` and a Registry.
- Know the Protobuf bytes: `tag = field_number << 3 | wire_type`, varint/ZigZag, length-delimited, 32/64-bit fixed. That encoding is *why* adding a field with a fresh number is safe and reusing a number is silent corruption.

- Protobuf-safe changes are additions with fresh numbers (`optional` / `repeated`), `reserved` on deletion, additive `enum` values with `UNSPECIFIED = 0` , and additive `oneof` cases. Breaking changes are number reuse, wire-type changes, cardinality changes on the same number, `oneof` removal, and `enum` value reuse — all caught by `buf breaking` .
- JSON-safe changes are additive optional fields and `x-` extensions; renames, removals, type changes, and tightening `required` / `enum` / `pattern` are breaking. Gate every OpenAPI change with `oasdiff breaking` .
- For any breaking shape, use expand-contract: expand (add new, keep old, dual-write), migrate (consumers prefer new, dual-read), contract (remove old, `reserved` the number/name, enforce with metrics on field presence). Apply it to wire *and* storage in the same window.
- Enforce mechanically: `buf breaking` vs `main` or BSR in CI (`FILE` for public APIs), `oasdiff breaking --fail-on ERR` for OpenAPI, and Schema Registry `FULL_TRANSITIVE` on Kafka/BSR publish — a breaking schema must not reach the topic, the gateway, or the cache.
- Operate mixed versions deliberately: deploy-order independence requires full compatibility; gate gateways/proxies for `ignore_unknown_fields` so they do not reject new fields they do not understand; track unknown-field counts, deserialize-failure rates, and field-presence metrics during expand-contract; and never `DROP COLUMN` / `reserved` until presence of the old shape is zero for longer than the retention/replay window.

Further reading

- Protocol Buffers — Encoding (varint, wire types, tag) and Language Guide (proto3). https://protobuf.dev/programming-guides/encoding/ / https://protobuf.dev/programming-guides/proto3/
- Protocol Buffers — Field presence and `optional` in proto3. https://protobuf.dev/programming-guides/field_presence/
- Buf — Breaking change detection (`buf breaking` , `WIRE` / `WIRE_JSON` / `FILE`). https://buf.build/docs/breaking/overview
- Buf Schema Registry (BSR) — schema governance and `buf breaking` against BSR. https://buf.build/docs/bsr/overview

- Confluent Schema Registry — compatibility types (`BACKWARD` / `FORWARD` / `FULL` / `TRANSITIVE`). https://docs.confluent.io/platform/current/schema-registry/fundamentals/schema-evolution.html
- Apicurio Registry — compatibility and artifact rules. https://www.apicurio.io/registry/docs/apicurio-registry/3.0.x/getting-started/assembly-configuring-the-registry.html
- `oasdiff` — OpenAPI breaking-change detection. https://github.com/oasdiff/oasdiff
- `openapi-diff` (OpenAPITools) — OpenAPI comparison. https://github.com/OpenAPITools/openapi-diff
- `openapi-compat` / Spectral — OpenAPI linting that catches compatibility-adjacent issues. https://github.com/stoplightio/spectral
- Avro — Specification and schema resolution. https://avro.apache.org/docs/current/specification/
- Thrift — IDL and protocol documentation. https://thrift.apache.org/docs/idl
- FlatBuffers / Cap'n Proto — zero-copy trade-offs and evolution. https://flatbuffers.dev/ / https://capnproto.org/language.html
- AIP-122 — Resource names, AIP-123 — Resource revisions (Google API Improvement Proposals) — expand-contract context. https://google.aip.dev/122 / https://google.aip.dev/123
- ConnectRPC — JSON/Protobuf wire negotiation and `Connect` protocol. https://connectrpc.com/docs/protocol/
- RFC 9457 — Problem Details (error envelopes referenced for compatibility of error `code` s — see Ch 7). https://www.rfc-editor.org/rfc/rfc9457.html
- Transcoding — `google.api.http` and `grpc-gateway` / Envoy `grpc_json_transcoder` (where wire-format negotiation meets the gateway). https://grpc-ecosystem.github.io/grpc-gateway/ / https://

www.envoyproxy.io/docs/envoy/latest/configuration/http/http_filters/grpc_json_transcoder_filter

Chapter 9 — API Governance, Linting, and Breaking-Change Detection

What this chapter covers. At ten services, API consistency is a matter of taste. At three hundred services owned by forty teams deploying fifty times a day, it is a matter of survival. Without governance, every team invents its own pagination, its own error envelope, its own naming convention, and its own definition of "breaking change" — and the cost compounds every time a consumer integrates. This chapter makes governance operational: not a committee that meets quarterly to produce a PDF, but a set of automated gates that make the right thing the easy thing. We define what API governance actually governs (naming, structure, lifecycle, compatibility), compare centralized, federated, and platform-enabled governance models, build a linting pipeline that enforces standards with Spectral and buf lint before a human ever reviews, wire breaking-change detection with `oasdiff`, `openapi-diff`, and `buf breaking` into CI so that no breaking change merges without an explicit major-version bump and migration guide, and show how to measure governance with catalogs and scorecards rather than compliance theatre. Every rule is anchored in runnable, version-pinned config — the same gates you run in CI on Monday morning.

Learning goals — after this chapter you should be able to:

- Compare centralized, federated, and platform-enabled API governance models and choose the right operating model for a given org size and service count — explaining where each model breaks down and how to avoid governance theatre.
- Author and enforce an API style guide as machine-checkable rules: Spectral rulesets for OpenAPI (naming, structure, pagination, error envelope, security) and buf lint rules for Protobuf (package versioning, field naming, `reserved` discipline).
- Classify any OpenAPI or Protobuf change as breaking, non-breaking, or risky — and explain why `required` field additions, type changes, and enum tightening are breaking even when they "look small."

- Wire linting and breaking-change detection into CI as blocking gates: Spectral in GitHub Actions, `buf breaking` against `main` or BSR, `oasdiff breaking` / `openapi-diff` for OpenAPI, with correct `fetch-depth`, baseline handling, and `fail-on` thresholds.
- Design the governance lifecycle (proposal → lint → breaking check → design review → publish → deprecate → sunset) with CODEOWNERS, branch protection, and catalog automation — and measure it with scorecards, not manual audits.
- Reason about governance through the distributed-systems lens: why shared standards are a coordination problem, how to scale review without becoming a bottleneck, and what to do when a governed API is also a wire format on a durable log.

Why governance, and why now

Every distributed system is a socio-technical system. Conway's law is not a metaphor — it is a prediction that, left unguided, your API surface will mirror your org chart's inconsistencies. One team paginates with `?page=2&per_page=50`, another with `?offset=100&limit=25`, a third with cursor `?after=eyJpZCI6MTAwfQ==`. One team returns errors as `{ "error": "not found" }`, another as `{ "code": "RESOURCE_NOT_FOUND", "message": "...", "details": [...] }` per RFC 9457 (see Chapter 7), a third as a bare HTTP status with no body. Consumers that integrate with five services write five adapters. Consumers that integrate with fifty give up and build a brittle hand-rolled client for each.

The cost is not aesthetic. Inconsistent naming forces consumers to memorize per-service conventions. Inconsistent pagination and filtering (see Chapter 6) forces them to reimplement iteration for every resource. Inconsistent error envelopes defeat the retry and circuit-breaker machinery (Chapter 7). Inconsistent versioning (see Chapter 5) means some teams version by URL path, some by header, some not at all — and the gateway must handle all three. And inconsistent evolution discipline means breaking changes ship because no gate caught them — the subject of this chapter's second half.

Governance is the answer, but only if it is automated, measured, and scaled to the org.

> **Distributed-systems lens.** API governance is a coordination problem. At ten services, coordination is cheap (one Slack channel, one staff engineer who reviews every design). At three hundred, the coordination cost of human-only review grows superlinearly. Automated linting and breaking-change gates are the equivalent of consensus for process: they let many teams make local decisions — adding a field, deprecating an endpoint — that are globally consistent without a global meeting. The governance pipeline is to team velocity what a schema registry (Chapter 10) is to data correctness: a shared invariant enforced mechanically.

What governance governs

A common failure mode is governing too little (only naming) or too much (every field's business semantics). The tractable surface is:

Domain	What is governed	Example rule
Naming	Resource, field, enum, path conventions	`snake_case` fields in JSON, `kebab-case` paths, `CamelCase` messages in proto; `GET /v1/orders/{order_id}` not `/getOrders`
Structure	Pagination, filtering, error envelope, idempotency keys	Every collection supports cursor pagination (`page_token` / `page_size`), errors use `application/problem+json` , mutations accept `Idempotency-Key` (Ch 6, 7)
Lifecycle	Versioning, deprecation, sunset, `Sunset` header	`Sunset: Sat, 31 Dec 2026 23:59:59 GMT` + `Deprecation: true` + `Sunset` link relation; removal only after traffic < 1% for one retention window (Ch 5)
Compatibility	What counts as breaking; how it is detected	Adding `required` is breaking; `buf breaking --against main` and `oasdiff breaking --fail-on ERR` block the PR

Domain	What is governed	Example rule
Security	Auth, scopes, rate-limit headers	Every operation declares `security: [bearerAuth: []]` or `apiKey`; `429` + `Retry-After` on rate-limited endpoints (Vol 9)
Documentation	Description, examples, `operationId`, contact	Every operation has `summary`, `description`, `operationId`, at least one `example`; every schema has `description`

What is *not* governed centrally: the domain model within those constraints. The orders team decides whether an order has a `promo_code` field; governance decides that the field is `promo_code` (not `promoCode` or `promo-code`), that it is optional when added, and that removing it follows expand-contract (Chapter 8).

Governance operating models

Centralized — the API platform team

One team owns the style guide, the linting config, and the design-review gate. Every new or changed API requires platform-team approval. Works brilliantly at 5–30 services: standards are coherent, enforcement is simple, drift is near zero. Fails at 100+ services because the platform team becomes a bottleneck and teams learn to route around it — shipping unreviewed APIs behind feature flags and asking forgiveness later.

Federated — guilds and RFCs

Standards are proposed as RFCs, discussed in an API guild (a cross-team community of practice), and adopted by consensus. Each domain team enforces the standard locally. Scales socially — ownership is distributed — but without automated enforcement, adoption is uneven. The team that disagreed with the pagination RFC quietly ignores it.

Platform-enabled — the model that scales

Standards are codified as machine-checkable rules (Spectral, buf lint) and shipped as a shared config package. CI enforces the rules automatically. Human review is reserved for the cases automation cannot judge — novel resource modeling, cross-domain consistency, deprecation planning. The platform team owns the *tooling*, not every decision.

Recommendation for a senior backend engineer: start centralized if you are under ~30 services, federate socially as you grow, and invest in platform-enabled automation the moment the API review queue has a wait time. The automation is not optional at scale — it is the only way to keep review latency flat while service count grows.

The canonical artifact is an **API style guide** — not a wiki page, but a lintable config. The next two sections build it.

Linting — making the style guide executable

Linting is the cheapest governance gate. It runs in under a second, requires no human, and catches the violations that would otherwise consume design-review time.

Spectral — OpenAPI linting (Stoplight Spectral 6.11+)

Spectral is the standard OpenAPI linter. It evaluates an OpenAPI document against a ruleset — built-in rules (`spectral:oas`) plus your org's extensions — and reports errors, warnings, and info.

```yaml
# .spectral.yaml — Acme API style guide (Spectral 6.11+, OpenAPI 3.1)
# Install: npm i -D @stoplight/spectral-cli@6.11.0 @stoplight/spectral-rulesets
extends: spectral:oas   # base OAS 3.x rules (valid schema, resolvable refs, etc.)

# Severity: error = blocks CI, warn = visible but non-blocking, off = disabled
rules:
  # ---- Naming ----
  path-kebab-case:
    description: "Paths MUST be kebab-case and lowercase"
    severity: error
    given: "$.paths[*]~"          # every path key
    then:
      function: pattern
      functionOptions:
        match: "^/v[0-9]+/[a-z0-9-]+(/[a-z0-9-]+|/\\{[a-z_]+\\})*$"

  operation-operationId:
    description: "Every operation MUST have an operationId (for codegen — Ch 10)"
    severity: error
    given: "$.paths[*][*]"
    then:
      field: operationId
      function: truthy

  operation-summary:
    description: "Every operation SHOULD have a summary"
    severity: warn
    given: "$.paths[*][*]"
    then:
      field: summary
      function: truthy

  field-snake-case:
    description: "JSON field names MUST be snake_case"
    severity: error
    given: "$.components.schemas[*].properties[*]~"
    then:
      function: pattern
      functionOptions:
        match: "^[a-z][a-z0-9_]*$"

  enum PascalCase-or-SCREAMING:
    description: "Enum values SHOULD be SCREAMING_SNAKE_CASE"
    severity: warn
    given: "$.components.schemas[*].properties[?(@.enum)]"
    then:
```

```
      field: enum
      function: schema
      functionOptions:
        schema:
          type: array
          items:
            type: string
            pattern: "^[A-Z][A-Z0-9_]*$"

  # ---- Structure ----
  pagination-required:
    description: "Collection GET MUST support cursor pagination (page_t
oken + page_size)"
    severity: error
    given: "$.paths[?(@property.match(/^\\/.*/) )].get"
    then:
      function: schema
      functionOptions:
        schema:
          type: object
          required: [parameters]
          properties:
            parameters:
              type: array
              contains:
                type: object
                properties:
                  name: { const: page_token }
                  in: { const: query }

  error-envelope:
    description: "Error responses SHOULD use application/problem+json (
RFC 9457, Ch 7)"
    severity: warn
    given: "$.paths[*][*].responses[?(@property >= '400')].content"
    then:
      field: "application/problem+json"
      function: truthy

  idempotency-key:
    description: "Non-idempotent mutations (POST) SHOULD accept Idempot
ency-Key"
    severity: warn
    given: "$.paths[*].post.parameters"
    then:
      function: schema
      functionOptions:
        schema:
          type: array
          contains:
            type: object
```

```
            properties:
              name: { const: Idempotency-Key }
              in: { const: header }

  # ---- Lifecycle ----
  sunset-header:
    description: "Deprecated operations MUST include Sunset header and
deprecation docs"
    severity: error
    given: "$.paths[*][*][?(@.deprecated == true)]"
    then:
      field: description
      function: pattern
      functionOptions:
        match: "(?i)sunset"

  # ---- Security ----
  operation-security:
    description: "Every operation MUST declare security (or explicitly
security: [])"
    severity: error
    given: "$.paths[*][*]"
    then:
      field: security
      function: truthy

  no-http-basic:
    description: "HTTP Basic auth MUST NOT be used"
    severity: error
    given: "$.components.securitySchemes[*]"
    then:
      field: type
      function: pattern
      functionOptions:
        notMatch: "http"
        # allow bearer/JWT, apiKey, oauth2, openIdConnect

  # ---- Documentation ----
  schema-description:
    description: "Every schema SHOULD have a description"
    severity: warn
    given: "$.components.schemas[*]"
    then:
      field: description
      function: truthy

  tag-description:
    description: "Every tag SHOULD have a description"
    severity: warn
    given: "$.tags[*]"
    then:
```

```
      field: description
      function: truthy
```

```
# Run locally (Spectral 6.11+, Node 20+)
npx @stoplight/spectral-cli@6.11.0 lint openapi.yaml --ruleset .spectra
l.yaml --format stylish

# Example output — two errors, one warning:
# openapi.yaml:42:7  error  path-kebab-case     Path "/v1/getOrders"
must be kebab-case
# openapi.yaml:87:11 error  field-snake-case    Property "promoCode"
must be snake_case
# openapi.yaml:103:9 warn   error-envelope      Error responses should
use application/problem+json
```

Distributing the ruleset. Publish it as a versioned npm package so every repo consumes the same rules without copy-paste drift:

```
// packages/acme-spectral-ruleset/package.json
{
  "name": "@acme/spectral-ruleset",
  "version": "2.4.0",
  "main": "ruleset.yaml",
  "publishConfig": { "registry": "https://registry.acme.internal" }
}
```

```
# Consumer repo — .spectral.yaml
extends: "@acme/spectral-ruleset"
# Per-repo overrides only with a comment and a linked RFC:
rules:
  pagination-required: off  # webhook collection — not paginated by
design (RFC-042)
```

buf lint — Protobuf linting (buf 1.40+)

For gRPC/Protobuf surfaces (Chapter 3), `buf lint` is the equivalent. It checks package structure, naming, field discipline, and breaking-change-adjacent rules (like `reserved` hygiene).

```yaml
# buf.yaml — module at api/orders (buf 1.40+, protobuf 5.x)
# Docs: https://buf.build/docs/configuration/v2/buf-yaml
version: v2
modules:
  - path: proto
lint:
  use:
    - DEFAULT              # sensible defaults (field naming, imports,
package)
    - PACKAGE_VERSION_SUFFIX  # require version suffix: api.orders.v1
  except: []               # never except without a // buf:lint:ignore
+ justification
  enum_zero_value_suffix: _UNSPECIFIED
  service_suffix: Service
  disallow_comment_ignores: false

breaking:
  use:
    - FILE                 # strictest: any file-level break is an
error
    # Alternatives: WIRE (wire-compat only), WIRE_JSON (+ JSON name),
PACKAGE
```

```
buf lint proto
# proto/acme/orders/v1/order.proto:14:10: Field name "promoCode" should
be lower_snake_case.
# proto/acme/orders/v1/order.proto:22:3: Enum zero value name "UNKNOWN"
should be suffixed with "_UNSPECIFIED".
```

Key rules that earn their keep:

Rule	What it catches	Why it matters
`PACKAGE_VERSION_SUFFIX`	`package acme.orders` without `.v1`	Versions are a directory — omitting them makes evolution impossible (Ch 5)
`FIELD_LOWER_SNAKE_CASE`	`promoCode` in proto (JSON name is auto `promoCode`)	Proto fields are `snake_case`; JSON mapping is handled by `protojson` — mixing them breaks codegen (Ch 10)

Rule	What it catches	Why it matters
`ENUM_ZERO_VALUE_SUFFIX`	`UNKNOWN = 0` vs `STATUS_UNSPECIFIED = 0`	Zero value is the default on missing field — it must be explicit (Ch 3, Ch 8)
`ENUM_FIRST_VALUE_ZERO`	`PENDING = 1` as first value	Proto3 enums must start at 0 — otherwise default is undefined
`IMPORT_USED` / `PACKAGE_SAME_DIRECTORY`	Unused imports, mixed packages per directory	Keeps the module graph clean for `buf breaking` and BSR

Breaking-change detection — failing the build before the incident

Linting catches style drift. Breaking-change detection catches *semantic* drift — the changes that are syntactically valid, pass every unit test, and break a consumer in production.

What is breaking — a precise taxonomy

Change	OpenAPI / JSON	Protobuf	Verdict
Add optional field	Non-breaking	Non-breaking (fresh number, `optional` / `repeated`)	Safe — old readers ignore
Add `required` field	**Breaking** — old writers omit it	**Breaking** if validated as required (proto3 fields are optional on wire)	Block unless major bump
Remove field (even optional)	**Breaking** if any client sends it	**Breaking** — must `reserved` number/ name	Expand-contract (Ch 8)

Change	OpenAPI / JSON	Protobuf	Verdict
Rename field	**Breaking** — old name disappears	**Breaking** — field number is identity, but JSON name change breaks `protojson`	Expand-contract
Change type (`string` → `number`, `int32` → `string`)	**Breaking**	**Breaking** — wire-type change	New field, new number
Tighten `enum` (remove value)	**Breaking** — old writer emits removed value	**Breaking** — `UNRECOGNIZED` handling required	Expand-contract or alias
Widen `enum` (add value)	Risky — old reader lacks default branch	Conditionally safe — if consumers handle `UNRECOGNIZED`	Warn — document handling
Tighten constraint (`maxLength`, `pattern`, `minimum`)	**Breaking** — old valid values now invalid	**Breaking** by validation (wire-compatible but semantically breaking)	Validate only after dual-write window
Remove `enum` value / `oneof` case	**Breaking**	**Breaking**	`reserved`
Change `optional` → `repeated` on same number	—	**Breaking** — cardinality change	New field
Add `x-` extension / additive header	Non-breaking	—	Safe

The critical insight: **wire-compatible is not semantically compatible**. A field can be wire-compatible (old bytes still parse) but semantically breaking (old values now fail validation). Breaking-change tools catch the structural half; validation-semantics review catches the other half —

which is why the governance flow has both an automated gate and a human review.

OpenAPI — `oasdiff` and `openapi-diff`

Two mature tools cover OpenAPI diffing. `oasdiff` (Go, `oasdiff/oasdiff` 1.10+) is the more granular; `openapi-diff` (Java, `OpenAPITools/openapi-diff` 2.1+) integrates with the Redocly/Java ecosystem. Use one.

```
# oasdiff 1.10+ — compare PR branch against main (Go 1.22+)
# Install: go install github.com/oasdiff/oasdiff@latest
# Or via Docker: docker run --rm -v $PWD:/specs oasdiff/oasdiff
breaking ...

# 1) Changelog — human-readable diff of every change (informational)
oasdiff changelog \
  https://raw.githubusercontent.com/acme/orders/main/openapi.yaml \
  ./openapi.yaml \
  --format json | jq .

# 2) Breaking gate — exits non-zero on ERR, zero on WARN/INFO
oasdiff breaking \
  https://raw.githubusercontent.com/acme/orders/main/openapi.yaml \
  ./openapi.yaml \
  --fail-on ERR \
  --composed          # also descend into allOf/anyOf/oneOf

# 3) With a local baseline (avoids network in CI — recommended)
git show origin/main:openapi.yaml > /tmp/baseline.yaml
oasdiff breaking /tmp/baseline.yaml ./openapi.yaml --fail-on ERR --
composed

# Example ERR output (blocks CI):
# breaking: added required field 'customer_id_v2' to schema Order — ERR
# breaking: removed enum value 'PENDING' from OrderStatus — ERR
# breaking: changed type of field 'quantity' from integer to string —
ERR
```

```
# Alternative: openapi-diff 2.1+ (Java 17+, Redocly ecosystem)
# Install: via Maven/Gradle or download the fat JAR
java -jar openapi-diff.jar \
  --fail-on-incompatible \
  /tmp/baseline.yaml ./openapi.yaml

# Output: INCOMPATIBLE — REST API backward compatibility broken
#   - Missing property: legacy_discount_code (removed)
#   - Changed required: customer_id is now required
```

`oasdiff` **stability levels** (what `--fail-on` thresholds mean):

Level	Meaning	CI action
`ERR`	Breaking — consumer that worked before will break	Block PR
`WARN`	Potentially breaking / risky — widened enum, new optional field that changes semantics	Surface as PR comment, do not block
`INFO`	Non-breaking additive change	Informational

Protobuf — `buf breaking` (buf 1.40+, BSR)

`buf breaking` compares the current module against a baseline — `main` on Git, or a published version on the Buf Schema Registry (BSR).

```
# buf.yaml — breaking config (continuing from lint above)
breaking:
  use: [FILE]       # strictest — any file-level incompatibility is an
error
  # WIRE      — only wire-compat breaks (reuse number, change wire
type)
  # WIRE_JSON — WIRE + JSON name changes (proto field → JSON key)
  # FILE      — WIRE_JSON + file-level (package, import, option
changes)
  # PACKAGE   — FILE + package-level (most strict — any cross-file
break)
  except: []
```

```
# Against git — most common in CI
buf breaking proto --against '.git#branch=main'

# Against BSR — when main has already published to the registry
buf breaking proto --against 'buf.build/acme/orders:main'

# Example output — blocks CI:
# proto/acme/orders/v1/order.proto:8:3: Field "2" on message "Order"
changed type
#   from "int32" to "string".
# proto/acme/orders/v1/order.proto:14:3: Field "4" on message "Order"
changed name
#   from "promo_code" to "promoCode".
# Failure: 2 breaking changes detected.
```

Choosing the baseline. In a trunk-based repo, `--against '.git#branch=main'` is correct. In a registry-centric workflow (many repos consuming the same proto), `--against 'buf.build/acme/orders:main'` is better — it compares against the *published* schema, not just the git branch, and catches the case where two concurrent PRs each add field number `7`.

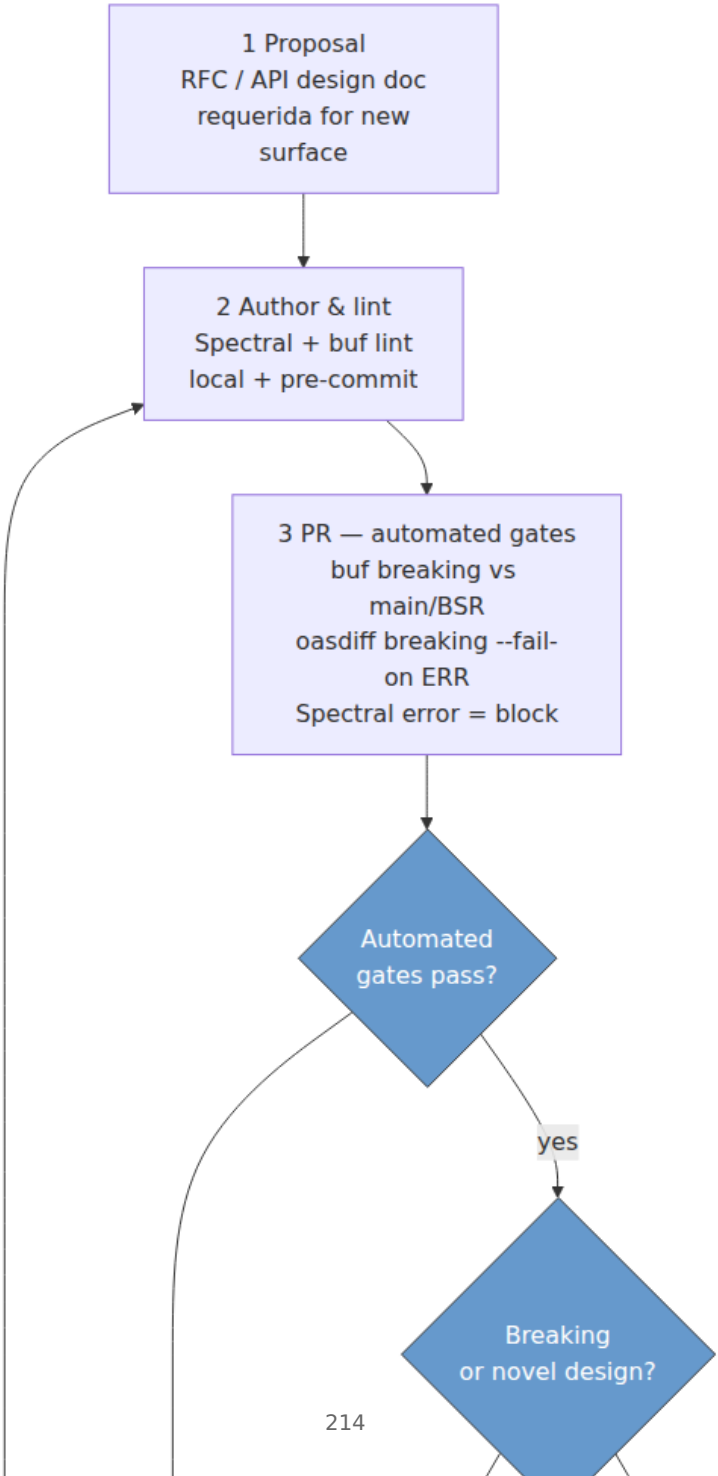
Complementary — Optic, Bump.sh, Swagger Diff

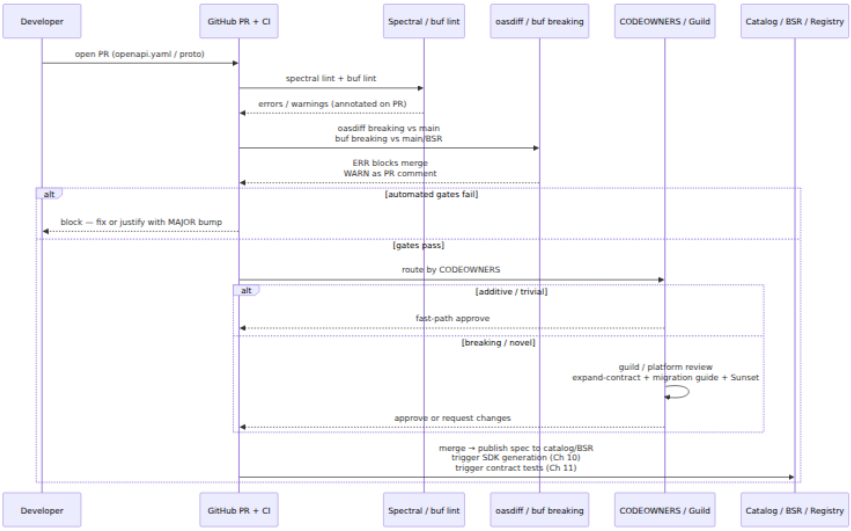
Tool	Surface	Strength	When to use
Optic (`useoptic/optic` 0.40+)	OpenAPI + live traffic	Compares spec against *observed* traffic — finds undocumented endpoints and drift	Alongside `oasdiff` when you want traffic-aware governance
Bump.sh	OpenAPI / AsyncAPI	Hosted diff + changelog + API catalog with breaking labels	When you want a hosted catalog + CI gate in one
`swagger-diff` (JS)	OpenAPI 2.0/3.x	Lightweight JS diff	Legacy or JS-native CI where Go/Java tooling is heavy

---

The governance pipeline — from proposal to sunset

Linting and breaking-change detection are gates. The lifecycle that connects them is:


1 Proposal
RFC / API design doc
requerida for new
surface
2 Author & lint
Spectral + buf lint
local + pre-commit
3 PR — automated gates
buf breaking vs
main/BSR
oasdiff breaking --fail-
on ERR
Spectral error = block
Automated
gates pass?
yes
Breaking
or novel design?



The `Sunset` / `Deprecation` lifecycle in HTTP headers (RFC 8594 + `draft-ietf-httpapi-deprecation-header`):

```
HTTP/1.1 200 OK
Deprecation: true
Sunset: Sat, 31 Dec 2026 23:59:59 GMT
Link: <https://docs.acme.internal/deprecations/orders-v1-legacy-
discount>; rel="sunset"
Link: <https://docs.acme.internal/migrations/orders-v1-to-v2>;
rel="deprecation"
Warning: 299 - "legacy_discount_code is deprecated, use promo_code
(field 4). Sunset 2026-12-31."

{
  "data": { "id": "ord_123", "promo_code": "SAVE20" },
  "meta": { "deprecation": "legacy_discount_code is deprecated —
migrate to promo_code by 2026-12-31" }
}
```

CI — the gates as code

```yaml
# .github/workflows/api-governance.yaml — Spectral + oasdiff + buf
(Node 20, Go 1.22, buf 1.40)
name: api-governance
on:
  pull_request:
    paths: ["openapi.yaml", "openapi/**", "proto/**", "buf.yaml", ".spe
ctral.yaml"]

jobs:
  spectral:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - name: Spectral lint (blocking on error)
        run: npx @stoplight/spectral-cli@6.11.0 lint openapi.yaml --
ruleset .spectral.yaml --format github

  oasdiff:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }  # full history — baseline is origin/
main
      - uses: actions/setup-go@v5
        with: { go-version: "1.22" }
      - run: go install github.com/oasdiff/oasdiff@latest
      - name: Baseline from main
        run: git show origin/main:openapi.yaml > /tmp/baseline.yaml
      - name: Breaking-change gate (fail on ERR)
        run: oasdiff breaking /tmp/baseline.yaml ./openapi.yaml --fail-
on ERR --composed
      - name: Changelog comment (informational)
        if: always()
        run: oasdiff changelog /tmp/baseline.yaml ./openapi.yaml --
format markdown > /tmp/changelog.md
      - uses: marocchino/sticky-pull-request-comment@v2
        if: always()
        with:
          path: /tmp/changelog.md

  buf:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - uses: bufbuild/buf-action@v1
        with:
```

```
          setup_only: true
      - run: buf lint proto
      - run: buf breaking proto --against '.git#branch=origin/main'
      # Alternative baseline — BSR (uncomment when publishing to BSR):
      # - run: buf breaking proto --against 'buf.build/acme/
orders:main'


# Optional — Optic traffic-aware check (when running against observed
traffic)
  # optic:
  #   runs-on: ubuntu-latest
  #   steps:
  #     - uses: actions/checkout@v4
  #     - run: npx @useoptic/optic@0.40.0 diff openapi.yaml --base
origin/main:openapi.yaml --check
```

Key CI details that prevent false confidence:

- `fetch-depth: 0` — without full history, `git show origin/main:openapi.yaml` and `buf breaking --against '.git#branch=main'` have no baseline. The default `fetch-depth: 1` silently makes every PR look non-breaking.
- `oasdiff breaking --fail-on ERR` — failing on `WARN` is too noisy (every additive field would block). Fail on `ERR` (structural breaks), surface `WARN` as a PR comment for reviewer judgment.
- `buf breaking` baseline choice — `.git#branch=origin/main` for trunk-based repos, `buf.build/acme/orders:main` for registry-centric workflows where concurrent PRs could collide on field numbers.
- `Spectral --format github` — annotates violations directly on the PR diff, so the developer fixes in context rather than hunting logs.

Pre-commit — shift left

```yaml
# .pre-commit-config.yaml — runs before push, same rules as CI
repos:
  - repo: https://github.com/stoplightio/spectral
    rev: v6.11.0
    hooks:
      - id: spectral
        entry: npx @stoplight/spectral-cli@6.11.0 lint
        args: [openapi.yaml, --ruleset, .spectral.yaml, --fail-
severity, error]
        files: openapi\.yaml$

  - repo: https://github.com/bufbuild/buf
    rev: v1.40.0
    hooks:
      - id: buf-lint
        entry: buf lint proto
        files: proto/.*\.proto$
```

CODEOWNERS and branch protection

```
# .github/CODEOWNERS — route API changes to the right reviewers
openapi.yaml              @acme/api-platform @acme/orders-owners
openapi/**                @acme/api-platform @acme/orders-owners
proto/                    @acme/api-platform @acme/orders-owners
buf.yaml                  @acme/api-platform
.spectral.yaml            @acme/api-platform
packages/acme-spectral-ruleset/  @acme/api-platform
```

Branch protection: require `api-governance` (all three jobs) to pass, require CODEOWNERS review on governed paths, and require linear history so the baseline is unambiguous. A breaking change that bumps `MAJOR` (Chapter 5) additionally requires platform-team approval — encode that as a separate required check that only the platform team can satisfy.

Catalogs and scorecards — measuring governance

Governance without measurement is theatre. The catalog is the source of truth; the scorecard is the feedback loop.

API catalog — a searchable inventory of every API surface, owner, lifecycle stage, and spec location. At minimum: Backstage (Spotify, OSS),

Bump.sh, or a homegrown registry backed by `openapi.yaml` / `buf.lock` discovery. Each entry links to the spec, the lint score, the last breaking-change result, the deprecation schedule, and the SDK (Chapter 10).

Scorecard — a per-service, per-team grade that rolls up the gates:

Dimension	Metric	Source
Lint compliance	Spectral `error` count = 0, `warn` count trending down	Spectral CI
Breaking-change hygiene	Breaking changes only with `MAJOR` bump + migration guide	`oasdiff` / `buf breaking` + changelog
Documentation	Every operation has `summary` / `description` / `operationId` + example	Spectral `operation-summary`, `schema-description`
Lifecycle	Deprecated surfaces have `Sunset` + `Link: rel=sunset` + traffic < threshold	Catalog + gateway metrics
Security	Every operation declares `security`, no `http: basic`, scopes reviewed	Spectral `operation-security`, `no-http-basic`
Adoption	SDK version adoption, consumer count, field-presence during expand-contract	Registry + gateway metrics (see Ch 8, Ch 10)

Scorecards work when they are *visible* — a dashboard that teams check, not a quarterly report. The best implementations post the scorecard as a PR comment or Slack summary on every merge, so the feedback is immediate. At scale, scorecard trends (not point-in-time grades) drive investment: a team whose `warn` count is climbing gets a nudge before it becomes a migration.

The distributed-systems lens — governance at scale

Review as a throughput problem

Human design review is a single-threaded resource. If every PR with a spec change requires a guild review, review latency grows with PR rate,

and teams learn to batch spec changes into large, hard-to-review PRs — the opposite of what you want. The mitigation is the **fast path**: additive, lint-clean, non-breaking changes (the majority) auto-pass the automated gates and require only CODEOWNERS approval. Only breaking or novel changes enter the guild queue. Measure the queue — if median time-to-approval exceeds a day, the fast path is too narrow.

The durable-log problem

When an API's wire format is also the storage or log format (Kafka topic, object store, database column — see Chapter 8 and Volume 10), governance has a longer tail. A breaking schema change that is safe for request/response (old consumers ignore the new field) may be breaking for the log (new consumers replay old bytes that lack the new `required` field). The governance gate for log-backed schemas must enforce `FULL` compatibility (Confluent `FULL_TRANSITIVE`, `buf breaking --against BSR` with `FILE`), not just backward compatibility. The catalog should mark log-backed schemas explicitly so reviewers apply the stricter check.

Versioning the governance itself

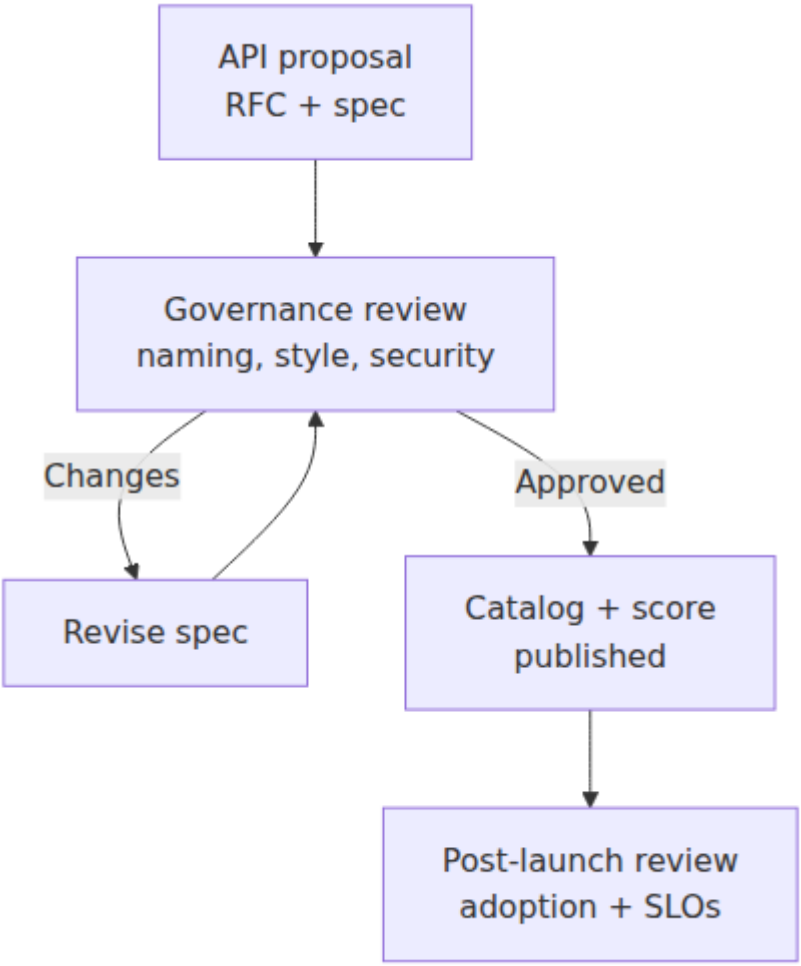
The Spectral ruleset and `buf.yaml` are versioned artifacts. A rule change (e.g., tightening `field-snake-case` or enabling `PACKAGE_VERSION_SUFFIX`) is itself a breaking change for consumers of the ruleset. Version the ruleset package (`@acme/spectral-ruleset@3.0.0`), publish a changelog, and give teams a migration window. Do not push a new `error`-severity rule without a `warn`-first deprecation window — or you will block every open PR simultaneously.

Anti-patterns (and what to do instead)

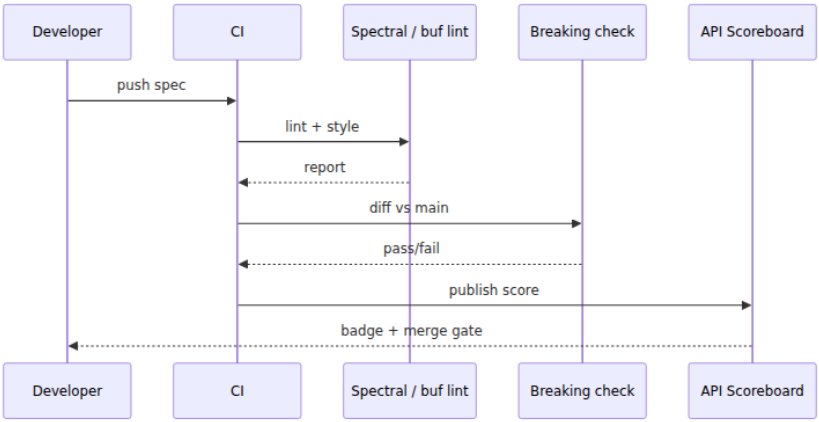
Anti-pattern	Why it hurts	Fix
Handbook-only governance — a PDF no CI enforces	Drifts immediately; the team that disagrees quietly ignores it	Codify every rule as a Spectral / buf lint rule; CI blocks on `error`

Anti-pattern	Why it hurts	Fix
`except:` / `x-spectral-ignore` without a linked RFC	Silences the gate that prevents incidents; spreads via copy-paste	`except` only with a `// reason: RFC-042` comment and a `warn`-first window
`fetch-depth: 1` in the breaking-change job	Baseline is empty — every PR looks non-breaking	`fetch-depth: 0` and `git show origin/main:openapi.yaml`
Failing CI on `WARN` (e.g., every additive field)	Noisy gate that teams learn to ignore or bypass	`--fail-on ERR` for blocking; `WARN` as a PR comment for reviewer judgment
One global review queue for every spec change	Bottleneck; teams batch changes into unreviewable PRs	Fast path for additive + lint-clean + non-breaking; guild only for breaking/novel
Governing business semantics centrally	Platform team cannot judge every domain's field meaning; review stalls	Govern structure/lifecycle/compat centrally; domain semantics stay with the owning team
Versioning the ruleset without a changelog	Teams blindsided by new `error` rules that block open PRs	Versioned package + changelog + `warn` window before promoting to `error`

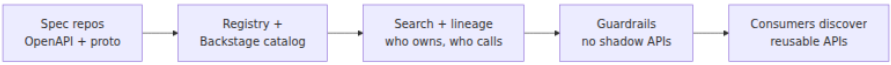
Governance Council Flow



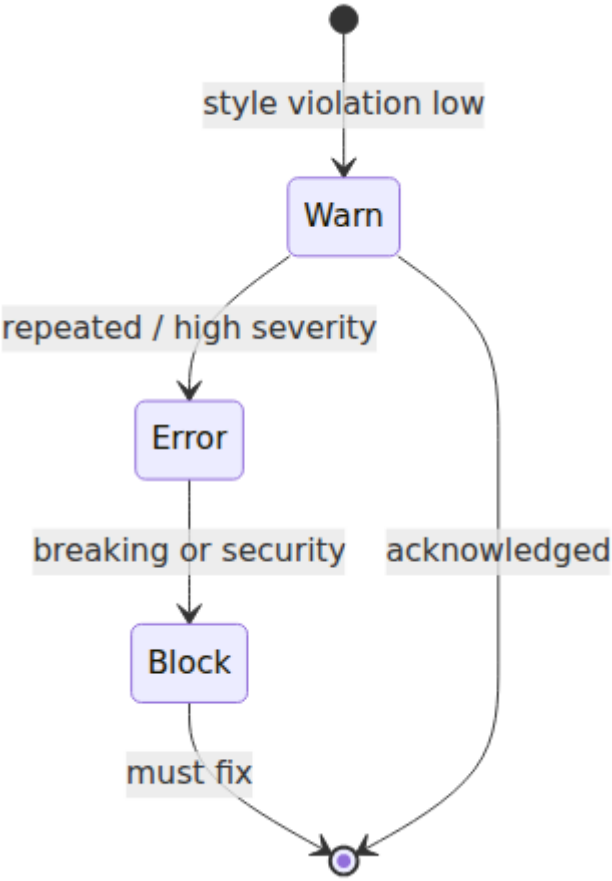
CI Governance Pipeline



Catalog Discovery



Style Guide Enforcement Levels



Key takeaways

- Governance at scale is platform-enabled, not committee-driven. Codify standards as machine-checkable rules (Spectral, buf lint), ship them as versioned packages, enforce them in CI, and reserve human review for breaking or novel changes — the fast path keeps review latency flat as service count grows.
- What you govern is naming, structure, lifecycle, compatibility, security, and documentation — not every domain field's business semantics. Keep the boundary crisp so the platform team owns the invariant and domain teams own the model.

- Linting is the cheapest gate. Spectral (`spectral:oas` + org extensions) catches naming, pagination, error-envelope, and security drift in under a second; `buf lint` (`DEFAULT` + `PACKAGE_VERSION_SUFFIX` + `ENUM_ZERO_VALUE_SUFFIX`) does the same for Protobuf — both run locally, pre-commit, and in CI.
- Breaking-change detection is the gate that prevents incidents. Classify every change (add optional = safe, add `required` / remove / rename / type change / tighten = breaking) and enforce with `oasdiff breaking --fail-on ERR --composed` for OpenAPI and `buf breaking --against '.git#branch=main'` (or BSR) for Protobuf — wire-compatible is not semantically compatible.
- Wire the gates correctly: `fetch-depth: 0` so the baseline exists, `--fail-on ERR` so the gate is not noisy, BSR baseline when concurrent PRs could collide on field numbers, and `Spectral --format github` so violations annotate the PR diff.
- Measure with catalogs and scorecards (lint compliance, breaking hygiene, docs, lifecycle, security, adoption) — posted on every PR, trended over time, not as a quarterly compliance report. Scorecard trends drive investment before drift becomes a migration.
- For log-backed schemas (Kafka, object store), enforce `FULL` / `FULL_TRANSITIVE` compatibility — request/response backward-compat is not enough when old bytes are replayed. Mark log-backed schemas in the catalog so reviewers apply the stricter check.
- Version the governance itself. The Spectral ruleset and `buf.yaml` are versioned artifacts — promote new `error` rules via a `warn` window with a changelog, or you will block every open PR at once.

Further reading

- Spectral — OpenAPI linting, custom rules, `spectral:oas` ruleset. https://docs.spectral.sh/ / https://github.com/stoplightio/spectral
- Buf — Lint (`buf lint`) and breaking-change detection (`buf breaking`), `WIRE` / `WIRE_JSON` / `FILE` / `PACKAGE` . https://buf.build/docs/lint/overview / https://buf.build/docs/breaking/overview
- Buf Schema Registry (BSR) — `buf breaking --against buf.build/...` , published-schema baseline. https://buf.build/docs/bsr/overview

- oasdiff — OpenAPI breaking-change detection (`breaking`, `changelog`, `diff`), `ERR` / `WARN` / `INFO` levels, `--composed`. https://github.com/oasdiff/oasdiff
- openapi-diff (OpenAPITools) — OpenAPI comparison, `--fail-on-incompatible`. https://github.com/OpenAPITools/openapi-diff
- Optic — Traffic-aware OpenAPI diff and governance (`optic diff --check`). https://www.useoptic.com/docs / https://github.com/useoptic/optic
- Bump.sh — Hosted API catalog, diff, and breaking-change labels. https://bump.sh/ / https://docs.bump.sh/
- Spectral + GitHub Actions — CI patterns for OpenAPI governance. https://docs.spectral.sh/guides/github-actions
- RFC 8594 — The Sunset HTTP Header Field. https://www.rfc-editor.org/rfc/rfc8594.html
- draft-ietf-httpapi-deprecation-header — The Deprecation HTTP Header Field. https://www.ietf.org/archive/id/draft-ietf-httpapi-deprecation-header-02.html
- Google API Improvement Proposals — AIP-122 (Resource names), AIP-128 (Pagination), AIP-193 (Errors), AIP-162 (Deprecation). https://google.aip.dev/
- Zalando RESTful API Guidelines / Microsoft API Guidelines — mature org-level style guides to mine for rules. https://opensource.zalando.com/restful-api-guidelines/ / https://github.com/microsoft/api-guidelines
- Backstage — API catalog and scorecards (Spotify, OSS). https://backstage.io/docs/features/software-catalog/ / https://backstage.io/docs/features/techdocs/

Chapter 10 — Schema Registry, Code Generation, and SDK Delivery

What this chapter covers. A schema that lives only in a Git repo is a promise without enforcement. A schema that lives in a registry — versioned, compatibility-checked, discoverable, and code-generated — is an invariant that every producer, consumer, gateway, and code-generated

client respects. This chapter closes the loop that Chapters 8 and 9 opened: Chapter 8 defined what compatibility *means* at the byte level, Chapter 9 built the CI gates that *detect* violations — this chapter builds the infrastructure that *prevents* them from being published and the delivery machinery that turns a published schema into type-safe clients in every language your consumers use. We dissect schema registries for the three surfaces you run — HTTP/REST (OpenAPI registries), gRPC/Protobuf (Buf Schema Registry), and event streaming (Confluent/Apicurio/Karica for Kafka) — their subject naming, compatibility enforcement, and evolution mechanics; build a code-generation pipeline with `buf generate` and OpenAPI Generator that produces idiomatic, versioned clients; and design SDK delivery as a product — versioned packages, changelogs, deprecation, and the CI/CD that publishes to npm/Maven Central/PyPI/Go without drift. Every pattern is anchored in runnable, version-pinned config and viewed through the distributed-systems lens where the registry is the single source of truth that keeps hundreds of services and clients consistent without a global lockstep deploy.

Learning goals — after this chapter you should be able to:

- Compare Confluent Schema Registry, Apicurio Registry, and Buf Schema Registry (BSR) on data model (subjects/artifacts), compatibility modes, storage, and when to use each — and sketch a subject-naming scheme for a multi-team Kafka + gRPC + REST estate.
- Configure compatibility enforcement (`BACKWARD` / `FORWARD` / `FULL` / `TRANSITIVE`) on Confluent/Apicurio and `buf breaking` on BSR, and explain why `FULL_TRANSITIVE` is the right default for topics you replay and `BACKWARD` for request/response.
- Wire a schema registry into producers and consumers: Confluent serializers (`KafkaAvroSerializer` / `ProtobufSerializer`), `buf` publishing (`buf push`), and Apicurio's Kafka SerDes — including schema references and canonical JSON for OpenAPI.
- Design a code-generation pipeline with `buf generate` (Protobuf → Go/Java/TypeScript/Python) and OpenAPI Generator (OpenAPI → typed clients), including `buf.gen.yaml` and `openapi-generator` config, templates, and CI wiring — and explain where codegen breaks down.

- Own SDK delivery end-to-end: versioning that tracks the API version (SemVer + API major), publishing to npm/Maven Central/PyPI/Go modules, changelog and migration-guide generation, and the GitHub Actions workflow that publishes on every spec merge without manual steps.
- Reason about the distributed-systems trade-offs: registry availability as a control-plane dependency, schema caching and stale-read handling, codegen staleness versus hand-written clients, and the catalog as the discovery surface that prevents shadow APIs.

Why a registry — from file to contract

In Chapter 9, the schema lived in Git: `openapi.yaml` and `proto/` at the head of `main`. That works for a single team. At fleet scale it has three failure modes:

1. **No single source of truth.** Two teams each vendor a copy of `orders.proto` at different commits. One adds field number `7`, the other also adds field number `7` with a different type. Both pass `buf breaking` against their own `main`, both merge, and the wire corrupts — the classic concurrent-evolution race that Git alone cannot catch.
2. **No publish-time enforcement.** A producer that embeds an Avro schema per message (or a JSON payload with no registry) can publish bytes that no consumer can deserialize. The break is discovered at consumption time — possibly during a replay months later.
3. **No discovery.** A new team that needs to call the orders service has no way to find the canonical spec, the supported versions, the deprecation schedule, or the generated client. They reverse-engineer the API from traffic or copy a stale example.

A schema registry solves all three by making the schema a *versioned, addressable, compatibility-checked artifact* — not just a file. Publishing is an API call (`POST /subjects/orders-value/versions` / `buf push`) that either succeeds (compatible) or fails (breaking) — before any bytes hit the wire. Consumers resolve schemas by ID, not by vendored copy. The catalog is the directory.

> **Distributed-systems lens.** The registry is *control plane*, not data plane. Producers and consumers cache schemas locally and fetch by ID; the

> registry does not sit on the hot path of every message. But its availability matters: if the registry is down, producers cannot publish *new* schemas (safe failure — they keep producing with the last schema), and consumers with cold caches cannot deserialize (mitigated by bundling schemas or by a registry-backed cache like Karapace/Apicurio's read-through). Treat the registry like etcd/ZooKeeper (Vol 6, Ch 8) — a strongly consistent coordination service whose writes are gated and whose reads are cached.

Schema registries compared

Capability	Confluent Schema Registry (CSR) 7.6+	Apicurio Registry 3.0+	Buf Schema Registry (BSR)	Kara Red Reg
Primary surface	Avro / Protobuf / JSON Schema on Kafka	Avro / Protobuf / JSON Schema / OpenAPI / AsyncAPI on Kafka + HTTP	Protobuf (Buf modules) for gRPC / Connect	Avro Prot JSO Sche CSR com API
Data model	*Subjects* (`topic-value` , `topic-key`) + versions + global/ compat config	*Artifacts* + groups + versions + rules; artifact types include `AVRO` , `PROTOBUF` , `OPENAPI` , `ASYNCAPI`	*Modules* (`buf.build/acme/orders`) + commits + tracks (`main`)	Subj (CSF com
Compatibility at publish	`BACKWARD` / `FORWARD` / `FULL` / `BACKWARD_TRANSITIVE` / `FORWARD_TRANSITIVE` / `FULL_TRANSITIVE` / `NONE`	`BACKWARD` / `FORWARD` / `FULL` + `BACKWARD_TRANSITIVE` / `FORWARD_TRANSITIVE` / `FULL_TRANSITIVE`	`buf breaking` against the registry's latest on `buf push` (enforced per module)	Sam CSR

Capability	Confluent Schema Registry (CSR) 7.6+	Apicurio Registry 3.0+	Buf Schema Registry (BSR)	Kara Red Reg
Storage	Kafka topic `_schemas` (single-partition, compacted)	SQL (PostgreSQL), Kafka (`kafkasql`), or in-memory	BSR-managed (hosted); Enterprise self-hosted	Post Kafk
Self-hosted	Yes — Java, Kafka-adjacent	Yes — Quarkus/Java, many storage backends	Enterprise self-hosted; otherwise hosted at `buf.build`	Yes - Pyth (Kar C++ (Red
HTTP/ OpenAPI registry	No (Avro/Protobuf/JSON Schema only)	Yes — `OPENAPI` artifacts, first-class	No (Protobuf only)	No
Codegen trigger	No (catalog-level)	No	`buf generate` + BSR-generated SDKs	No

Selection heuristic:

- **Kafka/Avro/Protobuf on Kafka:** Confluent CSR if you already run Confluent Platform; Apicurio if you want OpenAPI + Avro/Protobuf in one registry or need PostgreSQL storage without a Kafka dependency for the registry itself; Redpanda's registry if you run Redpanda.
- **gRPC/Protobuf across many repos:** BSR — it solves the concurrent-evolution race that CSR/Apicurio do not (module-level `buf breaking` on `buf push`), and `buf generate` is the native codegen.
- **HTTP/REST OpenAPI catalog:** Apicurio (`OPENAPI` artifacts) or a dedicated catalog (Backstage, Bump.sh, SwaggerHub) — CSR and BSR do not store OpenAPI.
- **Many teams, many surfaces:** Apicurio or a split — BSR for Protobuf, CSR/Apicurio for Kafka schemas, Backstage/Bump as the unified catalog that links to both. The unified catalog (see Chapter 9)

is what consumers actually browse; the registries are what CI publishes to.

Subject and artifact naming

A consistent naming scheme prevents the "which subject is the canonical one" confusion that plagues ungoverned registries.

```
# Confluent / Karapace — TopicNameStrategy (default) vs
RecordNameStrategy
# Recommended: TopicNameStrategy for topic-per-aggregate,
RecordNameStrategy for shared topics

orders-value          # topic 'orders', message value schema
(TopicNameStrategy)
orders-key            # topic 'orders', message key schema
com.acme.orders.OrderCreated  # RecordNameStrategy — schema FQN as
subject

# Apicurio — group + artifactId (group = team or domain, artifactId =
schema name)
group: orders
artifactId: OrderCreated        # AVRO or PROTOBUF artifact
artifactId: orders-openapi      # OPENAPI artifact — type=OPENAPI,
group=orders

# BSR — module path + track (module = repo-level unit, track = version
line)
buf.build/acme/orders           # module
buf.build/acme/orders:main      # track (like a branch — main, dev)
buf.build/acme/orders:1.4.0     # label (like a tag — SemVer label on a
commit)
```

Rule of thumb: one subject/artifact/module per *bounded context* (Vol 15, DDD — Ch 5), not per topic or per file. The orders team's `orders.proto` is one BSR module; its Kafka `OrderCreated` Avro schema is one Apicurio artifact; the same logical `Order` published to two topics (`orders`, `orders-retry`) shares the same schema via `RecordNameStrategy` or a referenced artifact rather than two forked copies.

Enforcement at publish time — compatibility that cannot be bypassed

Chapter 8 defined the compatibility taxonomy; Chapter 9 wired breaking-change detection into CI (`oasdiff`, `buf breaking` vs Git). The registry moves the gate one step further: even if a developer bypasses CI, the registry *refuses* an incompatible publish.

Confluent / Apicurio — compatibility modes

```
# --- Confluent Schema Registry 7.6+ ---
# Set default compatibility (global)
curl -X PUT http://registry:8081/config \
  -H 'Content-Type: application/vnd.schemaregistry.v1+json' \
  -d '{"compatibility": "FULL_TRANSITIVE"}'

# Per-subject override — orders-value needs FULL_TRANSITIVE (replayable
log)
curl -X PUT http://registry:8081/config/orders-value \
  -H 'Content-Type: application/vnd.schemaregistry.v1+json' \
  -d '{"compatibility": "FULL_TRANSITIVE"}'

# Per-subject for a request/response topic that is never replayed —
BACKWARD is enough
curl -X PUT http://registry:8081/config/rpc-replies-value \
  -H 'Content-Type: application/vnd.schemaregistry.v1+json' \
  -d '{"compatibility": "BACKWARD"}'

# Publish — rejected if incompatible with the configured mode
curl -X POST http://registry:8081/subjects/orders-value/versions \
  -H 'Content-Type: application/vnd.schemaregistry.v1+json' \
  -d @- <<'JSON'
{
  "schemaType": "AVRO",
  "schema": "{\"type\":\"record\",\"name\":\"Order\",\"namespace\":
\"com.acme.orders\",\"fields\":[{\"name\":\"order_id\",\"type\":
\"string\"},{\"name\":\"promo_code\",\"type\":[\"null\",\"string\"],
\"default\":null}]}"
}
JSON
# 409 Conflict if incompatible:
# {"error_code":409,"message":"Schema being registered is incompatible
with an earlier schema"}

# --- Apicurio Registry 3.0+ (compat via rules) ---
# Create artifact with a compatibility rule — FULL_TRANSITIVE
curl -X POST http://apicurio:8080/apis/registry/v3/groups/orders/
artifacts \
  -H 'Content-Type: application/json' \
  -d '{
    "artifactId": "OrderCreated",
    "artifactType": "AVRO",
    "firstVersion": { "content": { "content": "{\"type\":\"record\",
\"name\":\"OrderCreated\",...}" } }
  }'

# Add compatibility rule to an existing artifact
curl -X POST http://apicurio:8080/apis/registry/v3/groups/orders/
artifacts/OrderCreated/rules \
```

```
  -H 'Content-Type: application/json' \
  -d '{"config": "FULL_TRANSITIVE", "ruleType": "COMPATIBILITY"}'
```

Choosing the mode (see Chapter 8 for the wire-level why):

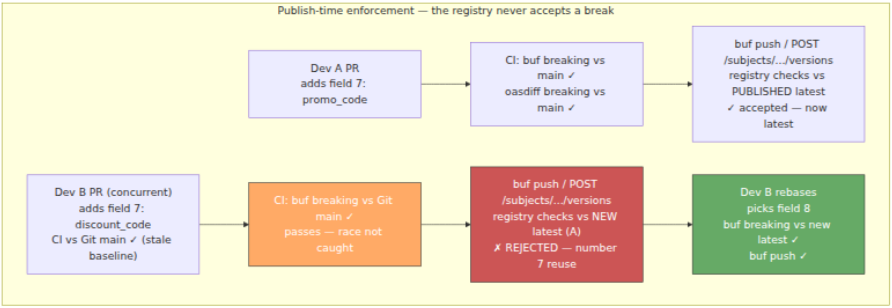
Mode	Question the registry asks on publish	Use when
`BACKWARD`	Can *old* consumers read *new* data? (`new` is superset of `old`)	Request/response, non-replayed topics — new producer is safe, old consumer ignorance is fine
`FORWARD`	Can *new* consumers read *old* data? (`old` is superset of `new`)	Rollback-sensitive, replay from old offsets after consumer upgrade
`FULL`	Both directions vs *latest* version	Rolling deploys in either direction (common default if you are unsure)
`FULL_TRANSITIVE`	Both directions vs *all* prior versions	Durable log you replay for months/years — never replays an incompatible generation (Kafka — Vol 10, Ch 3)
`NONE`	No check	Never on a shared topic — only for prototyping with a single producer

Default to `BACKWARD` for ephemeral request/response and `FULL_TRANSITIVE` for any topic whose bytes outlive the deploy that wrote them.

Buf Schema Registry — `buf push` as the gate

BSR enforces via the same `buf breaking` rules as CI, but against the *published* module — catching the concurrent-PR race that Git comparison misses.

```
# buf.yaml at api/orders (buf 1.40+) — breaking already configured (Ch
9)
# buf.yaml
# version: v2
# modules: [{ path: proto }]
# breaking: { use: [FILE] }

# Publish — fails if breaking vs the registry's latest on 'main'
buf push
# Failure example (concurrent PRs both added field 7):
# Failure: Field "7" on message "Order" already exists in the pushed
module.

# Push to a dev track first, promote to main after review (like a
feature branch for schemas)
buf push --label dev
# ... review ...
buf push --label 1.4.0   # tag the promotion

# Consumer pins to a label or track in buf.lock
# buf.lock — committed, so every build resolves the same schema
# version: v2
# deps:
#   - remote: buf.build/acme/orders
#     commit: 01h8x1abcdef1234567890ab
#     track: main
```



Wiring producers and consumers

Kafka — Confluent serializers (Java, Go, Python)

The serializer fetches (or registers) the schema on first write, caches the schema ID, and prepends the wire header. The consumer fetches by ID and deserializes — no schema per message.

```
// Java — Avro with Confluent Schema Registry (confluent-kafka 7.6+,
avro 1.11+)
// Producer — application.properties
// spring.kafka.producer.properties.schema.registry.url=http://
registry:8081
// spring.kafka.producer.value-
serializer=io.confluent.kafka.serializers.KafkaAvroSerializer

Properties producerProps = new Properties();
producerProps.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
"kafka:9092");
producerProps.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSer
ializer.class);
producerProps.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, KafkaAv
roSerializer.class);
producerProps.put(AbstractKafkaSchemaSerDeConfig.SCHEMA_REGISTRY_URL_CO
NFIG, "http://registry:8081");
producerProps.put(AbstractKafkaSchemaSerDeConfig.AUTO_REGISTER_SCHEMAS,
false); // governance: CI registers, not producers
producerProps.put(KafkaAvroSerializerConfig.AVRO_USE_LOGICAL_TYPE_CONVE
RTERS_CONFIG, true);

KafkaProducer<String, OrderCreated> producer = new KafkaProducer<>(prod
ucerProps);
// OrderCreated is Avro-generated (avro-maven-plugin) — type-safe,
registry-validated
OrderCreated event = OrderCreated.newBuilder()
    .setOrderId("ord_123")
    .setCustomerId("cus_456")
    .setPromoCode("SAVE20")   // nullable — default null keeps
FULL_TRANSITIVE compat
    .build();
producer.send(new ProducerRecord<>("orders", event.getOrderId().toStrin
g(), event));

// Consumer — symmetric, plus specific Avro reader for type safety
Properties consumerProps = new Properties();
consumerProps.put(ConsumerConfig.BOOTSTRAP_SERVERS_CONFIG,
"kafka:9092");
consumerProps.put(ConsumerConfig.GROUP_ID_CONFIG, "orders-projector");
consumerProps.put(ConsumerConfig.VALUE_DESERIALIZER_CLASS_CONFIG, Kafka
AvroDeserializer.class);
consumerProps.put(AbstractKafkaSchemaSerDeConfig.SCHEMA_REGISTRY_URL_CO
NFIG, "http://registry:8081");
consumerProps.put(KafkaAvroDeserializerConfig.SPECIFIC_AVRO_READER_CONF
IG, true);
// Deserialization failure = poison message — route to DLT (Vol 10, Ch
8), do not block the partition
```

```go
// Go — Protobuf on Kafka with Confluent's Go serializer (confluent-
kafka-go 2.4+, protobuf 1.34+)
// go.mod: github.com/confluentinc/confluent-kafka-go/v2,
google.golang.org/protobuf, github.com/confluentinc/schemaregistry

import (
    "github.com/confluentinc/confluent-kafka-go/v2/schemaregistry"
    "github.com/confluentinc/confluent-kafka-go/v2/schemaregistry/
serde"
    "github.com/confluentinc/confluent-kafka-go/v2/schemaregistry/
serde/protobuf"
    orderpb "acme/orders/proto/acme/orders/v1"
)

client, _ := schemaregistry.NewClient(schemaregistry.NewConfig("http://
registry:8081"))
ser, _ := protobuf.NewSerializer(client, serde.ValueSerde, protobuf.New
SerializerConfig())
des, _ := protobuf.NewDeserializer(client, serde.ValueSerde, protobuf.N
ewDeserializerConfig())

// Produce — serializer registers (or validates) the schema, caches ID,
prepends 5-byte header (magic + schema ID)
payload, _ := ser.Serialize("orders", &orderpb.OrderCreated{OrderId: "o
rd_123", PromoCode: proto.String("SAVE20")})
producer.Produce(&kafka.Message{TopicPartition: kafka.TopicPartition{To
pic: &topic, Partition: kafka.PartitionAny}, Value: payload}, nil)

// Consume — deserializer fetches by ID, caches, unknown fields
preserved (Ch 8)
msg, _ := consumer.ReadMessage(-1)
var event orderpb.OrderCreated
_ = des.DeserializeInto("orders", msg.Value, &event)
```

```python
# Python — Avro with Confluent's Python client (confluent-kafka 2.4+, fastavro)
from confluent_kafka import Producer
from confluent_kafka.schema_registry import SchemaRegistryClient
from confluent_kafka.schema_registry.avro import AvroSerializer, AvroDeserializer
from confluent_kafka.serialization import StringSerializer, SerializationContext, MessageField

sr = SchemaRegistryClient({"url": "http://registry:8081"})
avro_serializer = AvroSerializer(sr, schema_str=open("OrderCreated.avsc").read())
avro_deserializer = AvroDeserializer(sr, schema_str=open("OrderCreated.avsc").read())

producer = Producer({"bootstrap.servers": "kafka:9092"})
producer.produce(
    topic="orders",
    key=StringSerializer("utf_8")("ord_123", SerializationContext("orders", MessageField.KEY)),
    value=avro_serializer(event_dict, SerializationContext("orders", MessageField.VALUE)),
)
```

Wire header (Confluent wire format — 5 bytes prepended to every message):

```
0x00 | 4-byte big-endian schema ID | Avro/Protobuf/JSON Schema bytes
 ^magic byte       ^registry lookup key     ^payload validated against that schema
```

The consumer reads the magic byte, extracts the ID, fetches (or cache-hits) the schema, and deserializes. The schema is not in the message — only the ID — so payloads stay ~0.2–0.4× JSON size (Chapter 8).

Schema references — compose schemas without duplication:

```
// Avro — OrderCreated references Address (registered as a separate
subject)
{
  "type": "record",
  "name": "OrderCreated",
  "namespace": "com.acme.orders",
  "fields": [
    {"name": "order_id", "type": "string"},
    {"name": "shipping_address", "type": "com.acme.common.Address"}
  ]
}
```

```
# Register Address first, then OrderCreated with a reference
curl -X POST http://registry:8081/subjects/com.acme.common.Address/
versions \
  -H 'Content-Type: application/vnd.schemaregistry.v1+json' \
  -d '{"schemaType":"AVRO","schema":"{\"type\":\"record\",\"name\":
\"Address\",...}"}'

curl -X POST http://registry:8081/subjects/orders-value/versions \
  -H 'Content-Type: application/vnd.schemaregistry.v1+json' \
  -d '{
    "schemaType": "AVRO",
    "schema": "{\"type\":\"record\",\"name\":\"OrderCreated\",...}",
    "references":
[{"name":"com.acme.common.Address","subject":"com.acme.common.Address",
"version":1}]
  }'
```

gRPC — `buf push` and consumer resolution

No Kafka header — gRPC consumers resolve the schema via `buf.lock` at build time, not per-message. The registry's job is to be the *publish-time* gate and the *discovery* source for `buf generate`.

```
# Consumer repo — pin the schema version you build against
buf dep update
# resolves buf.lock to the latest commit on the tracked track
buf generate            # generates code from the pinned commit (see
codegen below)
```

OpenAPI — publishing to Apicurio / catalog

```
# Publish OpenAPI 3.1 to Apicurio as an OPENAPI artifact (Apicurio
Registry 3.0+)
curl -X POST http://apicurio:8080/apis/registry/v3/groups/orders/
artifacts \
  -H 'Content-Type: application/json' \
  -d @- <<'JSON'
{
  "artifactId": "orders-openapi",
  "artifactType": "OPENAPI",
  "firstVersion": {
    "content": { "contentType": "application/json", "content":
"{\"openapi\":\"3.1.0\",...}" },
    "isLatest": true
  }
}
JSON

# Add a compatibility rule — Apicurio can lint OpenAPI on publish via
rules
curl -X POST http://apicurio:8080/apis/registry/v3/groups/orders/
artifacts/orders-openapi/rules \
  -H 'Content-Type: application/json' \
  -d '{"ruleType":"COMPATIBILITY","config":"BACKWARD"}'
```

For Backstage/Bump catalogs, publishing is typically a `catalog-info.yaml` or a CI step that uploads `openapi.yaml` — the catalog links to the registry artifact, not a duplicate copy.

Code generation — from schema to type-safe client

Hand-written HTTP clients drift. A field renamed in the spec but not in the hand-written client is a bug that tests may not catch — especially when the field is optional and the test fixtures are stale. Code generation makes the schema the *single source of code*: the client and server stubs are derived, not authored, and a schema change that breaks the client is a compile error, not a runtime incident.

Protobuf — `buf generate` (buf 1.40+)

`buf generate` replaces the `protoc` + shell-script tangle with a declarative `buf.gen.yaml` that pins plugin versions and runs reproducibly in CI and locally.

```yaml
# buf.gen.yaml — generate Go, Java, TypeScript, Python from one proto module (buf 1.40+)
# Docs: https://buf.build/docs/generate/overview
version: v2
managed:
  enabled: true
  override:
    - file_option: go_package
      value: acme/orders/gen/go/acme/orders/v1;ordersv1
    - file_option: java_package
      value: com.acme.orders.v1
plugins:
  # Go — protobuf + gRPC/Connect
  - remote: buf.build/protocolbuffers/go:v1.34.2
    out: gen/go
    opt: paths=source_relative
  - remote: buf.build/connectrpc/go:v1.17.0
    out: gen/go
    opt: paths=source_relative
  # Java — protobuf + gRPC
  - remote: buf.build/protocolbuffers/java:v4.27.0
    out: gen/java
  - remote: buf.build/grpc/java:v1.66.0
    out: gen/java
  # TypeScript — protobuf-es + Connect (browser + Node)
  - remote: buf.build/bufbuild/es:v1.10.0
    out: gen/ts
    opt: target=ts
  - remote: buf.build/connectrpc/es:v1.6.1
    out: gen/ts
    opt: target=ts
  # Python — protobuf + gRPC
  - remote: buf.build/protocolbuffers/python:v5.27.0
    out: gen/python
  - remote: buf.build/grpc/python:v1.66.0
    out: gen/python

# Alternative — local plugins (when you need a custom protoc plugin):
# - plugin: buf.build/protocolbuffers/go:v1.34.2
#   out: gen/go
```

```
# Local — requires buf CLI 1.40+
buf generate          # reads buf.gen.yaml, fetches remote plugins,
writes gen/
buf generate --template buf.gen.yaml --path proto/acme/orders/v1/
order.proto

# CI — identical command, no local protoc installation
# .github/workflows/codegen.yaml (excerpt)
# - uses: bufbuild/buf-action@v1
#   with: { setup_only: true }
# - run: buf generate
# - run: git diff --exit-code gen/   # fail if generated code is stale
```

What `managed: enabled` does. Without it, every `.proto` needs `option go_package` and `option java_package` per file — a frequent source of "it generates but won't compile." Managed mode injects those options at generation time from `buf.gen.yaml`, so `.proto` files stay language-agnostic.

Server and client stubs (Go, ConnectRPC 1.17+):

```
// Server — implement the generated interface (gen/go/acme/orders/v1/
ordersv1connect)
type OrderServiceHandler struct{}

func (h *OrderServiceHandler) CreateOrder(
    ctx context.Context, req *connect.Request[ordersv1.CreateOrderReque
st],
) (*connect.Response[ordersv1.CreateOrderResponse], error) {
    // req.Msg.CustomerId, req.Msg.Quantity — type-safe, field presence
via proto3 optional
    if req.Msg.Quantity <= 0 {
        return nil, connect.NewError(connect.CodeInvalidArgument,
            errors.New("quantity must be > 0"))
    }
    // ...
    return connect.NewResponse(&ordersv1.CreateOrderResponse{OrderId: "
ord_123"}), nil
}

// Client — type-safe, no hand-written HTTP
client := ordersv1connect.NewOrderServiceClient(http.DefaultClient, "ht
tps://api.acme.internal")
resp, err := client.CreateOrder(ctx, connect.NewRequest(&ordersv1.Creat
eOrderRequest{
    CustomerId: "cus_456",
    Quantity:   2,
    PromoCode:  proto.String("SAVE20"),
}))
```

OpenAPI — OpenAPI Generator (7.8+)

OpenAPI Generator covers the REST surface that `buf generate` does not. It generates typed clients and server stubs for 50+ languages from an `openapi.yaml`.

```yaml
# openapi-generator-config.yaml — TypeScript + Go clients from one spec
(generator 7.8+)
# Docs: https://openapi-generator.tech/docs/generators
generatorName: typescript-fetch
outputDir: gen/ts
inputSpec: openapi.yaml
additionalProperties:
  npmName: "@acme/orders-client"
  npmVersion: "1.4.0"          # tracks the API version — see SDK
versioning below
  supportsES6: true
  withInterfaces: true
  useSingleRequestParameter: true
typeMappings:
  # map OpenAPI formats to idiomatic types
  date-time: Date
  ulid: string
importMappings: {}
templateDir: templates/typescript-fetch
# optional — override Mustache templates for org style

# --- Go client (second execution) ---
# generatorName: go
# outputDir: gen/go
# additionalProperties: { packageName: orders, isGoSubmodule: true }
```

```bash
# Install — version-pinned (OpenAPI Generator 7.8+, Java 17+, Node 20+)
npm i -D @openapitools/openapi-generator-cli@2.12.0
npx @openapitools/openapi-generator-cli@2.12.0 version-manager set 7.8.
0

# Generate — one command per target (or a batch script)
npx @openapitools/openapi-generator-cli generate -c openapi-generator-
config.yaml

# Via Docker — no Java locally
docker run --rm -v $PWD:/local openapitools/openapi-generator-
cli:v7.8.0 generate \
  -c /local/openapi-generator-config.yaml

# Verify — generated client compiles (fail CI if stale)
npm --prefix gen/ts run build
go vet ./gen/go/...

# Server stub (optional) — generate Echo/Gin/Chi stubs for Go, Spring
for Java
# generatorName: go-echo-server  /  spring
```

Custom templates — when the default Mustache output does not match org conventions (e.g., injecting `Idempotency-Key` headers, `Retry-After` handling, or tracing interceptors), vendor the templates:

```
# Extract default templates, then edit
npx @openapitools/openapi-generator-cli author template -g typescript-
fetch -o templates/typescript-fetch
# Edit templates/typescript-fetch/api.mustache to add idempotency,
tracing, etc.
```

Where codegen breaks down and what to do:

Limitation	Mitigation
Generated code is verbose / not idiomatic	Custom Mustache templates; post-process with `prettier` / `gofmt`; wrap generated clients in a thin hand-written facade that exposes the ergonomic surface
`oneOf` / `anyOf` polymorphism generates awkward unions	Model polymorphism as separate endpoints or use `discriminator` + codegen `oneOf` support (generator 7.8+ improved this)
Generator bugs / lag behind spec	Pin the generator version, vendor templates, and run `generated --check` in CI — do not float `latest`
"Generated code in Git" vs "generate at build"	Commit generated code — consumers need a readable diff on schema changes, and CI can `git diff --exit-code gen/` to catch staleness. For Go, the generated module is a real `go.mod` dependency.

SDK delivery — the schema's last mile

A generated client that lives only in the producer's repo is not an SDK. An SDK is a *versioned, published, documented package* that consumers install with `npm install` / `go get` / `pip install` and that tracks the API's version.

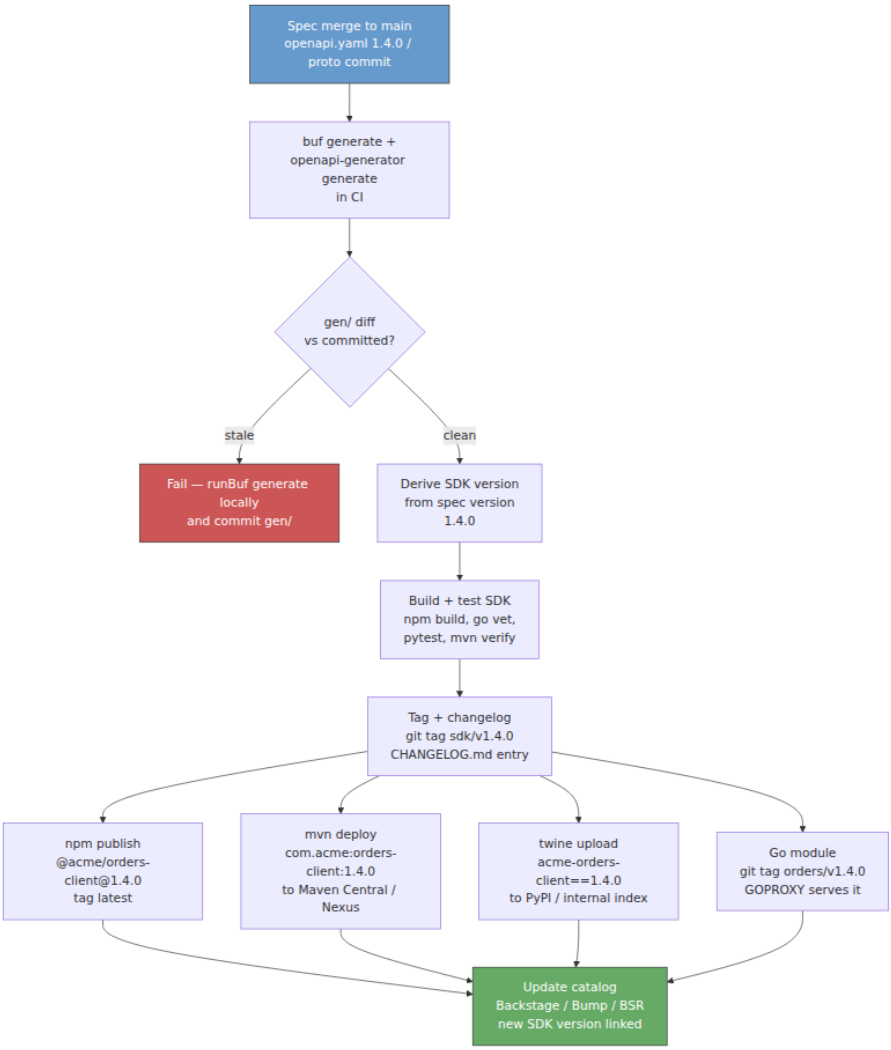
Versioning — API version drives SDK version

The SDK version must communicate compatibility at a glance. The convention that scales:

- **SDK major tracks API major.** `openapi.yaml` at `1.4.0` → SDK `@acme/orders-client@1.4.0`. API `2.0.0` (breaking per SemVer and Chapter 5) → SDK `2.0.0`. Consumers know that `1.x` → `2.0` is breaking without reading the changelog.
- **SDK minor/patch track additive changes.** New optional field → API `1.5.0` → SDK `1.5.0`. Bug fix in the SDK (retry logic, header handling) without a spec change → SDK `1.4.1`.
- **Pre-release for `dev` track.** `buf push --label dev` → SDK `1.5.0-dev.0` (npm `next` tag, Maven `SNAPSHOT`, Python `dev0`).

```
// gen/ts/package.json — generated client's package manifest (npm)
{
  "name": "@acme/orders-client",
  "version": "1.4.0",
  "description": "Acme Orders API — TypeScript client (generated from
openapi.yaml 1.4.0)",
  "repository": "github:acme/orders",
  "publishConfig": { "registry": "https://registry.acme.internal" },
  "files": ["dist/"],
  "main": "dist/index.js",
  "types": "dist/index.d.ts"
}
```

Publishing — one workflow, every registry


Spec merge to main
openapi.yaml 1.4.0 /
proto commit
buf generate +
openapi-generator
generate
in CI
gen/ diff
vs committed?
stale
clean
Fail — runBuf generate
locally
and commit gen/
Derive SDK version
from spec version
1.4.0
Build + test SDK
npm build, go vet,
pytest, mvn verify
Tag + changelog
git tag sdk/v1.4.0
CHANGELOG.md entry
npm publish
@acme/orders-
client@1.4.0
tag latest
mvn deploy
com.acme:orders-
client:1.4.0
to Maven Central /
Nexus
twine upload
acme-orders-
client==1.4.0
to PyPI / internal index
Go module
git tag orders/v1.4.0
GOPROXY serves it
Update catalog
Backstage / Bump / BSR
new SDK version linked

```yaml
# .github/workflows/sdk-publish.yaml — publish SDKs on spec merge (Node 20, Java 17, Python 3.12, buf 1.40)
name: sdk-publish
on:
  push:
    branches: [main]
    paths: ["openapi.yaml", "proto/**", "buf.gen.yaml", "openapi-generator-config.yaml", "gen/**"]

jobs:
  generate-and-verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: bufbuild/buf-action@v1
        with: { setup_only: true }
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - uses: actions/setup-java@v4
        with: { java-version: 17, distribution: temurin }
      - uses: actions/setup-python@v4
        with: { python-version: "3.12" }

      - run: buf generate
      - run:
npx @openapitools/openapi-generator-cli@2.12.0 generate -c openapi-generator-config.yaml
      - name: Fail if generated code is stale
        run: git diff --exit-code gen/
      - run:
npm --prefix gen/ts ci && npm --prefix gen/ts run build && npm --prefix gen/ts test
      - run: go vet ./gen/go/... && go test ./gen/go/...
      - run: pip install -e gen/python && pytest gen/python/tests

  publish-npm:
    needs: generate-and-verify
    runs-on: ubuntu-latest
    permissions: { contents: write, id-token: write }  # OIDC for npm provenance
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, registry-url: "https://registry.npmjs.org" }
      - run: npm --prefix gen/ts ci && npm --prefix gen/ts run build
      - name: Derive version from spec
        id: ver
        run: echo "version=$(yq .info.version openapi.yaml)" >> "$GITHUB_OUTPUT"
```

```
      - run: npm --prefix gen/ts version "$
{{ steps.ver.outputs.version }}" --no-git-tag-version
      - run: npm --prefix gen/ts publish --provenance --access public
        env: { NODE_AUTH_TOKEN: "${{ secrets.NPM_TOKEN }}" }

  publish-maven:
    needs: generate-and-verify
    runs-on: ubuntu-latest
    permissions: { contents: write }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: 17, distribution: temurin, server-id: cen
tral, server-username: MAVEN_USERNAME, server-password:
MAVEN_PASSWORD }
      - name: Derive version from spec
        id: ver
        run: echo "version=$(yq .info.version openapi.yaml)" >>
"$GITHUB_OUTPUT"
      - run: mvn -pl gen/java versions:set -DnewVersion="$
{{ steps.ver.outputs.version }}"
      - run: mvn -pl gen/java deploy -DskipTests
        env: { MAVEN_USERNAME: "${{ secrets.MAVEN_USERNAME }}",
MAVEN_PASSWORD: "${{ secrets.MAVEN_PASSWORD }}" }

  publish-python:
    needs: generate-and-verify
    runs-on: ubuntu-latest
    permissions: { contents: write, id-token: write }  # OIDC for PyPI
trusted publishing
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v4
        with: { python-version: "3.12" }
      - run: pip install build twine
      - name: Derive version from spec
        id: ver
        run: echo "version=$(yq .info.version openapi.yaml)" >>
"$GITHUB_OUTPUT"
      - run: python -m build gen/python --outdir dist/
      - uses: pypa/gh-action-pypi-publish@release/v1
        with: { packages-dir: dist/ }

  tag-and-catalog:
    needs: [publish-npm, publish-maven, publish-python]
    runs-on: ubuntu-latest
    permissions: { contents: write }
    steps:
      - uses: actions/checkout@v4
      - name: Derive version
        id: ver
```

```
        run: echo "version=$(yq .info.version openapi.yaml)" >>
"$GITHUB_OUTPUT"
      - run: |
          git tag "sdk/v${{ steps.ver.outputs.version }}"
          git push origin "sdk/v${{ steps.ver.outputs.version }}"
      - name: Append changelog (conventional commits → changelog)
        run: |
          npx conventional-changelog-cli -p angular -i CHANGELOG.md -s
          git add CHANGELOG.md && git commit -m "docs(changelog): sdk
v${{ steps.ver.outputs.version }}" && git push
      - name: Notify catalog (Backstage / Bump.sh)
        run: curl -X POST https://catalog.acme.internal/api/refresh -H
"Authorization: Bearer ${{ secrets.CATALOG_TOKEN }}"
```

Changelog and migration guide. Every SDK minor/major must link to a changelog entry. For breaking majors, a `MIGRATION.md` with before/after snippets is the difference between consumers upgrading in a week versus pinning the old version forever. Generate the changelog from conventional commits (`feat:`, `fix:`, `BREAKING CHANGE:`) and curate the migration guide by hand — automation cannot explain *why* a field was renamed.

Deprecation in the SDK. When the spec deprecates a field (`deprecated: true` in OpenAPI, `option deprecated = true` in proto), the generated client should surface it — JSDoc `@deprecated`, Java `@Deprecated`, Python `warnings.warn`. A custom Mustache template or a `buf generate` plugin option can inject deprecation annotations so consumers see the warning at compile/lint time, not at runtime.

The thin facade — generated core, hand-written surface

Raw generated clients are correct but rarely ergonomic. The pattern that scales is a thin hand-written facade that wraps the generated core:

```ts
// sdk/orders.ts — hand-written facade over gen/ts (TypeScript 5.4+)
import { OrdersApi, Configuration } from "../gen/ts/dist";

export class OrdersClient {
  private readonly api: OrdersApi;

  constructor(opts: { baseUrl: string; token: string; timeoutMs?: numbe
r }) {
    const cfg = new Configuration({
      basePath: opts.baseUrl,
      accessToken: opts.token,
      middleware: [tracingMiddleware, retryMiddleware({ maxRetries: 2 }
)],
      fetchApi: (url, init) => fetch(url, { ...init, signal: AbortSigna
l.timeout(opts.timeoutMs ?? 5000) }),
    });
    this.api = new OrdersApi(cfg);
  }

  // Ergonomic — single call, domain types, idempotency handled
  async createOrder(input: CreateOrderInput): Promise<Order> {
    const resp = await this.api.createOrder({
      createOrderRequest: toGenerated(input),
      idempotencyKey: input.idempotencyKey ?? crypto.randomUUID(),
    });
    return fromGenerated(resp);
  }
}
```

Consumers import from `sdk/orders.ts`, not `gen/ts/dist`. The generated code is an internal detail; the facade is the public surface that adds retries, tracing, timeouts, and ergonomic types. The facade is tested with contract tests (Chapter 11) so it does not drift from the spec.

The distributed-systems lens — registry as control plane

Availability and caching

The registry is not on the hot path per message — schemas are cached by ID — but its control-plane availability still matters:

- **Producer with warm cache:** keeps producing with the last schema ID even if the registry is down. No impact.

- **Producer publishing a new schema:** blocked until the registry recovers. Safe — it keeps producing with the old schema.
- **Consumer with warm cache:** keeps consuming. No impact.
- **Consumer with cold cache (new deploy, new topic):** cannot deserialize until the registry recovers. Mitigate by bundling schemas into the consumer image (generated code already has the schema) or by a local registry cache (Apicurio's `kafkasql` storage, Karapace's PostgreSQL, or a sidecar proxy like `confluent-schema-registry-proxy`).

Run the registry with the same care as etcd/ZooKeeper: replicated storage (Kafka `_schemas` topic with `replication.factor=3`, PostgreSQL with streaming replication), health checks, and alerting on publish latency and cache hit rate.

Staleness — generated code versus hand-written drift

A consumer that pins `buf.lock` at commit `abc123` and never updates is frozen at that schema. New fields added at `def456` are invisible until `buf dep update && buf generate`. The catalog and scorecard (Chapter 9) surface staleness — "47 consumers still on orders `1.2.0`, latest is `1.5.0`" — and the SDK's `Sunset` header tells them what breaks and when. Without that feedback loop, staleness silently accumulates until a breaking `2.0.0` forces a flag-day migration.

Shadow APIs and the catalog

The registry prevents *published* drift. It does not prevent *unpublished* drift — the team that ships an endpoint without registering it. The catalog's job is to make shadow APIs visible: every route the gateway serves (Envoy access logs, OpenAPI discovery) should have a corresponding catalog entry. A periodic reconciliation — "gateway serves `/v1/widgets` but no spec exists in the catalog" — is the same idea as the SBOM reconciliation in Companion Book 3, applied to APIs.

Anti-patterns (and what to do instead)

Anti-pattern	Why it hurts	Fix
Producers with `AUTO_REGISTER_SCHEMAS=true` on a shared topic	Any producer can publish any schema — compatibility gate bypassed; concurrent races create duplicate field numbers	`AUTO_REGISTER_SCHEMAS=false`; CI registers via `buf push` / `POST /subjects/.../versions` with compatibility checks
One subject per topic copy-pasted across teams	Same logical `Order` forked into `orders-value`, `orders-retry-value`, `orders-dlq-value` — three schemas to keep in sync	`RecordNameStrategy` or schema references; one canonical artifact, many topics

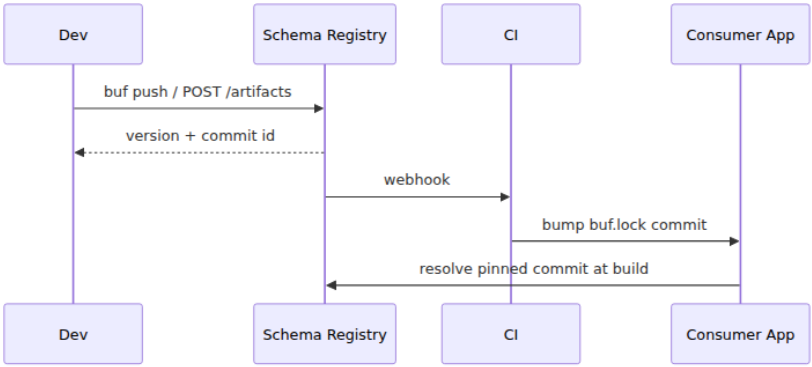
Anti-pattern	Why it hurts	Fix
`NONE` compatibility on a durable topic	Topic accepts incompatible schemas; replay fails months later when old bytes cannot be deserialized by the new consumer	`FULL_TRANSITIVE` for any replayable log; `BACKWARD` minimum for ephemeral topics
Generated code at `latest` with no pinned version	"Works on my machine" — CI and local generate different output; consumers cannot reproduce the build	Pin generator + plugin versions in `buf.gen.yaml` / `openapi-generator-config.yaml`; commit `buf.lock`
Publishing SDKs by hand (`npm publish` from a laptop)	Forgotten changelog, skipped tests, wrong version, no provenance	CI-only publish on `main` merge; `git diff --exit-code gen/` gate; OIDC provenance (`--provenance` / PyPI trusted publishing)
Raw generated client as the public SDK surface	Verbose, non-idiomatic, leaks generator quirks; no place to add retries/ tracing/ timeouts	Thin hand-written facade over generated core — generated code is internal, facade is public

Anti-pattern	Why it hurts	Fix
No catalog — "the Git repo is the catalog"	New teams cannot discover the API; shadow APIs proliferate; deprecation is invisible	Backstage / Bump.sh / Apicurio as the searchable catalog; gateway ↔ catalog reconciliation

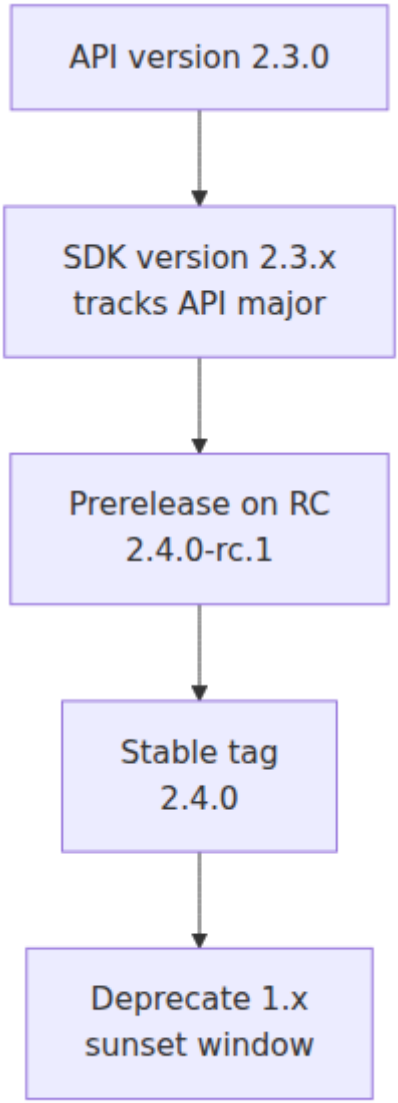
---

SDK Generation Pipeline

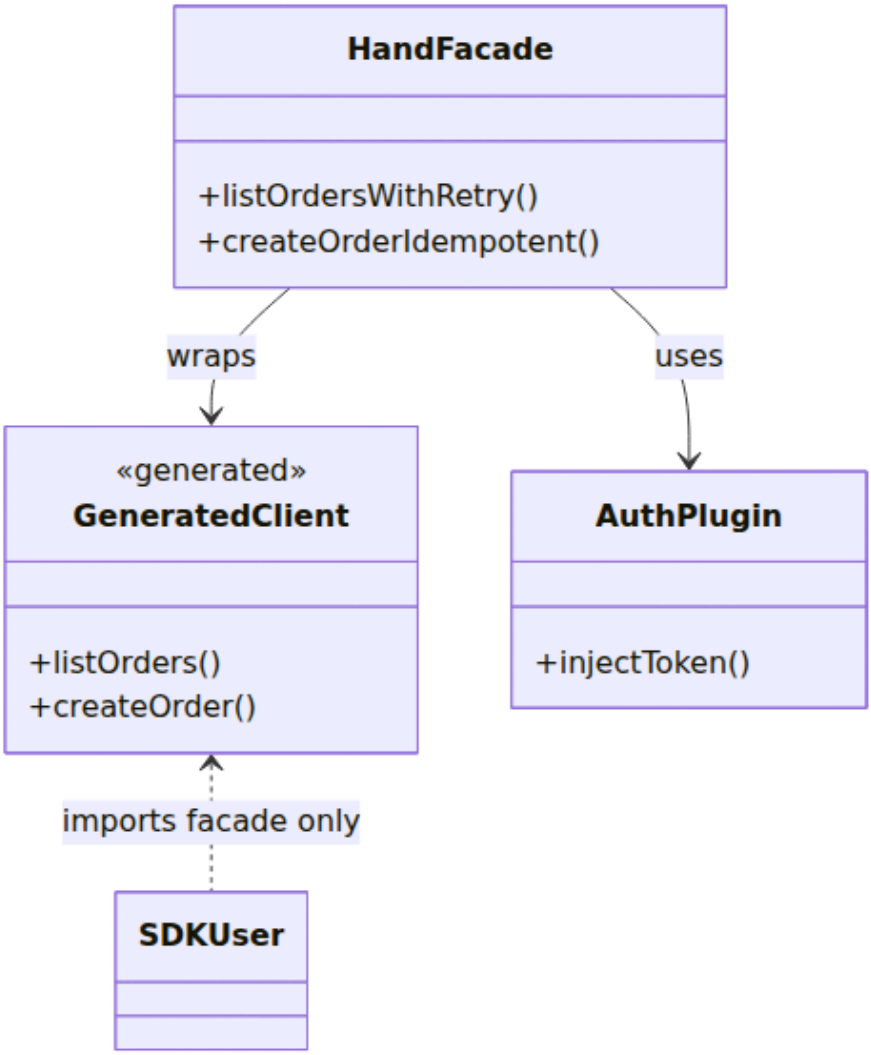
Schema Registry Publish and Pin



SDK Versioning



Thin Facade Pattern



Key takeaways

- A schema in Git is a promise; a schema in a registry is an invariant. The registry — Confluent/Apicurio for Kafka (Avro/Protobuf/JSON Schema), BSR for Protobuf/gRPC, Apicurio for OpenAPI — makes the schema versioned, compatibility-checked, and discoverable, and refuses incompatible publishes before any bytes hit the wire.

- Subject naming is architecture: one canonical artifact per bounded context (`RecordNameStrategy` / schema references for Kafka, one BSR module per domain, one Apicurio `OPENAPI` artifact per REST surface), not one copy per topic or per file — duplication is how concurrent evolution creates field-number collisions.
- Compatibility at publish is the gate that even bypassed CI cannot evade. Confluent/Apicurio `FULL_TRANSITIVE` for replayable logs and `BACKWARD` for ephemeral request/response; BSR's `buf push` enforces the same `buf breaking` rules against the *published* module, catching the concurrent-PR race that Git comparison misses.
- Producers and consumers wire to the registry via serializers that cache by schema ID (5-byte Confluent header: magic + ID) and via `buf.lock`-pinned generation — the registry is control plane, not data plane, and must be replicated and cached like any coordination service.
- Code generation (`buf generate` with `buf.gen.yaml` managed mode for Protobuf, OpenAPI Generator 7.8+ for REST) makes the schema the single source of code — generated clients are type-safe and break at compile time when the schema changes, not at runtime. Pin generator + plugin versions, commit `gen/`, and fail CI on staleness.
- SDK delivery is a product: SDK major tracks API major, minors track additive spec changes, CI publishes to npm/Maven Central/PyPI/Go on every `main` merge with OIDC provenance, a changelog, and a migration guide for majors. Wrap the generated core in a thin hand-written facade that adds retries, tracing, timeouts, and ergonomic types.
- Distinguish the registry (publish-time gate, schema storage) from the catalog (discovery, scorecards, deprecation visibility). The catalog — Backstage, Bump.sh, or Apicurio's UI — is what consumers browse; reconcile it against the gateway to surface shadow APIs. Staleness (`buf.lock` pin age, SDK adoption lag) is a metric, not an accident.

Further reading

- Confluent Schema Registry — subjects, compatibility, serializers, wire format. https://docs.confluent.io/platform/current/schema-registry/fundamentals/index.html

- Confluent — Schema evolution and compatibility types. https://docs.confluent.io/platform/current/schema-registry/fundamentals/schema-evolution.html
- Apicurio Registry — artifacts, groups, rules, Kafka SerDes, OpenAPI artifacts. https://www.apicurio.io/registry/docs/apicurio-registry/3.0.x/getting-started/assembly-intro-to-the-registry.html
- Buf Schema Registry (BSR) — modules, tracks, `buf push`, `buf breaking` against BSR. https://buf.build/docs/bsr/overview / https://buf.build/docs/bsr/module/track
- Buf — `buf generate`, `buf.gen.yaml`, managed mode, remote plugins. https://buf.build/docs/generate/overview / https://buf.build/docs/generate/managed
- Karapace — CSR-compatible registry for Kafka/Redpanda. https://docs.redpanda.com/current/manage/schema-reg/ / https://github.com/Aiven-Open/karapace
- OpenAPI Generator — generators, config, templates, `typescript-fetch` / `go` / `java`. https://openapi-generator.tech/docs/generators / https://openapi-generator.tech/docs/usage
- ConnectRPC — `buf generate` with `connectrpc/go` and `connectrpc/es` plugins. https://connectrpc.com/docs/
- Backstage — API catalog and discovery. https://backstage.io/docs/features/software-catalog/
- Bump.sh — API catalog, diff, and SDK hosting. https://bump.sh/ / https://docs.bump.sh/
- Conventional Commits + conventional-changelog — changelog generation for SDK releases. https://www.conventionalcommits.org/ / https://github.com/conventional-changelog/conventional-changelog
- npm provenance / PyPI trusted publishing / Maven Central — OIDC-backed publish. https://docs.npmjs.com/generating-provenance-

statements / https://docs.pypi.org/trusted-publishers/ / https://central.sonatype.org/publish/publish-guide/

Chapter 11 — Contract Testing and API Evolution in Practice

What this chapter covers. Chapters 8, 9, and 10 built the machinery that keeps schemas correct: the wire-format discipline that makes evolution safe, the linting and breaking-change gates that catch drift, and the registry + codegen pipeline that makes the schema the single source of truth. This chapter builds the machinery that proves the *implementation* honors that truth — and stays operable as it evolves. A schema that is compatible on paper but violated by a handler that returns a `string` where the spec promises an `int32` is a lie that the registry cannot catch. Contract testing is what catches it — not by testing the behavior of a service (unit/integration tests) nor by testing the whole graph end-to-end (E2E tests), but by testing the *contract* at the producer/consumer boundary so that both sides can evolve independently without a flag-day. We build the discipline from first principles: consumer-driven contracts (Pact) versus provider-driven schemas (OpenAPI + Dredd/Schemathesis), when to use each, how to wire them into CI as a verification matrix that scales with consumer count, and how to make API evolution a daily practice rather than a quarterly migration — expand-contract in code, deprecation and `Sunset` in headers, traffic-aware verification, and the observability that tells you when the old shape is safe to remove. Every pattern is anchored in runnable, version-pinned code (Pact 13+, OpenAPI + Schemathesis/Dredd, WireMock) and viewed through the distributed-systems lens where independent deployability is the invariant contract testing preserves.

Learning goals — after this chapter you should be able to:

- Distinguish contract tests from unit, integration, and E2E tests by what they isolate (the *interface* vs the *implementation* vs the *graph*), explain the test-pyramid economics that make E2E the wrong tool for contract verification at scale, and draw the boundary where contract tests own the guarantee.

- Author and verify consumer-driven contracts with Pact (consumer `given` / `uponReceiving` / `willRespondWith`, provider `verify` with state handlers), run them in CI with a Pact Broker (PactFlow or OSS), and interpret `can-i-deploy` as a deployment gate.
- Verify provider-driven contracts against an OpenAPI spec with Dredd, Schemathesis, and Prism — including `example` / `schema` alignment, property-based fuzzing, and the CI wiring that fails the build when the implementation diverges from the spec.
- Choose between consumer-driven (Pact) and provider-driven (OpenAPI) contract testing — or both — per surface (partner-facing REST, internal gRPC, event log) and explain how they compose rather than compete.
- Design API evolution as a production workflow: expand-contract with dual-write/dual-read, deprecation headers (`Deprecation` / `Sunset` / `Link: rel=sunset`), traffic-aware contract coverage, and the field-presence / error-rate metrics that tell you when to contract.
- Reason about contract testing through the distributed-systems lens: independent deployability without coordination, the verification matrix that replaces E2E combinatorial explosion, version-aware verification (provider verifies every consumer version it still serves), and why E2E is a deploy check but contracts are a merge check.

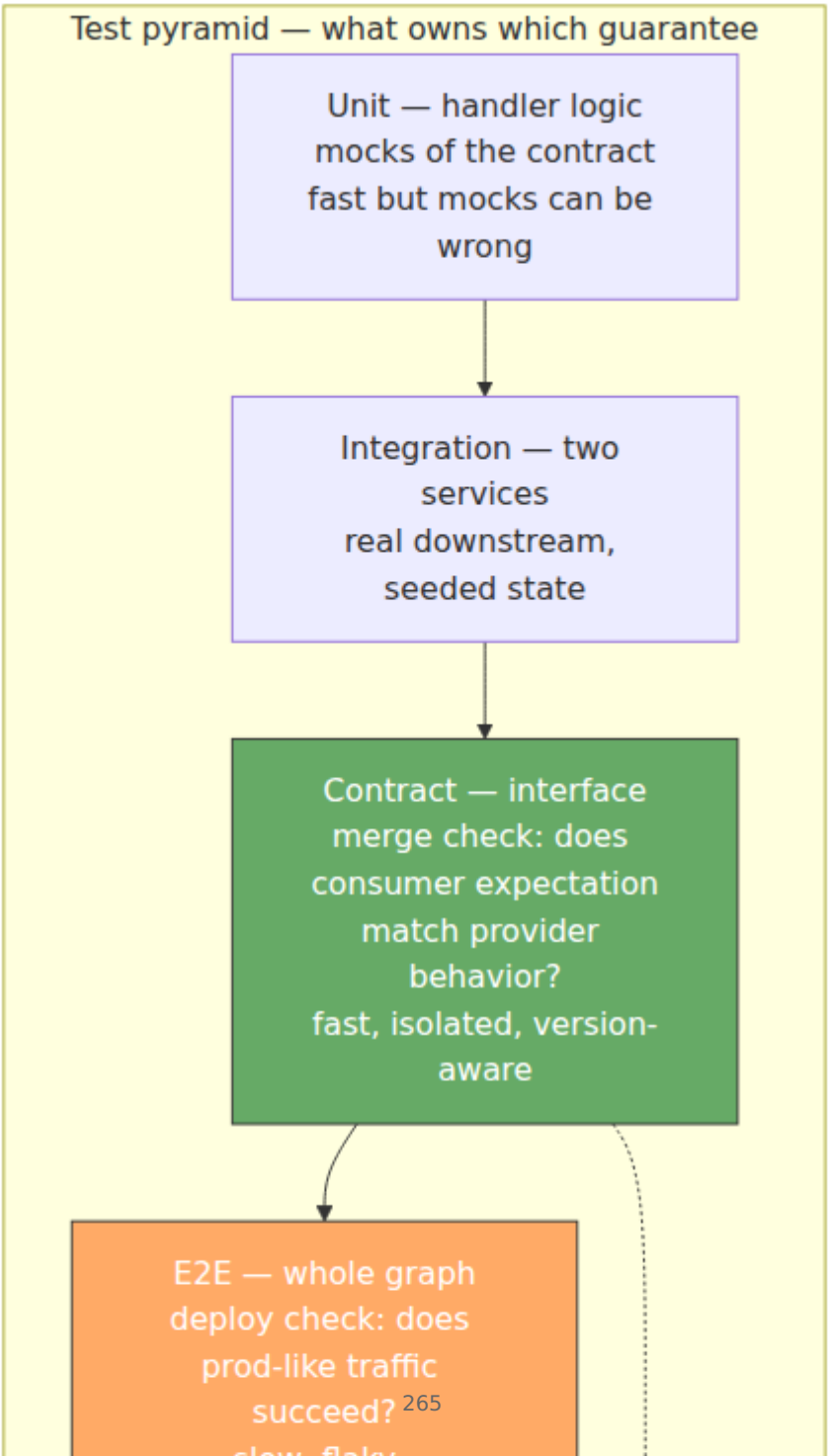
Why contracts need tests — the gap schemas leave

A schema is a *specification*. An implementation is a *behavior*. The two drift for mundane reasons: a handler returns `quantity: "2"` (string) where the schema promises `quantity: 2` (integer); a new code path returns a `404` body that omits the `code` field the error envelope requires (Chapter 7); a database migration changes a timestamp from `RFC 3339` to epoch millis and the handler faithfully serializes the new format without updating the spec. Linting (Chapter 9) checks the spec. Breaking-change detection checks the spec's evolution. Neither runs the implementation.

Three naive alternatives, and why they fail at scale:

Approach	What it proves	Why it fails at fleet scale
Unit tests against mocks/ hand-written stubs	The handler's logic, given a *mock* contract	Mocks are authored by the same team — they encode the same misunderstanding as the handler. The mock and the implementation can be consistently wrong.
Integration tests with a real downstream	The two services work *in this environment*	Requires the downstream to be running, seeded, and at the right version — flaky, slow, and couples deployments. Ten consumers × five provider versions = fifty environments.
E2E tests of the whole call graph	The graph works end-to-end	Slowest, flakiest, hardest to attribute (which hop broke?), and combinatorial — adding one service multiplies the graph. E2E is a deploy check, not a merge check.

Contract tests fill the precise gap: they prove the *consumer's expectation* and the *provider's behavior* agree on the *contract* — without requiring both to run together.


Test pyramid — what owns which guarantee
Unit — handler logic
mocks of the contract
fast but mocks can be
wrong
Integration — two
services
real downstream,
seeded state
Contract — interface
merge check: does
consumer expectation
match provider
behavior?
fast, isolated, version-
aware
E2E — whole graph
deploy check: does
prod-like traffic
succeed?

> **Distributed-systems lens.** Independent deployability is the invariant. In a system with fifty services, the cost of coordinating deploys ("deploy the provider, then deploy all ten consumers in order") is untenable. Contract tests preserve the ability for any service to deploy at any time — the consumer's contract is a *versioned expectation* that the provider verifies against every version it still serves. The provider does not need to be at the same commit as the consumer; it needs to satisfy every *published* contract that has not yet sunset.

Two families — consumer-driven versus provider-driven

Consumer-driven contracts (CDC) — Pact

The consumer writes the contract: "when I send *this* request, I expect *this* response." The consumer test generates a *pact* (a JSON file describing the interaction). The provider verifies that it can satisfy the pact. New consumers add new pacts; the provider verifies all of them. The name "consumer-driven" is literal — the consumer drives what is verified.

Best for: **partner-facing and team-to-team REST** where consumers are heterogeneous, the provider cannot enumerate every use case, and the consumer's actual usage (not the spec's theoretical surface) is what matters.

Provider-driven contracts — OpenAPI + Dredd/Schemathesis/Prism

The provider publishes the OpenAPI spec. Tools verify that the running provider *conforms* to the spec — every example validates against the schema, every response matches the declared type, and property-based fuzzing against the schema does not crash the handler.

Best for: **public REST** where the spec is the product, **event schemas** where the registry is the source of truth, and as a complement to Pact on internal surfaces.

They compose

Surface	Primary	Complement
Public REST (many external consumers)	Provider-driven (spec is the surface, consumers unknown)	Consumer-driven for key partners whose usage you want to lock
Internal REST between two teams	Consumer-driven (two sides, explicit expectations)	Provider-driven as a smoke test on the provider
gRPC / Protobuf	Provider-driven (proto is the contract, `buf breaking` already gates)	Consumer-driven via `pact-protobuf-plugin` when the consumer's field usage is narrower than the schema
Kafka / events	Provider-driven (registry + `FULL_TRANSITIVE`)	Consumer-driven (Pact event plugin or Avro-specific verification) when consumers project a subset of fields

The rest of the chapter builds both, then shows how evolution uses them.

Consumer-driven contracts — Pact in depth

Pact's model is four verbs: `given` (provider state), `uponReceiving` (request description), `withRequest` (request details), `willRespondWith` (expected response). The consumer test declares an interaction; Pact records it as a JSON pact; the provider replays it.

Consumer test — TypeScript (Pact 13+, Node 20+)

```
// pacts/orders-consumer.spec.ts — consumer: web front-end (Pact 13+, Vitest/Jest)
import { PactV4, MatchersV3, SpecificationVersion } from "@pact-foundation/pact";
import { OrdersClient } from "../src/orders-client"; // hand-written facade over generated client (Ch 10)

const { like, eachLike, regex, integer, uuid, iso8601DateTime } = MatchersV3;

const provider = new PactV4({
  consumer: "web-frontend",
  provider: "orders-service",
  spec: SpecificationVersion.SPECIFICATION_VERSION_V4,
  dir: "pacts/",                 // pact JSON written here for the broker
  pactfileWriteMode: "merge",
});

describe("Orders API — consumer contract (web-frontend → orders-service)", () => {
  it("creates an order and receives the created order", async () => {
    await provider
      .given("a customer exists with id cus_456")
      .uponReceiving("a request to create an order")
      .withRequest({
        method: "POST",
        path: "/v1/orders",
        // Pact matchers — flexible where the consumer is flexible, exact where it matters
        headers: { "Content-Type": "application/json", "Idempotency-Key": regex(".*", "[0-9a-f-]{8}-[0-9a-f-]{4}-[0-9a-f-]{4}-[0-9a-f-]{4}-[0-9a-f-]{12}") },
        body: {
          customer_id: "cus_456",
          quantity: integer(2),
          promo_code: like("SAVE20"),   // consumer accepts any string — like()
        },
      })
      .willRespondWith({
        status: 201,
        headers: { "Content-Type": "application/json" },
        body: {
          order_id: uuid("01h8x1abcdef1234567890abcd"),
          customer_id: "cus_456",
          quantity: integer(2),
          promo_code: like("SAVE20"),
          status: regex("PENDING", "PENDING|PAID|SHIPPED|CANCELLED"),
          created_at: iso8601DateTime("2026-03-10T14:22:31Z"),
```

```
        },
      })
      .executeTest(async (mockServer) => {
        const client = new OrdersClient({ baseUrl: mockServer.url, toke
n: "test-token" });
        const order = await client.createOrder({ customerId:
"cus_456", quantity: 2, promoCode: "SAVE20" });
        expect(order.orderId).toMatch(/^[0-9a-z]{26}$/);
        expect(order.status).toBe("PENDING");
      });
  });

  it("lists orders with cursor pagination", async () => {
    await provider
      .given("customer cus_456 has 3 orders")
      .uponReceiving("a request to list orders with pagination")
      .withRequest({
        method: "GET",
        path: "/v1/orders",
        query: { customer_id: "cus_456", page_size: "2" },
      })
      .willRespondWith({
        status: 200,
        headers: { "Content-Type": "application/json" },
        body: {
          data: eachLike({                              // consumer
expects an array of order shapes
            order_id: uuid("01h8x1abcdef1234567890abcd"),
            status: regex("PENDING", "PENDING|PAID|SHIPPED|CANCELLED"),
            quantity: integer(2),
          }, { min: 1 }),
          pagination: {
            next_page_token: like("eyJpZCI6Im9yZF8xMjMifQ=="),
            total_count: integer(3),
          },
        },
      })
      .executeTest(async (mockServer) => {
        const client = new OrdersClient({ baseUrl: mockServer.url, toke
n: "test-token" });
        const page = await client.listOrders({ customerId: "cus_456", p
ageSize: 2 });
        expect(page.data.length).toBeGreaterThan(0);
        expect(page.pagination.totalCount).toBe(3);
      });
  });
});
```

```
# Run consumer tests — writes pacts/web-frontend-orders-service.json
npx vitest run pacts/orders-consumer.spec.ts
# or: npm test -- pacts/

# Output — pact JSON (abridged, V4):
# {
#   "consumer": { "name": "web-frontend" },
#   "provider": { "name": "orders-service" },
#   "interactions": [{
#     "description": "a request to create an order",
#     "providerStates": [{ "name": "a customer exists with id
cus_456" }],
#     "request": { "method": "POST", "path": "/v1/orders", "body":
{...}, "matchingRules": {...} },
#     "response": { "status": 201, "body": {...}, "matchingRules":
{...} }
#   }]
# }
```

Matchers — the flexibility contract. `like("SAVE20")` means "any string matching this example" — not "exactly SAVE20." `regex`, `integer`, `uuid`, `iso8601DateTime`, `eachLike` encode the *shape* the consumer depends on, not the example value. This is what keeps pacts from being brittle: the consumer declares what varies (IDs, timestamps) and what is exact (`customer_id: "cus_456"` is exact — the consumer looks it up by that key).

Provider verification — Go + Pact Broker (Pact 13+, Go 1.22+)

The provider replays every interaction against its real handler. `given` maps to state setup; the request is sent to the handler under test; the response is matched.

```
// provider/pact_verify_test.go — provider: orders-service (Go 1.22+,
Pact 2.x Go DSL)
package provider_test

import (
    "fmt"
    "net/http"
    "testing"

    "github.com/pact-foundation/pact-go/v2/models"
    "github.com/pact-foundation/pact-go/v2/provider"
    "github.com/pact-foundation/pact-go/v2/utils"
)

func TestPactProvider(t *testing.T) {
    // 1) Start the provider under test (real handler, test database/
seed)
    //    In CI this is the built binary or the handler with a test
double for storage.
    srv := newTestServer(t) // returns *httptest.Server with the real
router
    defer srv.Close()

    // 2) State handlers — given("...") maps to setup/teardown
    stateHandlers := models.StateHandlers{
        "a customer exists with id cus_456": func(setup bool, s
models.ProviderState) error {
            if setup {
                return seedCustomer(s.Params["id"] /* or s.Params —
depending on Pact version */)
                // e.g. INSERT INTO customers (id) VALUES ('cus_456')
in test DB
            }
            return teardownCustomer("cus_456")
        },
        "customer cus_456 has 3 orders": func(setup bool, s models.Prov
iderState) error {
            if setup {
                return seedOrders("cus_456", 3)
            }
            return teardownOrders("cus_456")
        },
    }

    verifier := provider.NewVerifier()
    err := verifier.VerifyProvider(t, provider.VerifyRequest{
        Provider:              "orders-service",
        ProviderBaseURL:       srv.URL,
        PactBrokerURL:         utils.GetEnv("PACT_BROKER_URL", "http://
pact-broker:9292"),
```

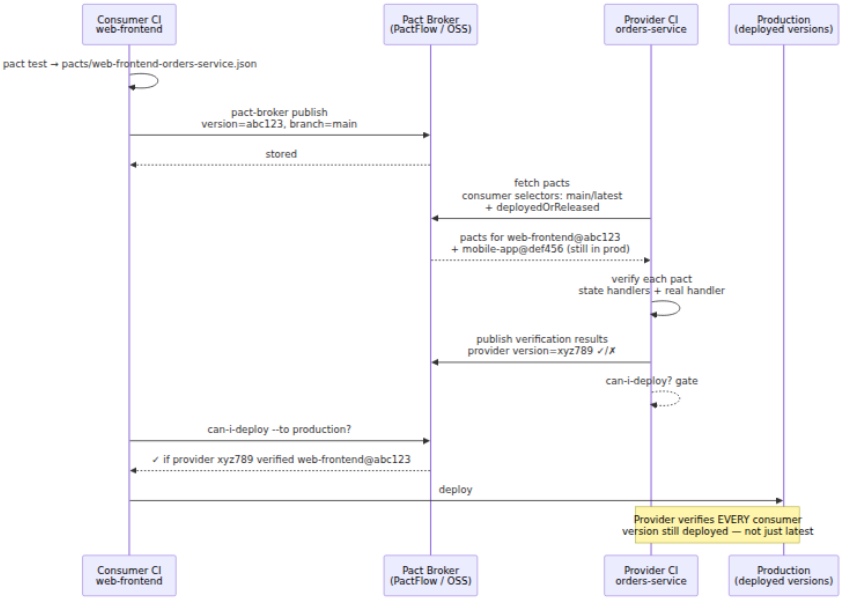
```
        BrokerToken:           utils.GetEnv("PACT_BROKER_TOKEN", ""),
        PublishVerificationResults: true,
        ProviderVersion:       utils.GetEnv("GIT_SHA", "local"),
        ProviderBranch:        utils.GetEnv("GIT_BRANCH", "main"),
        ConsumerVersionSelectors: []models.Selector{
            // Verify against every consumer version still in
production (not just latest)
            {Tag: "main", Latest: true},
            {Tag: "production", Latest: true},
            // Or: deployedOrReleased:true (PactFlow) — every version
with a deployed/prod tag
            {DeployedOrReleased: true},
        },
        StateHandlers: stateHandlers,
        // Optional — verify only pacts for this provider version's
branch (speed)
        // ProviderTags: []string{"main"},
    })
    if err != nil {
        t.Fatalf("pact verification failed: %v", err)
    }
}

func newTestServer(t *testing.T) *http.Server {
    // Wire the real router with a test double for storage — not mocks
of the contract
    // handlers.NewRouter(storage.NewMemoryStore()) or a test Postgres
    t.Helper()
    // ...
    return nil // placeholder — replace with real test server
}
func seedCustomer(id string) error  { fmt.Println("seed", id); return n
il }
func teardownCustomer(id string) error { return nil }
func seedOrders(customerID string, n int) error { return nil }
func teardownOrders(customerID string) error { return nil }
```

```yaml
# .github/workflows/pact-provider.yaml — verify on every provider PR
(Pact 13+, Go 1.22+)
name: pact-provider
on:
  pull_request:
    paths: ["provider/**", "openapi.yaml", "proto/**"]

jobs:
  verify:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_PASSWORD: test }
        ports: ["5432:5432"]
        options: --health-cmd="pg_isready" --health-interval=5s
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v5
        with: { go-version: "1.22" }
      - run: go test ./provider -run TestPactProvider -count=1
        env:
          PACT_BROKER_URL: ${{ secrets.PACT_BROKER_URL }}
          PACT_BROKER_TOKEN: ${{ secrets.PACT_BROKER_TOKEN }}
          GIT_SHA: ${{ github.sha }}
          GIT_BRANCH: ${{ github.head_ref || github.ref_name }}
```

Pact Broker and `can-i-deploy`

The Broker is the store of record for pacts and verification results. PactFlow (hosted) or the OSS `pact-broker` (Ruby, Docker `pactfoundation/pact-broker:2.113+`) both expose the same API.

```
# Publish consumer pacts to the broker (Pact CLI 2.x / PactFlow)
pact-broker publish pacts/ \
  --consumer-app-version "$(git rev-parse --short HEAD)" \
  --branch "$(git branch --show-current)" \
  --broker-base-url "$PACT_BROKER_URL" \
  --broker-token "$PACT_BROKER_TOKEN"

# Tag as deployed/released (so DeployedOrReleased selectors work)
pact-broker create-version-tag \
  --pacticipant web-frontend --version "$(git rev-parse --short HEAD)"
--tag production \
  --broker-base-url "$PACT_BROKER_URL" --broker-token "$PACT_BROKER_TOK
EN"

# Gate — can this version deploy? (fails if any required verification
is missing/red)
pact-broker can-i-deploy \
  --pacticipant web-frontend --version "$(git rev-parse --short HEAD)"
\
  --to-environment production \
  --broker-base-url "$PACT_BROKER_URL" --broker-token "$PACT_BROKER_TOK
EN"
# Exit 0 = all required pacts verified; non-zero = blocked — see which
pact failed

# Record deployment (so the matrix knows what is live)
pact-broker record-deployment \
  --pacticipant web-frontend --version "$(git rev-parse --short HEAD)"
--environment production \
  --broker-base-url "$PACT_BROKER_URL" --broker-token "$PACT_BROKER_TOK
EN"

# Docker — OSS broker (no PactFlow account needed)
# docker run -d -p 9292:9292 --name pact-broker pactfoundation/pact-
broker:2.113.0
# Broker UI: http://localhost:9292  — matrix view: which consumer
versions are verified by which provider versions
```



Version-aware verification is the key insight for distributed systems: the provider does not verify against "the consumer" — it verifies against *every pact version that is still deployed or released*. A consumer that ships weekly will have several versions in prod (canary, blue/green, mobile binaries). `DeployedOrReleased: true` (or `Tag: production` + `Tag: main`) ensures the provider cannot break an older consumer that has not yet upgraded.

Provider-driven contracts — the spec as the test

When the consumer set is open-ended (public API) or the spec is short-lived (events), verifying the implementation against its own spec is more direct than waiting for consumers to publish pacts.

Prism — mock server from OpenAPI (Stoplight Prism 5.6+)

Prism turns `openapi.yaml` into a mock server that validates requests and generates spec-compliant responses — useful as a local development double and as a validation proxy.

```
# Prism 5.6+ — mock server (Node 20+)
npx @stoplight/prism-cli@5.6.0 mock openapi.yaml --port 4010 --errors
# Mock server at http://localhost:4010
# --errors = return 422 when request fails schema validation (strict
mode)
# --validate-requests + --validate-responses = enforce both directions

# Validate that examples in the spec are schema-valid (catches spec
bugs)
npx @stoplight/prism-cli@5.6.0 mock openapi.yaml --validate-requests --
validate-responses --errors
curl -s http://localhost:4010/v1/orders/ord_123 | jq .
```

Dredd — implementation conformance (Dredd 14.3+)

Dredd replays the `example` and `schema` definitions in `openapi.yaml` against the running provider and asserts the responses match.

```
# dredd.yaml — Dredd 14.3+ (Node 20+), OpenAPI 3.1
# Docs: https://dredd.org/en/latest/
color: true
dry-run: null
hookfiles: ./dredd-hooks.js
language: nodejs
only: []
server: npm start          # or: go run ./cmd/orders-service
server-wait: 5
endpoint: http://localhost:3000
blueprint: openapi.yaml    # or: openapi.yaml (Dredd reads OpenAPI via
api-description)
```

```
// dredd-hooks.js — seed state before the transaction that needs it
(Dredd 14.3+)
const hooks = require('hooks');

hooks.before('/v1/orders > POST > 201 > application/json',
(transaction, done) => {
  // Seed the state that the 201 example assumes
  transaction.request.body = JSON.stringify({ customer_id: "cus_456", q
uantity: 2 });
  done();
});

hooks.beforeValidation('/v1/orders > GET > 200 > application/json', (tr
ansaction, done) => {
  // Loosen validation where the spec is intentionally flexible (e.g.,
total_count is int64)
  done();
});
```

```
npx dredd@14.3.0 --config dredd.yaml
# pass: POST /v1/orders 201 — response matches schema
# fail: GET /v1/orders/ord_xyz 200 — body missing required field
'status'
# Complete: 12 passing, 1 failing
```

Schemathesis — property-based fuzzing against the schema (Schemathesis 3.38+)

Schemathesis reads the schema and *generates* requests — valid, invalid, and edge-case — to find crashes, 500s, and contract violations that examples do not cover. It is the most powerful provider-driven tool for hardening.

```
# Schemathesis 3.38+ — property-based API testing (Python 3.11+)
pip install schemathesis==3.38.0 hypothesis==6.100.0

# 1) Smoke — every example in the spec returns a schema-valid response
schemathesis run openapi.yaml --base-url http://localhost:3000 --
checks all

# 2) Fuzz — generate valid + invalid requests from the schema and
assert:
#    no 500s, every response matches the schema, documented errors are
correct
schemathesis run openapi.yaml --base-url http://localhost:3000 \
  --checks all \
  --hypothesis-max-examples 200 \
  --workers 4

# 3) In CI — fail on any 500 or schema mismatch, report as JUnit for
GitHub
schemathesis run openapi.yaml --base-url http://localhost:3000 \
  --checks all \
  --hypothesis-max-examples 100 \
  --junit-xml junit.xml

# 4) Stateful — follow links (create then fetch) to exercise sequences
schemathesis run openapi.yaml --base-url http://localhost:3000 \
  --checks all --stateful links

# Example failure — handler returns string quantity where schema says
integer:
# FAILED — POST /v1/orders  — response body $.quantity: "2" is not of
type "integer"
```

```yaml
# .github/workflows/openapi-contract.yaml — Schemathesis + Dredd on
every provider PR
name: openapi-contract
on:
  pull_request:
    paths: ["openapi.yaml", "provider/**", "src/**"]

jobs:
  prism-validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npx @stoplight/prism-cli@5.6.0 mock openapi.yaml --
validate-requests --validate-responses --errors &
      - run: npx wait-on http://localhost:4010 && echo "Prism mock
validated spec examples"

  dredd:
    runs-on: ubuntu-latest
    services:
      postgres: { image: postgres:16, env: { POSTGRES_PASSWORD:
test }, ports: ["5432:5432"], options: --health-cmd="pg_isready" --
health-interval=5s }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm ci
      - run: npx dredd@14.3.0 --config dredd.yaml --reporter junit --
output dredd-junit.xml
      - uses: actions/upload-artifact@v4
        if: always()
        with: { name: dredd-junit, path: dredd-junit.xml }

  schemathesis:
    runs-on: ubuntu-latest
    services:
      postgres: { image: postgres:16, env: { POSTGRES_PASSWORD:
test }, ports: ["5432:5432"], options: --health-cmd="pg_isready" --
health-interval=5s }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v4
        with: { python-version: "3.11" }
      - run: pip install schemathesis==3.38.0 hypothesis==6.100.0
      - run: npm ci && npm start &
      - run: npx wait-on http://localhost:3000
      - run: schemathesis run openapi.yaml --base-url http://
```

```
localhost:3000 --checks all --hypothesis-max-examples 100 --junit-xml
junit.xml
      - uses: actions/upload-artifact@v4
        if: always()
        with: { name: schemathesis-junit, path: junit.xml }
```

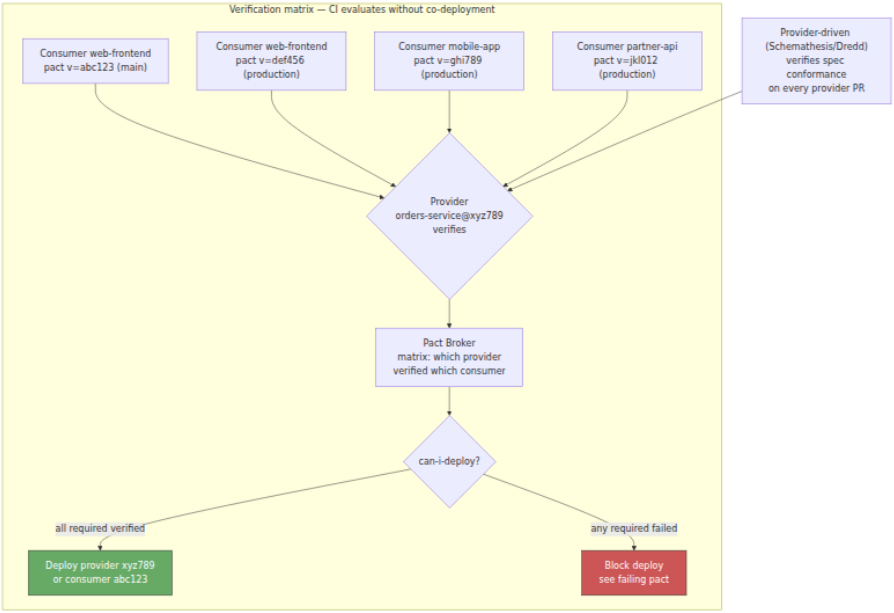
Legacy and staging traffic — Optic and coverage

Optic (`useoptic/optic` 0.40+) and similar tools compare the spec against *observed* traffic — finding endpoints the spec does not document and requests the tests do not cover. WireMock / Mountebank as record-replay proxies serve the same role for legacy services where the contract was never written down.

```
# Optic 0.40+ — diff spec vs observed traffic (informational,
complements Pact/Schemathesis)
npx @useoptic/optic@0.40.0 diff openapi.yaml --base origin/
main:openapi.yaml --check
# Undocumented endpoint: GET /v1/orders/internal/reindex — add to spec
or remove
```

The verification matrix — contracts at fleet scale

With N consumers and M provider versions, naive E2E needs N×M environments. Contracts collapse it to a verification matrix that CI evaluates without running both sides together:

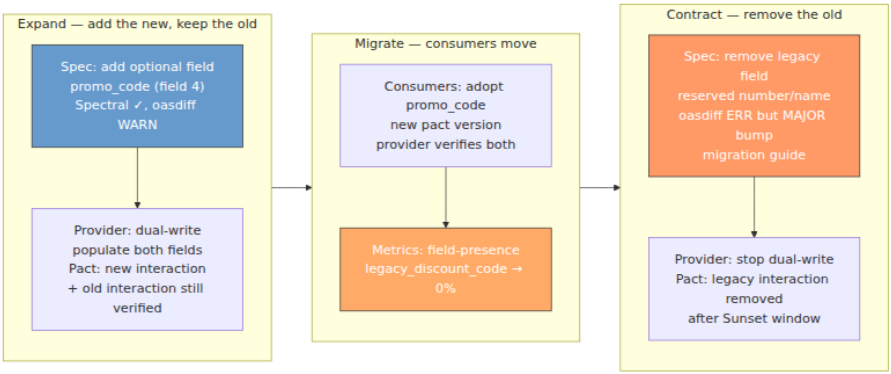


The key property: the matrix grows *additively* (one more pact per consumer release, one more verification per provider release), not *multiplicatively* (one more E2E environment per N×M combination). At ten consumers with five versions in prod, E2E needs fifty environments; contracts need ten pacts and one provider verification that iterates over ten pacts.

API evolution in practice — the daily discipline

Contract tests make evolution a merge-time concern. The workflow that keeps the system moving without flag-days:

1) Expand-contract — additive first, removal last (see Chapter 8)




```
// Consumer contract during expand — both fields expected during the
window
willRespondWith({
  status: 200,
  body: {
    order_id: uuid("01h8x1abcdef1234567890abcd"),
    promo_code: like("SAVE20"),                          // new
canonical
    legacy_discount_code: like("SAVE20"),                // old — still
verified until contract
  },
})
```

2) Deprecation headers — the removal schedule

```
HTTP/1.1 200 OK
Deprecation: true
Sunset: Sat, 31 Dec 2026 23:59:59 GMT
Link: <https://docs.acme.internal/deprecations/legacy-discount>;
rel="sunset"
Link: <https://docs.acme.internal/migrations/orders-1-to-2>;
rel="deprecation"
Warning: 299 - "legacy_discount_code is deprecated, use promo_code.
Sunset 2026-12-31."

{
  "data": { "order_id": "ord_123", "promo_code": "SAVE20" },
  "meta": { "warnings": ["legacy_discount_code is deprecated — migrate
to promo_code by 2026-12-31"] }
}
```

3) Traffic-aware coverage — what contracts do not cover

Contract tests verify the *declared* interactions. They do not verify what no consumer declared but production still sends. Complement with:

- **WireMock / production traffic shadowing** — record real traffic, replay against the new provider, diff responses (like Optic's traffic diff but for verification).
- **Schema coverage** — "which response fields are exercised by at least one contract" — a coverage report analogous to code coverage. Schemathesis's `--checks all` plus `eachLike` / `like` matchers drive coverage up; audit the gap.
- **Error-path contracts** — most pacts only cover the happy path. Add interactions for `400`, `404`, `422`, `429`, `503` (Chapter 7) — the error envelope is part of the contract.

4) Observability — when to contract

Do not guess when the old shape is safe to remove. Measure:

Signal	How	Contract when
Field-presence	`field_present{field="legacy_discount_code"}` from gateway or handler	0% for longer than the retention/replay window (Kafka) and the mobile-binary tail (public API)
Unknown-field count	Protobuf `unknown_fields` counter on the consumer or proxy	Spike confirms new producer is live; flat zero confirms old consumers are gone
Deserialize-failure rate	`InvalidProtocolBufferException` / `JsonMappingException` rate during rollout	Must stay flat during expand; rise is a break
Contract verification lag	Pact Broker matrix — which consumer versions are still `deployed`	No `production`-tagged consumer still verifies against the legacy interaction

Signal	How	Contract when
Gateway traffic by version	`http_requests{api_version="1"}` vs `api_version="2"`	Old version < 1% for one full release cycle

Only when all of the above agree is the old field safe to `reserved` / `x-sunset` / `DROP COLUMN` (Chapters 8, 10).

---

gRPC and event contracts — beyond REST

gRPC — Pact's Protobuf plugin

Pact's core is HTTP, but the Protobuf plugin (`pact-protobuf-plugin`) extends it to gRPC:

```
# Pact Protobuf plugin — consumer test for gRPC (Pact 13+, pact-protobuf-plugin 0.3+)
# Consumer declares the Protobuf interaction; Pact generates a pact with Protobuf payload
# Provider verifies via the gRPC handler (same state-handler pattern, gRPC transport)
```

For most gRPC estates, provider-driven verification (`buf breaking` + generated-code compile + handler unit tests against the generated types) is sufficient — the schema's `buf lint` + `buf breaking` already enforces compatibility, and the generated stubs make type mismatches compile errors. Add Pact's Protobuf plugin when the consumer projects a narrow subset of fields and you want to lock that projection.

Events — Avro/Protobuf on Kafka

Event contracts have two layers: the *registry* (Confluent/Apicurio `FULL_TRANSITIVE` — Chapter 10) guarantees wire compatibility, and the *contract test* guarantees the producer actually writes bytes the consumer can deserialize and the consumer actually handles every event the producer emits. Tools:

- **Pact's event plugin** — declare an event interaction (like an HTTP interaction but with a topic/queue address).

- **Spec-driven event verification** — generate events from the Avro/JSON Schema and assert the consumer's handler does not throw (similar to Schemathesis but for consumers).
- **Kafka-specific** — `kafka-schema-registry` + `avro-maven-plugin` + a test that produces with `KafkaAvroSerializer(autoRegister=false)` and consumes with `KafkaAvroDeserializer` — the same wiring as Chapter 10, but under test.

The distributed-systems lens — contracts as coordination

E2E is a deploy check; contracts are a merge check

E2E proves the *deployed* graph works — the canary analysis, the smoke test after a deploy, the synthetic that runs every minute in prod. It is valuable *after* merge. Contracts prove the *proposed* change will not break a peer — they run *before* merge, on the PR, without deploying either side. Confusing the two leads to the antipattern of "we need E2E to test the contract" — which means every PR waits for a full environment, and the feedback loop is hours, not minutes.

Independent deployability without coordination

The property contracts preserve is that any service can deploy at any time without coordinating with its peers — provided it satisfies every *published* contract that has not yet sunset. This is the service-boundary analogue of linearizability (Vol 6, Ch 3): the contract is the linearization point between consumer expectation and provider behavior, and the Broker's matrix is the log that proves they agreed.

The version problem that contracts solve

In a fleet with fifty services, a breaking provider change that is "compatible with the latest consumer" still breaks the four consumers that have not yet upgraded. Version-aware verification (`DeployedOrReleased: true`, `Tag: production`) is the fix — the provider verifies against every version still in production, not just `main`. The lifecycle that makes it tractable: consumers publish pacts tagged `production` on deploy, providers verify with `DeployedOrReleased`, and `can-i-deploy` gates both sides. When a consumer's `production` tag

moves past a contract, the old pact is no longer required — the provider can safely remove the legacy shape after the `Sunset` window.

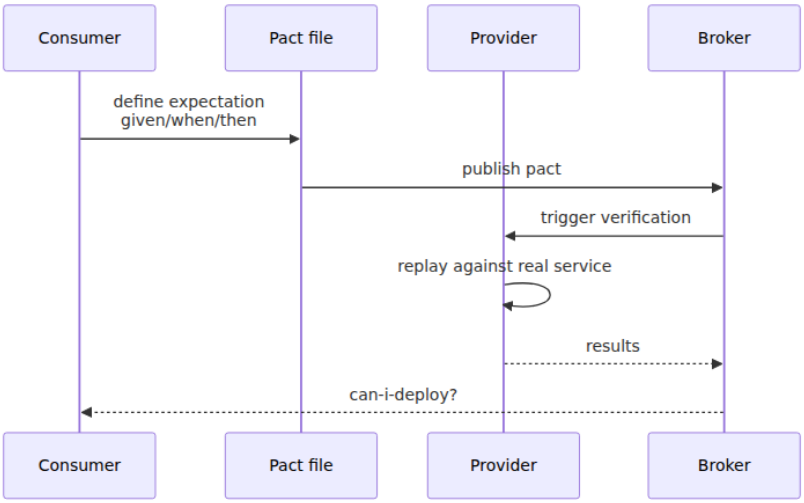
Anti-patterns (and what to do instead)

Anti-pattern	Why it hurts	Fix
E2E as the contract test — "spin up all ten services to test one field"	Slow, flaky, combinatorial, couples deployments; failures unattributable	Contract tests on the PR (Pact/Schemathesis — minutes, isolated); E2E as a post-deploy smoke
Mocks of the contract authored by the provider team	Mock and implementation share the same misunderstanding — consistently wrong	Consumer authors the pact (consumer-driven); provider verifies against it — two teams, one truth
Pacts without matchers — exact `promo_code: "SAVE20"`	Pact breaks on every new example value — brittle, teams disable it	`like("SAVE20")`, `regex`, `uuid`, `iso8601DateTime` — match the shape, not the example
Provider verifies only `main` latest	Breaks four consumers still on older versions in prod (canary, mobile binary tail)	`DeployedOrReleased: true` or `Tag: production` — verify every deployed version
Happy-path-only pacts — no error interactions	Error envelope drift (Chapter 7) undetected; retry/circuit-breaker breakage invisible	Add interactions for `400` / `404` / `422` / `429` / `503` — the error contract is part of the interface

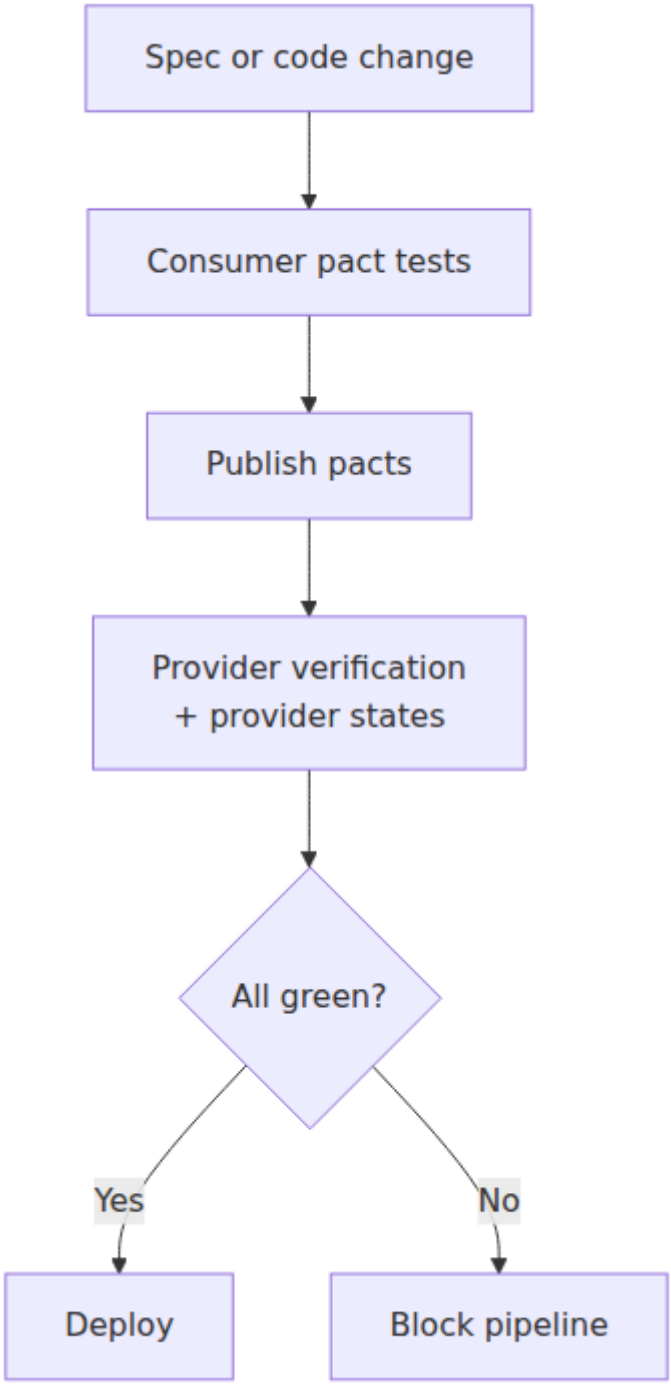
Anti-pattern	Why it hurts	Fix
`AUTO_REGISTER_SCHEMAS=true` with no contract test	Producer publishes incompatible bytes that pass the registry (`NONE` compat) but break the consumer	`AUTO_REGISTER_SCHEMAS=false` + contract tests + `FULL_TRANSITIVE` on replayable topics (Ch 10)
Contract tests that require a real downstream (integration tests relabeled)	Flaky, needs seeded state across services, couples CI	Consumer test uses `PactV4` mock server; provider test uses state handlers + test double for storage — no real peer required

---

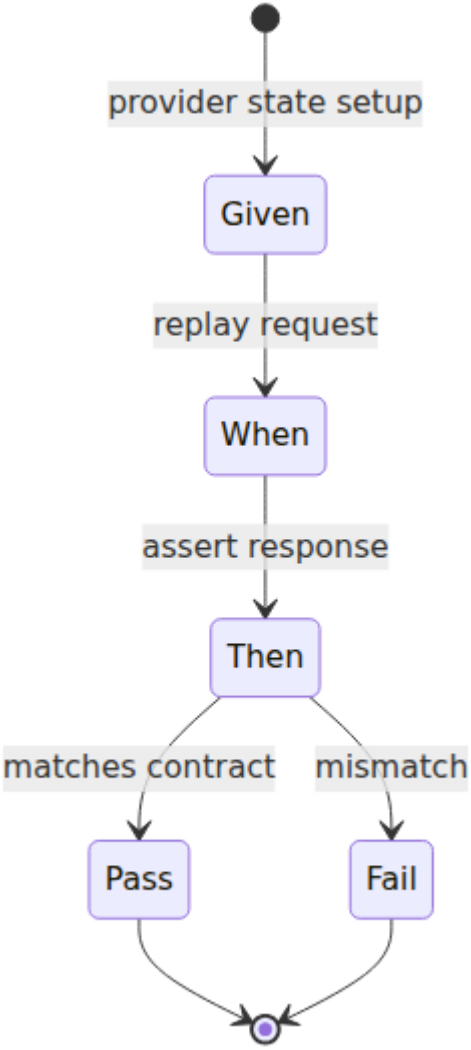
Pact Consumer-Driven Flow



Contract Test CI Pipeline


Spec or code change
Consumer pact tests
Publish pacts
Provider verification
+ provider states
All green?
Yes
No
Deploy
Block pipeline

Provider Verification States



Key takeaways

- A schema proves the *specification* is compatible; a contract test proves the *implementation* honors it. Schemas catch evolution errors in the spec; contracts catch drift between spec and handler —

neither replaces the other, and the registry cannot catch a handler that returns `string` where the spec promises `integer`.

- Contract tests isolate the *interface* — faster and more attributable than E2E, and not consistently wrong like mocks authored by the same team. Layer correctly: unit (handler logic) → contract (interface, merge check) → integration (two real services) → E2E (whole graph, deploy check).
- Consumer-driven contracts (Pact 13+) put the consumer in charge: `given` / `uponReceiving` / `withRequest` / `willRespondWith` with `like` / `regex` / `uuid` / `eachLike` matchers, pacts published to a Broker (PactFlow/OSS), provider verification via state handlers and `DeployedOrReleased:true`, and `can-i-deploy` as a deployment gate. Use when consumers are heterogeneous and the consumer's actual usage is what matters.
- Provider-driven contracts (`Dredd` 14.3+, `Schemathesis` 3.38+, `Prism` 5.6+) verify the running provider against its own OpenAPI spec — examples vs schema, response conformance, and property-based fuzzing that finds 500s and type mismatches no example covered. Best for public REST and as a complement to Pact internally.
- The verification matrix — not N×M E2E environments — is what scales. One pact per consumer release, one provider verification that iterates over every pact still `deployed` / `released` — additive, not multiplicative. `can-i-deploy` gates both consumer and provider deploys.
- Evolution is expand-contract with telemetry: add optional + dual-write, migrate consumers (new pact version), measure field-presence / unknown-field / deserialize-failure / gateway version traffic, and only `reserved` / `DROP` after the `Sunset` window and all signals agree. Deprecation headers (`Deprecation`, `Sunset`, `Link: rel=sunset`) make the schedule machine-readable.
- Cover the gaps: error-path contracts (`400` / `404` / `422` / `429` / `503`), traffic-aware diffing (Optic), WireMock/production-shadowing for undeclared usage, and schema coverage ("which response fields have at least one contract").
- For gRPC, `buf breaking` + generated-code compilation already enforce most of the contract — add `pact-protobuf-plugin` when the consumer's field projection is narrow. For Kafka, registry

(`FULL_TRANSITIVE`) + contract tests on both producer and consumer (Pact event plugin or Avro SerDes under test) close the loop.

Further reading

- Pact — Consumer-driven contracts, Pact V4 spec, matchers, Broker, `can-i-deploy`. https://docs.pact.io/ / https://docs.pact.io/pact_broker / https://docs.pact.io/pact_broker/can_i_deploy
- Pact — Language guides (JS/TS `@pact-foundation/pact` 13+, Go `pact-go` 2.x, Java `pact-jvm`, Python `pact-python`). https://docs.pact.io/implementation_guides/
- PactFlow — Hosted Pact Broker, `deployedOrReleased` selectors. https://docs.pactflow.io/docs/bi-directional-contracts
- Pact Protobuf Plugin — gRPC contract testing. https://github.com/pact-foundation/pact-protobuf-plugin
- Dredd — HTTP API testing against OpenAPI/API Blueprint. https://dredd.org/en/latest/ / https://github.com/apiaryio/dredd
- Schemathesis — Property-based OpenAPI testing (Hypothesis). https://schemathesis.readthedocs.io/en/stable/ / https://github.com/schemathesis/schemathesis
- Stoplight Prism — OpenAPI mock server, request/response validation. https://docs.stoplight.io/docs/prism / https://github.com/stoplightio/prism
- Optic — Traffic-aware OpenAPI diff and coverage. https://www.useoptic.com/docs / https://github.com/useoptic/optic
- WireMock / Mountebank — Record-replay and traffic shadowing for legacy and staging. https://wiremock.org/docs/ / http://www.mbtest.org/
- RFC 8594 — The Sunset HTTP Header Field. https://www.rfc-editor.org/rfc/rfc8594.html
- draft-ietf-httpapi-deprecation-header — The Deprecation HTTP Header Field. https://www.ietf.org/archive/id/draft-ietf-httpapi-deprecation-header-02.html
- *Consumer-Driven Contracts* — Ian Robinson (martinfowler.com). https://martinfowler.com/articles/consumerDrivenContracts.html
- *Testing Microservices: Contract Tests* — Pact / Atlassian. https://www.atlassian.com/continuous-delivery/principles/contract-tests